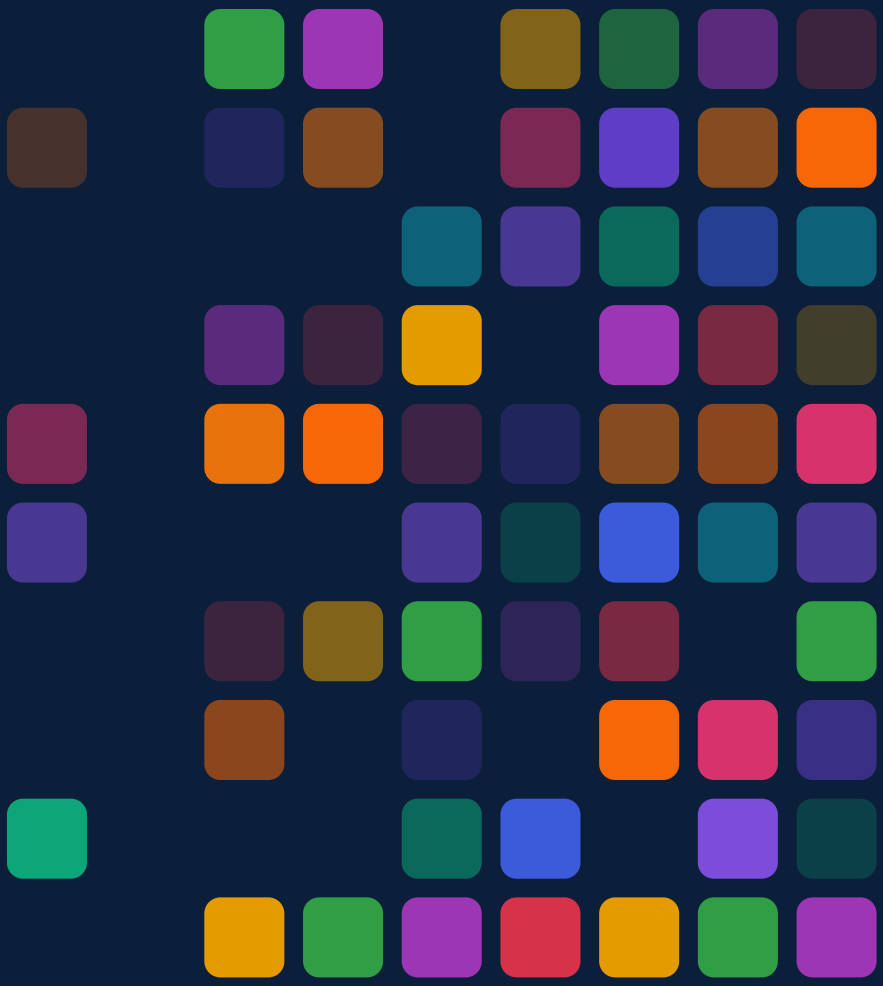


EDICIÓN 2026

GUÍA VISUAL DE

AWS

Amazon Web Services

Los conceptos esenciales para diseñar, construir y operar en la nube: de IAM y las redes a los contenedores y Kubernetes, las bases de datos, la mensajería, el cifrado y la entrega de contenido.

IAM · VPC · EC2 · ELB · ECS · EKS · Lambda · S3 · RDS · DynamoDB · SQS · SNS · CloudFront · CloudWatch · KMS · CloudFormation

ANTES DE EMPEZAR

Cómo leer esta guía

Esta guía recorre los servicios y conceptos de AWS que aparecen una y otra vez en cualquier proyecto real: identidad y permisos, redes, cómputo, contenedores y Kubernetes, *serverless*, almacenamiento, bases de datos, mensajería, entrega de contenido, observabilidad, seguridad y cifrado, infraestructura como código y costes. Cada capítulo parte del *porqué* de sus servicios, construye los conceptos paso a paso y termina con un resumen y preguntas de repaso; el último, *Arquitecturas de referencia*, junta todas las piezas en diseños completos.

Puedes leerla de principio a fin —el orden está pensado para que cada capítulo se apoye en los anteriores— o saltar directamente al servicio que necesites. Los capítulos de *Fundamentos*, *IAM* y *Redes* son la base de todo lo demás: si eres nuevo en AWS, empieza por ellos. IAM ocupa dos capítulos a propósito: el primero responde a *quién* (usuarios, grupos, roles y *permission sets*) y el segundo a *qué puede hacer* (políticas y evaluación de permisos). Es el tema que más dudas genera y el que más conviene dominar. También tienen espacio propio *Kubernetes y Amazon EKS*, justo después de *Contenedores*, y el cifrado con KMS, que ocupa la primera mitad de *Seguridad*. Al final, el *Glosario* y la *Chuleta de referencia rápida* reúnen términos, cifras clave y comandos para consultar en cualquier momento.

Cajas que encontrarás a lo largo del texto

NOTA

Aclaraciones y matices que conviene conocer.

CONSEJO

Atajos y recomendaciones prácticas del día a día.

CUIDADO

Errores habituales y comportamientos que sorprenden.

ANALOGÍA

Comparaciones con el mundo real para fijar conceptos.

CLAVE PARA LA CERTIFICACIÓN

Ideas que se repiten en las certificaciones de AWS (Cloud Practitioner, Solutions Architect, Developer, SysOps).

COSTES

Cómo se factura un servicio y dónde se esconden los sobrecostes.

BUENA PRÁCTICA

Recomendaciones alineadas con el AWS Well-Architected Framework.

Convenciones

- Los nombres de servicios y la terminología técnica se mantienen en inglés, como los verás en la consola y la documentación (*bucket*, *Security Group*, *listener*…), explicados la primera vez que aparecen.
- Los comandos usan la AWS CLI v2 (`aws ...`). Los identificadores como `123456789012` o `mi-bucket` son ejemplos.
- Los precios y límites cambian con frecuencia y dependen de la región: tómalos como orientación y consulta siempre la documentación oficial y la calculadora de precios de AWS.

PANORAMA

Mapa de servicios

AWS ofrece más de doscientos servicios, pero la inmensa mayoría de arquitecturas se construyen con un núcleo reducido. Este mapa agrupa por categoría los que trata esta guía, en el mismo orden que los capítulos; el color de cada categoría es el que verás en su capítulo y en los diagramas.

IDENTIDAD Y GOBIERNO

IAM usuarios, roles, políticas
IAM Identity Center acceso centralizado
Organizations multicuenta, SCP
STS credenciales temporales

REDES

VPC red privada virtual
Transit Gateway hub de redes
PrivateLink servicios privados
VPN / Direct Connect conexión híbrida

CÓMPUTO

EC2 máquinas virtuales
EBS / EFS discos y ficheros
EC2 Image Builder AMI doradas
Systems Manager operación de flotas

BALANCEO Y ESCALADO

ALB capa 7, HTTP/HTTPS
NLB capa 4, TCP/UDP
GWLB appliances de red
Auto Scaling elasticidad

CONTENEDORES

ECS orquestador de AWS
Fargate sin servidores
ECR registro de imágenes
App Runner web apps gestionadas

KUBERNETES

EKS Kubernetes gestionado
EKS Auto Mode nodos gestionados
Karpenter autoescalado de nodos
EKS Pod Identity roles IAM para pods

SERVERLESS

Lambda funciones bajo demanda
API Gateway API gestionadas
Step Functions orquestación
SAM despliegue serverless

ALMACENAMIENTO

S3 almacenamiento de objetos
S3 Glacier archivo
FSx sistemas de ficheros
AWS Backup copias centralizadas

BASES DE DATOS

RDS / Aurora relacionales
DynamoDB clave-valor NoSQL
ElastiCache caché en memoria
Redshift data warehouse

INTEGRACIÓN

SQS colas
SNS pub/sub
EventBridge bus de eventos
Kinesis streaming

EDGE Y DNS

CloudFront CDN global
Route 53 DNS gestionado
Global Accelerator red global de AWS
Lambda@Edge código en el borde

OBSERVABILIDAD

CloudWatch métricas, logs, alarmas
CloudTrail auditoría de API
X-Ray trazas distribuidas
AWS Config estado y cumplimiento

SEGURIDAD

KMS claves de cifrado
Secrets Manager secretos
ACM certificados TLS
WAF / Shield protección web
GuardDuty detección de amenazas

IAC Y DEVOPS

CloudFormation plantillas declarativas
CDK IaC programable
Terraform IaC multiproveedor
CodePipeline / CodeBuild CI/CD

COSTES

Cost Explorer análisis de gasto
Budgets presupuestos y alertas
Savings Plans descuentos por compromiso
Compute Optimizer ajuste de tamaño

CONTENIDO

Índice

1 Fundamentos de AWS y la nube 1

1.1 ¿Qué es la nube? 2
1.2 La infraestructura global de AWS 3
1.3 El modelo de responsabilidad compartida 4
1.4 AWS Well-Architected Framework 5
1.5 Formas de trabajar con AWS 6
1.6 Cuentas, usuario root e identificadores 6
1.7 Etiquetas y cuotas de servicio 7
1.8 Facturación básica y capa gratuita 7
1.9 Disponibilidad, tolerancia a fallos y recuperación ante desastres 8

2 IAM (I): identidades — usuarios, grupos, roles y permission sets 10

2.1 El mapa de IAM antes de los detalles 11
2.2 Principals, autenticación y autorización 12
2.3 Usuarios IAM 13
2.4 Grupos IAM 14
2.5 Roles IAM: identidades que se asumen 14
2.6 IAM Identity Center y los permission sets 18
2.7 Todas las piezas de un vistazo 21
2.8 Otras formas de federación 22
2.9 Preguntas frecuentes 22
2.10 ¿Qué uso cuando...? Ocho escenarios resueltos 22

3 IAM (II): permisos — políticas y evaluación 26

3.1 Del quién al qué: tres reglas de oro 27
3.2 Anatomía de una política, paso a paso 27
3.3 Los tipos de políticas 29
3.4 Cómo decide AWS: la lógica de evaluación 31
3.5 Condiciones útiles y variables de política 32
3.6 ABAC: permisos basados en etiquetas 33
3.7 Permissions boundaries: delegar sin escalada de privilegios 34
3.8 AWS Organizations: SCP y RCP 35
3.9 Depurar un AccessDenied paso a paso 38
3.10 Mínimo privilegio en la práctica 39
3.11 Ejercicios resueltos 39

4 Redes: VPC y conectividad 42

4.1 Direccionamiento IP y CIDR en cinco minutos 43
4.2 Amazon VPC: tu red privada en la nube 44
4.3 Subredes y zonas de disponibilidad 45
4.4 Tablas de rutas 45
4.5 Internet Gateway, IP públicas y NAT 46
4.6 Security Groups y Network ACL 48
4.7 VPC endpoints y PrivateLink 50
4.8 Conectar VPC: Peering, Transit Gateway y VPC Lattice 51
4.9 Conectividad híbrida 53
4.10 DNS dentro de la VPC 54
4.11 Visibilidad: VPC Flow Logs y Reachability Analyzer 54
4.12 Diseño de referencia: VPC Multi-AZ de tres capas 55

5 EC2: cómputo en la nube 58

5.1 ¿Qué es Amazon EC2? 59
5.2 Imágenes de máquina (AMI) 59
5.3 Tipos de instancia: familias y nomenclatura 60
5.4 Ciclo de vida de una instancia 62
5.5 Almacenamiento: EBS, instance store y EFS 62
5.6 Redes en EC2 64
5.7 Arranque, metadatos e identidad 65
5.8 Modelos de compra 66
5.9 Acceso a las instancias 67
5.10 AWS Systems Manager 68
5.11 Launch templates 68
5.12 Buenas prácticas 69

6 Balanceo de carga y Auto Scaling 70

6.1 ¿Por qué balancear la carga? 71
6.2 Tipos de balanceador 71
6.3 Listeners, reglas y target groups 72
6.4 Health checks 73
6.5 TLS, certificados y SNI 73
6.6 Cross-zone load balancing 73
6.7 Comportamiento fino del balanceador 74
6.8 Auto Scaling groups 74
6.9 Application Auto Scaling 77

7 Contenedores: ECS, Fargate, ECR y EKS 78

7.1 Contenedores en dos minutos 79
7.2 Amazon ECR: el registro de imágenes 80
7.3 Amazon ECS: conceptos clave 80
7.4 Capacidad: EC2, Fargate y Fargate Spot 81
7.5 Redes de las tareas 82
7.6 IAM en ECS: execution role y task role 83
7.7 Servicios: despliegue, balanceo y escalado 83
7.8 Comunicación entre servicios 84
7.9 Configuración, logs y depuración 85
7.10 Arquitectura de referencia 85
7.11 ¿ECS, EKS o App Runner? 86

8 Kubernetes y Amazon EKS 88

8.1 ¿Qué problema resuelve Kubernetes? 89
8.2 Arquitectura de un clúster 89
8.3 Los objetos fundamentales 90
8.4 Manos a la obra: YAML, kubectl y Helm 92
8.5 Escalado: pods y nodos 93
8.6 Amazon EKS: Kubernetes gestionado 93
8.7 Redes en EKS 95
8.8 Almacenamiento en EKS 96
8.9 IAM y EKS: dos direcciones que no debes confundir 97
8.10 Operación del clúster 99
8.11 Buenas prácticas y errores comunes 99

9 Serverless: Lambda, API Gateway y Step Functions 101

9.1 ¿Qué significa serverless? 102
9.2 AWS Lambda: funciones bajo demanda 102
9.3 Cómo se invoca una función 104
9.4 Permisos, red y configuración 105
9.5 Versiones, aliases y function URLs 106
9.6 Buenas prácticas con Lambda 106
9.7 Amazon API Gateway 107
9.8 AWS Step Functions 108
9.9 Desplegar con AWS SAM 109

10 Almacenamiento: S3 y compañía 111

10.1 Bloque, ficheros y objetos 112
10.2 Amazon S3: conceptos esenciales 112
10.3 Clases de almacenamiento 113
10.4 Lifecycle: mover y borrar automáticamente 114
10.5 Versionado, MFA Delete y Object Lock 114
10.6 Replicación entre buckets 115
10.7 Seguridad en S3 115
10.8 Rendimiento 117
10.9 Eventos de S3 117
10.10 Alojamiento web estático 118
10.11 Consultar datos y controlar costes 118
10.12 Más allá de S3 118

11 Bases de datos 121

11.1 Una base de datos para cada problema 122
11.2 Gestionada frente a autogestionada 122
11.3 Amazon RDS 123
11.4 Amazon Aurora 126
11.5 Amazon DynamoDB 128
11.6 Amazon ElastiCache y MemoryDB 132
11.7 Otras bases de datos de propósito específico 133
11.8 Migraciones: AWS DMS y SCT 134

12 Mensajería e integración: SQS, SNS, EventBridge y Kinesis 136

12.1 Acoplamiento y desacoplamiento 137
12.2 Amazon SQS: colas de mensajes 137
12.3 Amazon SNS: publicar y suscribir 140
12.4 El patrón fan-out: SNS + SQS 141
12.5 Amazon EventBridge: el bus de eventos 141
12.6 Amazon Kinesis Data Streams: streaming en tiempo real 142
12.7 Firehose, Flink, MSK y Amazon MQ 143
12.8 ¿Qué servicio elijo? 144
12.9 Patrones de integración 144

13 CloudFront, Route 53 y el borde de la red 146

13.1 ¿Por qué una CDN? 147
13.2 Distribuciones, orígenes y cache behaviors 147
13.3 La caché: cache key, políticas y TTL 148
13.4 Seguridad en el borde 150
13.5 Código en el borde: CloudFront Functions y Lambda@Edge 152
13.6 Logs y métricas 152
13.7 Route 53: el DNS gestionado de AWS 153
13.8 Patrón: sitio web estático con S3, CloudFront, Route 53 y ACM 155
13.9 Global Accelerator frente a CloudFront 155

14 Observabilidad: logging, métricas y auditoría 157

14.1 Por qué observar: los tres pilares 158
14.2 CloudWatch Metrics 158
14.3 CloudWatch Alarms 159
14.4 CloudWatch Logs 160
14.5 Dashboards y monitorización de aplicaciones 162
14.6 Trazas distribuidas con AWS X-Ray 162
14.7 Auditoría con AWS CloudTrail 163
14.8 AWS Config: configuración y cumplimiento 163
14.9 Reaccionar a eventos y otros logs 164
14.10 Logging centralizado multicuenta 164
14.11 ¿Qué herramienta para qué pregunta? 165
14.12 Buenas prácticas 165

15 Seguridad: cifrado con KMS, secretos y protección 167

15.1 Por qué cifrar 168
15.2 AWS KMS y sus tipos de clave 168
15.3 Quién puede usar una clave: key policies, IAM y grants 170
15.4 Cifrado envolvente (envelope encryption) 171
15.5 Contexto de cifrado, alias, rotación y ciclo de vida 172
15.6 KMS en los servicios de AWS 173
15.7 BYOK, CloudHSM, cifrado en cliente y costes 174
15.8 Secretos: Secrets Manager y Parameter Store 176
15.9 Certificados TLS con ACM 177
15.10 Protección perimetral: WAF, Shield y firewalls 177
15.11 Detección y respuesta: GuardDuty, Security Hub y más 179
15.12 Defensa en profundidad 180
15.13 Respuesta a incidentes 180

16 Infraestructura como código y DevOps 182

16.1 Por qué infraestructura como código 183
16.2 AWS CloudFormation 183
16.3 AWS CDK y AWS SAM 186
16.4 Terraform 187
16.5 CI/CD en AWS 188
16.6 Estrategias de despliegue 189
16.7 Entornos y buenas prácticas 189

17 Costes y optimización 191

17.1 El modelo de pago por uso 192
17.2 Los grandes conductores de coste 192
17.3 Herramientas de visibilidad y control 193
17.4 Estrategias de optimización 193
17.5 Modelos de compra 194
17.6 Trampas comunes 194
17.7 FinOps: el coste como responsabilidad compartida 195

18 Arquitecturas de referencia 196

18.1 Aplicación web de tres capas altamente disponible 197
18.2 Microservicios en contenedores con ECS Fargate 198
18.3 API serverless 199
18.4 Procesamiento asíncrono orientado a eventos 199
18.5 Sitio estático o SPA 200
18.6 Pipeline de datos y analítica 201
18.7 Estrategia multicuenta con Organizations 202
18.8 Recuperación ante desastres multirregión 203
18.9 Checklist de revisión Well-Architected 204

A Glosario 206

B Chuleta de referencia rápida 212

CAPÍTULO 1

Fundamentos de AWS y la nube

Qué es la nube, cómo se organiza la infraestructura global de AWS y qué reglas del juego conviene conocer antes de crear el primer recurso.

EN ESTE CAPÍTULO APRENDERÁS

1. Los modelos IaaS, PaaS y SaaS y el paso de CapEx a OpEx
2. Regiones, zonas de disponibilidad y ubicaciones de borde
3. El modelo de responsabilidad compartida
4. Los seis pilares del AWS Well-Architected Framework
5. Cuentas, usuario root, ARN, etiquetas y cuotas
6. Alta disponibilidad, tolerancia a fallos y recuperación ante desastres

SERVICIOS PROTAGONISTAS

Infraestructura global · AWS CLI · AWS CloudShell · Service Quotas · AWS Billing and Cost Management · AWS Well-Architected Tool

1.1 ¿Qué es la nube?

La **computación en la nube** (*cloud computing*) es la entrega bajo demanda de recursos de TI —servidores, almacenamiento, bases de datos, redes, software— a través de Internet y con pago por uso. En lugar de comprar servidores y mantenerlos durante años, alquilas la capacidad que necesitas a un proveedor que opera centros de datos gigantescos, y la devuelves cuando ya no te hace falta. La distinguen el **autoservicio bajo demanda** (creas recursos en minutos desde una consola o una API), el acceso por red, los recursos compartidos entre clientes aislados entre sí, la **elasticidad** y el **servicio medido**: pagas por horas, GB o peticiones.

Modelos de servicio: IaaS, PaaS y SaaS

Los servicios se clasifican según cuántas capas de la pila gestiona el proveedor por ti. Cuanto más subes, menos control tienes, pero también menos trabajo operativo:

- **IaaS** (*Infrastructure as a Service*): recibes máquinas virtuales, discos y redes, y gestionas todo lo demás desde el sistema operativo. Ejemplos: Amazon EC2 (servidores virtuales), EBS (discos) y VPC (redes privadas).
- **PaaS** (*Platform as a Service*): aportas código y datos; el proveedor se ocupa del sistema operativo, el runtime, los parches y el escalado. Ejemplo: AWS Elastic Beanstalk; servicios gestionados como Amazon RDS (bases de datos) o AWS Lambda (funciones sin servidores que administrar) siguen la misma filosofía.
- **SaaS** (*Software as a Service*): consumes una aplicación terminada, como un correo web o un CRM; en AWS, Amazon Connect (centro de contacto).



Figura 1 · Quién gestiona cada capa en cada modelo de servicio.

> **ANALOGÍA**
>
> **On-premises** es hacer la pizza en casa; **IaaS**, alquilar una cocina equipada; **PaaS**, pedirla a domicilio y servirla en tu mesa; **SaaS**, ir a la pizzería y elegir del menú.

CapEx, OpEx y elasticidad

Un centro de datos propio es un **gasto de capital** (CapEx): compras por adelantado servidores que amortizarás durante años y tienes que adivinar cuánta capacidad necesitarás. La nube lo convierte en **gasto operativo** (OpEx): pagas cada mes lo que consumes. Además aprovechas las economías de escala de AWS, ganas agilidad (un servidor tarda minutos, no semanas) y puedes desplegar en otros continentes en minutos.

La **escalabilidad** es la capacidad de crecer para atender más carga: en vertical (*scale up*, una máquina mayor) o en horizontal (*scale out*, más máquinas). La **elasticidad** va más allá: añadir y quitar capacidad

automáticamente según la demanda, de modo que pagas solo lo que usas en cada momento. En AWS la consigues con EC2 Auto Scaling (capítulo de Balanceo de carga y Auto Scaling) o con servicios *serverless*, que escalan solos.

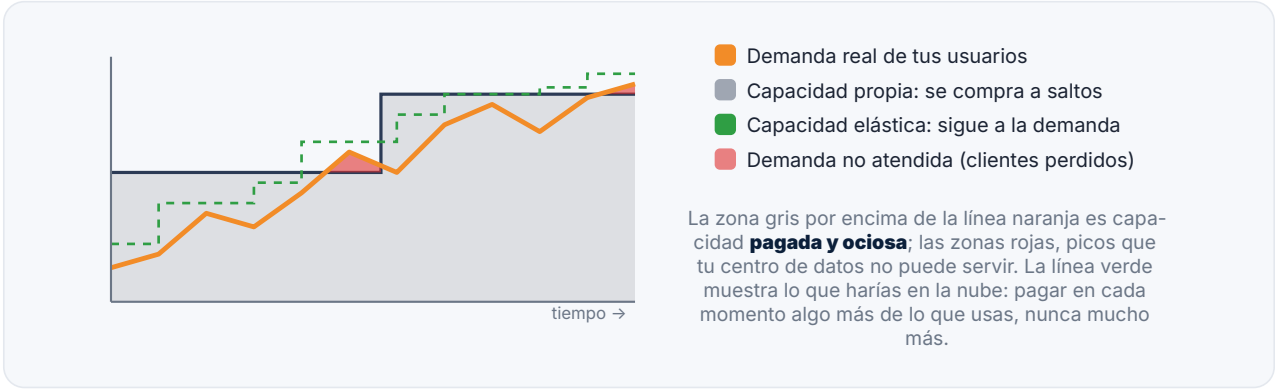


Figura 2 · Capacidad propia frente a capacidad elástica a lo largo del tiempo.

1.2 La infraestructura global de AWS

AWS organiza su infraestructura en una jerarquía sencilla: **regiones** que contienen **zonas de disponibilidad**, que a su vez agrupan **centros de datos**, más una red de **ubicaciones de borde** en cientos de ciudades. Hoy son más de 30 regiones y más de 100 zonas de disponibilidad, y la cifra crece cada año.

Una **región** es un área geográfica concreta con varios centros de datos, identificada por un código: `eu-west-1` (Irlanda), `eu-central-1` (Fráncfort), `eu-south-2` (España), `us-east-1` (Norte de Virginia). Cada región es **independiente**: tus datos no salen de ella salvo que tú los copies o replices. Las regiones abiertas a partir de marzo de 2019 vienen desactivadas (*opt-in*) y hay que habilitarlas antes de usarlas.

Cada región se divide en al menos tres **zonas de disponibilidad** (*Availability Zones*, AZ). Una AZ es uno o más centros de datos con energía, refrigeración y red redundantes, separada de las demás por muchos kilómetros (pero a menos de 100 km) para que un incendio o un corte eléctrico no afecte a dos a la vez. Las AZ se conectan con fibra dedicada y latencias de un solo dígito de milisegundos, lo que permite repartir una aplicación entre varias sin sacrificar rendimiento.

> ★ **CLAVE PARA LA CERTIFICACIÓN**
>
> Los nombres de AZ (`eu-west-1a`) se asignan de forma independiente en cada cuenta: tu `eu-west-1a` puede ser otra ubicación física que la de otra cuenta. Para coordinarte entre cuentas usa el **AZ ID** (`euw1-az1`).

Las **Local Zones** acercan un subconjunto de servicios (EC2, EBS, subredes…) a grandes ciudades, con latencias de milisegundos de un dígito; se nombran a partir de su región (`us-west-2-lax-1a`) y hay que habilitarlas. Las **Wavelength Zones** viven dentro de redes 5G de operadores. Con **AWS Outposts**, AWS instala y gestiona su hardware en tu centro de datos. Las **ubicaciones de borde** ejecutan CloudFront, Route 53, Global Accelerator, WAF y Shield; no lanzas instancias en ellas.

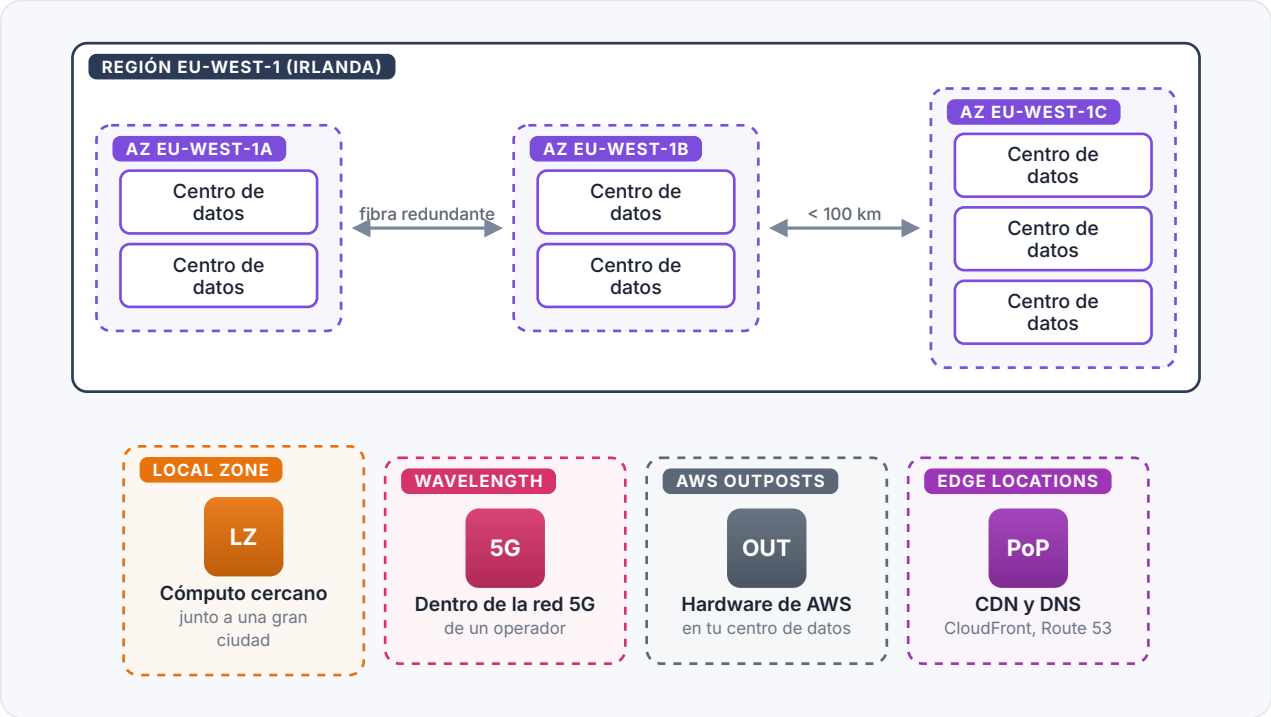


Figura 3 · Una región agrupa varias AZ aisladas, cada una con uno o más centros de datos. Fuera de las regiones, AWS acerca su infraestructura a los usuarios.

Cómo elegir una región

Sigue este orden: **1) cumplimiento** —si una ley, un contrato o tu política exige que los datos residan en un país o en la UE, no es negociable—; **2) latencia** —cerca de tus usuarios: para una tienda en España, `eu-south-2` (España) o `eu-west-3` (París)—; **3) servicios disponibles** —no todos los servicios ni tipos de instancia están en todas las regiones; las novedades suelen llegar antes a `us-east-1`—; y **4) precio**, que puede variar bastante entre regiones para el mismo recurso.

Servicios globales, regionales y zonales

Saber en qué nivel vive cada recurso te dice qué fallos puede sufrir y desde dónde puedes usarlo:

Globales	Regionales	Zonales
No dependen de una región	Resisten la caída de una AZ	Caen si cae su AZ
IAM, Route 53, CloudFront, Organizations. En la consola, el selector de región muestra «Global».	S3, DynamoDB, SQS, Lambda, VPC, KMS. S3 es regional, aunque los nombres de bucket son únicos en todo el mundo.	Instancias EC2, volúmenes EBS, subredes. Un volumen EBS solo se conecta a instancias de su AZ; para moverlo, usa un *snapshot* (regional).

1.3 El modelo de responsabilidad compartida

La seguridad se reparte entre AWS y tú. AWS responde de la **seguridad DE la nube**: edificios, hardware, red física y el software que hace funcionar sus servicios. Tú respondes de la **seguridad EN la nube**: lo que pones dentro y cómo lo configuras.



Figura 4 · Modelo de responsabilidad compartida.

Cuanto más gestionado es el servicio, más capas asume AWS (el hardware y las instalaciones son siempre suyos):

Quién se ocupa de…	EC2 (IaaS)	RDS (gestionado)	Lambda, S3
Datos, cifrado, permisos IAM, reglas de red, código	Tú	Tú	Tú
Sistema operativo y sus parches	Tú	AWS	AWS
Software de base (motor de base de datos, runtime)	Tú	AWS tú eliges versión y ventana	AWS
Alta disponibilidad y escalado	Tú varias AZ, Auto Scaling	Compartida tú activas Multi-AZ	AWS

CLAVE PARA LA CERTIFICACIÓN

«¿Quién parchea el sistema operativo?» En EC2, tú; en RDS y Lambda, AWS. Datos, permisos y cifrado son **siempre** tuyos: un bucket público por error es un fallo del cliente.

1.4 AWS Well-Architected Framework

El **Well-Architected Framework** reúne las buenas prácticas y preguntas que AWS ha destilado de miles de arquitecturas reales para evaluar un diseño y detectar riesgos a tiempo. Tiene seis pilares:

Excelencia operativa
Operar, observar y mejorar

- Operaciones como código.
- Cambios pequeños y reversibles; aprender de cada fallo.

Seguridad
Proteger datos y sistemas

- Identidad sólida y mínimo privilegio.
- Trazabilidad, seguridad en todas las capas, cifrado.

Fiabilidad
Funcionar y recuperarse

- Recuperación automática y probada.
- Escalar en horizontal; no adivinar capacidad.

Eficiencia del rendimiento
Los recursos adecuados

- Tecnologías avanzadas como servicio.
- Serverless, alcance global y experimentación.

Optimización de costes
Máximo valor por euro

- Gestión financiera de la nube (FinOps).
- Pagar por consumo y atribuir el gasto.

Sostenibilidad
Menor impacto ambiental

- Medir el impacto y maximizar la utilización.
- Hardware y servicios más eficientes.

> **CONSEJO**
>
> La **AWS Well-Architected Tool** de la consola guía gratis la revisión de una carga pilar a pilar, con **lentes** especializadas (Serverless, SaaS, IA generativa…). Recuerda que los pilares compiten: más fiabilidad suele costar más.

1.5 Formas de trabajar con AWS

En AWS **todo es una API**. Cada clic en la consola, cada comando y cada llamada de un SDK se convierten en una petición HTTPS a un *endpoint* del servicio (como `ec2.eu-west-1.amazonaws.com`), firmada con tus credenciales (Signature Version 4) y registrada por AWS CloudTrail (capítulo de Observabilidad). Lo que cambia es la herramienta: la **consola** web, para explorar y tareas puntuales; la **AWS CLI v2** (`aws <servicio> <acción>`), para consultas rápidas y scripts; **AWS CloudShell**, una terminal en el navegador ya autenticada con tu identidad de la consola, con la CLI instalada y 1 GB persistente por región; los **SDK** (boto3 para Python, JavaScript, Java, Go…), para integrar AWS en tus aplicaciones; y la **infraestructura como código** (CloudFormation, CDK, Terraform), la forma recomendada de gestionar entornos (capítulo de Infraestructura como código).

```bash
aws sts get-caller-identity     # ¿con qué identidad y en qué cuenta estoy?
aws s3 ls --profile prod --region eu-south-2    # perfil y región explícitos
```

1.6 Cuentas, usuario root e identificadores

Una **cuenta AWS** es el contenedor de tus recursos y, a la vez, un **límite de seguridad** y de **facturación**. Se identifica con 12 dígitos (`123456789012`); lo habitual es usar varias, agrupadas con AWS Organizations, que verás en el capítulo «IAM (II)».

El usuario root

Al crear la cuenta nace su **usuario root**, que entra con el correo y la contraseña del registro y tiene **acceso total e irrestricto**, facturación incluida. Las políticas de IAM no pueden limitarlo en su propia cuenta, así que es tu credencial más valiosa. Úsalo solo para las pocas tareas que lo exigen —cambiar el correo o la contraseña root, cerrar la cuenta, restaurar permisos si el único administrador se ha bloqueado, activar *MFA Delete* en S3…— y protégelo así:

1. **Activa MFA** (autenticación multifactor: un segundo factor además de la contraseña), mejor con una llave FIDO2 o una *passkey*, y registra más de un dispositivo. AWS la exige de forma progresiva al root desde 2024.
2. **No crees access keys** de root (las claves para usar la CLI o la API) y elimina las que existan.
3. Usa un **correo corporativo** compartido (una lista de distribución) y una contraseña larga guardada en un gestor.
4. Trabaja a diario con otra identidad (idealmente de IAM Identity Center) y **vigila** con una alerta cada inicio de sesión del root. Con Organizations puedes incluso eliminar las credenciales root de las cuentas miembro (gestión centralizada del acceso root, 2024).

Anatomía de un ARN

Cada recurso tiene un identificador único global, el **Amazon Resource Name** (ARN), que usarás en políticas de IAM, en la CLI y en plantillas:



Figura 5 · Partes de un ARN. La partición es `aws` en las regiones comerciales (`aws-cn` en China, `aws-us-gov` en GovCloud).

```text
arn:aws:iam::123456789012:role/app-web          # IAM es global: sin región
arn:aws:s3:::mis-fotos/2026/gato.jpg            # S3: sin región ni cuenta
arn:aws:s3:::mis-fotos/*                        # comodín: todos los objetos
```

1.7 Etiquetas y cuotas de servicio

Una **etiqueta** (*tag*) es un par clave-valor que adjuntas a un recurso, como `Entorno=prod` o `Proyecto=tienda`. Sirven para **repartir costes** (las *cost allocation tags* se activan en la consola de facturación), **automatizar** (apagar de noche todo lo que tenga `Horario=oficina`), **controlar el acceso** por atributos (ABAC, capítulo «IAM (II)») y **organizar** (Resource Groups, Tag Editor). La mayoría de servicios admiten hasta 50 etiquetas por recurso, que distinguen mayúsculas y minúsculas; el prefijo `aws:` está reservado. Define unas pocas obligatorias y aplícalas desde tu código de infraestructura.

Cada cuenta tiene además **cuotas** por servicio y, casi siempre, por región: protegen a AWS de abusos y a ti de errores caros. Unas son **ajustables** y se amplían pidiéndolo en la consola de **Service Quotas** (o con `aws service-quotas`); otras son **fijas**, parte del diseño del servicio. Revísalas antes de un lanzamiento, porque un aumento puede tardar desde minutos hasta días.

Cuota	Valor por defecto	¿Ajustable?
VPC por región (Amazon VPC)	5	✔
Ejecuciones concurrentes por región (AWS Lambda)	1000	✔
Duración máxima de una invocación (AWS Lambda)	15 minutos	✘
Usuarios IAM por cuenta	5000	✘

1.8 Facturación básica y capa gratuita

AWS factura **por uso**: tiempo de cómputo, GB-mes almacenados, peticiones y GB transferidos. La entrada de datos desde Internet suele ser gratuita; la salida a Internet, entre regiones y entre AZ se paga. Hay descuentos por compromiso (Savings Plans) y capacidad sobrante barata pero interrumpible (Spot), que verás en el capítulo de EC2; para controlar el gasto, **Cost Explorer**, **AWS Budgets** y la **Pricing Calculator** (capítulo de Costes).

La **capa gratuita** (*Free Tier*) cambió el 15 de julio de 2025: las cuentas nuevas reciben **créditos promocionales** y eligen entre un plan gratuito, limitado en tiempo y servicios, y un plan de pago; algunos servicios mantienen ofertas «siempre gratuitas». Las cuentas anteriores conservan sus ofertas de 12 meses hasta que vencen. Consulta las condiciones vigentes.

> **$ COSTES**
>
> Sorpresas habituales: recursos olvidados en otra región, NAT Gateways, IPv4 públicas (de pago por hora desde febrero de 2024), snapshots huérfanos y transferencia de salida. Crea un presupuesto con alertas en AWS Budgets **el primer día**.

1.9 Disponibilidad, tolerancia a fallos y recuperación ante desastres

Alta disponibilidad
Minimizar la caída

Ante un fallo puede haber un corte breve mientras conmuta. **Ej.:** RDS Multi-AZ conmuta a su réplica en uno o dos minutos.

Tolerancia a fallos
Ninguna interrupción visible

Redundancia activa: sigue funcionando aunque falle un componente. **Ej.:** instancias en tres AZ con margen para perder una.

Recuperación (DR)
Volver tras un evento grave

Disaster recovery: restaurar el servicio tras perder una región, un borrado masivo o un *ransomware*. Se mide con RTO y RPO.

La disponibilidad se expresa en «nueves»; cada nueve más divide por diez la caída permitida. La redundancia es la palanca: dos instancias independientes del 99 % solo fallan a la vez con probabilidad 0,01 × 0,01, así que juntas llegan al 99,99 %.

99,9 %	99,95 %	99,99 %	99,999 %
≈ 8,8 h de caída al año	≈ 4,4 h al año	≈ 53 min al año	≈ 5 min al año

La recuperación ante desastres se diseña con dos objetivos: el **RPO** (*Recovery Point Objective*), cuántos datos puedes perder —un RPO de 1 hora exige copias o réplicas al menos cada hora—, y el **RTO** (*Recovery Time Objective*), cuánto puede durar la caída.



Figura 6 · El RPO mira hacia atrás (datos perdidos); el RTO, hacia delante (tiempo hasta recuperar el servicio).

AWS describe cuatro estrategias, de más barata y lenta a más cara y rápida. Elige según el RTO y el RPO que exija el negocio, no por defecto la más sofisticada:



Figura 7 · Estrategias de recuperación ante desastres entre regiones.

CLAVE PARA LA CERTIFICACIÓN

«Lo más barato, RTO de horas» → *backup & restore*; «datos replicados y servidores apagados» → *pilot light*; «versión reducida siempre en marcha» → *warm standby*; «RTO y RPO casi cero» → *multi-site*. Y un plan de recuperación sin probar no es un plan.

RESUMEN DEL CAPÍTULO

- La nube ofrece recursos de TI bajo demanda y con pago por uso: convierte CapEx en OpEx y aporta elasticidad. IaaS, PaaS y SaaS difieren en cuántas capas gestiona el proveedor.
- Una región es un área geográfica independiente con al menos tres AZ aisladas y bien conectadas; Local Zones, Wavelength, Outposts y las ubicaciones de borde acercan la infraestructura a los usuarios.
- Elige región por cumplimiento, latencia, servicios y precio; distingue servicios globales (IAM), regionales (S3) y zonales (EC2, EBS).
- AWS se ocupa de la seguridad **de** la nube; tú, de la seguridad **en** la nube, con una frontera que depende del servicio.
- El Well-Architected Framework tiene seis pilares: excelencia operativa, seguridad, fiabilidad, eficiencia del rendimiento, costes y sostenibilidad.
- Todo es una API (consola, CLI, SDK, IaC). Protege el root con MFA y no lo uses a diario; ARN, etiquetas y cuotas son el vocabulario básico.
- Alta disponibilidad, tolerancia a fallos y recuperación ante desastres son cosas distintas; la última se diseña a partir del RTO y el RPO.

Preguntas de repaso

1. **¿Qué diferencia hay entre escalabilidad y elasticidad?**
   Respuesta: La escalabilidad es poder crecer para atender más carga; la elasticidad es ajustar la capacidad automáticamente, en ambos sentidos, según la demanda, pagando solo lo usado.
2. **Debes guardar datos de clientes europeos dentro de la UE y la mayoría de usuarios están en España. ¿Qué miras primero al elegir región?**
   Respuesta: El cumplimiento (una región de la UE) y después la latencia, por ejemplo `eu-south-2`; luego, servicios disponibles y precio.
3. **¿Por qué conviene desplegar una aplicación en al menos dos AZ?**
   Respuesta: Porque las AZ están aisladas físicamente: si una falla, la otra sigue sirviendo.
4. **En EC2, ¿quién aplica los parches del sistema operativo? ¿Y en RDS?**
   Respuesta: En EC2, el cliente. En RDS, AWS, dentro de la ventana de mantenimiento que elige el cliente.
5. **Tienes un volumen EBS en `eu-west-1a` y quieres conectarlo a una instancia de `eu-west-1b`. ¿Qué haces?**
   Respuesta: EBS es zonal: crea un snapshot del volumen (los snapshots son regionales) y restaura a partir de él un volumen nuevo en `eu-west-1b`.
6. **¿Por qué el ARN de un objeto S3 no lleva región ni cuenta?**
   Respuesta: Porque los nombres de bucket son únicos en todo el mundo: el nombre basta para identificarlo.
7. **El negocio tolera perder 5 minutos de datos y 30 minutos de caída, con presupuesto ajustado. ¿Qué estrategia de DR encaja?**
   Respuesta: Pilot light: datos replicados de forma continua (RPO de minutos) e infraestructura lista para arrancar en decenas de minutos, sin pagar una copia completa en marcha.

CAPÍTULO 2

IAM (I): identidades — usuarios, grupos, roles y permission sets

Quién es quién en AWS y cómo consigue cada identidad sus credenciales: el mapa que necesitas antes de escribir una sola política.

EN ESTE CAPÍTULO APRENDERÁS

1. Un modelo mental de cómo encajan todas las piezas de IAM
2. Qué es (y qué no es) un usuario, un grupo y un rol
3. AssumeRole, STS y cómo reciben sus credenciales EC2, Lambda o un pipeline
4. IAM Identity Center, permission sets y qué pasa al entrar por el portal
5. Federación: SAML, OIDC, GitHub Actions y Cognito
6. Qué identidad usar en cada escenario real

SERVICIOS PROTAGONISTAS

AWS IAM · AWS STS · IAM Identity Center · AWS Organizations · Amazon Cognito · IAM Roles Anywhere

2.1 El mapa de IAM antes de los detalles

Cada vez que alguien hace algo en AWS —un clic en la consola, un comando de la CLI, una llamada desde una aplicación— envía una **petición a una API**, y AWS debe responder a dos preguntas: **¿quién eres?** (autenticación) y **¿puedes hacer esto?** (autorización). **AWS Identity and Access Management** (IAM) gestiona ambas. Tiene fama de difícil porque mezcla piezas con nombres parecidos —usuarios, grupos, roles, políticas, permission sets, STS—, así que empecemos por el mapa completo. Toda petición recorre los mismos cuatro pasos:



Figura 8 · El recorrido de cualquier petición en AWS. Este capítulo trata los pasos 1 y 2 (quién eres y con qué credenciales); «IAM (II)», el paso 3 (qué puedes hacer).

Fíjate en el color de las credenciales: la gran idea de IAM moderno es que **casi nadie debería tener credenciales permanentes**. Las personas entran con la identidad que ya tienen en su empresa, gestionada por un **proveedor de identidad** (IdP, *identity provider*, como Microsoft Entra ID u Okta), y reciben credenciales temporales; las aplicaciones usan roles, que también las producen. Quien fabrica esas credenciales temporales es **AWS STS** (*Security Token Service*). Las access keys de largo plazo son el último recurso.

¿Y los grupos y los permission sets del título? No aparecen en el paso 1 porque **nunca hacen peticiones**: son formas de **repartir políticas**. Un grupo IAM las reparte entre los usuarios IAM de una cuenta; un permission set, entre las cuentas de una organización, donde se convierte en un rol.

La analogía que nos acompañará: un edificio de oficinas

Imagina una gran empresa con varias sedes: cada sede es una cuenta AWS, y la empresa entera, tu organización. Usaremos esta imagen en los dos capítulos de IAM:

En nuestro edificio…	…en AWS es
Una sede, con sus salas y su propia contabilidad	Una **cuenta AWS**
La llave maestra de la sede: abre todo, sin restricciones internas	El **usuario root**
Un empleado con tarjeta personal y permanente, válida en una sola sede	Un **usuario IAM**
Un departamento: sus miembros reciben los accesos del departamento	Un **grupo IAM**
Una acreditación de «Mantenimiento»: se recoge en el mostrador y caduca	Un **rol IAM**
La lista, junto al mostrador, de quién puede recoger esa acreditación	Su **trust policy**
Las puertas que abre esa acreditación	Su **permissions policy**
El vigilante que comprueba la lista y entrega la acreditación	**AWS STS**
El RR. HH. central: un alta por persona, válida para todas las sedes	**IAM Identity Center**
El modelo de acreditación «Desarrollador» o «Auditor», definido una vez	Un **permission set**
La orden «Ana recoge la acreditación Desarrollador en la sede dev»	Una **asignación**

En el capítulo «IAM (II)» completaremos el edificio con los carteles de las puertas (políticas de recurso) y las normas de la empresa que ninguna sede puede saltarse (SCP).

DÓNDE DEJA DE FUNCIONAR LA ANALOGÍA

- En AWS no se enseña la tarjeta solo al entrar: **cada petición** se autentica y se autoriza por separado.
- Las acreditaciones (roles) las recogen sobre todo **programas**: instancias EC2, funciones Lambda, pipelines.
- Un departamento (grupo) no es una persona: no entra en ningún sitio ni recoge acreditaciones.
- Recoger una acreditación no suma permisos a tu tarjeta: mientras actúas con ella, **solo cuenta ella**.
- En un edificio puedes pasear por los pasillos; en AWS, todo lo que no está permitido expresamente está prohibido.

2.2 Principals, autenticación y autorización

La **autenticación** demuestra quién eres: en la consola, con contraseña y MFA; en la CLI y los SDK, con una **firma criptográfica** (Signature Version 4) que se calcula con tu clave secreta sin que esta viaje nunca por la red. Puede hacerla el propio AWS o un IdP externo en el que AWS confía: eso es la **federación**, que verás más adelante. La **autorización** llega después: AWS reúne el **contexto de la petición** —quién la hace, la acción (`s3:GetObject`), el recurso (su ARN) y datos del entorno (IP, hora, región, etiquetas)— y lo evalúa contra las políticas que aplican.

Ese «quién» se llama **principal**: la entidad que firma una petición concreta. No lo confundas con las **identidades** que creas en IAM para adjuntarles permisos (usuarios, grupos y roles), porque no coinciden una a una:

Pieza	¿Quién firma sus peticiones?	ARN que verás en errores y en CloudTrail
Usuario root	Él mismo (evítalo en el día a día)	`arn:aws:iam::111122223333:root`
Usuario IAM	El propio usuario, con su contraseña o sus access keys	`arn:aws:iam::111122223333:user/ana`
Grupo IAM	**Nadie**: solo reparte políticas entre sus miembros	Tiene ARN (`…:group/Auditores`), pero nunca firma
Rol IAM	Nunca él: firma una **sesión del rol**, es decir, quien lo asumió	El rol: `arn:aws:iam::111122223333:role/app-web`
Sesión de un rol	Quien asumió el rol, con credenciales temporales	`arn:aws:sts::…:assumed-role/app-web/i-0a1b…`
Servicio de AWS	El servicio, cuando actúa en tu nombre	Un nombre de servicio, como `ec2.amazonaws.com`

El **usuario root** es un caso aparte: nace con la cuenta, tiene acceso total y las políticas de IAM no pueden limitarlo (solo las SCP, en las cuentas miembro de una organización). Es la llave maestra que se guarda en la caja fuerte; ya viste en el capítulo de Fundamentos cómo protegerlo.

El comando más útil de toda la CLI es el que te dice con qué identidad estás trabajando. Tras asumir un rol, el ARN es de tipo `assumed-role` y termina con el **nombre de sesión**, que permite saber quién o qué asumió el rol:

BASH

```
$ aws sts get-caller-identity
{
    "UserId": "AROA3XFRBF23EXAMPLE:ana",
    "Account": "222222222222",
    "Arn": "arn:aws:sts::222222222222:assumed-role/Desarrollador/ana"
}
```

2.3 Usuarios IAM

CONCEPTO

Usuario IAM — una identidad permanente dentro de una cuenta

QUÉ ES	Una identidad con **credenciales de largo plazo** en una sola cuenta: contraseña para la consola y/o hasta dos access keys para la API.
QUÉ NO ES	No es una persona de tu directorio corporativo: existe solo en la cuenta donde lo creas (para trabajar en otra, tendría que asumir un rol allí). Tampoco es el usuario root.
PARA QUÉ	Hoy, casos excepcionales: acceso de emergencia (*break-glass*), herramientas de terceros que no admiten roles ni federación, o cuentas aisladas sin Identity Center.
EJEMPLO	El usuario `emergencia-admin` , con MFA en una llave física guardada en una caja fuerte, para cuando falla el proveedor de identidad corporativo.
ERROR TÍPICO	Un usuario compartido por varias personas, o access keys copiadas en el código, en un fichero `.env` o en un repositorio Git.

En nuestro edificio, es un empleado con tarjeta permanente para **una** sede: si trabaja en diez, necesita diez tarjetas, y al irse hay que anular las diez. Sus credenciales:

Credencial	Para qué sirve	Lo que debes saber
Contraseña y MFA	Entrar en la consola	Aplica una política de contraseñas y añade siempre MFA: app TOTP, llave FIDO2 o *passkey* (hasta 8 dispositivos por usuario).
Access keys	Firmar peticiones desde la CLI, los SDK o la API	Un ID (`AKIA…`) y una clave secreta. **No caducan**: si se filtran, cualquiera actúa como ese usuario hasta que las desactives. Máximo dos por usuario, para rotarlas.

POR QUÉ EVITAR LAS ACCESS KEYS

Acaban donde no deben —el código, una imagen de contenedor, un repositorio público— y hay bots que las rastrean y las usan en minutos (típicamente, para minar criptomonedas). Al final del capítulo verás la alternativa sin claves para cada caso; si aun así necesitas una, rótala y vigila su último uso (`aws iam get-access-key-last-used`).

2.4 Grupos IAM

CONCEPTO

Grupo IAM un conjunto de usuarios IAM que comparten permisos

QUÉ ES	Una colección de usuarios IAM de una cuenta a la que adjuntas políticas; sus miembros heredan esos permisos.
QUÉ NO ES	**No es un principal**: sin credenciales, no inicia sesión, no asume roles ni aparece como `Principal` . No contiene otros grupos ni es un grupo de Identity Center.
PARA QUÉ	Gestionar permisos por función en vez de usuario a usuario: altas, bajas y cambios de puesto son meter o sacar a alguien del grupo.
EJEMPLO	El grupo `Auditores` con la política gestionada `SecurityAudit` : cuando llega un auditor nuevo, basta con añadirlo.
ERROR TÍPICO	Adjuntar políticas a cada usuario por separado, o intentar nombrar un grupo en la política de un bucket (no funciona: no es un principal).

¿Y si un usuario está en varios grupos (hasta 10)? Sus permisos son la **unión** de lo que permiten sus políticas propias y las de todos sus grupos, salvo lo que prohíba explícitamente alguna: un `Deny` gana siempre. En el ejemplo, `ec2:*` significa «todas las acciones de EC2»:



Figura 9 · Ana hereda los permisos de sus dos grupos, pero el Deny de su política propia anula una parte de lo que le da el grupo Desarrolladores.

2.5 Roles IAM: identidades que se asumen

Los roles son la pieza más importante de IAM y la que más cuesta entender al principio. Nacen de un problema: una instancia EC2, una función Lambda, un pipeline o una persona que entra en otra

cuenta necesitan permisos, pero no deberían guardar credenciales permanentes. La solución es una identidad que **no es de nadie**: un conjunto de permisos con una lista de quién puede usarlo. **Asumir** un rol es pedirle a AWS credenciales temporales que actúan como ese rol; mientras las usas, tienes los permisos del rol, no los tuyos.

CONCEPTO

Rol IAM una identidad sin credenciales propias que se «asume»

QUÉ ES	Una identidad con permisos pero **sin contraseña ni access keys**; quien autorice su trust policy la asume y recibe credenciales temporales de STS.
QUÉ NO ES	No es un usuario, no pertenece a una persona concreta y **no suma** permisos: mientras lo usas tienes los del rol, no los tuyos. No es un grupo.
PARA QUÉ	Dar permisos a servicios (EC2, Lambda, ECS), acceder a otras cuentas, federar identidades externas y elevar privilegios de forma puntual y auditada.
EJEMPLO	El rol `app-web`, que asumen las instancias EC2 de la web para leer el bucket de imágenes.
ERROR TÍPICO	Confundir sus dos políticas, o crear un usuario con access keys para una aplicación que podría usar un rol.

Las dos caras de un rol

Todo rol tiene dos políticas que responden a preguntas distintas. La **trust policy** (política de confianza) dice **quién puede asumirlo**: es una política de recurso —va pegada al propio rol, como la lista junto al mostrador— y por eso nombra a quién se aplica en su elemento `Principal`. La **permissions policy** (una o varias políticas de identidad) dice **qué puede hacer** quien lo asume.



Figura 10 · La trust policy controla la entrada al rol; la permissions policy, lo que se puede hacer una vez dentro. A la izquierda, ejemplos de quién puede figurar en una trust policy.

Aún no hace falta que domines la sintaxis JSON, que es el tema del capítulo «IAM (II)». Fíjate solo en que la trust policy nombra un `Principal` y la acción `sts:AssumeRole`, y la permissions policy, acciones y recursos:

Trust policy: la asumen las instancias EC2

```json
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {
      "Service": "ec2.amazonaws.com"
    },
    "Action": "sts:AssumeRole"
  }]
}
```

Permissions policy: lo que puede hacer

```json
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject"],
    "Resource":
      "arn:aws:s3:::fotos/*"
  }]
}
```

> ≈ **EN NUESTRO EDIFICIO**
>
> El rol es la acreditación de «Mantenimiento» que espera en el mostrador. La trust policy es la lista pegada al lado: «pueden recogerla los técnicos de la contrata X». La permissions policy, las puertas que abre: el cuarto de calderas, no la sala de dirección. Y STS es el vigilante que comprueba la lista, te entrega la acreditación y anota su hora de caducidad.

AssumeRole paso a paso

Asumir un rol es una conversación con STS. Los números de la figura corresponden a estos pasos:

1. El principal llama a `sts:AssumeRole` con el ARN del rol y un **nombre de sesión** que elige él.
2. STS comprueba la **trust policy**: ¿confía en quien llama? ¿Se cumplen sus condiciones (MFA, External ID…)?
3. Si todo encaja, STS devuelve **credenciales temporales**: `AccessKeyId` (empieza por `ASIA`), `SecretAccessKey`, `SessionToken` y `Expiration`.
4. El principal firma sus peticiones con ellas. Para AWS ahora es la sesión `assumed-role/NombreDelRol/NombreDeSesion`.
5. Cada petición se autoriza con la **permissions policy** del rol (y los límites que apliquen). Al caducar, se vuelve a asumir el rol; los SDK lo hacen solos.

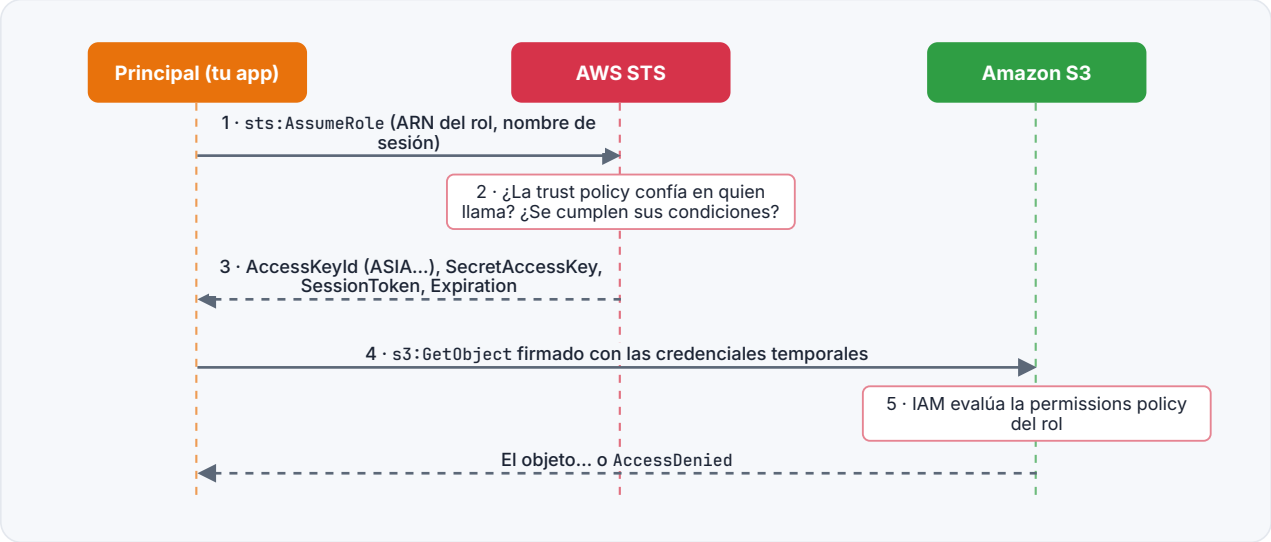


Figura 11 · AssumeRole visto como una secuencia de mensajes. Las credenciales temporales sustituyen a cualquier clave permanente.

¿Necesita quien llama un permiso propio para pedir el rol? Si ambos están en la misma cuenta y la trust policy lo nombra directamente a él, basta con ella. Si la trust policy confía en una cuenta entera (`…:root`) o el rol está en otra cuenta, la política de identidad de quien llama debe permitirle además `sts:AssumeRole` sobre ese rol: hacen falta los dos permisos, como verás en el capítulo «IAM (II)».

En la CLI rara vez llamarás a `assume-role` a mano: basta con describir el rol en un perfil de `~/.aws/config` y la CLI lo asume (y renueva) por ti. Sin Identity Center, este es el patrón menos malo para tu portátil: un usuario IAM con MFA que solo puede asumir roles, nunca access keys de administrador en `~/.aws/credentials`. Escribe los comentarios en su propia línea: al final de un valor se leerían como parte de él.

```ini
# Con --profile prod, la CLI asume el rol usando las credenciales del perfil dev
[profile prod]
role_arn       = arn:aws:iam::222222222222:role/Despliegue
source_profile = dev
# Pide el código del dispositivo MFA de Ana
mfa_serial     = arn:aws:iam::111111111111:mfa/ana
```

Cuánto duran las sesiones y el encadenamiento de roles

Una sesión de `AssumeRole` dura **una hora** por defecto. Quien llama puede pedir entre 15 minutos y la duración máxima configurada en el rol (`MaxSessionDuration`), que puede llegar a 12 horas. Si con las credenciales de un rol asumes otro, haces **encadenamiento de roles** (*role chaining*): es legítimo —por ejemplo, desde una cuenta central de automatización—, pero esa segunda sesión dura como máximo una hora, se pida lo que se pida. **Error típico:** un proceso de varias horas basado en roles encadenados que falla al cabo de una hora con credenciales caducadas.

Roles para servicios: cómo llegan las credenciales a tu código

Cuando quien asume el rol es un servicio de AWS (EC2, Lambda, ECS…), el patrón es siempre el mismo:

1. **Creas el rol**: su trust policy confía en el servicio (`ec2.amazonaws.com`, `lambda.amazonaws.com`…) y su permissions policy da lo que necesita tu código.
2. **Se lo entregas** al configurar el recurso (lanzar la instancia, crear la función). Para eso necesitas el permiso `iam:PassRole` sobre ese rol: sin esa barrera, quien pudiera crear una Lambda podría darle un rol de administrador y escalar privilegios a través de ella.
3. **El servicio asume el rol por ti**, deja las credenciales temporales donde los SDK saben buscarlas y las renueva antes de que caduquen.
4. **Tu código no contiene claves**: el SDK recorre su *cadena de proveedores de credenciales* (variables de entorno, perfiles de `~/.aws`, endpoint del contenedor, servicio de metadatos…) y usa las primeras que encuentra.

Quién	Cómo recibe el rol	Dónde encuentra el SDK las credenciales
Instancia EC2	Un **perfil de instancia** (*instance profile*) con un único rol; la consola lo crea por ti	En el servicio de metadatos (IMDS), siempre con IMDSv2 (capítulo de EC2)
Función Lambda	Su **rol de ejecución** (*execution role*)	En variables de entorno de la función (`AWS_ACCESS_KEY_ID`, `AWS_SESSION_TOKEN`…)
Tarea de ECS	Su *task role* (el agente usa el *task execution role*)	En un endpoint de credenciales de la tarea (capítulo de Contenedores)
Pod de EKS	EKS Pod Identity o IRSA	Del agente de Pod Identity o canjeando el token del pod (capítulo de Kubernetes y Amazon EKS)
CodePipeline, CodeBuild	Su **rol de servicio**	El servicio lo asume en cada ejecución; para desplegar en otra cuenta, asume allí un rol (capítulo de Infraestructura como código y DevOps)
GitHub Actions u otro CI	Federación OIDC (escenario 5)	Canjea el token OIDC del job con `AssumeRoleWithWebIdentity`
Servidor fuera de AWS	IAM Roles Anywhere	Canjea un certificado X.509 de tu PKI por credenciales temporales

Roles de servicio y roles vinculados a servicios

	Rol de servicio	Rol vinculado a un servicio
Qué es	Un rol normal que creas tú para que un servicio actúe en tu nombre	Un rol predefinido que crea y gestiona el propio servicio
Ejemplos	Rol de ejecución de una Lambda, *task role* de ECS, rol de una instancia EC2	`AWSServiceRoleForAutoScaling`, `AWSServiceRoleForECS`
Permisos	Los decides tú; su trust policy nombra al servicio (`lambda.amazonaws.com`)	Los define AWS y no puedes editarlos; las SCP no lo restringen
Borrado	Cuando quieras	Solo cuando el servicio ya no lo usa

Terceros y el External ID: el problema del «confused deputy»

Muchos proveedores (herramientas de costes, de seguridad, de monitorización) acceden a tu cuenta asumiendo un rol que tú creas y que confía en **su** cuenta. El riesgo se llama **confused deputy** (el «ayudante confundido»): un cliente malicioso del mismo proveedor le da **el ARN de tu rol** como si fuera el suyo, y el proveedor, que sí tiene permiso para asumirlo, acaba trabajando con tu cuenta en nombre del atacante.



Figura 12 · Sin External ID, el proveedor podría asumir tu rol por encargo de otro cliente. Con él, tu trust policy solo acepta el identificador que el proveedor generó para ti, y la petición hecha por encargo del atacante llega con el suyo.

La defensa es el **External ID**: el proveedor genera un identificador único para cada cliente y lo envía siempre al asumir el rol; tu trust policy lo exige con una condición. El atacante no puede hacer que el proveedor use **tu** identificador, así que la petición se deniega:

```json
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {"AWS": "arn:aws:iam::999999999999:root"},
    "Action": "sts:AssumeRole",
    "Condition": {"StringEquals": {"sts:ExternalId": "7f3a9c1e-cliente-1234"}}
  }]
}
```

> **NOTA**
>
> En `Principal`, `arn:aws:iam::999999999999:root` significa «la cuenta 999999999999», no su usuario root: delegas en esa cuenta y sus administradores deciden quién puede asumir tu rol.

2.6 IAM Identity Center y los permission sets

Las organizaciones bien gestionadas separan producción, desarrollo o cada equipo en **cuentas distintas**, la frontera de aislamiento más fuerte de AWS. Las agrupan con **AWS Organizations** bajo

una **cuenta de administración** (*management account*), que paga la factura y fija normas comunes (lo verás en el capítulo «IAM (II)»). Con veinte cuentas, dar a cada persona un usuario IAM en cada una significa veinte contraseñas, veinte juegos de claves permanentes y veinte bajas cuando alguien se va.

IAM Identity Center (antes AWS Single Sign-On) resuelve este problema: es la forma recomendada de dar acceso a las **personas** de tu organización a todas tus cuentas AWS (y a aplicaciones). Cada persona tiene una única identidad, con MFA, y desde un **portal de acceso** elige a qué cuenta entrar y con qué perfil, siempre con credenciales temporales. Se habilita en la cuenta de administración de la organización, y su gestión puede delegarse en una cuenta miembro. Sus piezas:

Fuente de identidad — dónde viven las personas: el directorio propio de Identity Center, un Active Directory o un IdP externo como Microsoft Entra ID, Okta o Google Workspace, conectado por SAML 2.0 y con **SCIM** para sincronizar usuarios y grupos automáticamente. Solo puede haber una a la vez.
Usuarios y grupos — las personas y sus agrupaciones (normalmente los mismos grupos que ya tienes en tu directorio corporativo).
Permission sets — plantillas de permisos que defines una vez.
Asignaciones — la combinación **quién + qué + dónde**: un usuario o grupo, un permission set y una cuenta AWS. Sin asignación, un permission set no concede nada.
Portal de acceso — la web (`https://d-xxxxxxxxxx.awsapps.com/start` o un alias propio) donde cada persona ve las cuentas y perfiles que tiene asignados.

CONCEPTO

Usuario / grupo de Identity Center — las personas de tu organización

QUÉ ES	Una persona del almacén de identidades de Identity Center (creada allí o sincronizada desde tu IdP), y los grupos que las reúnen para asignarles acceso.
QUÉ NO ES	No es un usuario IAM: no existe en ninguna cuenta, no tiene access keys permanentes ni le adjuntas políticas de IAM. Sus grupos **no** son grupos de IAM.
PARA QUÉ	Dar a cada empleado una única identidad con la que entra, con MFA, en todas las cuentas y aplicaciones que se le asignen.
EJEMPLO	Ana entra en el portal con su cuenta corporativa de Microsoft Entra ID y ve las cuentas `dev` y `prod`.
ERROR TÍPICO	Crear además un usuario IAM para la misma persona «por si acaso», o dar de alta personas a mano cuando tu IdP puede sincronizarlas por SCIM.

Como los nombres se parecen tanto, conviene poner frente a frente las dos familias de usuarios y grupos:

	Usuarios y grupos IAM	Usuarios y grupos de Identity Center
Dónde viven	Dentro de una cuenta AWS concreta	En Identity Center (creados allí o sincronizados desde tu IdP por SCIM)
Credenciales	Permanentes: contraseña y access keys	Ninguna permanente en AWS: tras iniciar sesión, las temporales del rol de cada cuenta
Cómo reciben permisos	Políticas de IAM adjuntas al usuario o a sus grupos	Asignaciones: usuario o grupo + permission set + cuenta
Alcance	Una sola cuenta	Todas las cuentas de la organización que asignes
Altas y bajas	Cuenta por cuenta	Una vez, en tu directorio corporativo

CONCEPTO

Permission set una plantilla de permisos para muchas cuentas

QUÉ ES	Una plantilla definida en Identity Center con políticas gestionadas (de AWS o tuyas), una política inline opcional, un permissions boundary opcional y la duración de la sesión (de 1 a 12 horas).
QUÉ NO ES	No es un rol ni una política que se adjunte: por sí mismo no concede nada hasta que lo **asignas** a una cuenta. No es un principal.
PARA QUÉ	Definir perfiles —«Desarrollador», «SoloLectura», «Administrador»— una sola vez y desplegarlos en tantas cuentas como necesites.
EJEMPLO	`PowerUser` = política gestionada `PowerUserAccess` + sesión de 8 horas, asignado al grupo `Developers` en las cuentas `dev` y `test`.
ERROR TÍPICO	Tocar a mano el rol que genera en cada cuenta (lo gestiona Identity Center), o usar una política propia que no existe con ese nombre en alguna cuenta asignada.

Qué pasa por debajo cuando entras por el portal

Aquí está la pieza que lo conecta todo. Cuando asignas un permission set a una cuenta, Identity Center crea en esa cuenta un **rol IAM** llamado `AWSReservedSSO_<PermissionSet>_<sufijo>`, con las políticas del permission set, y un proveedor SAML (`AWSSSO_…_DO_NOT_DELETE`, uno por cuenta), que es el único en el que confía la trust policy del rol: ese rol solo puede asumirse a través de Identity Center. Hay **un rol por permission set y cuenta**, no por persona: si cincuenta personas tienen `PowerUser` en dev, todas asumen el mismo rol y CloudTrail las distingue por el nombre de sesión. Si cambias el permission set, Identity Center actualiza el rol en todas las cuentas asignadas.

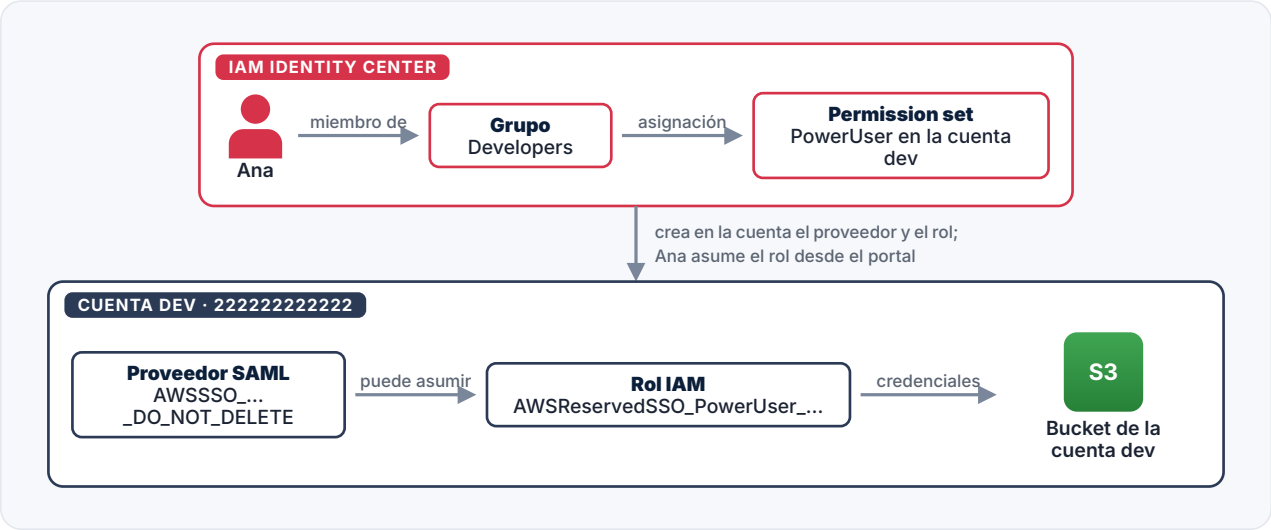


Figura 13 · De la persona al recurso. Ana pertenece al grupo Developers, que tiene asignado el permission set PowerUser en la cuenta dev. Al hacer la asignación, Identity Center creó en esa cuenta un proveedor SAML y el rol AWSReservedSSO_PowerUser_..., que solo confía en ese proveedor.

Así es una jornada de Ana:

1. Abre el portal de acceso y se autentica en la fuente de identidad (su IdP corporativo), con MFA. Empieza su **sesión del portal**.
2. El portal le muestra las cuentas y permission sets que tiene asignados, directamente o a través de sus grupos.
3. Elige «dev → PowerUser». Mediante una federación SAML (`AssumeRoleWithSAML`) con el proveedor que creó en la cuenta dev, Identity Center le consigue credenciales temporales del rol `AWSReservedSSO_PowerUser_…`, y la consola se abre en esa cuenta.

4 Trabaja como la sesión `assumed-role/AWSReservedSSO_PowerUser_…/ana`, con los permisos del permission set. Esas credenciales caducan según la duración de sesión del permission set; mientras siga abierta su sesión del portal, obtiene otras sin volver a autenticarse.

En la CLI ocurre lo mismo. La configuración se hace una vez con `aws configure sso`, que crea un perfil como este; después, cada día, `aws sso login --profile dev-poweruser` abre el navegador para autenticarte y guarda un token en caché (`~/.aws/sso/cache`). Con él, la CLI pide al portal las credenciales temporales del rol de esa cuenta y las renueva sola:

```ini
[profile dev-poweruser]
sso_session    = empresa
sso_account_id = 222222222222
# El nombre del permission set
sso_role_name  = PowerUser

[sso-session empresa]
sso_start_url           = https://d-1234567890.awsapps.com/start
sso_region              = eu-west-1
sso_registration_scopes = sso:account:access
```

≈ **EN NUESTRO EDIFICIO**

Identity Center es el RR. HH. central de la empresa: da de alta a Ana una sola vez y decide en qué sedes trabaja. El permission set es el modelo de acreditación «Desarrollador»; la asignación, la orden «Ana puede recoger esa acreditación en la sede dev». Con esa orden, RR. HH. deja en el mostrador de la sede dev una acreditación concreta —el rol `AWSReservedSSO_…`—, y Ana la recoge cada mañana desde el portal.

2.7 Todas las piezas de un vistazo

Pieza	Qué es	Credenciales	Úsala para…	No la confundas con…
Usuario IAM	Identidad permanente de una persona o app, en una sola cuenta	Permanentes: contraseña y access keys (`AKIA…`)	Solo excepciones: emergencia, herramientas sin roles	Un usuario de Identity Center: fuera de las cuentas y sin claves permanentes
Grupo IAM	Conjunto de usuarios IAM de una cuenta; no firma peticiones	Ninguna	Repartir políticas entre usuarios IAM	Un grupo de Identity Center, que recibe asignaciones en muchas cuentas
Rol IAM	Identidad sin credenciales propias que se asume; actúa mediante sesiones	Temporales (`ASIA…`), para quien lo asume	Servicios, otras cuentas, federación	Un usuario: el rol no es de nadie y sus credenciales caducan
Trust policy	La política de recurso de un rol: quién puede asumirlo	—	Todo rol tiene una	La permissions policy: qué puede hacer quien lo asume
Usuario de Identity Center	Una persona de tu organización, con una sola identidad para todo	Ninguna propia: las temporales del rol de cada cuenta	Acceso de personas a todas las cuentas	Un usuario IAM (ver arriba)
Grupo de Identity Center	Conjunto de usuarios de Identity Center, a menudo sincronizado desde tu IdP	Ninguna	Asignar permission sets por equipos	Un grupo IAM (ver arriba)
Permission set	Plantilla de permisos de Identity Center; no concede nada hasta que se asigna	Ninguna	Definir perfiles una vez para muchas cuentas	El rol `AWSReservedSSO_…` que crea en cada cuenta asignada: el que se asume

2.8 Otras formas de federación

La **federación** consiste en que AWS confíe en un proveedor de identidad externo: las identidades viven fuera y, tras autenticarse allí, reciben credenciales temporales de AWS asumiendo un rol. El IdP se lo comunica a AWS con un protocolo estándar: **SAML 2.0** (aserciones en XML, típico de los directorios corporativos) u **OIDC** (*OpenID Connect*, tokens JSON firmados llamados JWT, típico de plataformas como GitHub o Kubernetes). Identity Center es la federación recomendada para empleados, pero hay otros casos:

Mecanismo	Para quién	Cómo obtiene las credenciales
IAM Identity Center	Empleados, en muchas cuentas	Portal o CLI → asume el rol `AWSReservedSSO_…` de cada cuenta
SAML 2.0 directo con IAM	Empleados en una cuenta (opción heredada)	Aserción SAML del IdP → `AssumeRoleWithSAML`
Proveedor OIDC en IAM	Cargas externas: GitHub Actions, GitLab, Kubernetes…	Token OIDC (JWT) → `AssumeRoleWithWebIdentity`
Amazon Cognito	Usuarios finales de tus aplicaciones	Un *user pool* autentica; un *identity pool* canjea el token por credenciales de un rol limitado
IAM Roles Anywhere	Servidores y cargas fuera de AWS	Un certificado X.509 de tu PKI de confianza → credenciales temporales de un rol

2.9 Preguntas frecuentes

? ¿Un grupo puede asumir un rol o aparecer como `Principal`?
No: no es un principal. Para que sus miembros asuman un rol, adjunta al grupo una política que permita `sts:AssumeRole` sobre él y haz que la trust policy confíe en la cuenta.

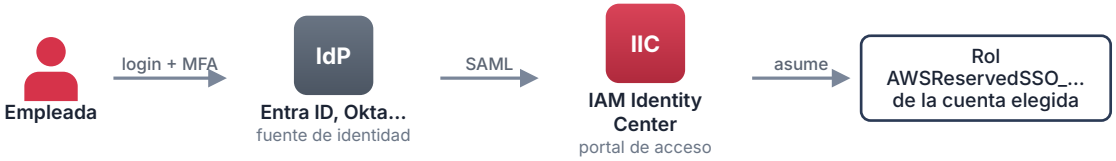
? ¿Necesito AWS Organizations para usar Identity Center?
Para dar acceso a cuentas AWS, sí: se habilita en la cuenta de administración de la organización. Sin Organizations solo puede dar acceso a aplicaciones.

? ¿Por qué el portal no me pide la contraseña cada hora?
Hay dos relojes: la sesión del portal (por defecto, 8 horas) y las credenciales de cada rol (la duración del permission set). Mientras la primera siga abierta, obtienes credenciales nuevas sin volver a autenticarte.

? ¿Cómo nombro el rol de un permission set en una política de recurso?
Su ARN lleva una ruta (`…/aws-reserved/sso.amazonaws.com/…`) y un sufijo aleatorio que cambia si se recrea: en vez de copiarlo, usa la condición `aws:PrincipalArn` con comodines (capítulo «IAM (II)»).

2.10 ¿Qué uso cuando…? Ocho escenarios resueltos

ESCENARIO 1 **Una persona de tu empresa entra en la consola** — IAM Identity Center



Personas y grupos en Identity Center (sincronizados por SCIM), permission sets y asignaciones a **grupos**. **Evita:** un usuario IAM por persona y por cuenta.

ESCENARIO 2 **Una aplicación en EC2 lee objetos de S3** — Rol + perfil de instancia



El rol permite `s3:GetObject` sobre `arn:aws:s3:::fotos/*` y el código no contiene claves: el SDK las encuentra solo. **Evita:** access keys en la instancia, en variables de entorno o en la AMI.

ESCENARIO 3 **Una función Lambda escribe en DynamoDB** — Rol de ejecución



El rol de ejecución lleva `dynamodb:PutItem` sobre la tabla y permisos para escribir logs; Lambda lo asume y pasa las credenciales a tu código en variables de entorno. Quién puede **invocar** la función lo decide otra cosa: su política de recurso (capítulo de Serverless).

ESCENARIO 4 **Una tarea de ECS lee un secreto de Secrets Manager** — Task role / execution role



Una tarea tiene **dos** roles: el *execution role*, con el que ECS la prepara (imagen, logs y secretos que se inyectan como variables de entorno), y el *task role*, el de tu código. Si tu código pide el secreto con el SDK, el permiso va en el task role; si se inyecta desde la definición de tarea, en el execution role. Los detalles, en el capítulo de Contenedores.

ESCENARIO 5 **GitHub Actions despliega en AWS sin access keys** — Federación OIDC



Registra en IAM el proveedor OIDC `token.actions.githubusercontent.com` y crea un rol que acepte solo los tokens de **tu** repositorio y rama: sin la condición sobre `sub`, cualquier repositorio de GitHub podría asumirlo.

```json
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {"Federated":
      "arn:aws:iam::123456789012:oidc-provider/token.actions.githubusercontent.com"},
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": {
        "token.actions.githubusercontent.com:aud": "sts.amazonaws.com"},
      "StringLike": {
        "token.actions.githubusercontent.com:sub": "repo:org/web:ref:refs/heads/main"}
    }
  }]
}
```

En el workflow, el job declara el permiso para pedir el token y asume el rol:

```yaml
permissions:
  id-token: write        # permite al job pedir el token OIDC
steps:
  - uses: aws-actions/configure-aws-credentials@v4
    with:
      role-to-assume: arn:aws:iam::123456789012:role/github-deploy
      aws-region: eu-west-1
  - run: aws s3 sync ./dist s3://mi-web-estatica
```

ESCENARIO 6 La cuenta A lee un bucket de la cuenta B

Política de recurso o rol en B



Hay dos caminos. **Con un rol en B** (el del diagrama): A asume `lector-b` y trabaja «dentro» de B, sin sus propios permisos durante la sesión; útil si hay que tocar muchos recursos. **Con una política de recurso**: la política del bucket permite el acceso al rol de A y la política de identidad de ese rol permite `s3:GetObject` sobre el bucket; entre cuentas hacen falta **los dos permisos** (lo verás con detalle en el capítulo «IAM (II)»).

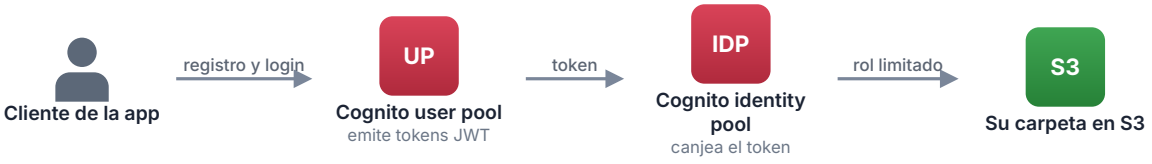
ESCENARIO 7 Un proveedor SaaS externo audita tu cuenta

Rol con External ID

Un rol cuya trust policy confía en la cuenta del proveedor **y** exige el External ID que te ha dado (sección del *confused deputy*), con solo los permisos necesarios, como la política gestionada `SecurityAudit`. Entregas al proveedor el ARN del rol, nunca access keys, y revocas el acceso cuando quieras borrando el rol o cambiando su trust policy.

ESCENARIO 8 Los usuarios finales de tu app móvil

Amazon Cognito



Tus clientes no son empleados: ni IAM ni Identity Center. Un **user pool** de Cognito gestiona registro, inicio de sesión y MFA; si la app debe llamar directamente a AWS, un **identity pool** canjea el token por credenciales temporales de un rol muy limitado. A menudo basta con que tu API valide el token.

RESUMEN DEL CAPÍTULO

- Toda petición recorre cuatro pasos: identidad → credenciales → políticas → recurso. Los grupos y los permission sets no hacen peticiones: reparten políticas.
- Un principal es quien firma una petición: un usuario, una sesión de rol, el root o un servicio. Los grupos nunca firman nada; los roles, a través de sus sesiones.
- Los usuarios IAM tienen credenciales permanentes (contraseña, access keys `AKIA…`): resérvalos para casos excepcionales.
- Un rol no tiene credenciales propias: su trust policy dice quién puede asumirlo y su permissions policy, qué puede hacer. STS entrega credenciales temporales (`ASIA…` + token + caducidad).
- Los servicios asumen su rol por ti (perfil de instancia en EC2, rol de ejecución en Lambda, task role en ECS) y dejan las credenciales donde el SDK las busca; entregar un rol exige `iam:PassRole`. El External ID protege los roles para terceros.
- IAM Identity Center da a cada persona una identidad única. Una asignación (usuario o grupo + permission set + cuenta) crea en esa cuenta un rol `AWSReservedSSO_…`, uno por permission set y cuenta, que la persona asume desde el portal o con `aws sso login`.
- Federación: SAML con los IdP corporativos, OIDC con cargas externas como GitHub Actions, IAM Roles Anywhere para servidores fuera de AWS y Amazon Cognito para los clientes de tus apps.

Preguntas de repaso

1 **¿Por qué un grupo IAM no puede aparecer como `Principal` en la política de un bucket?**
Respuesta: Porque no es un principal: no se autentica ni firma peticiones. Solo sirve para adjuntar políticas a sus usuarios.

2 **¿Qué dos políticas tiene un rol y qué pregunta responde cada una?**
Respuesta: La trust policy (¿quién puede asumirlo?) y la permissions policy (¿qué puede hacer quien lo asume?).

3 **Tu equipo copia access keys en un servidor EC2 para que la aplicación lea de S3. ¿Qué propones?**
Respuesta: Un rol con lectura sobre ese bucket, entregado con un perfil de instancia; el SDK toma credenciales temporales del IMDS y las claves desaparecen.

4 **¿Cómo obtiene sus permisos una función Lambda y dónde los encuentra su código?**
Respuesta: Lambda asume el rol de ejecución de la función y le pasa las credenciales temporales en variables de entorno; el SDK las encuentra solo, sin claves en el código.

5 **Ana tiene asignado el permission set `ReadOnly` en las cuentas `dev` y `prod`, y Luis, el mismo en `dev`. ¿Cuántos roles ha creado Identity Center?**
Respuesta: Dos, uno por cuenta (`AWSReservedSSO_ReadOnly_<sufijo>`): hay un rol por permission set y cuenta, no por persona. Ana y Luis asumen el mismo rol en `dev` y CloudTrail los distingue por el nombre de sesión.

6 **¿Qué ocurre por debajo cuando Ana elige «dev → PowerUser» en el portal?**
Respuesta: Identity Center le obtiene credenciales temporales del rol `AWSReservedSSO_PowerUser_…` que creó en la cuenta dev, mediante una federación SAML con el proveedor que también creó allí. Ana trabaja como la sesión `assumed-role/AWSReservedSSO_PowerUser_…/ana`, con los permisos del permission set, hasta que caducan.

7 **Un workflow de GitHub Actions debe desplegar en tu cuenta. ¿Cómo se autentica sin guardar access keys?**
Respuesta: Con federación OIDC: registras en IAM el proveedor `token.actions.githubusercontent.com` y creas un rol cuya trust policy solo acepta los tokens de tu repositorio y rama (condición sobre `sub`). El job lo asume con `AssumeRoleWithWebIdentity`.

CAPÍTULO 3

IAM (II): permisos — políticas y evaluación

Qué puede hacer cada identidad, sobre qué recursos y en qué condiciones: cómo se escriben las políticas y cómo decide AWS cada petición.

EN ESTE CAPÍTULO APRENDERÁS

1. Leer y escribir políticas JSON elemento a elemento
2. Los tipos de políticas y cómo se combinan sus límites
3. La lógica de evaluación, en la misma cuenta y entre cuentas
4. Condiciones, variables y control de acceso por etiquetas (ABAC)
5. Permissions boundaries, SCP y RCP para poner límites
6. Depurar un AccessDenied y aplicar el mínimo privilegio

SERVICIOS PROTAGONISTAS

AWS IAM · AWS Organizations · IAM Access Analyzer · IAM Policy Simulator · AWS CloudTrail · AWS Control Tower

3.1 Del quién al qué: tres reglas de oro

En el capítulo «IAM (I)» viste **quién** hace cada petición y **cómo** obtiene sus credenciales (los pasos 1 y 2 del mapa de IAM). Este capítulo trata el paso 3: **qué puede hacer**. En nuestro edificio, ya sabemos quién lleva cada tarjeta o acreditación; ahora toca decidir qué puertas abre cada una, qué dicen los carteles de cada sala y qué normas de la empresa están por encima de todo. Todo se apoya en tres reglas que conviene grabarse a fuego:



Figura 14 · Las tres reglas de la autorización en AWS. La tercera no tiene excepciones: ningún Allow puede anular un Deny explícito.

Los permisos se expresan con **políticas**: documentos JSON que se adjuntan a identidades o a recursos.

CONCEPTO

Política un documento JSON que permite o deniega

QUÉ ES	Un documento con una o varias sentencias (*statements*) que permiten o deniegan acciones sobre recursos, opcionalmente bajo condiciones.
QUÉ NO ES	No es una identidad ni un principal, y no hace nada hasta que la adjuntas a una identidad o a un recurso. Tampoco es un rol: un rol **tiene** políticas.
PARA QUÉ	Conceder permisos (Allow) o limitarlos (Deny, permissions boundaries, SCP...).
EJEMPLO	La política gestionada `ReadOnlyAccess`, o una tuya que permite leer solo el bucket `facturas`.
ERROR TÍPICO	Poner `"Action": "*"` y `"Resource": "*"` «para probar» y no volver a quitarlo nunca.

3.2 Anatomía de una política, paso a paso

Esta política permite leer el bucket `facturas`, pero solo desde la red de la oficina. Recorre sus elementos en orden:

JSON

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "LeerFacturas",
    "Effect": "Allow",
    "Action": [
      "s3:GetObject",
      "s3:ListBucket"
    ],
    "Resource": [
      "arn:aws:s3:::facturas",
      "arn:aws:s3:::facturas/*"
    ],
    "Condition": {
      "IpAddress": {
        "aws:SourceIp": "203.0.113.0/24"
      }
    }
  }]
}
```

1. **Version**: la versión del lenguaje, no de tu política. Usa siempre `"2012-10-17"`; la antigua `2008-10-17` no admite variables.
2. **Statement**: la lista de sentencias. Cada una se evalúa por separado.
3. **Sid**: un nombre opcional para la sentencia; útil para leer y depurar.
4. **Effect**: `Allow` o `Deny`.
5. **Action**: qué operaciones de la API (`servicio:Acción`).
6. **Resource**: sobre qué recursos, por su ARN.
7. **Condition** (opcional): cuándo aplica la sentencia.

En las políticas de recurso aparece además **Principal**: a quién se aplica.

Action: qué operaciones

Cada acción tiene la forma `servicio:Operación` (`s3:GetObject`, `ec2:StartInstances`, `dynamodb:Query`) y no distingue mayúsculas. Admite comodines: `s3:Get*` (todas las que empiezan por `Get`), `s3:*` (todo S3) o `*` (todo AWS). Las acciones son operaciones de la **API**, no pantallas de la consola: ver una página de la consola suele requerir varias acciones `List*` y `Describe*`. La lista completa de acciones, tipos de recurso y claves de condición de cada servicio está en la **Service Authorization Reference** de la documentación de AWS: tenla siempre a mano.

Resource: sobre qué recursos

Los recursos se indican con su ARN, y también admiten comodines (`*` y `?`). Cada acción se autoriza contra un **tipo de recurso** concreto, y aquí está el error más típico de S3: `s3:ListBucket` actúa sobre el **bucket** (`arn:aws:s3:::facturas`), mientras que `s3:GetObject` actúa sobre los **objetos** (`arn:aws:s3:::facturas/*`). Por eso el ejemplo lista los dos ARN. Algunas acciones no admiten permisos a nivel de recurso (por ejemplo, `ec2:DescribeInstances` o `s3:ListAllMyBuckets`) y exigen `"Resource": "*"`.

Condition: cuándo aplica

El bloque `Condition` añade requisitos sobre el contexto de la petición, con la estructura `{"Operador": {"clave": "valor"}}`. Dos reglas de lógica: **operadores o claves distintos se combinan con Y** (deben cumplirse todos) y **varios valores de una misma clave se combinan con O** (basta con uno). Si una condición no se cumple, la sentencia simplemente **no aplica**.

Operadores	Para comparar…	Ejemplo
`StringEquals`, `StringLike`, `StringNotEquals`	Texto (con `*` y `?` en `Like`)	`"aws:PrincipalTag/Equipo": "pagos"`
`ArnEquals`, `ArnLike`	ARN	`"aws:SourceArn": "arn:aws:sns:eu-west-1:111122223333:*"`
`IpAddress`, `NotIpAddress`	Rangos IP (CIDR)	`"aws:SourceIp": "203.0.113.0/24"`
`Bool`, `Null`	Verdadero/falso; si existe la clave	`"aws:SecureTransport": "false"`
`NumericLessThan`, `DateGreaterThan` …	Números y fechas	`"aws:MultiFactorAuthAge": "3600"`

Dos modificadores completan el lenguaje: el sufijo `…IfExists` («compruébalo solo si la clave existe») y los prefijos `ForAnyValue:` / `ForAllValues:` para claves con varios valores, como `aws:TagKeys`.

Principal: a quién (solo en políticas de recurso)

Las políticas de identidad no llevan `Principal`: se aplican a la identidad a la que se adjuntan. Las de recurso sí, y admiten una cuenta (`{"AWS": "arn:aws:iam::111122223333:root"}`), un rol o un usuario concretos, un servicio (`{"Service": "lambda.amazonaws.com"}`), un proveedor federado o `"*"` (cualquiera, incluido el público anónimo: úsalo solo con condiciones que lo acoten). Nunca un grupo.

CÓMO LEER CUALQUIER POLÍTICA

Primero mira si tiene `Principal`: si lo tiene, es una política de recurso y dice **quién** puede tocar ese recurso; si no, es de identidad y habla de quien la lleva. Después traduce cada sentencia a una frase: «`Effect` a `Principal` hacer `Action` sobre `Resource` si `Condition`». La del ejemplo diría: «Permite listar el bucket `facturas` y descargar sus objetos, solo desde la red 203.0.113.0/24».

NotAction y NotResource, con cuidado

`NotAction` significa «todas las acciones **excepto** estas», y `NotResource`, «todos los recursos excepto estos». Combinados con `Deny` son muy útiles —«deniega todo salvo lo necesario para configurar la MFA», «deniega todo fuera de estas regiones salvo los servicios globales»—; verás ambos ejemplos en este capítulo.

> **CUIDADO**
>
> `"Effect": "Allow"` con `NotAction` concede **todo lo demás**, incluidos los servicios que AWS lance mañana. Un `Allow` + `NotAction: "iam:*"` no es «todo menos IAM con prudencia»: es administrador de casi todo. Úsalo solo si lo entiendes y, preferiblemente, nunca con `Allow`.

3.3 Los tipos de políticas

No todas las políticas hacen lo mismo. Unas **conceden** permisos y otras solo **limitan** lo que las primeras pueden conceder:

Tipo	Se adjunta a	¿Concede?	Para qué	En nuestro edificio
De identidad (gestionada o inline)	Usuarios, grupos y roles	✔ Sí	Los permisos del día a día	Lo que abre tu tarjeta o tu acreditación
De recurso	Recursos: bucket S3, cola SQS, clave KMS, función Lambda, trust policy de un rol…	✔ Sí	Acceso entre cuentas y desde servicios	El cartel de la puerta: «solo Contabilidad»
Permissions boundary	Usuarios y roles	✘ No, limita	Delegar la creación de roles con un techo	Un tope grabado en la tarjeta: nunca abrirá más, la programe quien la programe
SCP (Organizations)	Raíz, OU o cuenta	✘ No, limita	Guardarraíles para los principals de una cuenta	Las normas de la empresa, que ninguna sede puede saltarse
RCP (Organizations)	Raíz, OU o cuenta	✘ No, limita	Perímetro para los recursos de una cuenta	«En estas salas no entra nadie de fuera de la empresa»
Session policy	Una sesión (al asumir un rol o federarse)	✘ No, limita	Reducir los permisos de una sesión concreta	Un pase de un día con menos permisos que tu acreditación
ACL	Algunos recursos (S3, heredado)	✔ Sí	Compatibilidad; desactivadas por defecto en S3	Un sistema antiguo de llaves; evítalo

Identidad frente a recurso: la misma regla en dos sitios

La diferencia que más cuesta al principio es **dónde vive** la política. Una política de identidad va pegada a quien actúa y habla de lo que **él** puede hacer: no lleva `Principal`. Una política de recurso va pegada a lo que se toca y dice **quién** puede tocarlo: lleva `Principal`. Estas dos sentencias conceden exactamente lo mismo:

En el rol app-web (de identidad)

```json
{
  "Effect": "Allow",
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3:::facturas/*"
}
```

En el bucket facturas (de recurso)

```json
{
  "Effect": "Allow",
  "Principal": {"AWS":
    "arn:aws:iam::111122223333:role/app-web"},
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3:::facturas/*"
}
```

En la misma cuenta basta con cualquiera de las dos; entre cuentas hacen falta las dos, como verás en la lógica de evaluación. En el edificio: tu tarjeta dice qué puertas abres tú; el cartel de la puerta, quién puede entrar.

Gestionadas o inline

Las políticas de identidad, a su vez, pueden ser de tres clases:

Gestionada por AWS
p. ej. ReadOnlyAccess

Las crea y mantiene AWS; cómodas para empezar y para funciones típicas (administrador, solo lectura, auditoría). Son amplias, y AWS las amplía cuando lanza servicios nuevos.

Gestionada por el cliente
reutilizable y versionada

La creas tú, la adjuntas a varias identidades y guarda hasta cinco versiones para volver atrás. Es la opción recomendada para el mínimo privilegio.

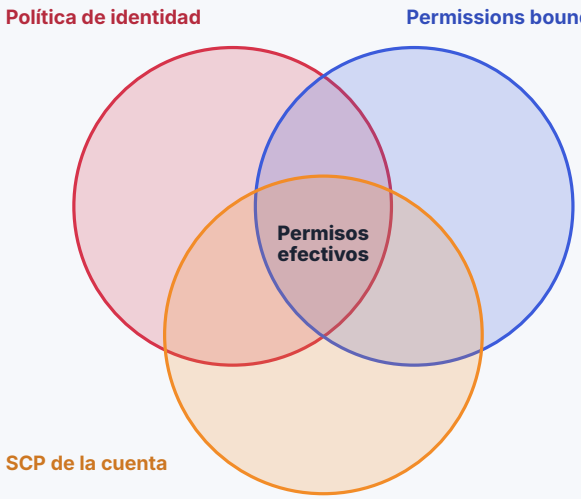
Inline
pegada a una sola identidad

Vive dentro de un usuario, grupo o rol y se borra con él. Útil para una relación estricta uno a uno; difícil de auditar si abusas de ella.

Las políticas de un permission set no son una clase aparte: Identity Center las adjunta como políticas de identidad (gestionadas o inline) al rol `AWSReservedSSO_…` de cada cuenta, y se evalúan igual que cualquier otra.

Cómo se combinan

Cuando aplican varias, los permisos efectivos son la **intersección** de lo que concede la política de identidad y los límites que no conceden nada (boundary, SCP y, si existe, la session policy), siempre que ninguna lo deniegue explícitamente:



Ejemplo: la política de identidad permite S3 y SQS; el boundary permite S3 y DynamoDB; la SCP permite todo salvo regiones fuera de la UE. Resultado: **solo S3, solo en la UE**.

Las **políticas de recurso** son el otro camino: en la misma cuenta pueden conceder acceso por sí solas, como verás en la lógica de evaluación.

Figura 15 · Solo se permite lo que cae dentro de todas las políticas aplicables. Un Deny explícito en cualquiera de ellas lo bloquea.

3.4 Cómo decide AWS: la lógica de evaluación

Para cada petición, AWS reúne el **contexto** (principal, acción, recurso, datos del entorno y etiquetas) y todas las políticas que aplican, y sigue siempre el mismo razonamiento. Para predecir si una petición se permite, recorre este diagrama de arriba abajo: la primera casilla de color a la que llegues da la respuesta.



Figura 16 · Lógica de evaluación de una petición dentro de una cuenta (simplificada). (*) Entre cuentas, además, debe permitirlo la política de identidad del principal.

Dos matices importantes. Primero, en la misma cuenta, un Allow en una política de recurso puede bastar aunque la identidad no tenga ninguna política que lo permita (la interacción con boundaries y session policies tiene detalles adicionales que recoge la documentación). Segundo, hay dos excepciones en las que la política de recurso es **obligatoria**: la **trust policy** de un rol (nadie puede asumirlo si no lo permite) y la **key policy** de una clave KMS (las políticas de IAM solo funcionan si la key policy lo delega en la cuenta).

> ★ **CLAVE PARA LA CERTIFICACIÓN**
>
> Las SCP, las RCP y los permissions boundaries **nunca conceden** permisos: solo recortan. Si una pregunta dice «se ha adjuntado una SCP que permite S3» y el usuario no tiene ninguna política de identidad, la respuesta es que **no puede** usar S3.

Misma cuenta frente a entre cuentas

Cuando el principal y el recurso están en **cuentas distintas**, la regla cambia: hacen falta **dos** Allow, uno en cada lado. La cuenta del principal debe permitir la acción en su política de identidad (y sus SCP no deben impedirlo), y la cuenta del recurso debe permitirla en la política del recurso (y sus RCP no deben impedirlo).

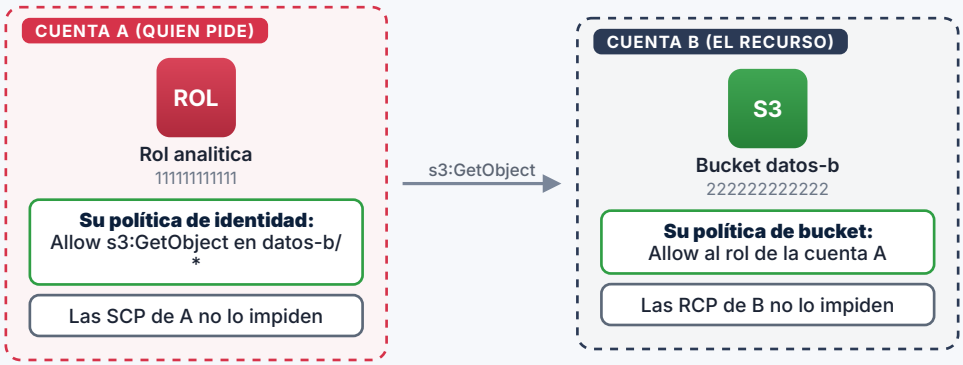


Figura 17 · Acceso entre cuentas con una política de recurso: cada cuenta controla su lado y hacen falta los dos permisos.

Política de identidad del principal	Política del recurso	Misma cuenta	Entre cuentas
Allow	No dice nada	✔ Permitido	✘ Denegado
No dice nada	Allow a ese principal	✔ Permitido	✘ Denegado
Allow	Allow a ese principal	✔ Permitido	✔ Permitido
Allow	Deny	✘ Denegado	✘ Denegado

Esta es la política del bucket en la cuenta B. Fíjate en que nombra al rol concreto de A; nombrar `arn:aws:iam::111111111111:root` delegaría en toda la cuenta A, que decidiría con sus propias políticas quién entra:

JSON

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "LecturaDesdeAnalitica",
    "Effect": "Allow",
    "Principal": {"AWS": "arn:aws:iam::111111111111:role/analitica"},
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::datos-b/*"
  }]
}
```

NOTA

Si en lugar de esto la cuenta A **asume un rol** de la cuenta B (el camino «con un rol en B» del escenario 6 de «IAM (I)»), solo la llamada a `AssumeRole` es entre cuentas y necesita los dos permisos: la política de identidad en A y la trust policy en B. Después, la sesión del rol pertenece a B y sus peticiones se evalúan como cualquier otra de B.

3.5 Condiciones útiles y variables de política

Las condiciones son lo que convierte una política correcta en una política **precisa**. Estas claves globales son las que más verás:

Clave de condición	Qué compara	Uso típico
`aws:SourceIp`	IP pública de origen	Solo desde la oficina (no aplica si la petición llega por un VPC endpoint)
`aws:SourceVpce`, `aws:SourceVpc`	Endpoint o VPC de origen	Bucket accesible solo desde tu red privada
`aws:PrincipalOrgID`	Organización del principal	Política de recurso: «cualquiera de mi organización»
`aws:RequestedRegion`	Región de destino	SCP que limita las regiones permitidas
`aws:MultiFactorAuthPresent`	Si la sesión usó MFA	Exigir MFA para acciones sensibles
`aws:SecureTransport`	Si la petición usa TLS	Denegar el acceso sin HTTPS a un bucket
`aws:PrincipalTag/…`, `aws:ResourceTag/…`, `aws:RequestTag/…`, `aws:TagKeys`	Etiquetas del principal, del recurso o de la petición	Control de acceso por atributos (ABAC)
`aws:SourceArn`, `aws:SourceAccount`	En nombre de qué recurso o cuenta actúa un servicio	Evitar el *confused deputy* entre servicios
`aws:CalledVia`, `aws:ViaAWSService`	Si la petición la hace un servicio en tu nombre	Permitir ciertas acciones solo a través de un servicio

Un ejemplo clásico: denegar **todo** salvo lo necesario para configurar la MFA cuando la sesión no se ha autenticado con ella. Se usa `BoolIfExists` porque, con access keys de largo plazo, la clave `aws:MultiFactorAuthPresent` ni siquiera existe; con un simple `Bool`, la condición no se cumpliría y esas peticiones se escaparían del Deny:

```json
{
  "Sid": "DenegarSinMFA",
  "Effect": "Deny",
  "NotAction": [
    "iam:CreateVirtualMFADevice", "iam:EnableMFADevice", "iam:ListMFADevices",
    "iam:GetUser", "sts:GetSessionToken"
  ],
  "Resource": "*",
  "Condition": {"BoolIfExists": {"aws:MultiFactorAuthPresent": "false"}}
}
```

Las **variables de política** se sustituyen en cada petición por un valor del contexto. Con `${aws:username}` una sola política sirve para que cada usuario IAM gestione **sus propias** credenciales; con `${aws:PrincipalTag/Equipo}`, para comparar etiquetas. Solo funcionan con `"Version": "2012-10-17"`.

```json
{
  "Sid": "GestionarMisPropiasClaves",
  "Effect": "Allow",
  "Action": ["iam:ChangePassword", "iam:CreateAccessKey", "iam:ListAccessKeys"],
  "Resource": "arn:aws:iam::*:user/${aws:username}"
}
```

3.6 ABAC: permisos basados en etiquetas

El modelo clásico (RBAC, por roles) obliga a escribir una política por equipo y por recurso: «el equipo de pagos puede parar las instancias A, B y C». Con **ABAC** (*attribute-based access control*) escribes **una sola** política que compara etiquetas: «puedes parar una instancia si su etiqueta `Equipo` coincide con la tuya». Cuando se crea una instancia nueva bien etiquetada, nadie tiene que tocar ninguna política.

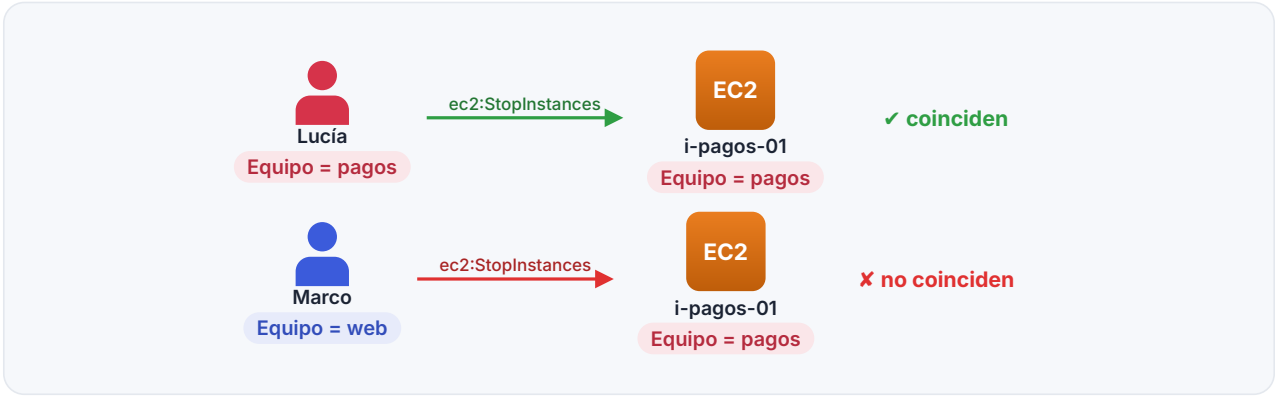


Figura 18 · La misma política para Lucía y para Marco: lo que cambia son sus etiquetas.

JSON

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["ec2:StartInstances", "ec2:StopInstances"],
    "Resource": "arn:aws:ec2:*:*:instance/*",
    "Condition": {
      "StringEquals": {"aws:ResourceTag/Equipo": "${aws:PrincipalTag/Equipo}"}
    }
  }]
}
```

CUIDADO

En ABAC, las etiquetas **son** permisos. Si cualquiera puede cambiar la etiqueta `Equipo` de una instancia (o la suya propia), puede darse acceso a sí mismo. Restringe quién puede etiquetar con condiciones sobre `aws:RequestTag` y `aws:TagKeys` en `ec2:CreateTags`, y protege las etiquetas de las identidades (en Identity Center pueden venir del IdP corporativo como atributos de sesión).

3.7 Permissions boundaries: delegar sin escalada de privilegios

Imagina que quieres que los desarrolladores creen los roles de sus propias Lambdas sin pasar por el equipo de seguridad. El peligro es evidente: quien puede crear roles y adjuntarles políticas podría crear un rol de administrador, dárselo a una Lambda (gracias a `iam:PassRole`) y escalar privilegios.

CONCEPTO

Permissions boundary — el techo de permisos de un usuario o rol

QUÉ ES	Una política gestionada que fijas como **límite máximo** de un usuario o rol: sus permisos efectivos son la intersección entre sus políticas de identidad y el boundary.
QUÉ NO ES	No concede nada por sí misma: un rol con boundary y sin políticas no puede hacer nada. No se aplica a grupos.
PARA QUÉ	Delegar la gestión de IAM (crear roles para aplicaciones) sin riesgo de que alguien se conceda más de lo permitido.
EJEMPLO	El boundary `LimiteApps` permite S3, DynamoDB y CloudWatch Logs; todo rol que creen los desarrolladores debe llevarlo.
ERROR TÍPICO	Fijar el boundary pero permitir a los desarrolladores quitarlo o editarlo: se lo saltarían en un minuto.

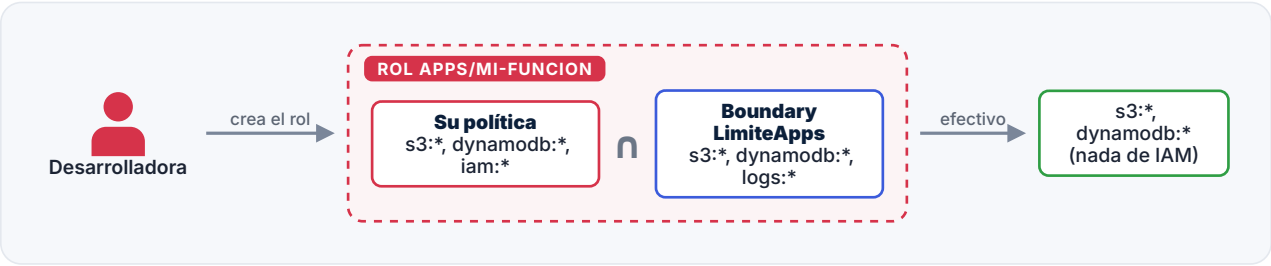


Figura 19 · Aunque la desarrolladora adjunte al rol una política muy amplia, el boundary obligatorio recorta el resultado.

La política de los desarrolladores solo les deja crear roles **si llevan el boundary**, pasar a Lambda solo roles de su ruta y nunca quitar ni modificar el límite:

JSON

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "CrearRolesSoloConLimite",
      "Effect": "Allow",
      "Action": ["iam:CreateRole", "iam:PutRolePermissionsBoundary",
                 "iam:AttachRolePolicy", "iam:PutRolePolicy"],
      "Resource": "arn:aws:iam::111122223333:role/apps/*",
      "Condition": {"StringEquals": {
        "iam:PermissionsBoundary": "arn:aws:iam::111122223333:policy/LimiteApps"}}
    },
    {
      "Sid": "PasarSoloRolesDeApps",
      "Effect": "Allow",
      "Action": "iam:PassRole",
      "Resource": "arn:aws:iam::111122223333:role/apps/*",
      "Condition": {"StringEquals": {"iam:PassedToService": "lambda.amazonaws.com"}}
    },
    {
      "Sid": "NoQuitarNiEditarElLimite",
      "Effect": "Deny",
      "Action": ["iam:DeleteRolePermissionsBoundary", "iam:CreatePolicyVersion",
                 "iam:DeletePolicy", "iam:SetDefaultPolicyVersion"],
      "Resource": ["arn:aws:iam::111122223333:role/*",
                   "arn:aws:iam::111122223333:policy/LimiteApps"]
    }
  ]
}
```

3.8 AWS Organizations: SCP y RCP

AWS Organizations agrupa tus cuentas en una jerarquía: una **raíz** (*root*), **unidades organizativas** (OU) que pueden anidarse y **cuentas miembro**, todas gobernadas desde la **cuenta de administración** (*management account*), que paga la factura consolidada (con descuentos por volumen compartidos) y aplica las políticas de la organización. Las dos más importantes para IAM son las SCP y las RCP.

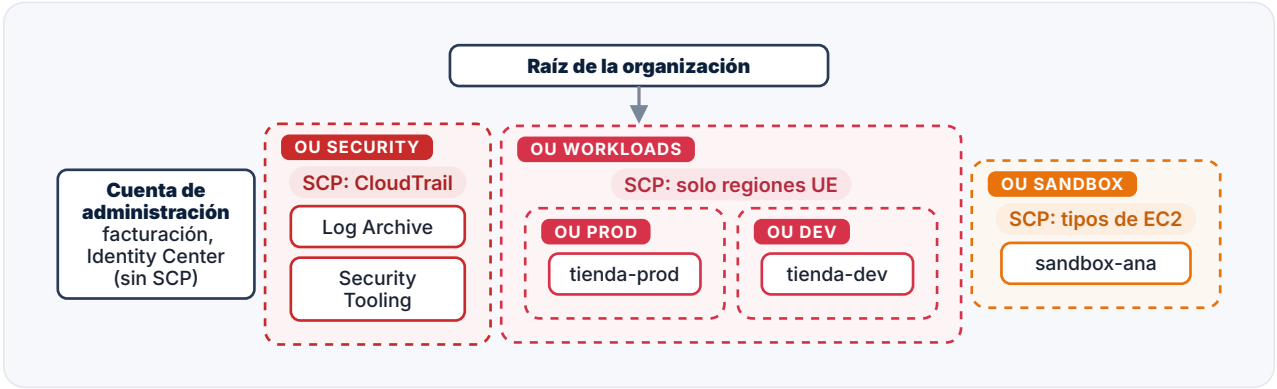


Figura 20 · Una organización típica. Las SCP se adjuntan a la raíz, a las OU o a las cuentas y se heredan hacia abajo; no afectan a la cuenta de administración.

CONCEPTO

SCP los guardarraíles de los principals de una cuenta

QUÉ ES	Una política de Organizations que fija el **máximo** de lo que pueden hacer los usuarios y roles de las cuentas a las que se aplica, incluido su usuario root.
QUÉ NO ES	No concede permisos: dentro de la cuenta sigue haciendo falta una política de identidad. No afecta a la cuenta de administración ni a los roles vinculados a servicios.
PARA QUÉ	Imponer reglas de la organización que ningún administrador de cuenta puede saltarse: regiones, servicios prohibidos, no desactivar la auditoría.
EJEMPLO	Denegar cualquier acción fuera de `eu-west-1` y `eu-south-2`, salvo los servicios globales.
ERROR TÍPICO	Quitar la SCP por defecto `FullAWSAccess` sin poner otra que permita: todo queda denegado en las cuentas afectadas.

Una acción solo se permite si las SCP la permiten **en cada nivel** desde la raíz hasta la cuenta, y un Deny en cualquier nivel se hereda hacia abajo. Por eso el patrón más habitual es dejar `FullAWSAccess` y añadir SCP de **denegación**:

JSON

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "SoloRegionesUE",
      "Effect": "Deny",
      "NotAction": ["iam:*", "organizations:*", "sts:*", "route53:*", "cloudfront:*",
                    "support:*", "budgets:*"],
      "Resource": "*",
      "Condition": {"StringNotEquals": {
        "aws:RequestedRegion": ["eu-west-1", "eu-south-2"]}}
    },
    {
      "Sid": "ProtegerAuditoria",
      "Effect": "Deny",
      "Action": ["cloudtrail:StopLogging", "cloudtrail:DeleteTrail",
                 "organizations:LeaveOrganization"],
      "Resource": "*"
    }
  ]
}
```

La lista de `NotAction` está abreviada: la documentación de AWS ofrece la lista completa de servicios globales que deben quedar fuera de la restricción de regiones.

Las **RCP** (*resource control policies*, 2024) son la otra mitad: limitan lo que se puede hacer **sobre los recursos** de tus cuentas, sin importar quién lo pida, aunque sea un principal de fuera de la organiza-

ción. Son la pieza clave de un **perímetro de datos**. Esta RCP impide que nadie ajeno a la organización acceda a los buckets, salvo los servicios de AWS que actúan en tu nombre:

```json
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "PerimetroDeDatosS3",
    "Effect": "Deny",
    "Principal": "*",
    "Action": "s3:*",
    "Resource": "*",
    "Condition": {
      "StringNotEqualsIfExists": {"aws:PrincipalOrgID": "o-a1b2c3d4e5"},
      "BoolIfExists": {"aws:PrincipalIsAWSService": "false"}
    }
  }]
}
```

Con las RCP ya tienes los tres «techos» de AWS. Se confunden a menudo, sobre todo las SCP con los permissions boundaries, así que conviene verlos frente a frente:

	SCP	RCP	Permissions boundary
Dónde se define	En Organizations: raíz, OU o cuenta	En Organizations: raíz, OU o cuenta	En IAM: un usuario o un rol concreto
A quién limita	A los principals de las cuentas, incluido su root	A los recursos de las cuentas, los use quien los use	Solo a esa identidad
¿Afecta a identidades externas?	✘ No	✔ Sí	✘ No
¿Concede permisos?	✘ No	✘ No	✘ No
Uso típico	Guardarraíles: regiones, servicios prohibidos, auditoría intocable	Perímetro de datos	Delegar la creación de roles
Servicios	Todos	Un subconjunto (al lanzarse: S3, STS, KMS, SQS y Secrets Manager)	Todos
Excepciones	Cuenta de administración y roles vinculados a servicios	Ídem	—
Por defecto	`FullAWSAccess`	`RCPFullAWSAccess`	Sin boundary, sin techo



AWS CONTROL TOWER

AWS Control Tower automatiza una **landing zone** multicuenta como la de la figura: crea las cuentas de logs y auditoría, aplica **controles** preventivos (políticas de Organizations como las SCP), detectivos (reglas de AWS Config) y proactivos (hooks de CloudFormation), y ofrece una **Account Factory** para crear cuentas ya configuradas. Organizations admite además otros tipos de políticas (de etiquetas, de copias de seguridad, declarativas…). El diseño de las OU lo verás en el capítulo de Arquitecturas de referencia.

3.9 Depurar un AccessDenied paso a paso

Tarde o temprano verás un mensaje como este. No lo resuelvas a ciegas añadiendo permisos: léelo.

```text
An error occurred (AccessDenied) when calling the GetObject operation:
User: arn:aws:sts::111122223333:assumed-role/app-web/i-0a1b2c3d4e5f67890
is not authorized to perform: s3:GetObject on resource:
"arn:aws:s3:::facturas/2026/enero.pdf" because no identity-based policy
allows the s3:GetObject action
```

1. **Lee el mensaje entero**: quién (aquí, una sesión del rol `app-web` lanzada por una instancia), qué acción y sobre qué recurso. Muchos servicios añaden **qué tipo de política** lo ha impedido.
2. **Confirma la identidad** con `aws sts get-caller-identity`. Un perfil, un rol o una cuenta equivocados son la causa más común.
3. **Busca el evento en CloudTrail** (`errorCode` = `AccessDenied` o `UnauthorizedOperation`) para ver la identidad, la acción y los recursos exactos. Ojo: las lecturas de objetos de S3 son *eventos de datos* y solo se registran si los has activado.
4. **Recorre las capas** del diagrama de evaluación: Deny explícito → SCP/RCP → política de recurso → identidad → boundary → sesión. Entre cuentas, revisa los dos lados.
5. **Reproduce y valida**: el simulador de políticas comprueba qué permitiría una política; IAM Access Analyzer valida su sintaxis y buenas prácticas.

Si el mensaje dice…	Mira aquí
«…because no identity-based policy allows the … action»	Falta un Allow en las políticas del usuario, de sus grupos o del rol
«…with an explicit deny in an identity-based policy»	Un Deny en esas mismas políticas
«…because no resource-based policy allows…»	La política del recurso (muy típico entre cuentas)
«…with an explicit deny in a service control policy»	Una SCP de la organización: habla con el equipo de plataforma
«…with an explicit deny in a resource control policy»	Una RCP: el perímetro de datos de la organización
«…because no permissions boundary allows…»	El permissions boundary de la identidad
«…because no session policy allows…»	La session policy usada al asumir el rol o federarse

EL EJEMPLO, RESUELTO

El mensaje lo dice casi todo: quien pide es la sesión del rol `app-web` (la abrió la instancia `i-0a1b…`) y **ninguna política de identidad** permite `s3:GetObject` sobre ese objeto. Abres el rol y ves que solo permite `arn:aws:s3:::fotos/*`. Si la aplicación debe leer facturas, añades `arn:aws:s3:::facturas/*` a su política; si no, el fallo está en el código, que pide el bucket equivocado. Si el mensaje hubiera dicho «explicit deny in a service control policy», ningún cambio en el rol lo arreglaría.

```bash
aws sts get-caller-identity
aws iam simulate-principal-policy \
  --policy-source-arn arn:aws:iam::111122223333:role/app-web \
  --action-names s3:GetObject --resource-arns arn:aws:s3:::facturas/2026/enero.pdf
aws sts decode-authorization-message --encoded-message <mensaje>   # p. ej., errores de EC2
aws accessanalyzer validate-policy --policy-type IDENTITY_POLICY \
  --policy-document file://politica.json
```

IAM Access Analyzer	Qué hace
Validación de políticas	Antes de desplegar, devuelve errores, avisos de seguridad y sugerencias; sus comprobaciones personalizadas responden preguntas como «¿esta política concede acceso público?».
Acceso externo	Detecta recursos (buckets, roles, claves KMS, funciones Lambda, colas, secretos…) accesibles desde fuera de tu cuenta u organización.
Acceso sin usar (de pago)	Señala roles, access keys, contraseñas y permisos que no se usan, para retirarlos.
Generación de políticas	Propone una política de mínimo privilegio a partir de las acciones que un rol ha usado realmente según CloudTrail.

3.10 Mínimo privilegio en la práctica

El principio de **mínimo privilegio** dice que cada identidad debe tener solo los permisos que necesita, y nada más. Nadie acierta a la primera, así que se aplica como un ciclo:

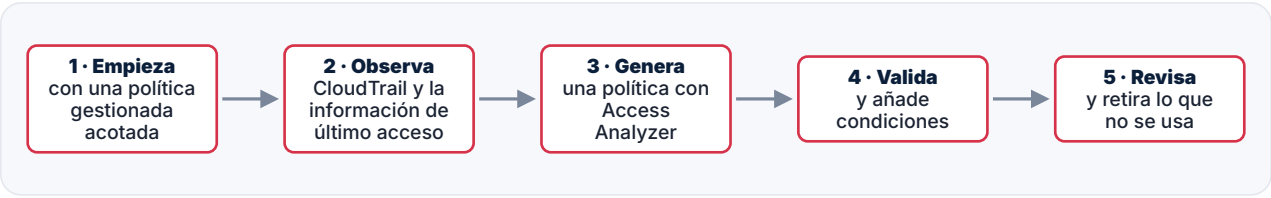


Figura 21 · El ciclo del mínimo privilegio: empezar amplio pero acotado, observar el uso real y recortar.

Para revisar periódicamente tienes el **informe de credenciales**, un CSV con el estado de contraseñas, MFA y access keys de cada usuario que se obtiene con `aws iam generate-credential-report`; la **información de último acceso**, que muestra qué servicios (y en algunos casos qué acciones) ha usado cada identidad y cuándo; y los hallazgos de acceso sin usar de Access Analyzer.

Haz

- Personas con Identity Center; cargas de trabajo con roles.
- Políticas gestionadas por el cliente, acotadas a recursos concretos.
- Condiciones (MFA, organización, red, etiquetas) donde aporten.
- Guardarraíles con SCP/RCP y boundaries para delegar.
- Revisiones periódicas con Access Analyzer.

Evita

- `"Action": "*"` sobre `"Resource": "*"` fuera de administradores.
- Access keys de largo plazo y usuarios compartidos.
- `Allow` con `NotAction`.
- `Principal: "*"` sin condiciones en políticas de recurso.
- Usar la cuenta de administración para cargas de trabajo.

3.11 Ejercicios resueltos

Antes de leer la solución, decide tú: ¿se permite la petición? Recuerda las tres reglas y el diagrama de evaluación.

EJERCICIO 1 Laura ejecuta `aws s3 ls s3://fotos`. Su única política es esta.

```json
{
  "Effect": "Allow",
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3:::fotos/*"
}
```

DENEGADO

Listar es `s3:ListBucket` y se autoriza contra el ARN del **bucket** (`arn:aws:s3:::fotos`), que esta sentencia no cubre: deny implícito. Puede descargar objetos si conoce su nombre, pero no listarlos.

EJERCICIO 2 Ana (grupos Desarrollo y Seguridad-prod) intenta terminar una instancia con la etiqueta `Entorno=prod`.

JSON
```
[{"Effect": "Allow", "Action": "ec2:*",
  "Resource": "*"},
 {"Effect": "Deny",
  "Action": "ec2:TerminateInstances",
  "Resource": "*",
  "Condition": {"StringEquals":
    {"aws:ResourceTag/Entorno": "prod"}}}]
```

DENEGADO

La primera sentencia (grupo Desarrollo) lo permite, pero la segunda (grupo Seguridad-prod) lo deniega explícitamente y el Deny gana. Con una instancia `Entorno=dev`, la condición no se cumple, el Deny no aplica y la acción se **permite**.

EJERCICIO 3 Un usuario lanza desde casa (IP `198.51.100.7`) un `aws s3 cp` a un bucket.

JSON
```
{
  "Effect": "Allow",
  "Action": "s3:*",
  "Resource": "*",
  "Condition": {"IpAddress":
    {"aws:SourceIp": "203.0.113.0/24"}}
}
```

DENEGADO

La condición no se cumple, así que la sentencia no aplica y no queda ningún Allow: deny implícito. Ojo: si la petición llegara a S3 por un VPC endpoint, `aws:SourceIp` tampoco serviría: esa clave no está presente en las peticiones que llegan por un endpoint.

EJERCICIO 4 El rol `analitica` (cuenta A) lee un objeto de `datos-b` (cuenta B), cuya política de bucket no menciona a la cuenta A.

JSON
```
{
  "Effect": "Allow",
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3:::datos-b/*"
}
```

DENEGADO

Es una política de identidad en la cuenta A: entre cuentas hace falta **también** un Allow en la política del bucket de B. Soluciones: añadirlo en B o hacer que A asuma un rol de B.

EJERCICIO 5 Un administrador con `AdministratorAccess` lanza una instancia en `us-east-1`. Su cuenta está en la OU Workloads, con la SCP de regiones de este capítulo.

JSON
```
{
  "Effect": "Deny",
  "NotAction": ["iam:*", "sts:*", "..."],
  "Resource": "*",
  "Condition": {"StringNotEquals":
    {"aws:RequestedRegion":
      ["eu-west-1", "eu-south-2"]}}
}
```

DENEGADO

`ec2:RunInstances` no está en `NotAction` y la región no es de la UE: la SCP lo deniega explícitamente. Da igual que sea administrador, e incluso el usuario root de la cuenta miembro: solo la cuenta de administración queda fuera del alcance de las SCP.

EJERCICIO 6 Pablo no tiene ninguna política de identidad y descarga `informes/q3.pdf`. La política del bucket `informes`, de su misma cuenta, dice:

JSON
```
{
  "Effect": "Allow",
  "Principal": {"AWS":
  "arn:aws:iam::111122223333:user/pablo"},
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3:::informes/*"
}
```

PERMITIDO

En la misma cuenta, el Allow de una política de recurso que nombra a Pablo basta, aunque él no tenga ninguna política propia (siempre que no haya un Deny explícito ni lo impida una SCP o una RCP). Si Pablo fuera de otra cuenta, necesitaría además un Allow en su política de identidad.

RESUMEN DEL CAPÍTULO

- Tres reglas: todo empieza denegado; un Allow explícito lo abre; un Deny explícito gana siempre.
- Una sentencia combina `Effect`, `Action`, `Resource` y, opcionalmente, `Condition`; las políticas de recurso añaden `Principal`. Para leerla, conviértela en una frase. Usa siempre `"Version": "2012-10-17"`.
- En las condiciones, claves distintas se combinan con Y y los valores de una clave, con O. `NotAction` con `Allow` es peligroso.
- Las políticas de identidad (en quien actúa) y de recurso (en lo que se toca) conceden; los boundaries, las SCP, las RCP y las session policies solo limitan. Los permisos efectivos son la intersección.
- En la misma cuenta basta un Allow (de identidad o de recurso, salvo trust policies y claves KMS); entre cuentas hacen falta los dos.
- Las condiciones globales (`aws:PrincipalOrgID`, `aws:SourceIp`, `aws:MultiFactorAuthPresent` ...) y ABAC con etiquetas dan precisión sin multiplicar políticas.
- Tres techos: las SCP limitan a los principals de las cuentas; las RCP, a sus recursos; un permissions boundary, a una sola identidad, y sirve para delegar la creación de roles sin escalada de privilegios. Ni las SCP ni las RCP afectan a la cuenta de administración.
- Ante un AccessDenied: lee el mensaje, confirma la identidad, busca en CloudTrail, recorre las capas y valida con el simulador y Access Analyzer.

Preguntas de repaso

1. **Un usuario sin ninguna política de identidad está en una cuenta cuya SCP permite `s3:*`. ¿Puede leer un bucket de su cuenta?**
   Respuesta: No, salvo que la política del bucket le conceda acceso a él directamente. Las SCP no conceden permisos: solo limitan.

2. **Un rol tiene una política que permite `s3:*` y `sqs:*`, y un boundary que permite `s3:*` y `dynamodb:*`. ¿Puede enviar un mensaje a SQS?**
   Respuesta: No: los permisos efectivos son la intersección (solo S3). El boundary no concede DynamoDB, porque la política del rol tampoco lo permite.

3. **La cuenta A necesita leer un bucket de la cuenta B con una política de recurso. ¿Qué dos permisos hacen falta?**
   Respuesta: Un Allow en la política de identidad del principal de A y un Allow para ese principal en la política del bucket de B (y que las SCP de A y las RCP de B no lo impidan).

4. **Quieres que los desarrolladores creen roles para sus Lambdas sin poder convertirse en administradores. ¿Qué mecanismo usas?**
   Respuesta: Un permissions boundary obligatorio (condición `iam:PermissionsBoundary` al crear roles), `iam:PassRole` limitado a su ruta y un Deny que impida quitar o editar el boundary.

5. **Quieres impedir que cualquier identidad ajena a tu organización acceda a tus buckets, aunque una política de bucket lo permita por error. ¿SCP o RCP?**
   Respuesta: Una RCP: limita los recursos de tus cuentas sea quien sea el principal. Las SCP solo afectan a los principals de tus propias cuentas.

6. **Recibes «...with an explicit deny in a service control policy». ¿Añadir permisos al rol lo arregla?**
   Respuesta: No. El bloqueo está en una SCP de Organizations; ningún permiso en la cuenta puede anular un Deny explícito. Hay que cambiar la SCP o mover la cuenta.

CAPÍTULO 4

Redes: VPC y conectividad

Construye tu propia red privada en AWS: direcciones, subredes, rutas, salida a Internet, cortafuegos y conexiones con otras redes y con tu centro de datos.

EN ESTE CAPÍTULO APRENDERÁS

1. Calcular rangos CIDR y planificar direcciones sin solapamientos
2. Diseñar subredes públicas y privadas repartidas en varias AZ
3. Entender tablas de rutas, Internet Gateway y NAT Gateway
4. Combinar Security Groups y Network ACL con criterio
5. Acceder en privado a servicios de AWS con VPC endpoints
6. Conectar VPC entre sí y con tu red local

SERVICIOS PROTAGONISTAS

Amazon VPC · NAT Gateway · VPC endpoints · AWS PrivateLink · AWS Transit Gateway · Amazon VPC Lattice · AWS Site-to-Site VPN · AWS Direct Connect · Route 53 Resolver · VPC Flow Logs

4.1 Direccionamiento IP y CIDR en cinco minutos

Casi todo lo que despliegas en AWS —una instancia EC2, una base de datos RDS, una tarea de ECS, un balanceador o una función Lambda conectada a tu red— recibe una dirección IP dentro de una red que diseñas tú. Antes de construirla, repasemos cómo se escriben y se reparten esas direcciones.

Una dirección **IPv4** tiene 32 bits que se escriben como cuatro números de 0 a 255 (octetos) separados por puntos, como `10.0.1.25`. La notación **CIDR** (*Classless Inter-Domain Routing*) describe un bloque de direcciones con una dirección base y un **prefijo**: en `10.0.0.0/16`, los primeros 16 bits son fijos e identifican la red, y los 16 restantes quedan libres para numerar equipos. Cuantos más bits fijos, más pequeño es el bloque: un /*n* contiene 2^{32-n} direcciones. Para redes privadas se usan los rangos de la RFC 1918, que no se enrutan en Internet: `10.0.0.0/8`, `172.16.0.0/12` y `192.168.0.0/16`.

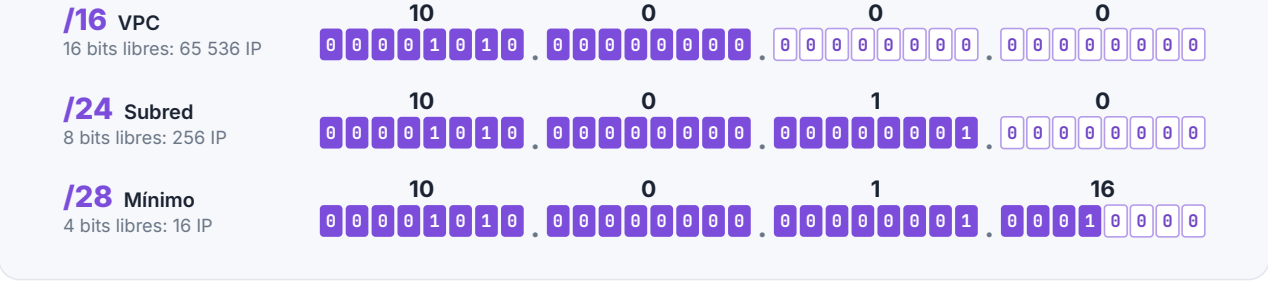


Figura 22 · Cada bit del prefijo queda fijo (en color) y los restantes numeran direcciones (en blanco). Del /16 al /28, el bloque pasa de 65 536 a 16 direcciones.

Prefijo	Direcciones	Utilizables en AWS	Uso típico
/16	65 536	65 531	VPC grande: es el máximo que admite AWS
/20	4096	4091	Subredes de la VPC por defecto; subredes para contenedores
/24	256	251	La subred «clásica»: fácil de leer y de calcular
/26	64	59	Subred pequeña (por ejemplo, para bases de datos)
/28	16	11	El mínimo permitido: subredes de tránsito o para *endpoints*

Las cinco direcciones reservadas

En cada subred, AWS reserva las cuatro primeras direcciones y la última. Para la subred `10.0.0.0/24`:

.0	**.1**	**.2**	**.3**	**.255**
Dirección de red	Router de la VPC	DNS de Amazon	Uso futuro	Difusión

La última se reserva aunque la VPC no admite difusión (*broadcast*). Por eso una /24 ofrece 251 direcciones utilizables y una /28, solo 11. Y recuerda que balanceadores, NAT Gateways, endpoints o tareas de Fargate crean interfaces de red (ENI) en tus subredes: cada una consume una dirección.

> ★ **CLAVE PARA LA CERTIFICACIÓN**
>
> Las cinco IP reservadas son un clásico. Si te piden «al menos 30 instancias en una subred», una /27 no basta (32 − 5 = 27 utilizables): necesitas una /26.

4.2 Amazon VPC: tu red privada en la nube

Una **VPC** (*Virtual Private Cloud*) es una red virtual aislada lógicamente del resto de clientes de AWS, en la que decides el rango de direcciones, las subredes, las rutas y los cortafuegos. Es un recurso **regional**: abarca todas las zonas de disponibilidad (AZ) de la región, mientras que cada subred vive en una sola AZ.

FICHA DEL SERVICIO

Amazon VPC

Red virtual privada y aislada dentro de una región

ÁMBITO	Regional; cada subred pertenece a una única AZ
TAMAÑO	IPv4 de /16 a /28, ampliable con bloques secundarios; IPv6 opcional
FACTURACIÓN	La VPC, las subredes, las tablas de rutas, los Security Groups y las NACL son gratis. Se paga por NAT Gateway, endpoints de interfaz, VPN, Transit Gateway, IPv4 públicas y transferencia de datos
CUOTAS	5 VPC por región y 5 bloques IPv4 por VPC por defecto (ambas ajustables)

≈ **ANALOGÍA**

Una VPC es como una planta de oficinas que alquilas en exclusiva: tú numeras los despachos (direcciones IP), levantas tabiques para separar zonas (subredes), decides dónde está la puerta a la calle (Internet Gateway), pones un control en la entrada de cada zona (NACL) y un vigilante en la puerta de cada despacho (Security Group).

VPC por defecto frente a VPC personalizada

Cada región trae una **VPC por defecto** lista para usar: CIDR `172.31.0.0/16`, una subred /20 en cada AZ y un Internet Gateway con ruta `0.0.0.0/0`, así que todas sus subredes son públicas y las instancias reciben IPv4 pública. Es perfecta para pruebas, no para producción. Para cualquier entorno serio diseñas una **VPC personalizada**: CIDR coordinado con el resto de tus redes, subredes públicas y privadas con tablas de rutas explícitas y salida a Internet controlada (NAT, endpoints).

Elegir el CIDR sin solapamientos

El CIDR principal de una VPC no se puede cambiar. Elígelo pensando en todas las redes con las que algún día tendrás que hablar: otras VPC, tu centro de datos, las oficinas o redes de socios. Si dos redes conectadas comparten direcciones, el router no sabe adónde enviar los paquetes; AWS ni siquiera permite un peering entre VPC con CIDR solapados.

Lo habitual es un **plan de direccionamiento** central: por ejemplo, `10.0.0.0/16` para producción, `10.1.0.0/16` para preproducción y `10.2.0.0/16` para desarrollo, evitando el `192.168.0.0/16` de las oficinas. En organizaciones grandes, **Amazon VPC IP Address Manager** (IPAM) automatiza el reparto entre cuentas y regiones.

✓ **CONSEJO**

Renumerar una red en producción es un proyecto doloroso. Ante la duda, sé generoso: las direcciones privadas no cuestan nada, y un /16 por VPC es una elección habitual.

Bloques secundarios e IPv6

Si te quedas corto, puedes **asociar bloques CIDR secundarios** a la VPC y crear subredes en ellos. No pueden solaparse y hay combinaciones restringidas: si el bloque principal pertenece a `10.0.0.0/8`, no puedes añadir bloques de los otros rangos RFC 1918, pero sí del rango `100.64.0.0/10`, muy usado para dar direcciones a los pods de Kubernetes (lo verás en el capítulo «Kubernetes y Amazon EKS»).

La VPC también puede ser **dual stack**: le asocias un bloque IPv6 /56 proporcionado por Amazon (o tus propias direcciones) y cada subred recibe un /64. Las IPv6 que asigna Amazon son públicas (globalmente únicas): lo que decide si son accesibles desde Internet son las rutas y los cortafuegos, y no hace falta NAT para salir. Además, no tienen el coste por hora de las IPv4 públicas. También puedes crear subredes solo IPv6.

4.3 Subredes y zonas de disponibilidad

Una **subred** es un trozo del CIDR de la VPC que vive en una única AZ; al lanzar una instancia eliges la subred y, con ella, la AZ. Como una subred no puede abarcar varias AZ, la alta disponibilidad se consigue **repitiendo** cada capa en dos o, mejor, tres AZ. Para alinear subredes entre cuentas, guíate por el AZ ID (`euw1-az1`) y no por el nombre, que cambia de una cuenta a otra (capítulo de Fundamentos).

El CIDR de una subred debe estar contenido en el de la VPC (entre /16 y /28) y no puede solaparse con el de otras subredes. El atributo *Auto-assign public IPv4 address* decide si las instancias nuevas reciben una IP pública. La Figura 23 muestra un plan típico de tres capas en tres AZ, que reutilizaremos al final del capítulo.



Figura 23 · Plan de direcciones para una VPC 10.0.0.0/16 con tres capas en tres AZ. La capa de aplicación recibe bloques grandes porque los contenedores consumen muchas IP.

4.4 Tablas de rutas

Cada subred está asociada a exactamente una **tabla de rutas** (*route table*): una lista de reglas que, para cada destino (un rango CIDR), indica el siguiente salto o **target** (un gateway, un peering, una interfaz de red…). Una misma tabla puede servir a varias subredes. Toda VPC tiene una **tabla principal** (*main*), que se aplica a las subredes sin asociación explícita.

> **BUENA PRÁCTICA**
>
> Deja la tabla principal como privada (solo la ruta local) y asocia explícitamente cada subred a su tabla. Así, una subred nueva que olvides asociar nunca quedará expuesta a Internet por accidente.

Todas las tablas incluyen la **ruta local** (el CIDR de la VPC hacia `local`), que permite que cualquier subred alcance a cualquier otra dentro de la VPC. No se puede borrar: dentro de una VPC, el aislamiento entre subredes no lo dan las rutas, sino los Security Groups y las NACL. Esta es la tabla de una subred privada típica:

Destino	Target	Para qué sirve
`10.0.0.0/16`	`local`	Tráfico dentro de la VPC (siempre presente)
`10.20.0.0/16`	`pcx-0a1b2c3d` (peering)	Llegar a la VPC de servicios compartidos
`192.168.0.0/16`	`tgw-0f1e2d3c` (Transit Gateway)	Llegar al centro de datos
`pl-1a2b3c4d` (*prefix list* de S3)	`vpce-09a8b7c6` (gateway endpoint)	Hablar con S3 sin pasar por el NAT
`0.0.0.0/0`	`nat-0123abcd` (NAT Gateway)	Todo lo demás: salida a Internet

Cuando varias rutas coinciden con un destino, gana la **más específica**, la de prefijo más largo (*longest prefix match*). Con la tabla anterior:

- `10.0.3.7` coincide con `10.0.0.0/16` y con `0.0.0.0/0`; gana el /16: tráfico local.
- `10.20.5.9` coincide con `10.20.0.0/16` y con `0.0.0.0/0`; viaja por el peering.
- Una IP de S3 figura en la prefix list, cuyos bloques son más específicos que `/0`: va por el endpoint.
- `8.8.8.8` solo coincide con `0.0.0.0/0`: sale por el NAT Gateway.

Otros targets habituales: Internet Gateway (`igw-`), Egress-only IGW (`eigw-`), *virtual private gateway* de una VPN (`vgw-`) o una interfaz de red (`eni-`), como la de un *appliance* de seguridad.

4.5 Internet Gateway, IP públicas y NAT

Internet Gateway y subredes públicas

El **Internet Gateway** (IGW) conecta la VPC con Internet. Es gestionado, redundante y escala solo: no es un cuello de botella ni un punto único de fallo, y no tiene coste propio. Cada VPC admite uno como máximo. Para IPv4 traduce 1:1 la IP privada de la instancia a su IP pública: el sistema operativo solo ve la privada. Y si quieres un freno de emergencia, **VPC Block Public Access** (2024) bloquea por cuenta y región el tráfico de los Internet Gateways, con exclusiones para las VPC o subredes que deban ser públicas.

¿Qué hace **pública** a una subred? Solo una cosa: que su tabla de rutas tenga una ruta `0.0.0.0/0` (o `::/0`) hacia un IGW. No existe ninguna casilla «subred pública», y el nombre que le pongas no cambia nada. Una subred **privada** es, sencillamente, la que no tiene esa ruta: nada de lo que vive en ella es alcanzable directamente desde Internet, aunque tuviera IP pública. Para que una instancia sea accesible desde Internet deben cumplirse cuatro condiciones:

1. La tabla de rutas de su subred tiene una ruta hacia el Internet Gateway.
2. La instancia tiene una IPv4 pública o una Elastic IP (o una dirección IPv6).
3. Su Security Group permite el tráfico entrante (por ejemplo, TCP 443).
4. La NACL de la subred permite la entrada y también la salida de las respuestas.

IP pública frente a Elastic IP

Una **IPv4 pública automática** se asigna al lanzar la instancia desde el *pool* de Amazon y se pierde al detenerla o terminarla: tras un *stop/start* tendrá otra. Una **Elastic IP** es una dirección estática que reservas en tu cuenta y región; puedes asociarla a una instancia o interfaz, moverla a otra en segundos, y es tuya hasta que la liberes.

COSTES

Desde el 1 de febrero de 2024, AWS cobra **todas** las IPv4 públicas por hora, estén en uso o no: las asignadas automáticamente a EC2, las Elastic IP y también las de NAT Gateways, balanceadores y otros servicios. La tarifa es de 0,005 USD por hora y dirección (unos 3,6 USD al mes): poco por IP, mucho multiplicado por cientos. Localiza las tuyas con *Public IP Insights* (en VPC IPAM), libera las Elastic IP que no uses y valora IPv6.

NAT Gateway: salida a Internet para subredes privadas

Las instancias de una subred privada no tienen IP pública, pero a menudo necesitan salir a Internet (parches, API externas). El **NAT Gateway** lo resuelve: se crea en una subred pública con una Elastic IP, y la tabla de rutas de las subredes privadas envía `0.0.0.0/0` hacia él. Traduce las conexiones salientes a su IP pública y deja volver las respuestas, pero nunca acepta conexiones iniciadas desde Internet.

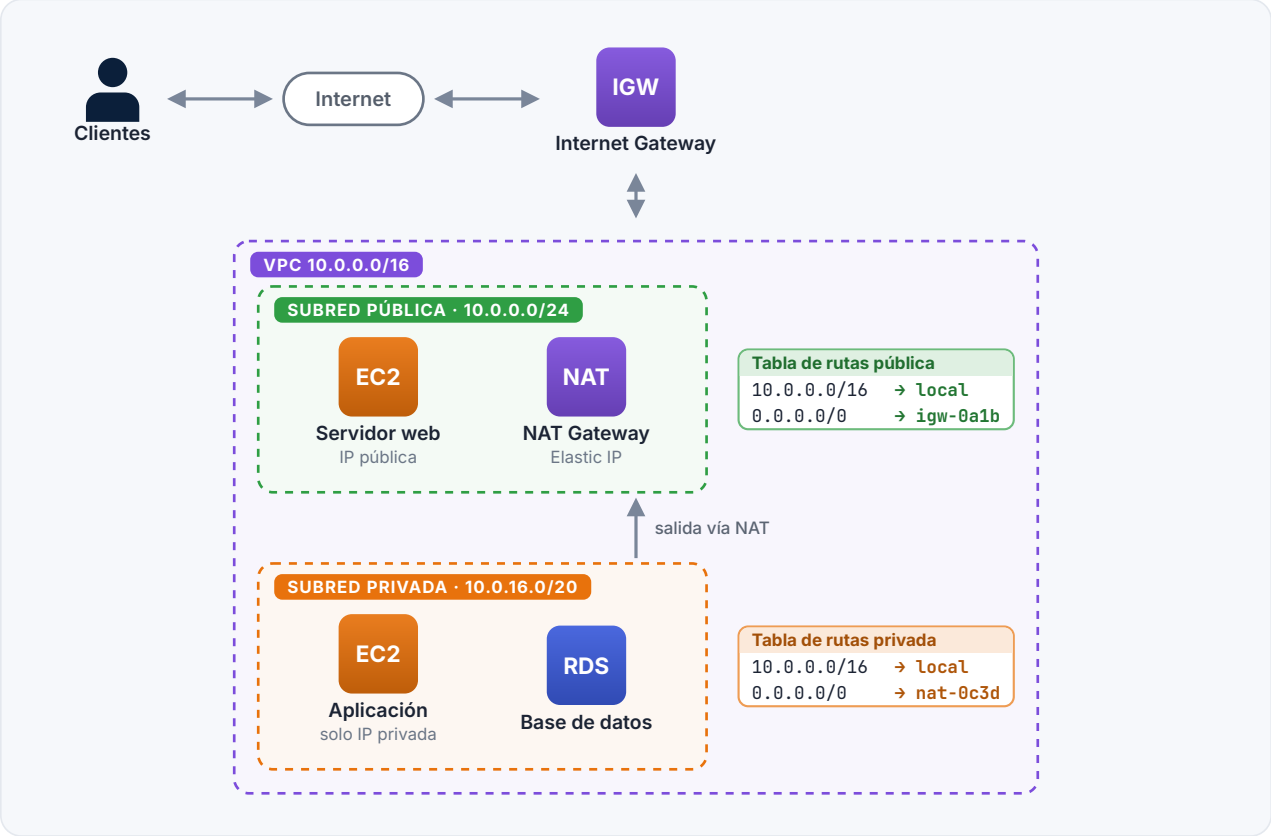


Figura 24 · Subred pública y subred privada. El Internet Gateway, conectado a la VPC, da entrada y salida a la subred pública; la privada solo puede salir, a través del NAT Gateway.

Con estas piezas, cada subred encaja en uno de tres tipos según adónde envía su ruta por defecto (`0.0.0.0/0`, «todo lo demás»):

Tipo de subred	Ruta por defecto hacia	¿Entrada desde Internet?	¿Salida a Internet?	Qué suele alojar
Pública	Internet Gateway	Sí, si el recurso tiene IP pública y los cortafuegos lo permiten	Sí	Balanceadores, NAT Gateways
Privada	NAT Gateway	No	Solo las conexiones que inicia ella	Aplicaciones, contenedores, Lambda en la VPC
Aislada	Ninguna: solo la ruta local	No	No	Bases de datos, cachés

Antes del NAT Gateway se usaban **NAT instances**: instancias EC2 configuradas para hacer NAT (con el *source/destination check* desactivado) que tú administras, parcheas y haces redundantes. Su ancho de banda es el del tipo de instancia y admiten Security Groups; el NAT Gateway, en cambio, está gestionado, es redundante dentro de su AZ, escala solo de 5 a 100 Gbps y no admite Security Groups (solo lo filtra la NACL de su subred). Hoy las NAT instances solo tienen sentido en laboratorios o con presupuestos mínimos.

> **ALTA DISPONIBILIDAD: UN NAT POR AZ**
>
> Un NAT Gateway clásico es **zonal**: si su AZ cae, todas las subredes que lo usan se quedan sin salida. El patrón recomendado es un NAT Gateway por AZ y una tabla de rutas privada por AZ que apunte a su NAT local; de paso evitas pagar tráfico entre AZ. Desde finales de 2025 existe además un modo **regional**: un único NAT Gateway que se extiende automáticamente a las AZ donde tienes recursos y que no necesita subred pública.

> **COSTES**
>
> Cada NAT Gateway se paga por hora de existencia y por GB procesado, además de su IPv4 pública y de la transferencia a Internet. En us-east-1 ronda los 0,045 USD por hora y por GB (varía según la región): tres NAT suponen unos 100 USD al mes antes de mover un solo byte. Lo que más infla la factura es el tráfico hacia S3, DynamoDB o ECR que cruza el NAT sin necesidad: los **VPC endpoints** lo evitan.

Egress-only Internet Gateway

Con IPv6 no hace falta NAT para salir a Internet, porque las direcciones que asigna Amazon ya son públicas. Para que las instancias puedan iniciar conexiones IPv6 hacia Internet sin aceptar conexiones entrantes se usa un **Egress-only Internet Gateway**, con la ruta `::/0 → eigw-…`. Como el NAT, recuerda las conexiones que salen y solo deja volver sus respuestas (es *stateful*), pero no traduce direcciones y no tiene coste por hora. Existen también NAT Gateways **privados**, sin IP pública, para conectar redes con CIDR solapados.

4.6 Security Groups y Network ACL

AWS te da dos cortafuegos complementarios que actúan en puntos distintos del camino. La **Network ACL** (NACL) vigila la frontera de una subred: filtra todo lo que entra en ella o sale de ella. El **Security Group** (SG) va pegado a cada interfaz de red (ENI): la de una instancia, un balanceador, una base de datos RDS… Una petición que llega de Internet a una instancia supera los dos controles, y la diferencia decisiva aparece en la respuesta:

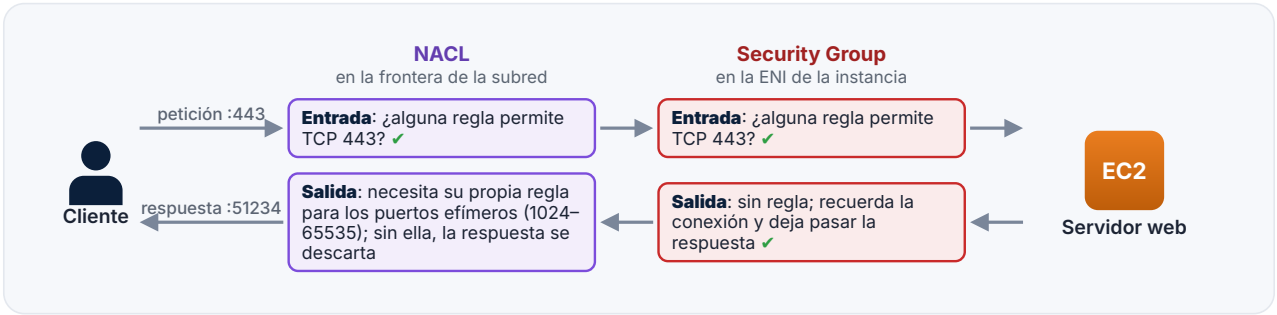


Figura 25 · Recorrido de una petición HTTPS y de su respuesta. La NACL, en la frontera de la subred, revisa cada sentido por separado; el Security Group recuerda la conexión y deja salir la respuesta sin necesidad de regla de salida.

Característica	Security Group	Network ACL
Se aplica a	Interfaces de red (ENI) de los recursos	Subredes completas
Estado	**Stateful**: la respuesta se permite sola	**Stateless**: hay que permitir ida y vuelta
Reglas	Solo *allow*	*Allow* y *deny*
Evaluación	Todas las reglas a la vez	Por número, de menor a mayor; decide la primera que coincide
Origen o destino	CIDR, prefix list u otro Security Group	Solo CIDR
Por defecto	Uno nuevo: sin entrada, toda la salida	La NACL por defecto lo permite todo; una nueva lo deniega todo
Asociación	Hasta 5 por ENI (ajustable)	Exactamente una por subred
Bloquear una IP concreta	✘	✔

Security Groups encadenados: una aplicación de tres capas

La gran ventaja de los SG es que una regla puede tener como origen **otro Security Group**: «permite el puerto 5432 a cualquier recurso que lleve `sg-app`». La regla sigue a los recursos aunque cambien de IP o pasen de dos a cien instancias.

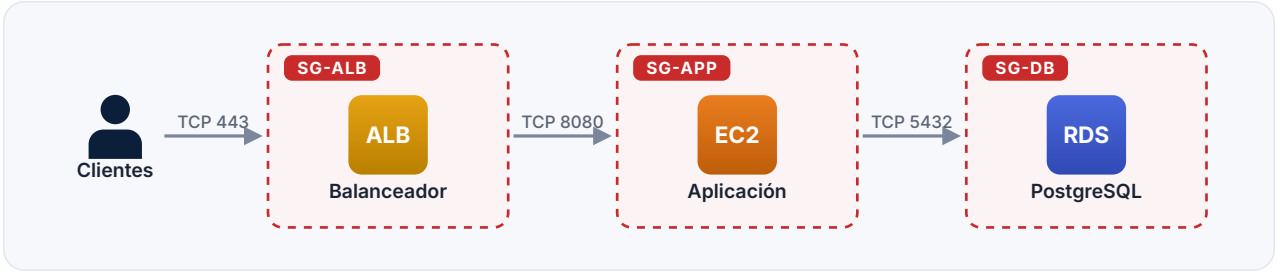


Figura 26 · Security Groups encadenados: cada capa solo acepta tráfico de la capa anterior, identificada por su SG y no por sus direcciones IP.

Security Group	Entrada	Salida
`sg-alb`	TCP 443 desde `0.0.0.0/0` y `::/0`	TCP 8080 hacia `sg-app`
`sg-app`	TCP 8080 desde `sg-alb`	TCP 5432 hacia `sg-db` ; TCP 443 hacia `0.0.0.0/0` (API, parches)
`sg-db`	TCP 5432 desde `sg-app`	Ninguna: las respuestas salen solas

```bash
aws ec2 authorize-security-group-ingress --group-id sg-0db1111aaaa2222bb \
  --protocol tcp --port 5432 --source-group sg-0app333cccc4444dd
```

Los cambios se aplican al instante. Puedes referenciar SG de una VPC emparejada de la misma región y, desde 2024, de VPC conectadas por Transit Gateway (en reglas de entrada). Cada SG admite por defecto 60 reglas de entrada y 60 de salida (ajustable).

NACL y puertos efímeros

Al ser stateless, una NACL evalúa por separado el paquete de ida y el de vuelta. La respuesta de un servidor se dirige al **puerto efímero** (temporal) que eligió el cliente al abrir la conexión: Linux usa por defecto el rango 32768–60999, Windows 49152–65535, y los balanceadores y el NAT Gateway 1024–65535. Por eso la NACL debe permitir 1024–65535 en el sentido de las respuestas. Ejemplo para una subred que solo aloja el balanceador:

Regla	Sentido	Protocolo y puertos	Origen / destino	Acción
90	Entrada	Todo	`203.0.113.0/24`	DENY: red maliciosa conocida
100	Entrada	TCP 443	`0.0.0.0/0`	ALLOW: clientes HTTPS
110	Entrada	TCP 1024–65535	`10.0.0.0/16`	ALLOW: respuestas de la capa de aplicación
*	Entrada	Todo	`0.0.0.0/0`	DENY implícito
100	Salida	TCP 1024–65535	`0.0.0.0/0`	ALLOW: respuestas a los clientes
110	Salida	TCP 8080	`10.0.0.0/16`	ALLOW: hacia la capa de aplicación
*	Salida	Todo	`0.0.0.0/0`	DENY implícito

CLAVE PARA LA CERTIFICACIÓN

SG: stateful, solo allow, en cada ENI. NACL: stateless, allow y deny, en la frontera de la subred, reglas numeradas. Si te piden bloquear una IP o un rango concreto, la respuesta es una NACL (o AWS WAF, si el tráfico es HTTP), nunca un Security Group.

BUENA PRÁCTICA

Haz de los SG tu control principal y deja las NACL casi abiertas, para bloqueos gruesos (rangos concretos, aislar subredes sensibles). Replicar toda la lógica en las NACL multiplica los errores con los puertos efímeros.

4.7 VPC endpoints y PrivateLink

Una instancia de una subred privada que llama a S3, SQS o Secrets Manager habla con endpoints públicos de AWS: sin más, necesita un NAT Gateway y paga su procesamiento. Los **VPC endpoints** permiten llegar a esos servicios por la red privada de AWS, sin Internet Gateway, sin NAT y sin IP pública. Hay dos tipos principales:

	Gateway endpoint	Interface endpoint (PrivateLink)
Servicios	Solo S3 y DynamoDB	La mayoría de servicios de AWS, de terceros y los tuyos propios
Funcionamiento	Una ruta en la tabla de rutas (prefix list → `vpce-`)	Una ENI con IP privada en cada subred elegida
DNS	Sin cambios: se usan los nombres públicos	DNS privado: el nombre del servicio resuelve a IP privadas
Accesible desde on-premises u otras VPC	✘	✔
Security Groups	✘	✔
Coste	**Gratis**	Por hora y AZ + por GB procesado

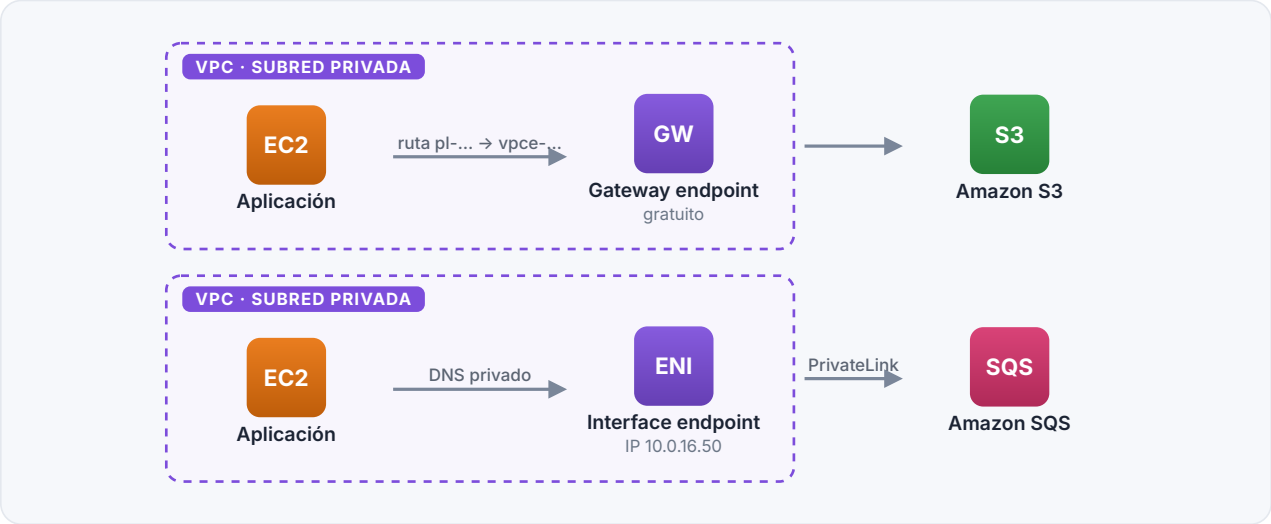


Figura 27 · Dos formas de llegar a servicios de AWS sin NAT: el gateway endpoint es una ruta en la tabla de rutas; el interface endpoint es una interfaz de red con IP privada.

S3 y, desde 2024, DynamoDB admiten también interface endpoints, útiles para llegar a ellos desde tu red local. El DNS privado de los interface endpoints requiere los atributos `enableDnsSupport` y `enableDnsHostnames` (sección de DNS).

Endpoint policies

Cada endpoint admite una **endpoint policy**: una política JSON que limita qué se puede hacer **a través de él**. No concede permisos (siguen haciendo falta los de IAM y los del recurso), pero evita, por ejemplo, que alguien use tu red para copiar datos a un bucket ajeno:

```json
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "SoloBucketsDeLaEmpresa",
    "Effect": "Allow",
    "Principal": "*",
    "Action": ["s3:GetObject", "s3:PutObject", "s3:ListBucket"],
    "Resource": ["arn:aws:s3:::empresa-datos", "arn:aws:s3:::empresa-datos/*"]
  }]
}
```

En sentido contrario, la **bucket policy** puede exigir que las peticiones lleguen por tu endpoint con la condición `aws:SourceVpce`. Para crear un gateway endpoint basta con indicar el servicio (`com.amazonaws.eu-west-1.s3`) y las tablas de rutas que deben recibir la ruta.

PrivateLink para tus propios servicios

Con **AWS PrivateLink** también puedes publicar un servicio propio: lo colocas detrás de un Network Load Balancer, creas un **endpoint service** y los consumidores crean un interface endpoint hacia él. La conexión es unidireccional (solo inicia el consumidor), no expone el resto de tu red y funciona aunque los CIDR se solapen. Un tercer tipo de endpoint, el de **Gateway Load Balancer**, intercala *appliances* de inspección (cortafuegos, IDS) en el camino del tráfico.

4.8 Conectar VPC: Peering, Transit Gateway y VPC Lattice

VPC Peering

Un **VPC Peering** une dos VPC de forma privada, como si fueran una sola red, aunque sean de cuentas o regiones distintas, y sin ningún gateway que dimensionar. Tras aceptar la conexión, añades en cada lado la ruta hacia el CIDR del otro (`10.1.0.0/16 → pcx-…`). Sus reglas son estrictas:

- Los CIDR de ambas VPC no pueden solaparse.

- **No es transitivo**: si A está emparejada con B, y B con C, A no llega a C a través de B.
- Tampoco hay *edge-to-edge routing*: A no puede usar el IGW, el NAT, la VPN ni el gateway endpoint de B.
- La malla crece deprisa: *n* VPC necesitan $n(n-1)/2$ peerings (10 VPC, 45 conexiones).
- No tiene coste por hora: pagas la transferencia de datos, gratuita dentro de la misma AZ y con la tarifa habitual entre AZ o entre regiones.



Figura 28 · El peering no es transitivo: cada pareja de VPC que deba comunicarse necesita su propia conexión.

Transit Gateway

Con decenas de VPC más conexiones a tu centro de datos, la malla de peerings se vuelve inmanejable. **AWS Transit Gateway** (TGW) es un router regional gestionado con topología *hub-and-spoke*: cada VPC, VPN o Direct Connect se conecta una sola vez al TGW (un **attachment**) y este enruta entre todos de forma transitiva.

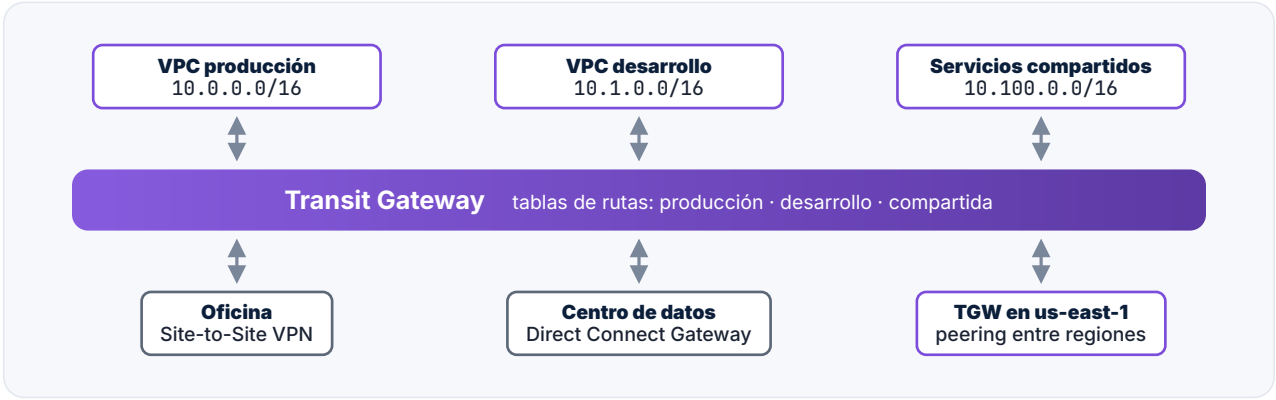


Figura 29 · Transit Gateway como hub: cada red se conecta una sola vez y las tablas de rutas del TGW deciden quién llega a quién.

FICHA DEL SERVICIO

AWS Transit Gateway

Router regional gestionado para interconectar VPC y redes locales

ÁMBITO	Regional; se comparte entre cuentas con AWS RAM y se une a TGW de otras regiones mediante peering
ATTACHMENTS	VPC, Site-to-Site VPN, Direct Connect Gateway, peering de TGW y Connect (SD-WAN)
SEGMENTACIÓN	Varias tablas de rutas: producción y desarrollo aisladas entre sí, ambas con acceso a servicios compartidos
FACTURACIÓN	Por hora de cada attachment + por GB procesado

Para redes globales gestionadas mediante políticas, **AWS Cloud WAN** lleva la misma idea a varias regiones desde una sola consola.

> **CLAVE PARA LA CERTIFICACIÓN**
>
> Peering: uno a uno, no transitivo, sin coste por hora. Transit Gateway: hub transitivo para muchas VPC y redes locales, con coste por attachment y por GB. Si el enunciado habla de «cientos de VPC» o de «simplificar la conectividad», la respuesta es Transit Gateway.

VPC Lattice

Amazon VPC Lattice ataca el problema desde la capa de aplicación: en lugar de unir redes, conecta **servicios**. Defines una *service network*, registras en ella tus servicios (instancias, contenedores, funciones Lambda…) y asocias las VPC consumidoras. Lattice se encarga del descubrimiento, del enrutamiento HTTP, HTTPS y gRPC, de la autorización con políticas de IAM y de la observabilidad, incluso entre cuentas y con CIDR solapados; desde finales de 2024 también da acceso a recursos TCP, como una base de datos RDS. Resumiendo: Peering y Transit Gateway conectan **redes** (y exigen CIDR sin solapar); PrivateLink expone **un servicio** concreto; Lattice gestiona la comunicación entre **muchos servicios**.

4.9 Conectividad híbrida

Conectar AWS con tu centro de datos u oficinas es el escenario **híbrido**. Dispones de tres herramientas, que a menudo se combinan:

Site-to-Site VPN — túneles IPsec cifrados sobre Internet entre tu router (**customer gateway**) y, en el lado de AWS, un **virtual private gateway** (VGW, asociado a una VPC) o un Transit Gateway. Cada conexión incluye dos túneles hacia endpoints distintos de AWS y admite rutas estáticas o BGP.
Client VPN — acceso remoto para personas, compatible con OpenVPN, con autenticación mediante Active Directory, SAML (tu proveedor de identidad) o certificados.
Direct Connect — un enlace físico privado entre tu red y AWS a través de una ubicación de Direct Connect, dedicado o alojado (*hosted*) mediante un socio. No va cifrado por defecto: usa MACsec en las conexiones dedicadas compatibles o una VPN por encima. Llega a tus VPC mediante *virtual interfaces* y un **Direct Connect Gateway**.

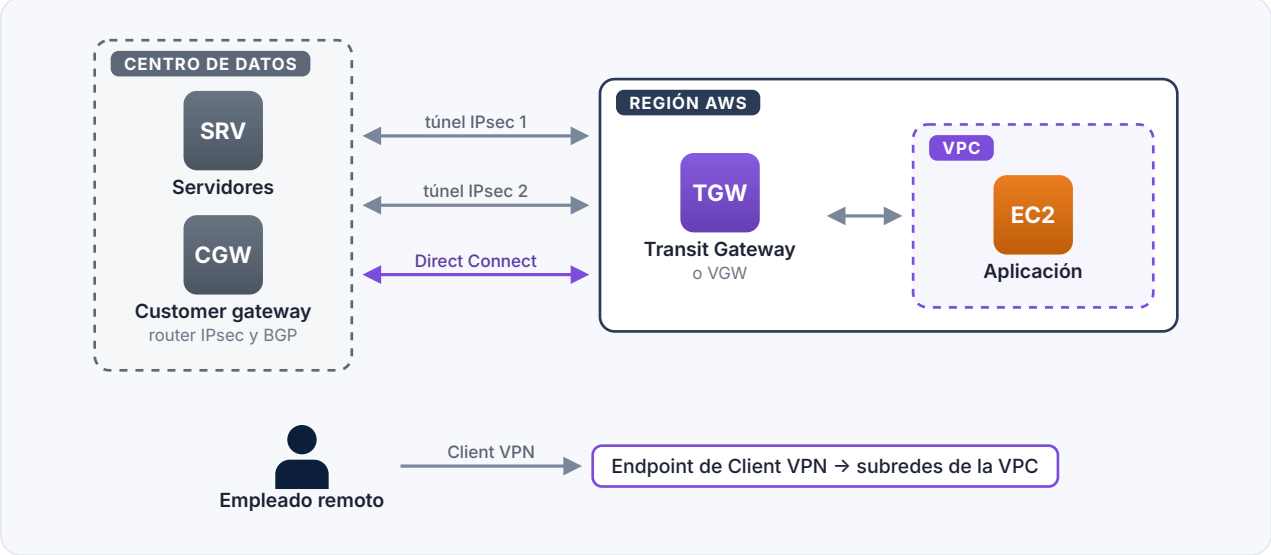


Figura 30 · Conectividad híbrida: dos túneles VPN sobre Internet, un enlace de Direct Connect y acceso de usuarios remotos con Client VPN.

	Site-to-Site VPN	Client VPN	Direct Connect
Para quién	Redes: centros de datos, oficinas	Personas: portátiles	Redes con mucho tráfico
Medio	IPsec sobre Internet	TLS (OpenVPN) sobre Internet	Enlace físico privado (sin cifrar por defecto)
Ancho de banda	Hasta 1,25 Gbps por túnel (5 Gbps en los túneles grandes de TGW y Cloud WAN)	Por usuario	Dedicado: de 1 a 400 Gbps; alojado: de 50 Mbps a 25 Gbps
Puesta en marcha	Minutos	Minutos	Semanas
Facturación	Por hora de conexión + datos	Por hora de asociación y de conexión	Por hora de puerto + GB salientes

BUENA PRÁCTICA

Un patrón clásico de resiliencia: Direct Connect como camino principal y una Site-to-Site VPN como respaldo. Para cargas críticas, AWS recomienda además dos conexiones de Direct Connect en ubicaciones distintas.

4.10 DNS dentro de la VPC

Cada VPC incluye un servidor DNS gestionado, el **Route 53 Resolver** (también llamado *Amazon DNS server*), en la dirección base del CIDR más dos (en `10.0.0.0/16`, la `10.0.0.2`) y en `169.254.169.253`. Resuelve nombres públicos de Internet, los nombres internos de EC2 (`ip-10-0-1-25.eu-west-1.compute.internal`) y tus **zonas privadas** de Route 53. Su comportamiento depende de dos atributos de la VPC: `enableDnsSupport` decide si el Resolver responde a las instancias (activado por defecto) y `enableDnsHostnames`, si las instancias con IP pública reciben un nombre DNS público (activado en la VPC por defecto, pero no en las que creas por CLI o con infraestructura como código). Ambos deben estar en `true` para usar zonas privadas de Route 53 y el DNS privado de los interface endpoints: es un olvido clásico, que se corrige con `aws ec2 modify-vpc-attribute`.

Para el **DNS híbrido**, el Resolver ofrece endpoints: uno **de entrada** (*inbound*), para que tus servidores DNS locales resuelvan nombres privados de AWS, y uno **de salida** (*outbound*) con **reglas de reenvío**, para que la VPC resuelva dominios corporativos como `corp.ejemplo.com` contra tus DNS internos. **Route 53 Resolver DNS Firewall** permite además bloquear consultas a dominios maliciosos. Las zonas, los tipos de registro y las políticas de enrutamiento de Route 53 se tratan en el capítulo «CloudFront, Route 53 y el borde de la red».

4.11 Visibilidad: VPC Flow Logs y Reachability Analyzer

VPC Flow Logs registra metadatos de las conexiones IP —quién habla con quién, por qué puertos, cuántos bytes y si se aceptó o se rechazó—, no el contenido de los paquetes (para eso está *Traffic Mirroring*). Se activa por VPC, subred o interfaz, envía los registros a CloudWatch Logs, S3 o Amazon Data Firehose y agrega cada flujo en ventanas de 1 o 10 minutos. Así es un registro en el formato por defecto (versión 2):



Figura 31 · Anatomía de un registro de VPC Flow Logs: una conexión HTTPS aceptada hacia una instancia. Los campos `start` y `end` son marcas de tiempo Unix; el protocolo 6 es TCP.

```bash
aws ec2 create-flow-logs --resource-type VPC --resource-ids vpc-0abc1234 \
  --traffic-type ALL --log-destination-type s3 --max-aggregation-interval 60 \
  --log-destination arn:aws:s3:::mis-flow-logs/vpc-prod/
```

Un diagnóstico clásico: para el mismo intento de conexión SSH aparecen estos dos registros (abreviados):

```text
... 198.51.100.7 10.0.1.25 51234 22 6 4 240 ... ACCEPT OK
... 10.0.1.25 198.51.100.7 22 51234 6 4 240 ... REJECT OK
```

La entrada se aceptó, pero la respuesta se rechazó. Un Security Group nunca bloquearía una respuesta, porque es stateful: el culpable es la NACL, que no permite la salida hacia los puertos efímeros del cliente. En el capítulo de Observabilidad verás cómo consultar estos registros con CloudWatch Logs Insights o Athena.

Reachability Analyzer responde a «¿puede A hablar con B?» analizando la **configuración** (rutas, SG, NACL, endpoints, peerings, Transit Gateway…) entre un origen y un destino, por ejemplo del Internet Gateway a una instancia por el puerto 443, y señala el componente que bloquea el camino. No envía paquetes reales, así que funciona antes incluso de desplegar la aplicación; se paga por análisis. Su hermano, **Network Access Analyzer**, busca accesos no deseados a gran escala («¿qué es accesible desde Internet?»).

4.12 Diseño de referencia: VPC Multi-AZ de tres capas

Juntemos todas las piezas. La Figura 32 muestra la VPC que encontrarás, con variaciones, en la mayoría de aplicaciones web en producción; usa el plan de direcciones de la Figura 23. Las decisiones de diseño, una a una:

- **Tres AZ y tres capas** (pública, aplicación y datos), replicadas en todas las AZ.
- **Tablas de rutas**: una pública (`0.0.0.0/0 → igw`), una privada por AZ (`0.0.0.0/0 → NAT de su AZ`) y una de datos sin salida a Internet (subredes aisladas).
- **Nada con IP pública salvo el balanceador y los NAT**; la administración, con Session Manager (AWS Systems Manager), sin bastiones ni SSH abierto.
- **Security Groups encadenados** (`sg-alb → sg-app → sg-db`), NACL casi abiertas, endpoints para no pagar NAT de más y Flow Logs desde el primer día.

> **CONSEJO**
>
> El asistente *VPC and more* de la consola crea en un minuto la VPC con subredes públicas y privadas en varias AZ, tablas de rutas, IGW, NAT Gateways y un gateway endpoint de S3. Úsalo para aprender; en producción, define la red como código (capítulo de Infraestructura como código y DevOps).

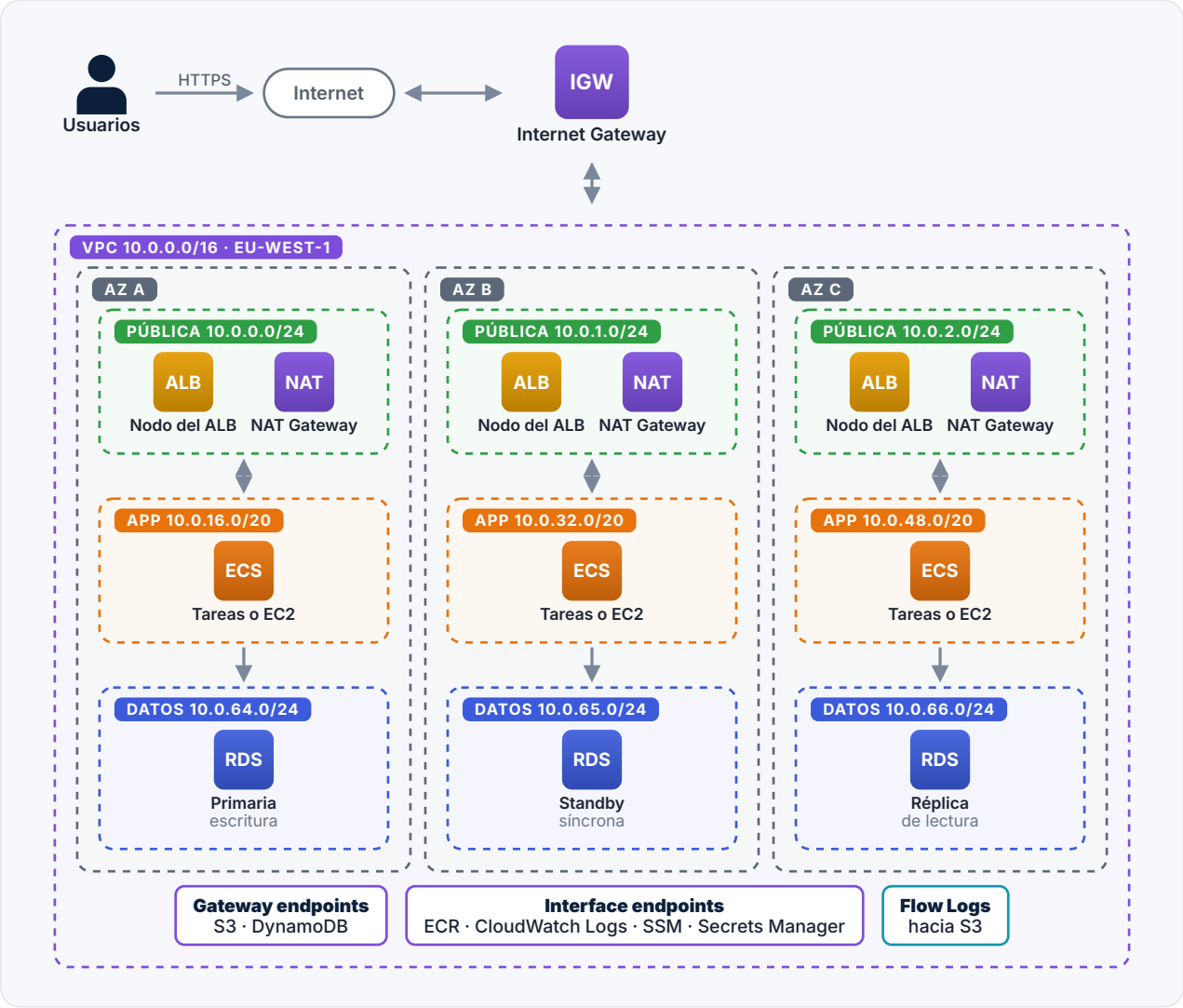


Figura 32 · Diseño de referencia: VPC de tres capas en tres AZ. El ALB tiene un nodo en cada subred pública y envía las peticiones a la capa de aplicación, que sale a Internet por el NAT Gateway de su propia AZ (de ahí la flecha doble) y consulta una base de datos Multi-AZ. Los endpoints evitan pasar por el NAT.

RESUMEN DEL CAPÍTULO

- Una **VPC** es una red privada regional; cada subred vive en una sola AZ y pierde **5 IP** reservadas. Elige un CIDR (de /16 a /28) que no se solape con tus otras redes: no se puede cambiar, solo ampliar con bloques secundarios.
- Las **tablas de rutas** deciden el camino: gana la ruta más específica y la ruta local no se puede borrar.
- La ruta por defecto (`0.0.0.0/0`) define el tipo de subred: **pública** si va a un Internet Gateway (la instancia necesita además IP pública y cortafuegos que la dejen pasar), **privada** si sale por un **NAT Gateway** y **aislada** si no tiene salida.
- El NAT Gateway se paga por hora y por GB, y conviene uno por AZ (o el modo regional); en IPv6 se usa el Egress-only IGW. Todas las IPv4 públicas se cobran por hora desde 2024.
- La **NACL** filtra en la frontera de la subred, sin estado (reglas allow y deny numeradas: no olvides los puertos efímeros); el **Security Group** filtra en cada ENI, con estado, solo con reglas allow y puede referenciar a otros SG.
- Los **gateway endpoints** (S3, DynamoDB) son gratuitos; los **interface endpoints** (PrivateLink) cubren el resto y se pagan por hora y GB. **Peering** une dos VPC sin transitividad; **Transit Gateway** es el hub transitivo; **VPC Lattice** conecta servicios.

- Para el entorno híbrido: **Site-to-Site VPN** (IPsec sobre Internet, lista en minutos), **Client VPN** (para personas) y **Direct Connect** (enlace dedicado, tarda semanas).
- El **Route 53 Resolver** da DNS en cada VPC; **Flow Logs** y **Reachability Analyzer** son tus herramientas de diagnóstico.

Preguntas de repaso

1. **¿Cuántas direcciones utilizables tiene una subred /26 en AWS?**
   Respuesta: 59: el bloque tiene 64 direcciones y AWS reserva 5.
2. **Una instancia tiene IP pública pero no responde desde Internet. ¿Qué compruebas?**
   Respuesta: Que la tabla de rutas de su subred tenga `0.0.0.0/0` hacia el IGW, que su Security Group permita el puerto, que la NACL permita la entrada y la salida (puertos efímeros) y que el servicio esté escuchando.
3. **¿Por qué se recomienda un NAT Gateway por AZ?**
   Respuesta: Porque un NAT zonal depende de su AZ: si cae, las subredes de otras AZ que lo usan pierden la salida. Además, evita cargos de tráfico entre AZ.
4. **Tu NACL permite la entrada al puerto 443, pero los clientes no reciben respuesta. ¿Qué falta?**
   Respuesta: Una regla de salida hacia los puertos efímeros (1024–65535): la NACL es *stateless* y no permite las respuestas automáticamente.
5. **¿Cómo accedes a S3 desde una subred privada sin NAT y sin coste adicional?**
   Respuesta: Con un gateway endpoint de S3 asociado a la tabla de rutas de esa subred.
6. **Las VPC A y B están emparejadas, y también B y C. ¿Puede A hablar con C?**
   Respuesta: No: el peering no es transitivo. Necesitas un peering A–C o un Transit Gateway.

CAPÍTULO 5

EC2: cómputo en la nube

Servidores virtuales en minutos: elige imagen, tipo de instancia, almacenamiento, red y modelo de compra, y opéralos sin abrir un solo puerto SSH.

EN ESTE CAPÍTULO APRENDERÁS

1. Qué es una instancia y cómo nace a partir de una AMI
2. Leer nombres como `m7g.xlarge` y elegir la familia adecuada
3. El ciclo de vida de una instancia y qué se factura en cada estado
4. Almacenamiento: EBS, instance store y EFS
5. Red, user data, metadatos IMDSv2 y roles de instancia
6. Modelos de compra, acceso seguro y Systems Manager

SERVICIOS PROTAGONISTAS

Amazon EC2 · Amazon EBS · Amazon EFS · AWS Systems Manager · EC2 Image Builder · AWS Nitro System

5.1 ¿Qué es Amazon EC2?

Antes de la nube, conseguir un servidor implicaba pedirlo, esperar semanas, instalarlo en un rack y amortizarlo durante años. **Amazon Elastic Compute Cloud (EC2)** lo convierte en una llamada a una API: en uno o dos minutos tienes una **instancia** —una máquina virtual, o un servidor físico completo en los tipos *bare metal*— con el sistema operativo, la CPU, la memoria, el disco y la red que elijas, y dejas de pagar el cómputo cuando la paras o la eliminas. A cambio de esa flexibilidad, y según el modelo de responsabilidad compartida que vimos en el capítulo de Fundamentos, tú gestionas el sistema operativo invitado (parches incluidos), el software, los datos y las reglas de acceso; AWS, el hardware, la virtualización y la red física.

FICHA DEL SERVICIO

Amazon EC2

Capacidad de cómputo redimensionable en la nube

TIPO	IaaS: máquinas virtuales y servidores bare metal bajo demanda
ÁMBITO	Cada instancia vive en una subred y, por tanto, en una única zona de disponibilidad
FACTURACIÓN	Por segundo (mínimo 60 s) en Linux y Windows; por hora en algunas AMI comerciales. Aparte: EBS, IPv4 públicas y transferencia de datos
LÍMITES CLAVE	Cuotas de vCPU por región y grupo de familias (ajustables en Service Quotas)

750+ tipos de instancia disponibles

60 s mínimo facturable; después, por segundo

2 min de aviso antes de interrumpir una Spot

90 % de descuento máximo con Spot

Da igual que lances la instancia desde la consola, la CLI, CloudFormation o un grupo de Auto Scaling: siempre decides los mismos ocho ingredientes, que este capítulo recorre uno por uno.

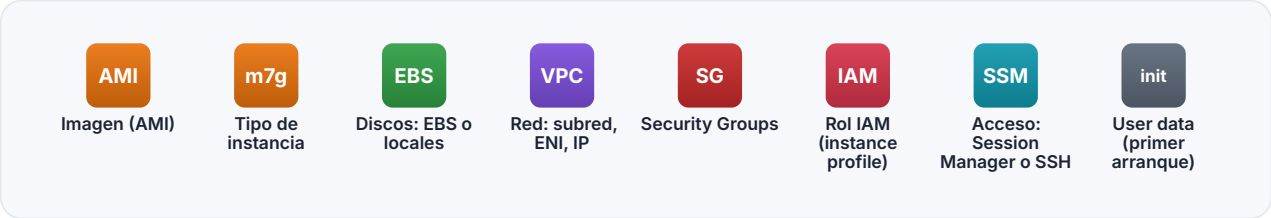


Figura 33 · Los ocho ingredientes de una instancia. Una *launch template* los guarda todos en una plantilla versionada.

5.2 Imágenes de máquina (AMI)

Una **Amazon Machine Image (AMI)** es la plantilla de la que nace el disco raíz: sistema operativo, software preinstalado y configuración. Por dentro, una AMI respaldada por EBS es una o varias **snapshots** más los permisos de lanzamiento (qué cuentas pueden usarla) y el **block device mapping** (qué volúmenes crear). Las AMI son **regionales**: el mismo sistema tiene un ID `ami-...` distinto en cada región, y para llevar una AMI propia a otra región hay que copiarla.

Públicas
AWS, editores y comunidad

Amazon Linux 2023, Ubuntu, Windows Server, RHEL... Cualquiera puede publicar *community AMIs*: comprueba siempre el propietario.

AWS Marketplace
Software de terceros listo para usar

Firewalls, bases de datos o sistemas endurecidos; la licencia se suma a tu factura de EC2, normalmente por hora.

Propias
Creadas por ti

Desde una instancia configurada (`create-image`), con EC2 Image Builder o con Packer. Privadas por defecto; puedes compartirlas con otras cuentas.

Golden AMI
La imagen oficial de la empresa

AMI propia endurecida, parcheada, con los agentes corporativos y versionada. Todas las instancias parten de ella: arranques rápidos y homogéneos.

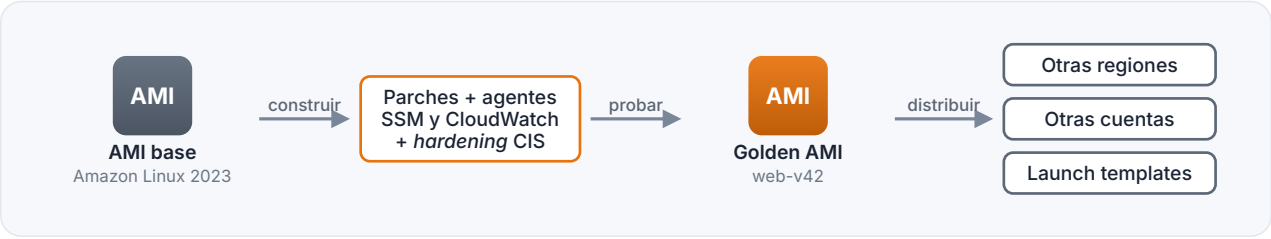


Figura 34 · Canalización de una *golden AMI*. EC2 Image Builder automatiza construcción, pruebas y distribución, y puede repetirla periódicamente para incorporar parches.

No copies IDs de AMI a mano: los **parámetros públicos** de Systems Manager siempre apuntan a la última versión.

```bash
# ID de la última Amazon Linux 2023 para ARM64 (Graviton) en la región actual
aws ssm get-parameter --query Parameter.Value --output text \
  --name /aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-arm64
# Crear una AMI propia y copiarla a otra región
aws ec2 create-image --instance-id i-0abc1234def567890 --name web-v42
aws ec2 copy-image --source-region eu-west-1 --region eu-central-1 \
  --source-image-id ami-0123456789abcdef0 --name web-v42
```

Dos detalles que sorprenden: `create-image` reinicia la instancia por defecto para obtener un sistema de ficheros consistente (`--no-reboot` lo evita, con riesgo), y **dar de baja** (*deregister*) una AMI no borra sus snapshots, que se siguen facturando hasta que las eliminas.

5.3 Tipos de instancia: familias y nomenclatura

El **tipo de instancia** fija la combinación de CPU, memoria, almacenamiento local, red y aceleradores. AWS los agrupa en familias según el tipo de carga para el que están optimizados:

Familia	Optimizada para	Ejemplos	Casos de uso típicos
T	General, CPU *burstable*	`t4g`, `t3`, `t3a`	Desarrollo y webs con poca carga
M	Equilibrada (1 vCPU : 4 GiB)	`m7g`, `m7i`, `m8g`	Aplicaciones y backends
C	Cómputo (1 vCPU : 2 GiB)	`c7g`, `c7i`, `c8g`	Lotes, vídeo, juegos, inferencia en CPU
R	Memoria (1 vCPU : 8 GiB)	`r7g`, `r7i`, `r8g`	Bases de datos y cachés en memoria
X	Mucha memoria (16+ GiB/vCPU)	`x2idn`, `x8g`	SAP HANA, bases de datos en memoria
I	NVMe local con muchas IOPS	`i4i`, `i4g`	NoSQL, OLTP, motores de búsqueda
D	HDD local denso	`d3`, `d3en`	HDFS, almacenamiento distribuido
P	GPU para entrenamiento y HPC	`p4d`, `p5`	Entrenamiento de IA a gran escala
G	GPU para gráficos e inferencia	`g5`, `g6`, `g6e`	Renderizado, *streaming*, inferencia
Inf, **Trn**	Chips de IA de AWS (Inferentia, Trainium)	`inf2`, `trn2`	Inferencia y entrenamiento de IA a menor coste
Hpc	HPC con red EFA	`hpc7g`, `hpc7a`	Simulación (CFD), meteorología

Dentro de cada familia, el nombre completo no es arbitrario: se lee por partes.

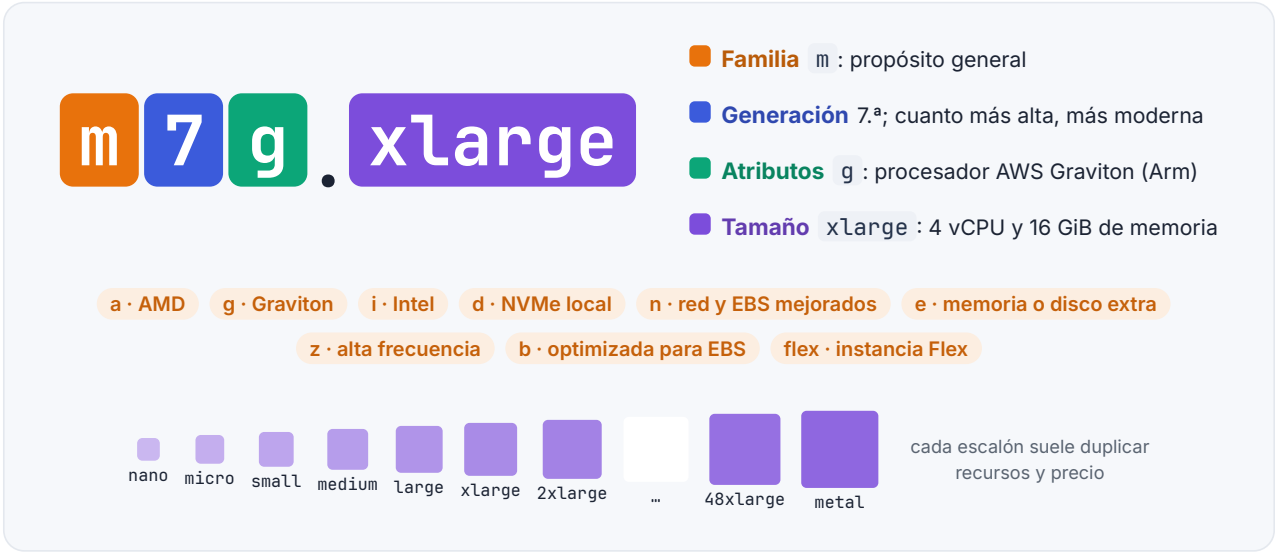


Figura 35 · Anatomía de un nombre de tipo de instancia, letras de atributo frecuentes y escalera de tamaños.

CONSEJO

Truco: C de *Compute*, R de *RAM*, I de *IOPS*, D de *Dense storage*, G de *Graphics*. Elige la generación más reciente y deja que AWS Compute Optimizer te recomiende el tamaño según el uso real.

Instancias T y créditos de CPU

Las instancias **T** son baratas porque no garantizan el 100 % de la CPU: tienen una **línea base** (el 10 % por vCPU en una `t3.micro`) y acumulan **créditos de CPU** mientras trabajan por debajo; en los picos los gastan. Un crédito es una vCPU al 100 % durante un minuto.

Modo standard
Por defecto en T2

Sin créditos, la CPU queda limitada a la línea base: barato y predecible, pero la aplicación se ralentiza.

Modo unlimited
Por defecto en T3, T3a y T4g

Puedes superar la línea base sin créditos; si la media de 24 h la excede, pagas un recargo por los créditos excedentes.

CUIDADO

Una T con CPU alta sostenida en modo *unlimited* acaba costando más que una M equivalente. Si `CPUCreditBalance` vive en cero o `CPUSurplusCreditsCharged` crece, cambia de familia.

Graviton y el sistema Nitro

AWS Graviton son los procesadores Arm (`arm64`) diseñados por AWS (Graviton2, 3 y 4; letra `g`): mejor relación precio-rendimiento y menor consumo que las x86 comparables, también en RDS, ElastiCache, Lambda o Fargate. Java, Python, Node.js, Go o .NET funcionan sin cambios; solo necesitas binarios o imágenes de contenedor `arm64` (o multiarquitectura).

Por debajo de las instancias modernas está el **AWS Nitro System**: AWS sacó las funciones de virtualización (red VPC, acceso a EBS, almacenamiento local, gestión) de la CPU principal y las llevó a **tarjetas Nitro** dedicadas, con un hipervisor mínimo y un chip de seguridad como raíz de confianza. Consecuencias prácticas:

- Casi todos los recursos del servidor quedan para tus instancias, y existen tipos `.metal` sin hipervisor.

- Por diseño, ningún operador de AWS puede iniciar sesión en el host ni leer la memoria de tus instancias; **Nitro Enclaves** crea entornos aislados, sin red ni disco persistente, para datos muy sensibles.
- Los volúmenes EBS aparecen como dispositivos NVMe (`/dev/nvme1n1`) y la red usa siempre el adaptador ENA.

5.4 Ciclo de vida de una instancia

Una instancia pasa por estados bien definidos. Conocerlos evita sustos, tanto en la factura como en los datos.

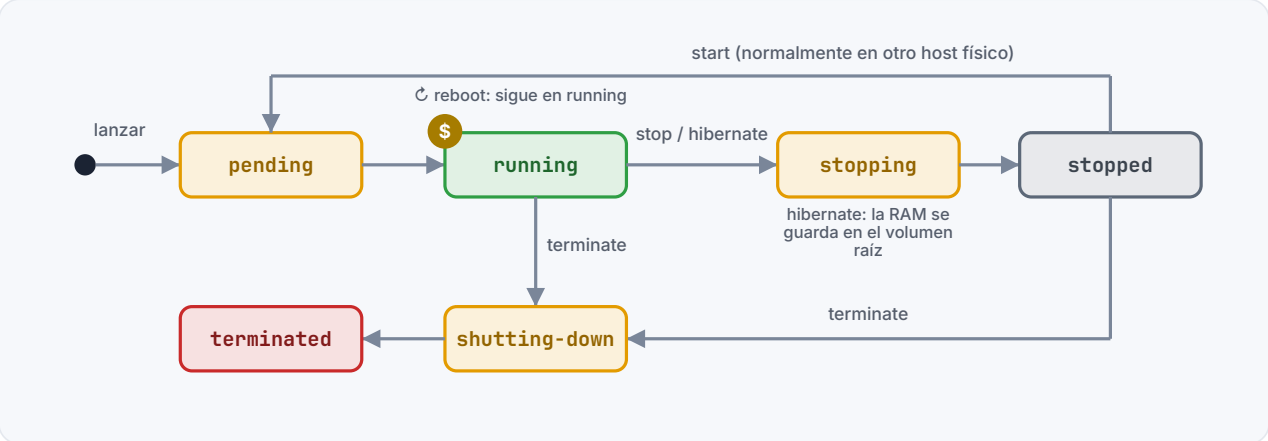


Figura 36 · Estados de una instancia EC2. Solo se factura el cómputo en `running` (y en `stopping` cuando hibernas). En `stopped` sigues pagando los volúmenes EBS y las IPv4 públicas asignadas.

Acción	Qué se conserva	IPv4 pública automática	Instance store	RAM
Reboot	Todo: mismo host, IP y volúmenes	Se mantiene	Se conserva	Se pierde
Stop / start	Volúmenes EBS, ID, IP privada y Elastic IP	Cambia	Se pierde	Se pierde
Hibernate	Lo mismo que *stop* más la RAM	Cambia	Se pierde	Se restaura
Terminate	Nada, salvo volúmenes con `DeleteOnTermination=false`	Se libera	Se pierde	Se pierde

Al terminar se borra por defecto el volumen raíz y se conservan los demás volúmenes EBS. Activa la **protección contra terminación** en instancias críticas; un `shutdown` desde el sistema operativo **para** la instancia salvo que lo configures. La **hibernación** ayuda con arranques lentos o cachés calientes: exige un volumen raíz cifrado con espacio para la RAM, solo existe en ciertos tipos y AMI y dura como máximo 60 días.

5.5 Almacenamiento: EBS, instance store y EFS

Una instancia puede usar discos de red persistentes (EBS), discos locales efímeros (instance store) y sistemas de ficheros compartidos (EFS). El diagrama resume dónde vive cada uno.

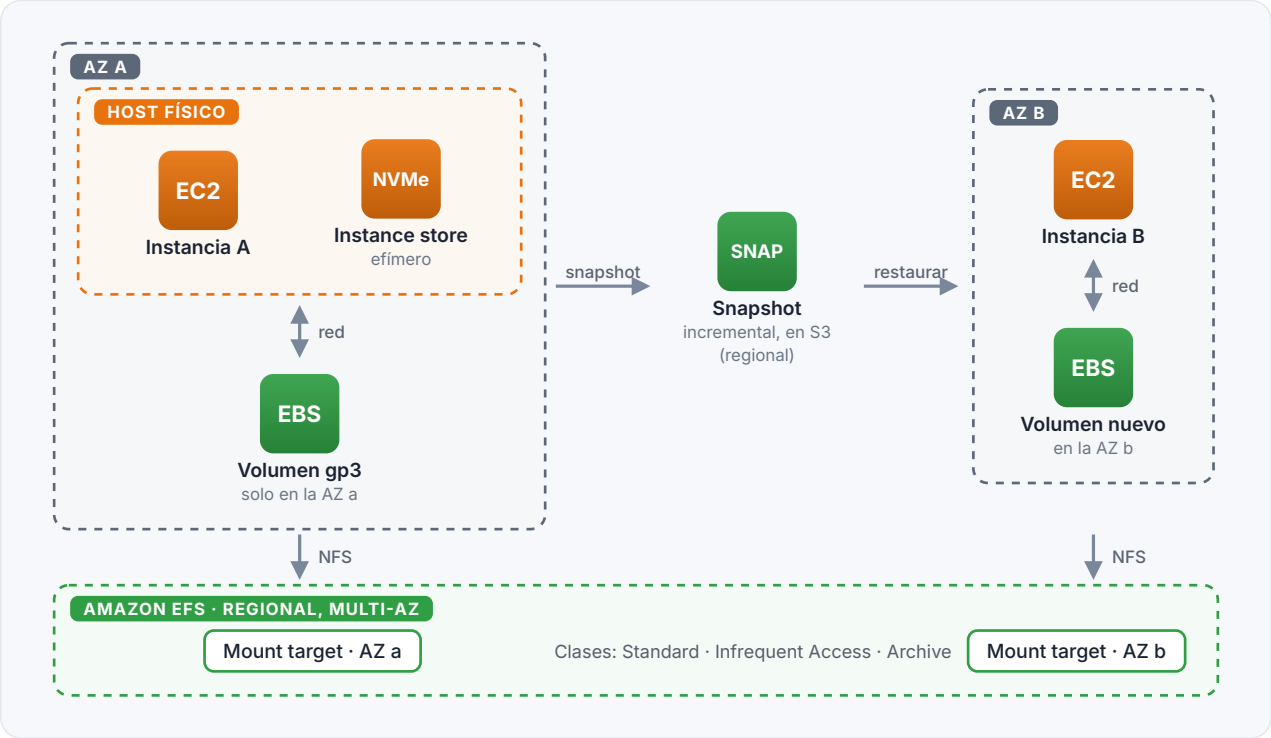


Figura 37 · Un volumen EBS pertenece a una AZ; para llevarlo a otra se pasa por un snapshot regional. El instance store vive en el host físico y EFS se comparte entre AZ mediante *mount targets*.

Amazon EBS: discos de red persistentes

Elastic Block Store (EBS) ofrece volúmenes de bloque conectados por la red que sobreviven a la instancia. Cada volumen vive en **una sola AZ** (y se replica dentro de ella) y solo se conecta a instancias de esa AZ. Con *Elastic Volumes* amplías el tamaño o cambias el tipo y el rendimiento en caliente.

Tipo	Medio	Rendimiento	Arran-que	Casos de uso
gp3	SSD	3000 IOPS y 125 MiB/s incluidos con cual-quier tamaño; más IOPS y *throughput* apro-visionables sin agrandar el volumen	✔	Por defecto: discos raíz, aplicaciones, bases de da-tos medianas
gp2	SSD	3 IOPS por GiB (de 100 a 16 000), ráfagas de 3000 en volúmenes pequeños	✔	Heredado: migra a gp3
io2 Block Ex-press	SSD	Hasta 256 000 IOPS y 4000 MiB/s, latencia submilisegundo, durabilidad del 99,999 %	✔	Bases de datos críticas
st1	HDD	Throughput secuencial de hasta 500 MiB/s	✘	Big data, logs
sc1	HDD	Hasta 250 MiB/s; el menor coste por GB	✘	Datos fríos

Snapshots — copias incrementales que AWS guarda en S3 (no aparecen en tus *buckets*). Son regiona-les: con ellas creas volúmenes en cualquier AZ, las copias a otra región o las compartes. Automatízalas con AWS Backup o Data Lifecycle Manager; la *Recycle Bin* protege de borrados accidentales.
Cifrado — con KMS cifra los datos en reposo, en tránsito hacia la instancia y los snapshots. Activa el **cifrado por defecto** en cada región. Un volumen sin cifrar no se cifra «en el sitio»: snapshot, copia cifrada y volumen nuevo.
Multi-Attach — solo io1/io2, hasta 16 instancias Nitro de la misma AZ, y con un sistema de ficheros de clúster (ext4 o XFS se corromperían).

> **CONSEJO**
>
> gp3 es hasta un 20 % más barato por GB que gp2 y desacopla rendimiento y tamaño: migra con Elastic Volumes sin parar la instancia.

Instance store y Amazon EFS

El **instance store** son discos (normalmente NVMe) físicamente conectados al host: rapidísimos e incluidos en el precio de los tipos que los tienen (letra `d`, familias I y D), pero **efímeros**. Sus datos sobreviven a un reinicio y se pierden al parar, hibernar o terminar la instancia, o si falla el hardware. Úsalos para cachés, temporales o datos replicados en otros nodos (Cassandra, Kafka), nunca para algo que no puedas regenerar.

Amazon EFS, en cambio, es un sistema de ficheros NFS totalmente gestionado que montan a la vez miles de instancias Linux, contenedores o funciones Lambda: crece y decrece solo, se paga por GB almacenado y, en modo regional, guarda los datos en varias AZ, con un *mount target* por AZ protegido por un Security Group (puerto 2049). Para Windows (SMB), HPC u otros protocolos existe **Amazon FSx**; ambos se tratan en el capítulo de Almacenamiento.

Característica	EBS	Instance store	EFS
Tipo y ámbito	Bloque, en una AZ	Bloque, en el host físico	Ficheros (NFS), regional
Persistencia	Independiente de la instancia	Se pierde al parar o terminar	Independiente
Acceso simultáneo	Una instancia (Multi-Attach en io1/io2)	Solo su instancia	Miles de clientes
Se paga por	GB aprovisionado (+ IOPS y throughput)	Incluido en la instancia	GB usado (+ accesos IA/Archive)
Caso típico	Disco raíz, bases de datos	Cachés, temporales	Contenido compartido, *home*

5.6 Redes en EC2

Toda instancia se conecta a la VPC con al menos una **Elastic Network Interface (ENI)**: una tarjeta de red virtual con IP privada principal, IP secundarias opcionales, MAC y sus propios Security Groups (5 por ENI por defecto, ampliable). La principal (`eth0`) no se separa; las secundarias pueden moverse a otra instancia de la misma AZ. El máximo de ENI e IP depende del tipo.

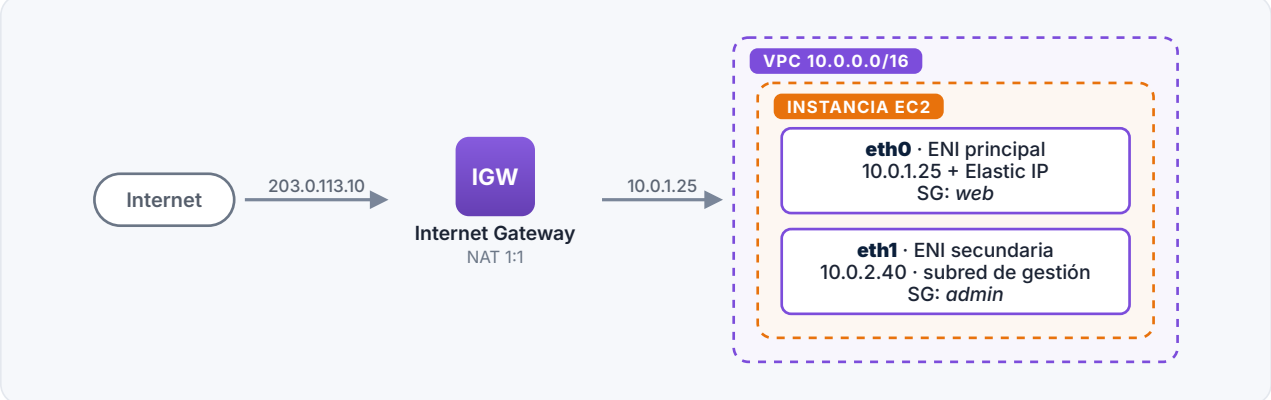


Figura 38 · El sistema operativo solo ve IP privadas: el Internet Gateway traduce 1:1 la IPv4 pública (automática o Elastic IP) a la privada de la ENI.

Cada instancia conserva su IP privada aunque la pares; además puede tener una IPv4 pública automática (cambia en cada *stop/start*), una Elastic IP estática o direcciones IPv6. El capítulo de Redes

explica cuándo usar cada una y por qué toda IPv4 pública se factura por hora; en la práctica, no expongas cada instancia: pon un balanceador delante.

Los **Security Groups** filtran el tráfico de cada ENI con estado (los estudiamos en el capítulo de Redes). Todas las instancias actuales usan **enhanced networking** con el **Elastic Network Adapter (ENA)**, que mediante SR-IOV ofrece decenas o cientos de Gbps según el tipo. Para HPC y entrenamiento distribuido, el **Elastic Fabric Adapter (EFA)** añade a ENA un modo *OS-bypass* con el que MPI o NCCL hablan con el hardware de red sin pasar por el kernel; ese tráfico solo funciona dentro de una misma subred.

Placement groups

Por defecto, EC2 reparte tus instancias por el hardware como mejor le parece. Un **placement group**, sin coste adicional, te deja influir en esa colocación:



Cluster
Juntas en hardware cercano de **una sola AZ**: mínima latencia y máximo ancho de banda (HPC con EFA). Un fallo del hardware compartido puede afectar a todas.

Spread
Cada instancia en un rack distinto, **máximo 7 por AZ** y grupo. Para pocas instancias críticas que no deben caer juntas (quórum, primario y réplica).

Partition
Particiones (**hasta 7 por AZ**) que no comparten racks; cientos de instancias. HDFS, HBase, Cassandra o Kafka leen su partición en los metadatos.

Figura 39 · Estrategias de *placement group*. Cada rectángulo vertical es un rack con su propia red y alimentación.

> ★ **CLAVE PARA LA CERTIFICACIÓN**
>
> ¿Mínima latencia entre nodos HPC? Placement group *cluster* + EFA. ¿IP pública fija? Elastic IP. ¿Conmutar una IP a un servidor de respaldo? ENI secundaria movida a otra instancia de la misma AZ. ¿Instancias críticas sin hardware compartido? Placement group *spread*.

5.7 Arranque, metadatos e identidad

User data y cloud-init

El **user data** es un script (o una configuración de cloud-init) que la instancia ejecuta como `root` en su **primer arranque**; en Linux lo interpreta **cloud-init** y en Windows, EC2Launch. Admite hasta 16 KB y su salida queda en `/var/log/cloud-init-output.log`.

```bash
#!/bin/bash
# User data: se ejecuta como root en el primer arranque (Amazon Linux 2023)
dnf install -y nginx
echo "<h1>Hola desde $(hostname)</h1>" > /usr/share/nginx/html/index.html
systemctl enable --now nginx
```

Cuidado: el user data no es secreto (lo lee cualquier proceso de la instancia y quien tenga `ec2:DescribeInstanceAttribute`); las contraseñas, léelas al arrancar de Secrets Manager o Parameter Store.

Roles de instancia (instance profile)

Una aplicación en EC2 que llama a S3 o DynamoDB **nunca** debe llevar *access keys* en un fichero. En su lugar, asocias a la instancia un **rol IAM** mediante un **instance profile**, un contenedor con exactamente un rol (los roles y su trust policy se explican en el capítulo «IAM (I)»). EC2 publica en el servicio de metadatos (IMDS, a continuación) credenciales temporales de ese rol, que se renuevan solas y que

los SDK y la CLI encuentran automáticamente: tu código no guarda ninguna clave. La consola crea el instance profile por ti (con la CLI: `create-instance-profile` y `add-role-to-instance-profile`), y el rol se puede cambiar en caliente.

Metadatos de instancia: IMDSv2

Cada instancia consulta datos sobre sí misma (ID, tipo, AZ, IP, user data, credenciales de su rol) en el **Instance Metadata Service (IMDS)**, accesible solo desde dentro en `169.254.169.254`. Con **IMDSv2** primero se pide un *token* con un `PUT` (válido de 1 segundo a 6 horas) y después se envía en cada consulta:

```bash
TOKEN=$(curl -sX PUT "http://169.254.169.254/latest/api/token" \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
curl -sH "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/placement/availability-zone
```

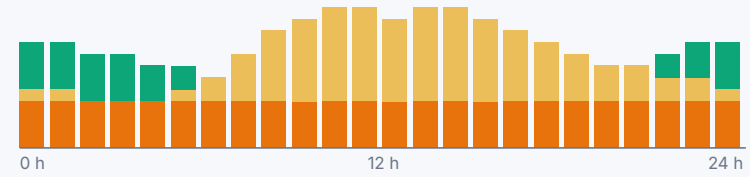
¿Por qué? Con IMDSv1 bastaba un `GET`: una vulnerabilidad SSRF (*Server-Side Request Forgery*) o un proxy mal configurado permitían robar las credenciales del rol. El `PUT` con cabecera propia es mucho más difícil de provocar, y la respuesta no pasa de un salto de red (*hop limit*).

> **BUENA PRÁCTICA**
>
> Exige IMDSv2 (`HttpTokens=required`) en las launch templates y por defecto en la cuenta (`modify-instance-metadata-defaults`); Amazon Linux 2023 ya lo exige. Sube el hop limit a 2 solo si hay contenedores que usen el IMDS.

5.8 Modelos de compra

La misma instancia puede costar muy distinto según cómo la compres; es la palanca de ahorro más potente de EC2 (el capítulo de Costes amplía la estrategia).

Modelo	Compromiso	Ahorro orientativo	¿Interrupción?	Ideal para
On-Demand	Ninguno; pagas por segundo	Precio de referencia	✘	Cargas impredecibles, pruebas, picos
Savings Plans	Gasto por hora durante 1 o 3 años	Hasta 72 % (EC2 Instance) o 66 % (Compute)	✘	Base estable; Compute SP cubre también Fargate y Lambda
Reserved Instances	Tipo y región (o AZ), 1 o 3 años	Hasta 72 % (Standard)	✘	Cargas muy estables; hoy se prefieren los Savings Plans
Spot	Ninguno	Hasta 90 %	Sí, aviso de 2 min	Lotes, CI, contenedores, cargas tolerantes a fallos
Dedicated Instances	Ninguno (también reservables)	Más caras	✘	Hardware no compartido con otras cuentas
Dedicated Hosts	Por servidor físico	Más caros	✘	Licencias por socket o núcleo (BYOL), macOS
Capacity Reservations	Capacidad en una AZ, sin plazo mínimo	Ninguno (se combinan con SP o RI)	✘	Garantizar capacidad para eventos o DR



Spot
Lotes nocturnos que pueden reintentarse

On-Demand
Picos variables del horario laboral

Savings Plans / RI
Capacidad mínima encendida 24/7

Figura 40 · Estrategia típica: la base 24/7 con Savings Plans, los picos diurnos con On-Demand y los lotes nocturnos con Spot.

Spot aprovecha la capacidad libre de EC2, con un precio que varía poco a poco (ya no hay pujas). Cuando AWS la necesita, avisa con **2 minutos** de antelación en los metadatos (`spot/instance-action`) y en EventBridge (a veces antes llega una **rebalance recommendation**), y la instancia se termina, se para o se hiberna. Diversifica tipos y AZ y deja elegir a Auto Scaling o EC2 Fleet con `price-capacity-optimized`.

> **COSTES**
>
> Los Compute Savings Plans siguen aplicándose aunque cambies de familia, región o SO, o pases a Fargate o Lambda. Las Capacity Reservations se cobran **las uses o no**.

5.9 Acceso a las instancias

La forma clásica de entrar en un Linux es SSH con un **par de claves**: EC2 inyecta la clave pública en la instancia y tú custodias la privada (AWS no guarda copia). Obliga a abrir el puerto 22, a rotar ficheros `.pem` y a mantener bastiones.

	SSH con par de claves	EC2 Instance Connect	Session Manager
Puerto entrante	22 abierto en el SG	22 (desde tu IP o desde un EC2 Instance Connect Endpoint)	Ninguno
Claves	Las custodias y rotas tú	Temporales, válidas 60 s	No hay claves
Quién entra	Quien tenga la clave privada	Quien tenga permisos IAM	Quien tenga permisos IAM
Auditoría	Logs del sistema operativo	CloudTrail registra cada envío de clave	CloudTrail + registro de sesión en S3 o CloudWatch Logs
Subredes privadas	Bastión o VPN	Con EC2 Instance Connect Endpoint	Sí, vía NAT o VPC endpoints

```bash
ssh -i ~/.ssh/mi-clave.pem ec2-user@203.0.113.10                # SSH clásico
aws ec2-instance-connect ssh --instance-id i-0abc1234def567890  # Instance Connect
aws ssm start-session --target i-0abc1234def567890              # Session Manager
aws ssm start-session --target i-0abc1234def567890 \
  --document-name AWS-StartPortForwardingSessionToRemoteHost \
  --parameters '{"host":["db.interna.local"],"portNumber":["5432"],
                 "localPortNumber":["5432"]}'        # túnel a una BD privada
```

BUENA PRÁCTICA

Usa **Session Manager** por defecto: sin puertos entrantes ni claves, con acceso por IAM y sesiones auditadas. Necesita el SSM Agent (preinstalado en las AMI de Amazon Linux, Ubuntu y Windows de AWS), un rol con la política `AmazonSSMManagedInstanceCore` y salida hacia los endpoints de Systems Manager (`ssm` , `ssmmessages` y, según la versión del agente, `ec2messages`), por NAT o con VPC endpoints.

5.10 AWS Systems Manager

Session Manager es solo una de las capacidades de **AWS Systems Manager**, la navaja suiza para operar flotas de servidores sin entrar en ellos uno a uno.

FICHA DEL SERVICIO

AWS Systems Manager

Operación centralizada de instancias y servidores

REQUISITOS	SSM Agent, permisos IAM y conectividad con los endpoints de Systems Manager
ÁMBITO	Regional; gestiona EC2 y también servidores on-premises o de otras nubes (nodos híbridos)
FACTURACIÓN	Muchas capacidades no tienen coste adicional en EC2; algunas avanzadas sí

Run Command — ejecuta un documento (como `AWS-RunShellScript`) en cientos de instancias elegidas por etiquetas, con control de concurrencia y de errores.
Patch Manager — aplica parches del sistema operativo según *patch baselines* y ventanas de mantenimiento, e informa del cumplimiento.
Parameter Store — configuración y secretos jerárquicos (`SecureString` , cifrados con KMS); lo ampliamos en el capítulo de Seguridad. Hay más: Inventory, State Manager, Automation o Fleet Manager.

BASH

```bash
# Reiniciar nginx en todas las instancias web de producción
aws ssm send-command --document-name AWS-RunShellScript \
  --targets "Key=tag:App,Values=web" "Key=tag:Entorno,Values=prod" \
  --parameters 'commands=["systemctl restart nginx"]' \
  --max-concurrency 25% --max-errors 1
```

5.11 Launch templates

Una **launch template** guarda todos los parámetros de lanzamiento (AMI, tipo, Security Groups, instance profile, volúmenes, metadatos, user data, opciones de Spot) en una plantilla **versionada**. Cada cambio crea una versión (puedes marcar una como *default*, usar `$Latest` o fijar un número). La usan `run-instances`, Auto Scaling y EC2 Fleet, y sustituye a las antiguas *launch configurations*, obsoletas. `UserData` va en base64:

JSON

```json
{
  "ImageId": "ami-0123456789abcdef0",
  "InstanceType": "m7g.large",
  "IamInstanceProfile": { "Name": "web-profile" },
  "SecurityGroupIds": ["sg-0123456789abcdef0"],
  "MetadataOptions": { "HttpTokens": "required", "HttpPutResponseHopLimit": 1 },
  "BlockDeviceMappings": [{ "DeviceName": "/dev/xvda",
    "Ebs": { "VolumeSize": 20, "VolumeType": "gp3", "Encrypted": true } }],
  "UserData": "IyEvYmluL2Jhc2gKZG5mIGluc3RhbGwgLXkgbmdpbngK"
}
```

```bash
aws ec2 create-launch-template --launch-template-name web \
  --launch-template-data file://lt.json
aws ec2 run-instances --launch-template LaunchTemplateName=web,Version='$Latest'
```

5.12 Buenas prácticas

- **Seguridad**: IMDSv2 obligatorio, roles de instancia, Session Manager y cifrado de EBS por defecto.
- **Inmutabilidad**: instancias como ganado, no mascotas: golden AMI + launch templates, y reemplazar en vez de parchear a mano.
- **Fiabilidad**: varias AZ con Auto Scaling y balanceadores; snapshots automáticos con AWS Backup.
- **Coste**: Compute Optimizer, Graviton, Savings Plans + Spot, gp3; borra volúmenes sueltos, snapshots viejos y Elastic IP sin usar.
- **Operación**: etiqueta todo, parchea con Patch Manager y envía métricas y logs a CloudWatch.

RESUMEN DEL CAPÍTULO

- EC2 da máquinas virtuales facturadas por segundo; tú gestionas el sistema operativo y lo de encima. Nacen de una **AMI** (regional); las **golden AMI** homogeneizan los despliegues.
- `m7g.xlarge` = familia, generación, atributos y tamaño. Las T usan créditos de CPU; Graviton mejora el precio-rendimiento y Nitro sostiene todo.
- Solo se factura el cómputo en `running`; parar conserva EBS, pero libera la IPv4 pública automática y borra el instance store.
- **EBS** es un disco de red persistente ligado a una AZ (gp3 por defecto); el **instance store** es local y efímero; **EFS** es NFS compartido y Multi-AZ.
- Las IPv4 públicas (también las Elastic IP) se pagan por hora; ENA y EFA aceleran la red y los placement groups controlan la colocación. **IMDSv2** y los **roles de instancia** dan credenciales sin claves en disco.
- Savings Plans para la base, On-Demand para los picos, Spot (aviso de 2 minutos) para lo tolerante a interrupciones.
- Accede con **Session Manager**, opera con Systems Manager y lanza siempre desde **launch templates** versionadas.

Preguntas de repaso

1. **Necesitas mover un volumen EBS de `eu-west-1a` a `eu-west-1b`. ¿Cómo lo haces?**
   Respuesta: Creas un snapshot (regional) y, a partir de él, un volumen nuevo en `eu-west-1b`. Un volumen EBS no se conecta a instancias de otra AZ.
2. **¿Qué significa cada parte de `c7gn.2xlarge`?**
   Respuesta: Familia C (cómputo), 7.ª generación, atributos `g` (Graviton) y `n` (red mejorada), tamaño `2xlarge`.
3. **Paras y vuelves a arrancar una instancia. ¿Qué cambia?**
   Respuesta: Conserva EBS, IP privada y Elastic IP; cambia la IPv4 pública automática y se pierden la RAM y el instance store. Parada no paga cómputo, pero sí EBS.
4. **¿Por qué IMDSv2 es más seguro que IMDSv1?**
   Respuesta: Exige un token obtenido con un `PUT` y enviado en una cabecera, difícil de provocar con un ataque SSRF, y la respuesta con el token no sale de la instancia (hop limit).
5. **Un clúster HPC necesita la mínima latencia entre nodos. ¿Qué configuras?**
   Respuesta: Un placement group cluster en una sola AZ, con tipos de instancia que admitan EFA.
6. **¿Qué modelo de compra eliges para un proceso nocturno que puede reintentarse?**
   Respuesta: Spot (hasta un 90 % de descuento), diversificando tipos y AZ, porque tolera interrupciones con 2 minutos de aviso.

CAPÍTULO 6

Balanceo de carga y Auto Scaling

Reparte el tráfico entre muchas instancias sanas y deja que la flota crezca y se encoja sola según la demanda.

EN ESTE CAPÍTULO APRENDERÁS

1. Por qué balancear y qué aporta Elastic Load Balancing
2. Elegir entre ALB, NLB y GWLB
3. Listeners, reglas, target groups y health checks
4. TLS con ACM, cross-zone, stickiness y drenaje de conexiones
5. Auto Scaling groups: capacidad, políticas y health checks
6. Lifecycle hooks, instance refresh, warm pools y Spot

SERVICIOS PROTAGONISTAS

Elastic Load Balancing · Application Load Balancer · Network Load Balancer · Gateway Load Balancer · Amazon EC2 Auto Scaling · AWS Certificate Manager

6.1 ¿Por qué balancear la carga?

Una aplicación servida por un único servidor tiene tres problemas: si cae, la aplicación cae (punto único de fallo); si el tráfico crece, solo puedes comprar una máquina más grande (escalado vertical, con techo); y cada despliegue obliga a cortar el servicio. Un **balanceador de carga** ofrece un único punto de entrada (un nombre DNS), reparte las peticiones entre varios servidores —los **targets**— de varias zonas de disponibilidad y deja de enviar tráfico a los que fallan sus comprobaciones de salud. Con **Auto Scaling**, que lanza y retira servidores según la demanda, obtienes una flota elástica que se repara sola.

≈ **ANALOGÍA**

Piensa en el maître de un restaurante grande: los clientes solo ven la puerta (el nombre DNS); él los reparte entre las mesas de varios camareros, deja de asignar mesas al que se ha puesto enfermo y, si se forma cola, el gerente (Auto Scaling) llama a más camareros.

FICHA DEL SERVICIO

Elastic Load Balancing

Balanceadores de carga gestionados, elásticos y Multi-AZ

TIPOS	Application (ALB), Network (NLB) y Gateway (GWLB); el Classic (CLB) es de la generación anterior
ÁMBITO	Regional: el balanceador tiene nodos en cada AZ que habilites y escala solo
FACTURACIÓN	Por hora y por unidades de capacidad consumidas (LCU en ALB, NLCU en NLB, GLCU en GWLB)
SE INTEGRA CON	Auto Scaling, ECS, EKS, Lambda, ACM, AWS WAF (ALB), CloudFront y Global Accelerator

6.2 Tipos de balanceador

Característica	Application LB	Network LB	Gateway LB
Capa OSI	7 (aplicación)	4 (transporte)	3 (red)
Protocolos	HTTP, HTTPS, HTTP/2, gRPC, WebSocket	TCP, UDP, TLS, TCP_UDP	Todo el tráfico IP, encapsulado en GENEVE (puerto 6081)
Enrutamiento	Por host, ruta, cabeceras, método, *query string* o IP de origen	Por listener (protocolo y puerto)	Transparente hacia los appliances
IP estática	✘ Solo nombre DNS; sus IP cambian	✔ Una por AZ (admite Elastic IP)	No aplica: se usa a través de *GWLB endpoints*
Security Groups	✔	✔ Desde 2023, solo si se asignan al crearlo	✘
Targets	Instancias, IP y funciones Lambda	Instancias, IP y un ALB	Instancias e IP
IP del cliente	En la cabecera `X-Forwarded-For`	Se conserva (o Proxy Protocol v2)	Se conserva: el paquete llega intacto
Casos de uso	Webs, API, microservicios y contenedores	TCP/UDP de alto rendimiento, IP fija, PrivateLink, IoT, juegos	Firewalls, IDS/IPS e inspección de tráfico de terceros

El **ALB** es la opción por defecto para el tráfico web: entiende HTTP, así que enruta por contenido, termina TLS, autentica usuarios o devuelve respuestas fijas. El **NLB** reenvía conexiones TCP/UDP sin mirar dentro, con latencia muy baja; es la base de **AWS PrivateLink**. El **GWLB** inserta de forma transparente una flota de appliances de seguridad en el camino del tráfico, que llega a él por rutas hacia un GWLB endpoint. El **Classic Load Balancer** (CLB), de la generación anterior, funciona en capa 4 y 7 pero sin reglas, target groups ni targets Lambda: úsalo solo en cargas heredadas y planifica su migración a ALB o NLB (la consola incluye un asistente).

CONSEJO

¿Necesitas IP fija **y** reglas HTTP? Pon un NLB delante de un ALB (target group de tipo `alb`) o usa Global Accelerator. Y si creas un NLB, asígnale Security Groups desde el principio: no se pueden añadir después a un NLB creado sin ellos.

6.3 Listeners, reglas y target groups

Listener — escucha en un protocolo y un puerto (por ejemplo, HTTPS:443) y aplica reglas a lo que llega. Un balanceador puede tener varios.
Regla — condiciones y acciones, evaluadas por prioridad (primero el número más bajo); si ninguna coincide se aplica la regla por defecto. Las reglas por contenido son exclusivas del ALB.
Target group — conjunto de destinos con su protocolo, su puerto y su health check. Un mismo target group puede usarse en varias reglas.
Target — cada destino registrado: una instancia EC2 (por ID), una IP (contenedores en modo `awsvpc`, servidores on-premises vía VPN o Direct Connect), una función Lambda (en ALB) o un ALB completo detrás de un NLB.
La Figura 41 muestra un listener típico. Las condiciones disponibles son `host-header`, `path-pattern`, `http-header`, `http-request-method`, `query-string` y `source-ip`; dentro de una regla se combinan con «y». El *forward ponderado* reparte el tráfico entre varios target groups por pesos, la base de los despliegues *canary* y *blue/green*. Las acciones `authenticate-cognito` y `authenticate-oidc` (solo en listeners HTTPS) delegan el inicio de sesión en Amazon Cognito o en cualquier proveedor OpenID Connect antes de dejar pasar la petición.

BASH

```
# Target group para contenedores en modo awsvpc (target type ip)
aws elbv2 create-target-group --name tg-api --protocol HTTP --port 8080 \
  --vpc-id vpc-0abc1234 --target-type ip --health-check-path /health
# Regla: lo que llegue a api.ejemplo.com va al target group de la API
aws elbv2 create-rule --listener-arn "$LISTENER_ARN" --priority 10 \
  --conditions Field=host-header,Values=api.ejemplo.com \
  --actions Type=forward,TargetGroupArn="$TG_API_ARN"
# Listener HTTP que solo redirige a HTTPS
aws elbv2 create-listener --load-balancer-arn "$ALB_ARN" --protocol HTTP \
  --port 80 --default-actions \
  'Type=redirect,RedirectConfig={Protocol=HTTPS,Port=443,StatusCode=HTTP_301}'
```

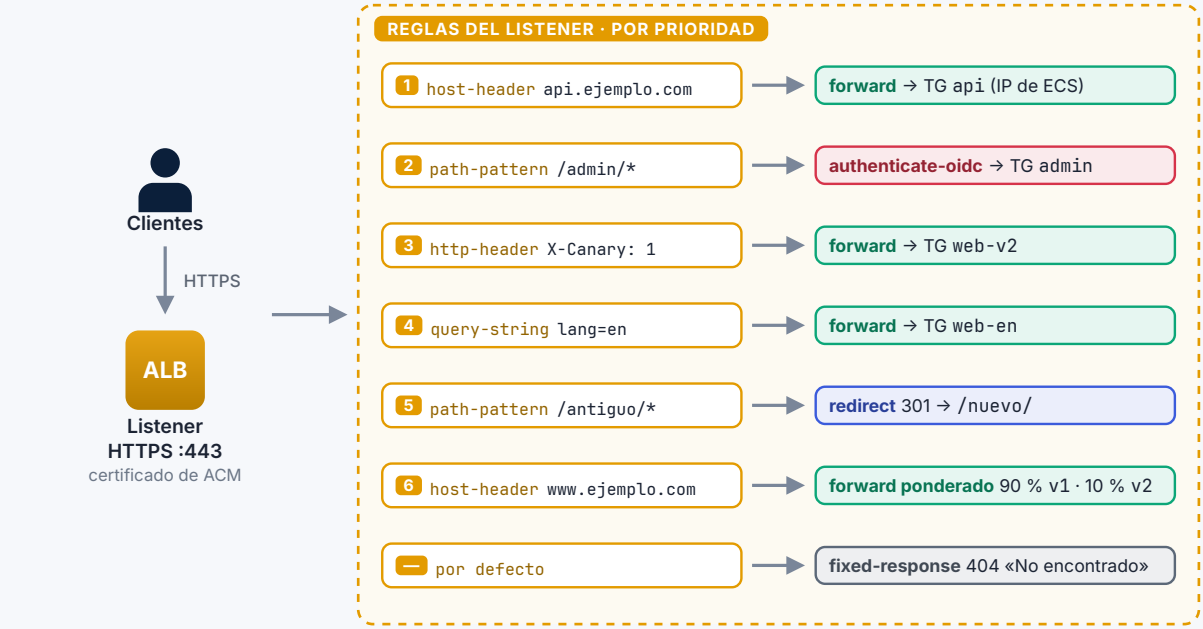


Figura 41 · Un listener de ALB evalúa sus reglas por prioridad: condiciones de host, ruta, cabeceras o query string, y acciones *forward* (simple o ponderado), *authenticate*, *redirect* y *fixed-response*.

6.4 Health checks

El balanceador comprueba periódicamente cada target y solo le envía tráfico si está sano. En el target group defines el protocolo, el puerto y la ruta (por ejemplo, `HTTP` y `/health`), el intervalo y el *timeout* (30 s y 5 s por defecto en ALB), los umbrales de éxitos y fallos consecutivos para cambiar de estado (5 y 2) y los códigos que cuentan como éxito (`200` o un rango como `200-399`). Si **todos** los targets están no sanos, el ALB reparte entre todos igualmente (*fail open*): prefiere servir algo a no servir nada.

> **CUIDADO**
>
> Un health check «profundo» que comprueba también la base de datos parece buena idea hasta que la base de datos tiene un problema: todos los targets fallan a la vez y el balanceador deja de servir incluso las páginas que no la necesitan. Comprueba que **tu proceso** responde y vigila las dependencias con métricas y alarmas.

6.5 TLS, certificados y SNI

El ALB y el NLB pueden **terminar TLS**: descifran en el balanceador y descargan ese trabajo de tus servidores. El certificado sale normalmente de **AWS Certificate Manager (ACM)**, que emite certificados públicos sin coste y los renueva automáticamente si los validaste por DNS; debe estar en la misma región que el balanceador (ACM se explica en el capítulo de Seguridad). La **política de seguridad** del listener fija versiones y cifrados (elige una con TLS 1.3). Con **SNI** (*Server Name Indication*), un listener sirve varios dominios con certificados distintos, elegidos según el nombre que pide el cliente. Para cifrar de extremo a extremo, usa un target group HTTPS; el ALB admite además TLS mutuo (mTLS).

6.6 Cross-zone load balancing

Cada AZ habilitada tiene su nodo del balanceador, y el DNS reparte a los clientes por igual entre los nodos. ¿Qué pasa si las AZ tienen distinto número de targets? Depende del **cross-zone load balancing**:

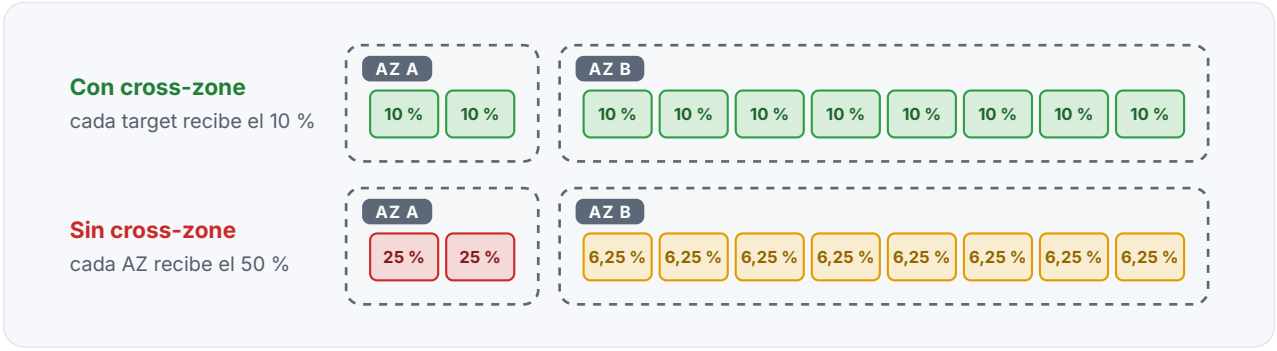


Figura 42 · Dos targets en la AZ a y ocho en la AZ b. Con cross-zone, el tráfico se reparte por igual entre los diez; sin él, cada AZ recibe la mitad y los dos targets de la AZ a quedan sobrecargados.

En el **ALB** viene siempre activado en el balanceador (puedes desactivarlo por target group) y no cobra la transferencia entre AZ. En el **NLB** y el **GWLB** viene **desactivado** por defecto y, si lo activas, pagas la transferencia de datos entre AZ.

6.7 Comportamiento fino del balanceador

Deregistration delay — al retirar un target, el balanceador deja de enviarle peticiones nuevas pero espera a que terminen las que tiene en curso (300 s por defecto, de 0 a 3600 s). Es el antiguo *connection draining*.
Slow start — en el ALB, un target nuevo recibe una parte creciente del tráfico durante 30 a 900 s, para que caliente cachés.
Sticky sessions — el ALB fija un cliente a un target con una cookie propia (`AWSALB`, de 1 segundo a 7 días) o de tu aplicación; el NLB, por IP de origen. Desequilibran la carga: mejor una aplicación sin estado con la sesión en ElastiCache o DynamoDB.
X-Forwarded-For — el ALB sustituye la IP de origen por la suya y añade la del cliente en esa cabecera, junto a `X-Forwarded-Proto` y `X-Forwarded-Port`. Configura tus logs para leerlas.
Internet-facing o interno — el primero tiene IP públicas y vive en subredes públicas; el *internal* solo tiene IP privadas. Patrón típico: ALB público → capa web → ALB interno → capa de aplicación.
Access logs — el ALB (y el NLB con listeners TLS) puede dejar en S3, cada 5 minutos, el detalle de cada petición: IP del cliente, latencias, ruta, códigos. Desactivados por defecto; analízalos con Athena.

> **BUENA PRÁCTICA**
>
> Vigila en CloudWatch `TargetResponseTime`, `HTTPCode_Target_5XX_Count` (errores de tu aplicación), `HTTPCode_ELB_5XX_Count` (del balanceador) y `UnHealthyHostCount` (capítulo de Observabilidad).

6.8 Auto Scaling groups

Un **Auto Scaling group (ASG)** mantiene una flota de instancias EC2 del tamaño adecuado: lanza instancias cuando falta capacidad o alguna falla, y las termina cuando sobra. Cada ASG combina tres cosas: **qué** lanzar (una launch template, que vimos en el capítulo de EC2), **dónde** (una lista de subredes, idealmente en varias AZ) y **cuántas** (capacidad mínima, deseada y máxima).

FICHA DEL SERVICIO

Amazon EC2 Auto Scaling

Flotas de EC2 que se escalan y se reparan solas

FACTURACIÓN	Sin coste adicional: pagas las instancias y las alarmas de CloudWatch que use
CAPACIDAD	Mínimo ≤ deseada ≤ máximo; el ASG mantiene la deseada y las políticas la mueven entre los límites
REPARACIÓN	Sustituye las instancias que fallan los health checks y reparte la flota entre las AZ

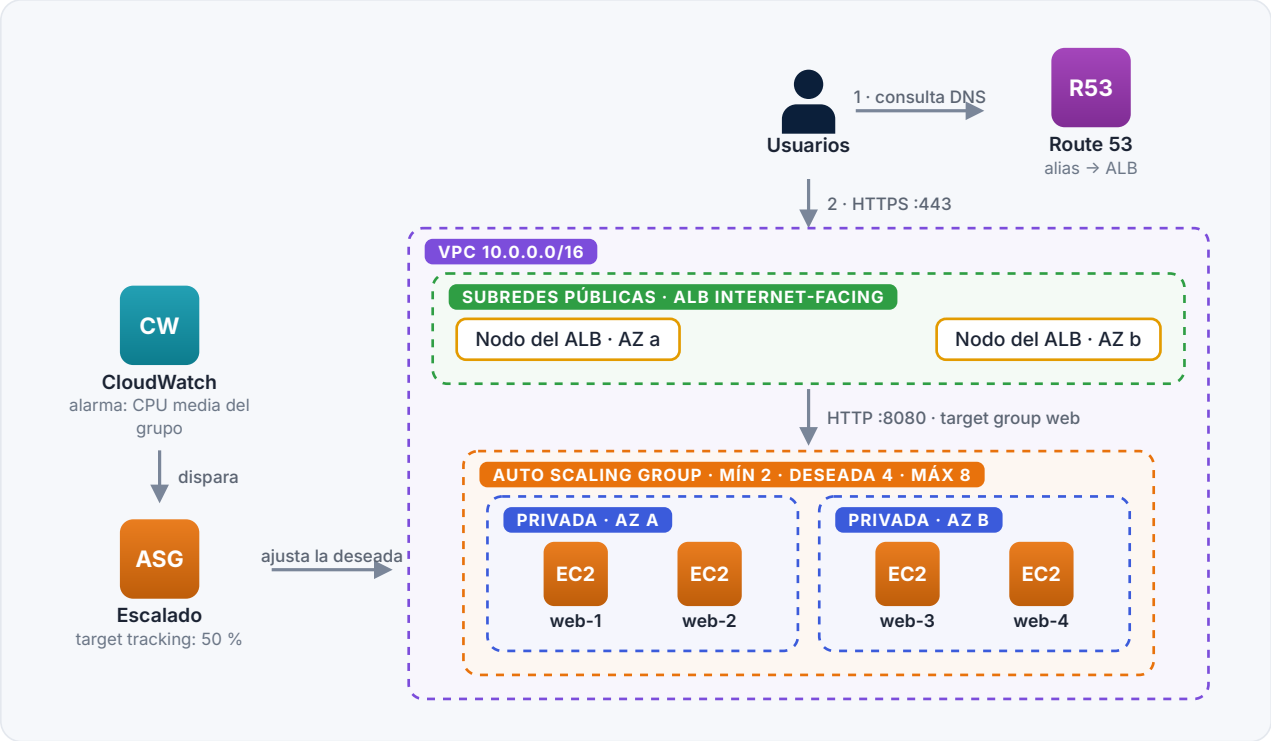


Figura 43 · Arquitectura clásica. El usuario resuelve el nombre en Route 53 (1) y se conecta por HTTPS a los nodos del ALB *internet-facing* en las subredes públicas de dos AZ (2), que reparten el tráfico entre las instancias del ASG en subredes privadas. Una alarma de CloudWatch y una política de escalado ajustan la capacidad deseada.

Políticas de escalado

Política	Cómo funciona	Cuándo usarla
Target tracking	Mantiene una métrica en un objetivo (CPU media del 50 %, peticiones por target…) como un termostato; crea las alarmas por ti	La opción por defecto
Step scaling	Alarmas con escalones: +1 instancia si la CPU pasa del 60 %, +3 si pasa del 80 %	Respuesta proporcional al pico
Simple scaling	Un ajuste por alarma y espera al *cooldown*	Heredada
Scheduled	Cambia mínimo, deseada y máximo a una hora (*cron*)	Horario laboral, campañas
Predictive	Aprende del histórico con *machine learning* y escala por adelantado	Cargas cíclicas, arranques lentos

```bash
# Target tracking: mantener la CPU media del grupo en el 50 %
aws autoscaling put-scaling-policy --auto-scaling-group-name web-asg \
  --policy-name cpu-50 --policy-type TargetTrackingScaling \
  --target-tracking-configuration '{"TargetValue": 50.0,
    "PredefinedMetricSpecification":
      {"PredefinedMetricType": "ASGAverageCPUUtilization"}}'
# Programada: mínimo de 6 instancias los días laborables a las 7:00 UTC
aws autoscaling put-scheduled-update-group-action \
  --auto-scaling-group-name web-asg --scheduled-action-name laborable \
  --recurrence "0 7 * * 1-5" --min-size 6
```

Para que el grupo no «oscile», el **cooldown** (300 s por defecto) hace esperar a una política simple tras cada ajuste, y el **instance warm-up** evita que las instancias recién lanzadas cuenten en la métrica agregada hasta que han arrancado.

Health checks y ciclo de vida

Por defecto, el ASG solo usa los **health checks de EC2** (la máquina funciona y pasa los *status checks*). Detrás de un balanceador, activa también los **health checks de ELB** para sustituir instancias cuyo servidor web no responde aunque la máquina siga viva. El *health check grace period* da tiempo a que una instancia nueva arranque antes de evaluarla.

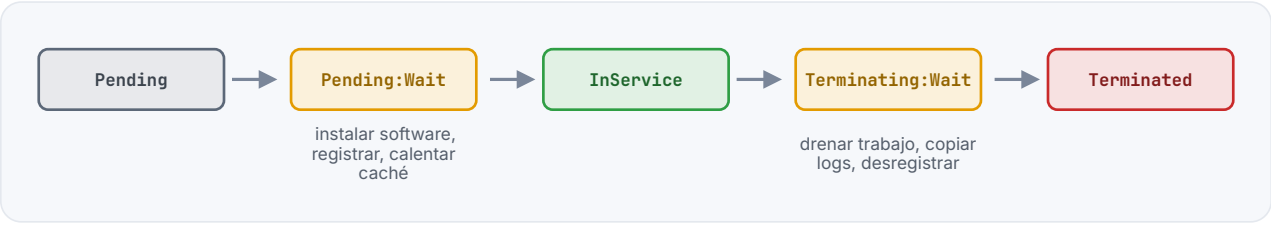


Figura 44 · Los *lifecycle hooks* pausan una instancia al entrar o salir del grupo para que ejecutes acciones propias (por defecto, hasta 1 hora) antes de continuar.

- **Lifecycle hooks**: la instancia espera en `Pending:Wait` o `Terminating:Wait` hasta que tu script o una función Lambda llaman a `complete-lifecycle-action`, o vence el plazo.
- **Instance refresh**: sustituye las instancias por tandas para aplicar una nueva AMI o plantilla, manteniendo un mínimo sano (90 % por defecto), con vuelta atrás automática si la activas.
- **Warm pools**: instancias ya inicializadas y paradas (o en marcha, o hibernadas) que el ASG usa al escalar para evitar arranques lentos; las paradas solo cuestan su EBS.
- **Terminación**: al reducir, el ASG elige la AZ con más instancias y, dentro de ella, por defecto, la de plantilla más antigua. Puedes proteger instancias con *scale-in protection*.

Mezclar tipos de instancia y Spot

Con una **mixed instances policy**, un mismo ASG combina varios tipos de instancia y modelos de compra: una base On-Demand fija y, por encima, un porcentaje On-Demand y el resto en Spot, diversificado entre tipos y AZ. **Capacity Rebalancing** sustituye por adelantado las instancias Spot con riesgo alto de interrupción.

```json
"MixedInstancesPolicy": {
  "LaunchTemplate": {
    "LaunchTemplateSpecification": { "LaunchTemplateName": "web",
                                     "Version": "$Latest" },
    "Overrides": [{ "InstanceType": "m7g.large" }, { "InstanceType": "m6g.large" },
                  { "InstanceType": "c7g.xlarge" }] },
  "InstancesDistribution": { "OnDemandBaseCapacity": 2,
    "OnDemandPercentageAboveBaseCapacity": 25,
    "SpotAllocationStrategy": "price-capacity-optimized" } }
```

Las dos primeras instancias son On-Demand; por encima, una de cada cuatro es On-Demand y tres son Spot, repartidas entre tres tipos Graviton (la AMI debe ser `arm64`). Cuantos más tipos y AZ admitas,

menos interrupciones sufrirás; si además cubres la base On-Demand con Savings Plans (capítulo de EC2), la flota cuesta una fracción de una flota fija dimensionada para el pico.

6.9 Application Auto Scaling

Application Auto Scaling aplica el mismo modelo (recurso con mínimo y máximo, y políticas de target tracking, step o programadas) a recursos que no son instancias EC2: tareas de servicios ECS, capacidad de tablas DynamoDB, réplicas de Aurora, concurrencia aprovisionada de Lambda, endpoints de SageMaker o Spot Fleet, entre otros. Lo veremos con ECS en el capítulo de Contenedores.

RESUMEN DEL CAPÍTULO

- Un balanceador da un único punto de entrada, reparte el tráfico entre targets de varias AZ y aparta los que fallan sus health checks.
- **ALB** (capa 7) para HTTP: reglas por host, ruta, cabeceras, método o query string, y acciones `forward`, `redirect`, `fixed-response` y `authenticate`. **NLB** (capa 4) para TCP/UDP, IP estática y PrivateLink. **GWLB** (capa 3) para appliances. El CLB es heredado.
- Listener → reglas → target groups → targets (instancias, IP, Lambda o un ALB detrás de un NLB).
- Health checks superficiales; si todos fallan, el ALB hace fail open.
- TLS con certificados de ACM (en la región del balanceador) y SNI para varios dominios en un listener.
- Cross-zone siempre activo en ALB; opcional y con coste entre AZ en NLB y GWLB. Deregistration delay (300 s por defecto), slow start, stickiness y `X-Forwarded-For`.
- Un ASG mantiene la capacidad deseada entre un mínimo y un máximo, en varias AZ, a partir de una launch template; target tracking es la política por defecto.
- Health checks de ELB en el ASG, lifecycle hooks, instance refresh y warm pools para operar flotas; mixed instances para combinar On-Demand y Spot.

Preguntas de repaso

1. **Tu aplicación web necesita enrutar `/api/*` a un servicio y el resto a otro. ¿Qué balanceador usas?**
   Respuesta: Un ALB, con una regla `path-pattern` para `/api/*` que reenvíe a un target group y la regla por defecto apuntando al otro.
2. **Un cliente necesita incluir en su firewall las IP fijas de tu servicio TCP. ¿Qué haces?**
   Respuesta: Usar un NLB, que tiene una IP estática por AZ y admite Elastic IP (o poner Global Accelerator delante).
3. **Tras un despliegue, los usuarios ven errores al retirar instancias antiguas. ¿Qué ajuste revisas?**
   Respuesta: El *deregistration delay* del target group, para que las peticiones en curso terminen antes de retirar el target, además de los health checks.
4. **Tienes 2 targets en una AZ y 8 en otra detrás de un NLB, y los dos primeros están saturados. ¿Por qué?**
   Respuesta: En NLB el cross-zone viene desactivado: cada AZ recibe la mitad del tráfico, así que cada target de la primera AZ recibe el 25 %. Activa cross-zone o equilibra los targets.
5. **Tu ASG no sustituye instancias cuyo nginx se ha caído. ¿Qué falta?**
   Respuesta: Activar los health checks de ELB en el ASG; por defecto solo usa los de EC2, que solo comprueban que la máquina funciona.
6. **¿Qué política de escalado eliges para mantener la CPU media en torno al 50 %?**
   Respuesta: Target tracking sobre la métrica `ASGAverageCPUUtilization` con objetivo 50; el ASG crea y gestiona las alarmas.

CAPÍTULO 7

Contenedores: ECS, Fargate, ECR y EKS

Empaqueta tu aplicación una vez y ejecútala en cualquier parte: registro de imágenes, orquestación y cómputo sin servidores para contenedores.

EN ESTE CAPÍTULO APRENDERÁS

1. Imágenes, contenedores y el registro Amazon ECR
2. Clúster, task definition, tarea y servicio en ECS
3. EC2, Fargate y Fargate Spot como capacidad
4. Modos de red e IAM: execution role frente a task role
5. Despliegues, balanceo, escalado, secretos y logs
6. Cuándo elegir ECS, EKS o App Runner

SERVICIOS PROTAGONISTAS

Amazon ECS · AWS Fargate · Amazon ECR · AWS Cloud Map · Amazon EKS · AWS App Runner

7.1 Contenedores en dos minutos

«En mi máquina funciona» es el problema clásico: la aplicación depende de una versión concreta del lenguaje, de librerías del sistema y de ficheros de configuración que nunca son idénticos en desarrollo, pruebas y producción. Un **contenedor** empaqueta la aplicación con todas sus dependencias en una unidad que se ejecuta igual en cualquier sitio. A diferencia de una máquina virtual, no lleva un sistema operativo completo: comparte el kernel del host y se aísla con mecanismos del propio kernel, así que arranca en segundos y ocupa megas en lugar de gigas.



Figura 45 · Una máquina virtual incluye un sistema operativo completo; los contenedores comparten el kernel del host y solo empaquetan la aplicación y sus librerías.

Imagen — plantilla inmutable de solo lectura, formada por capas, que contiene la aplicación y sus dependencias. Sigue el estándar OCI y se identifica por repositorio y etiqueta (`web-api:1.4.2`) o por *digest* (`sha256:...`).
Contenedor — una imagen en ejecución, con su sistema de ficheros efímero, sus procesos y su red.
Registro — almacén de imágenes del que se descargan (*pull*) y al que se suben (*push*): Docker Hub, Amazon ECR...
Orquestador — decide dónde se ejecuta cada contenedor, lo reinicia si falla, lo conecta al balanceador y lo escala: Amazon ECS o Kubernetes (Amazon EKS).

DOCKERFILE

```dockerfile
# Etapa 1: compilar con todas las herramientas
FROM golang:1.23 AS build
WORKDIR /src
COPY . .
RUN CGO_ENABLED=0 go build -o /app ./cmd/api
# Etapa 2: imagen final mínima, sin compiladores ni shell
FROM gcr.io/distroless/static-debian12
COPY --from=build /app /app
USER nonroot
EXPOSE 8080
ENTRYPOINT ["/app"]
```

CONSEJO

Las construcciones en varias etapas (*multi-stage builds*) como esta producen imágenes pequeñas: menos superficie de ataque, menos vulnerabilidades que parchear y descargas más rápidas al escalar. Y con `USER nonroot` el proceso no se ejecuta como `root` dentro del contenedor.

7.2 Amazon ECR: el registro de imágenes

FICHA DEL SERVICIO

Amazon ECR

Registro de imágenes de contenedor totalmente gestionado

TIPO	Registro privado por cuenta y región (y ECR Public para imágenes públicas); un repositorio por imagen
SEGURIDAD	Permisos con IAM y políticas de repositorio; cifrado en reposo con AES-256 o KMS
FACTURACIÓN	Por GB almacenado al mes y por transferencia de datos de salida (gratuita dentro de la misma región)
INTEGRACIÓN	ECS, EKS, Lambda (imágenes de contenedor), App Runner, CodeBuild y Amazon Inspector

Etiquetas inmutables — con `IMMUTABLE`, nadie puede sobrescribir `web-api:1.4.2` con otro contenido. Despliega siempre etiquetas concretas, nunca `latest`.
Escaneo de vulnerabilidades — el básico analiza paquetes del sistema operativo al subir la imagen; el mejorado, con Amazon Inspector, escanea de forma continua también las dependencias de los lenguajes.
Lifecycle policies — borran automáticamente imágenes antiguas o sin etiqueta (por ejemplo, «las no etiquetadas a los 14 días; de las demás, conserva las 50 más recientes»), para no pagar almacenamiento de más.
Replicación y *pull through cache* — copian imágenes a otras regiones o cuentas, y cachean en tu registro imágenes de registros públicos (Docker Hub, Quay, GitHub, ECR Public...).

BASH

```
REG=123456789012.dkr.ecr.eu-west-1.amazonaws.com
aws ecr create-repository --repository-name web-api \
  --image-tag-mutability IMMUTABLE --image-scanning-configuration scanOnPush=true
# Docker se autentica con un token temporal (válido 12 horas)
aws ecr get-login-password --region eu-west-1 \
  | docker login --username AWS --password-stdin "$REG"
docker build -t "$REG/web-api:1.4.2" . && docker push "$REG/web-api:1.4.2"
```

7.3 Amazon ECS: conceptos clave

Amazon Elastic Container Service (ECS) es el orquestador de contenedores propio de AWS: tú describes qué contenedores quieres y cuántas copias, y ECS decide dónde ejecutarlos, los vigila, los sustituye si fallan y los registra en el balanceador.

FICHA DEL SERVICIO

Amazon ECS

Orquestación de contenedores totalmente gestionada

PLANO DE CONTROL	Gestionado por AWS y sin coste: pagas solo la capacidad (instancias EC2 o Fargate)
ÁMBITO	Regional; las tareas se reparten entre las AZ de las subredes que elijas
CAPACIDAD	AWS Fargate (sin servidores) o instancias EC2 propias con el agente de ECS
INTEGRACIÓN	ELB, Auto Scaling, IAM, CloudWatch, Secrets Manager, Cloud Map, CodeDeploy

Clúster — agrupación lógica de servicios, tareas y capacidad. Puede mezclar Fargate e instancias EC2.

Task definition — plantilla JSON **versionada** (familia + revisión, como `web-api:7`) que describe uno o varios contenedores: imagen, CPU y memoria, puertos, variables, secretos, logs, roles IAM y modo de red.
Tarea — una instancia en ejecución de una task definition. Sus contenedores comparten ciclo de vida y, en `awsvpc`, la misma IP.
Servicio — mantiene N tareas en marcha (*desired count*), sustituye las que fallan, gestiona los despliegues y las registra en un balanceador.
Agente de ECS — proceso que corre en cada instancia EC2 del clúster, la registra y arranca o para sus contenedores. En Fargate lo gestiona AWS.

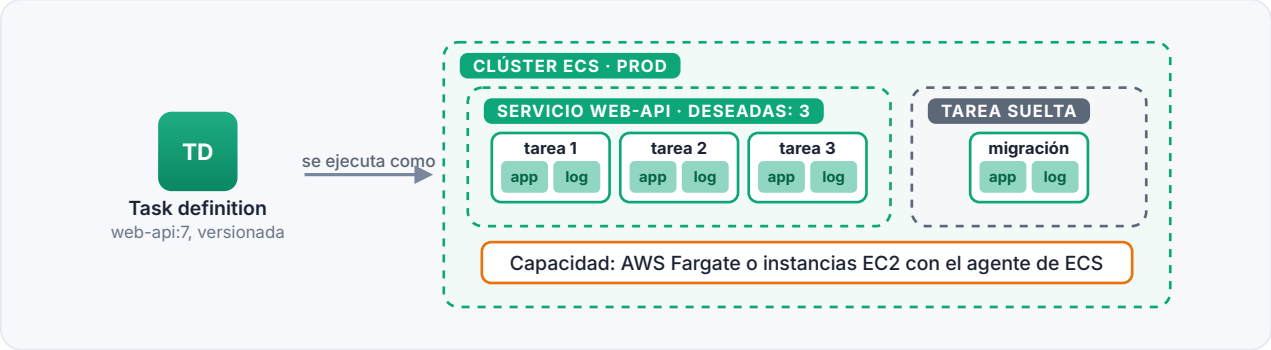


Figura 46 · Jerarquía de ECS: un servicio mantiene varias copias de una task definition en un clúster; también puedes lanzar tareas sueltas, como una migración de base de datos.

Así se ve una task definition para Fargate con un contenedor, un secreto y logs en CloudWatch:

```json
{
  "family": "web-api", "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "512", "memory": "1024",
  "runtimePlatform": { "cpuArchitecture": "ARM64",
                       "operatingSystemFamily": "LINUX" },
  "executionRoleArn": "arn:aws:iam::123456789012:role/ecsTaskExecutionRole",
  "taskRoleArn": "arn:aws:iam::123456789012:role/web-api-task",
  "containerDefinitions": [{
    "name": "api", "essential": true,
    "image": "123456789012.dkr.ecr.eu-west-1.amazonaws.com/web-api:1.4.2",
    "portMappings": [{ "containerPort": 8080 }],
    "environment": [{ "name": "APP_ENV", "value": "prod" }],
    "secrets": [{ "name": "DB_PASSWORD", "valueFrom":
      "arn:aws:secretsmanager:eu-west-1:123456789012:secret:prod/db-AbCdEf" }],
    "logConfiguration": { "logDriver": "awslogs",
      "options": { "awslogs-group": "/ecs/web-api",
                   "awslogs-region": "eu-west-1",
                   "awslogs-stream-prefix": "api" } }
  }]
}
```

7.4 Capacidad: EC2, Fargate y Fargate Spot

Las tareas necesitan CPU y memoria de algún sitio. Con **ECS sobre EC2** las pones tú: un grupo de Auto Scaling con la AMI optimizada para ECS, que también debes parchear y dimensionar. Con **AWS Fargate** no hay servidores a la vista: declaras la CPU y la memoria de cada tarea y AWS la ejecuta en un entorno aislado, facturando por segundo. **Fargate Spot** usa capacidad sobrante con descuento, a cambio de poder interrumpir la tarea. Fargate admite también Graviton (`ARM64`, como la task definition anterior), más barato, y los Compute Savings Plans cubren su uso. Desde 2025 existe además un punto intermedio, **ECS Managed Instances**: ECS lanza y mantiene por ti instancias EC2 en tu cuenta (elección del tipo, parches, sustitución) y conservas toda la gama de EC2, GPU incluidas.

0,25–16	120 GB	200 GiB	70 %
vCPU por tarea en Fargate	memoria máxima por tarea	almacenamiento efímero máximo (20 por defecto)	descuento máximo de Fargate Spot

Característica	ECS sobre EC2	Fargate	Fargate Spot
Servidores	Los gestionas tú (AMI, parches, escalado)	Los gestiona AWS	Los gestiona AWS
Facturación	Instancias EC2, aunque estén medio vacías	vCPU y GB de memoria por segundo (mínimo 1 min)	Igual, con hasta un 70 % de descuento
Interrupciones	No (salvo que uses Spot de EC2)	✘	Sí: aviso de 2 minutos y `SIGTERM`
Modos de red	`awsvpc`, `bridge`, `host`, `none`	Solo `awsvpc`	Solo `awsvpc`
GPU y tipos concretos	✔	✘	✘
Tareas daemon	✔ Una por instancia	✘	✘
Ideal para	Flotas grandes y estables, GPU, requisitos especiales	La opción por defecto: sin servidores que mantener	Lotes, *workers* y entornos no productivos

Los **capacity providers** deciden dónde va cada tarea. Un servicio puede usar una estrategia que combine varios: con `FARGATE` (base 2, peso 1) y `FARGATE_SPOT` (peso 3), las dos primeras tareas van a Fargate normal y, de las siguientes, una de cada cuatro va a Fargate y tres a Spot. Con EC2, un capacity provider asociado a un ASG escala las instancias según las tareas pendientes (*managed scaling*) y evita terminar instancias que aún ejecutan tareas.

7.5 Redes de las tareas

El **modo de red** de la task definition decide cómo se conectan los contenedores (Figura 47). **awsvpc** es el modo recomendado (y el único en Fargate): cada tarea recibe su propia ENI con IP privada de la subred, así que aplicas Security Groups por tarea y ves su tráfico en los VPC Flow Logs. En **bridge**, los contenedores usan una red virtual dentro de la instancia y se publican en puertos del host; si dejas el puerto del host vacío (o a 0), Docker asigna uno alto aleatorio: es el *dynamic port mapping*, que permite varias copias de la misma tarea en una instancia. **host** elimina esa capa y da el máximo rendimiento de red, pero dos tareas no pueden usar el mismo puerto en la misma instancia.

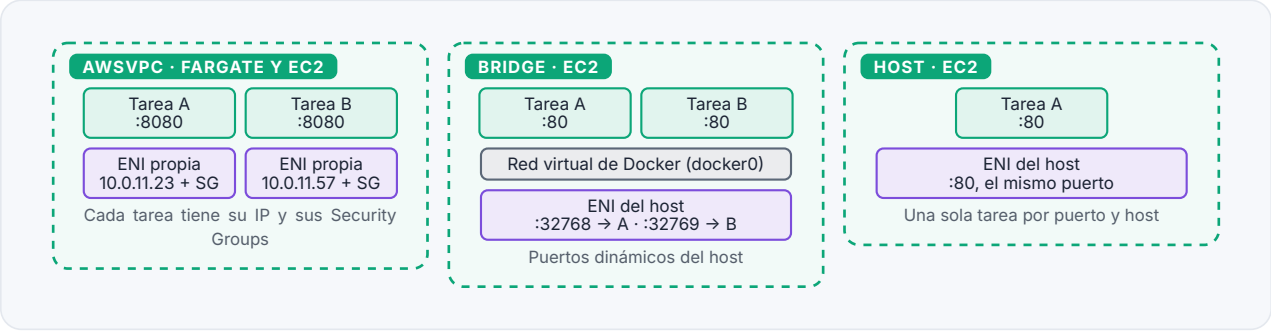


Figura 47 · Modos de red de ECS. En `awsvpc` cada tarea es un ciudadano más de la VPC; en `bridge` las tareas comparten la red del host mediante puertos dinámicos; en `host` usan directamente los puertos del host.

CONSEJO

En EC2 con `awsvpc`, el número de tareas por instancia está limitado por las ENI que admite el tipo de instancia. Activa el **ENI trunking** (ajuste de cuenta `awsvpcTrunking`) para multiplicarlo en los tipos compatibles.

7.6 IAM en ECS: execution role y task role

Una tarea puede llevar dos roles IAM distintos (qué es un rol y cómo se asume se explica en el capítulo «IAM (I)»), y confundirlos es uno de los errores más habituales en ECS:

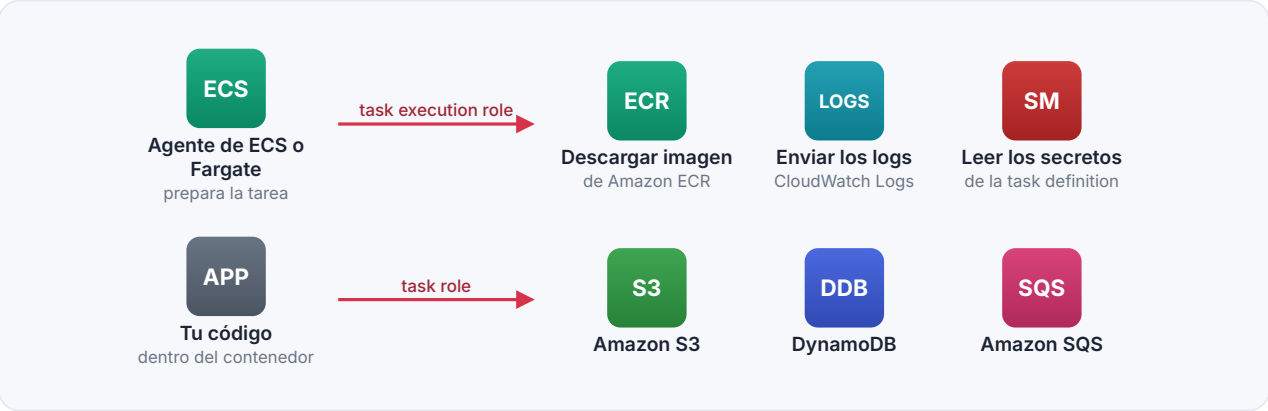


Figura 48 · El *task execution role* lo usa ECS para preparar la tarea; el *task role* lo usa tu código. Ambos confían en el principal `ecs-tasks.amazonaws.com`.

Task execution role — lo asume el agente de ECS (o Fargate) **antes** de arrancar tus contenedores: descarga la imagen de ECR, crea el flujo de logs y lee los secretos referenciados en la task definition. La política gestionada `AmazonECSTaskExecutionRolePolicy` cubre ECR y CloudWatch Logs; los secretos requieren permisos adicionales.
Task role — lo usa **tu aplicación** para llamar a otros servicios. El SDK obtiene credenciales temporales de un endpoint local de la tarea, igual que una instancia EC2 las obtiene del IMDS. Usa un rol por servicio, con el mínimo privilegio.
Rol de instancia — solo en ECS sobre EC2; permite al agente registrar la instancia en el clúster. No lo uses para dar permisos a las aplicaciones.

ERROR TÍPICO

Tu código recibe `AccessDenied` al leer de S3: le diste el permiso al **execution role**, pero la aplicación usa el **task role**. Y al revés: si la tarea ni siquiera arranca (`CannotPullContainerError` o `ResourceInitializationError`), revisa el **execution role** y que la subred tenga salida hacia ECR y Secrets Manager (NAT Gateway o VPC endpoints).

7.7 Servicios: despliegue, balanceo y escalado

Un servicio mantiene el número deseado de tareas repartidas entre AZ. La estrategia `REPLICA` (la habitual) mantiene N copias; `DAEMON` ejecuta exactamente una por instancia EC2, útil para agentes de monitorización (no existe en Fargate). En EC2 puedes además influir en la colocación con estrategias (`spread` por AZ, `binpack` para llenar instancias) y restricciones.

Rolling update
Por defecto

ECS sustituye las tareas por tandas, entre `minimumHealthyPercent` (100 % por defecto) y `maximumPercent` (200 %). Con el **circuit breaker**, si las tareas nuevas no se estabilizan, vuelve solo a la versión anterior.

Blue/green
Con CodeDeploy o nativo

Levanta la versión nueva junto a la antigua con un segundo target group y desplaza el tráfico de golpe, por porcentajes (*canary*) o poco a poco (*linear*), con vuelta atrás inmediata. Desde 2025 ECS lo ofrece también de forma nativa.

Alarmas
Vuelta atrás por métricas

Asocia alarmas de CloudWatch al despliegue (errores 5XX, latencia): si saltan durante el despliegue, ECS revierte automáticamente.

Para recibir tráfico, el servicio se registra en un **target group** del balanceador (capítulo de Balanceo de carga y Auto Scaling). En `awsvpc`, el target group es de tipo `ip` y registra la IP de cada tarea (obligatorio en Fargate); en `bridge`, de tipo `instance`, y el ALB registra cada instancia con el puerto dinámico de cada tarea. El Security Group de las tareas debe permitir el puerto del contenedor **solo desde el Security Group del ALB**, y el *health check grace period* evita que ECS mate tareas que aún están arrancando.

```bash
NET='awsvpcConfiguration={subnets=[subnet-0a1b,subnet-0c2d],securityGroups=[sg-0e3f]}'
aws ecs create-service --cluster prod --service-name web-api \
  --task-definition web-api:7 --desired-count 4 \
  --capacity-provider-strategy capacityProvider=FARGATE,base=2,weight=1 \
      capacityProvider=FARGATE_SPOT,weight=3 \
  --network-configuration "$NET" \
  --load-balancers targetGroupArn=$TG_ARN,containerName=api,containerPort=8080 \
  --deployment-configuration 'deploymentCircuitBreaker={enable=true,rollback=true}'
```

El número de tareas se escala con **Application Auto Scaling** (también en el capítulo de Balanceo de carga y Auto Scaling): registras el servicio como recurso escalable y le asocias una política, normalmente de target tracking sobre la CPU media (`ECSServiceAverageCPUUtilization`), la memoria o las peticiones por target del ALB.

```bash
aws application-autoscaling register-scalable-target --service-namespace ecs \
  --scalable-dimension ecs:service:DesiredCount \
  --resource-id service/prod/web-api --min-capacity 2 --max-capacity 20
# Después, put-scaling-policy con una política de target tracking (CPU al 60 %)
```

7.8 Comunicación entre servicios

Cuando un servicio llama a otro (la web a la API de pagos, por ejemplo), necesita encontrarlo aunque sus tareas cambien de IP en cada despliegue. Hay tres opciones:

	Service Discovery	Service Connect	ALB interno
Cómo se invoca	Nombre DNS en AWS Cloud Map: `pagos.prod.local`	Alias corto: `http://pagos:8080`	Nombre DNS del ALB
Quién reparte	El cliente, con lo que devuelve el DNS	Un proxy que ECS añade a cada tarea	El balanceador
Extras	Registro y health checks automáticos	Reintentos, detección de destinos con fallos y métricas por servicio en CloudWatch	Reglas HTTP, TLS, WAF

Service Connect es la opción recomendada para comunicar servicios ECS entre sí: usa un *namespace* de Cloud Map, pero los clientes no dependen del DNS ni de su caché. AWS App Mesh, la malla de servicios que se usaba antes, llegó al fin de su soporte el 30 de septiembre de 2026.

7.9 Configuración, logs y depuración

Secretos y configuración. Nunca metas contraseñas en la imagen ni en variables `environment` en claro. En la sección `secrets` de la task definition referencias un secreto de **Secrets Manager** (por ARN, incluso una clave concreta de un secreto JSON) o un parámetro de **SSM Parameter Store**: ECS lo lee con el execution role y lo inyecta como variable de entorno al arrancar el contenedor. Si cambias el secreto, las tareas en marcha no se enteran: fuerza un nuevo despliegue. Para mucha configuración no sensible puedes usar `environmentFiles` (un fichero `.env` en S3).

Logs. El driver `awslogs` envía la salida estándar de cada contenedor a CloudWatch Logs (grupo, región y prefijo del *stream* en `logConfiguration`). Para enviarlos a varios destinos (S3, Amazon Data Firehose, OpenSearch o herramientas de terceros), **FireLens** añade a la tarea un contenedor con Fluent Bit que enruta los logs (driver `awsfirelens`). Fija una retención en cada grupo de logs (por defecto se guardan para siempre) y activa **Container Insights** para tener métricas de CPU, memoria y red por servicio y tarea; lo ampliaremos en el capítulo de Observabilidad.

ECS Exec. Para entrar en un contenedor en ejecución sin SSH ni puertos abiertos, ECS Exec usa por debajo el canal de Session Manager. Requiere activar `enableExecuteCommand` en el servicio, permisos `ssmmessages` en el task role y el plugin de Session Manager en tu equipo; cada sesión queda registrada en CloudTrail.

```bash
aws ecs update-service --cluster prod --service web-api \
  --enable-execute-command --force-new-deployment
aws ecs execute-command --cluster prod --task "$TASK_ID" --container api \
  --interactive --command "/bin/sh"
```

7.10 Arquitectura de referencia

Juntando todas las piezas, este es el despliegue típico de una API en contenedores: un ALB público, un servicio ECS en Fargate repartido en subredes privadas de dos AZ y una base de datos RDS Multi-AZ, con acceso privado a ECR, CloudWatch Logs y Secrets Manager mediante VPC endpoints.

> **$ COSTES**
>
> Si las tareas de subredes privadas descargan imágenes a través de un NAT Gateway, pagas su procesamiento por GB en cada despliegue y cada escalado. Los VPC endpoints de ECR (`ecr.api` y `ecr.dkr`), el endpoint *gateway* de S3 (gratuito, ahí están las capas) y el de CloudWatch Logs suelen salir más baratos y mantienen el tráfico dentro de AWS.

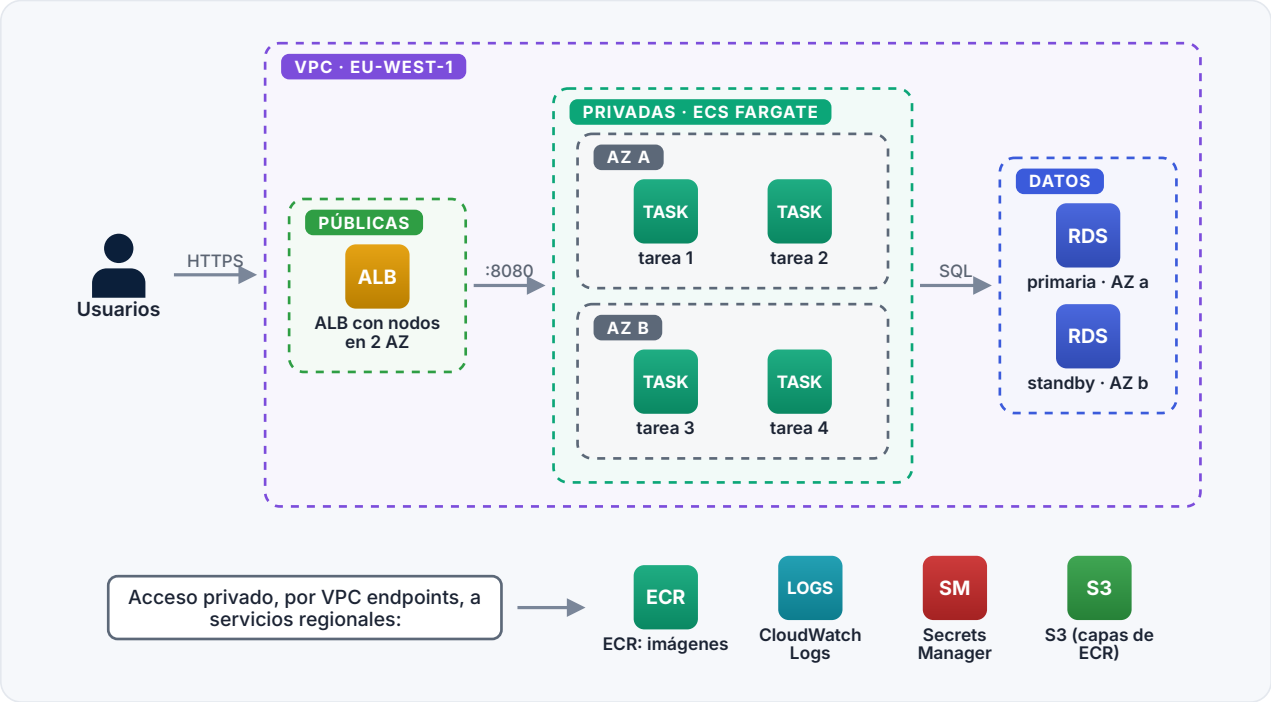


Figura 49 · Arquitectura de referencia: ALB público → servicio ECS en Fargate en subredes privadas → RDS. Las tareas descargan la imagen de ECR, envían logs y leen secretos a través de VPC endpoints, sin salir a Internet.

7.11 ¿ECS, EKS o App Runner?

Amazon Elastic Kubernetes Service (EKS) es la otra gran opción para orquestar contenedores en AWS: Kubernetes gestionado. AWS opera el plano de control de Kubernetes, el orquestador de código abierto más extendido, y tú eliges dónde corren los contenedores (EC2, Fargate o EKS Auto Mode). Lo estudiaremos a fondo en el capítulo «Kubernetes y Amazon EKS»; aquí basta con saber cuándo conviene cada uno:

Criterio	Amazon ECS	Amazon EKS
Modelo	Orquestador propio de AWS, con una API sencilla	Kubernetes estándar y certificado
Curva de aprendizaje	Baja: pocos conceptos	Alta: Kubernetes y su ecosistema
Plano de control	Sin coste	Tarifa por clúster y hora
Portabilidad	Ligado a AWS	Manifiestos portables a otros Kubernetes, on-premises o en otras nubes
Ecosistema	Integración nativa con los servicios de AWS	Helm, operadores y herramientas CNCF
Mantenimiento	Sin versiones que actualizar	Actualizar la versión de Kubernetes y sus complementos
Elígelo si	Quieres simplicidad y todo tu entorno está en AWS	Ya usas Kubernetes, necesitas portabilidad o su ecosistema

En el otro extremo está **AWS App Runner**, aún más sencillo: le das una imagen de contenedor (o el código fuente) y se encarga de construir, desplegar, servir con HTTPS, escalar según las peticiones y conectarse a tu VPC. Es cómodo para webs y API sin equipo de plataforma, a cambio de mucho menos control sobre la red y el despliegue; antes de elegirlo para un proyecto nuevo, revisa su estado y sus límites en la documentación.

RESUMEN DEL CAPÍTULO

- Un contenedor empaqueta la aplicación y sus dependencias y comparte el kernel del host; nace de una imagen OCI inmutable.
- **ECR** guarda las imágenes con etiquetas inmutables, escaneo de vulnerabilidades, lifecycle policies y replicación.
- En **ECS**, una **task definition** versionada describe los contenedores; una **tarea** es una ejecución y un **servicio** mantiene N tareas detrás de un balanceador.
- La capacidad la ponen instancias EC2 propias o **Fargate** (sin servidores, por segundo); **Fargate Spot** abarata hasta un 70 % con aviso de 2 minutos.
- `awsvpc` da a cada tarea su ENI y sus Security Groups (único modo en Fargate); `bridge` usa puertos dinámicos y `host`, los del host.
- El **execution role** prepara la tarea (ECR, logs, secretos); el **task role** da permisos a tu código. Confundirlos es el error más común.
- Despliegues rolling con circuit breaker, blue/green y vuelta atrás por alarmas; target groups de tipo `ip`; escalado con Application Auto Scaling; **Service Connect** entre servicios.
- Secretos inyectados desde Secrets Manager o Parameter Store; logs a CloudWatch con `awslogs` o FireLens.
- **ECS** es lo más sencillo y no cobra el plano de control; **EKS** ofrece Kubernetes estándar y portable; **App Runner**, la máxima sencillez para webs y API.

Preguntas de repaso

1. **Tu aplicación en ECS recibe `AccessDenied` al escribir en DynamoDB, aunque el execution role tiene el permiso. ¿Qué pasa?**
   Respuesta: Los permisos de la aplicación van en el **task role**; el execution role solo lo usa ECS para descargar la imagen, enviar logs y leer los secretos.
2. **¿Qué tipo de target group necesitas para un servicio en Fargate detrás de un ALB?**
   Respuesta: De tipo `ip`, porque en `awsvpc` (el único modo de Fargate) cada tarea tiene su propia IP.
3. **¿Qué diferencia hay entre una tarea y un servicio en ECS?**
   Respuesta: Una tarea es una ejecución de una task definition; un servicio mantiene un número deseado de tareas, sustituye las que fallan, gestiona despliegues y las registra en el balanceador.
4. **¿Cuándo elegirías ECS sobre EC2 en lugar de Fargate?**
   Respuesta: Cuando necesitas GPU o tipos de instancia concretos, tareas daemon, acceso al host, o una flota grande y estable en la que llenar instancias sale más barato.
5. **Un despliegue deja tareas que se reinician en bucle. ¿Cómo evitas que el servicio se quede roto?**
   Respuesta: Activando el deployment circuit breaker con `rollback`: ECS detecta que las tareas nuevas no se estabilizan y vuelve a la versión anterior.
6. **¿Cómo inyectas la contraseña de la base de datos en un contenedor sin incluirla en la imagen?**
   Respuesta: Con la sección `secrets` de la task definition, apuntando a Secrets Manager o Parameter Store; ECS la lee con el execution role y la expone como variable de entorno.
7. **Tu equipo ya usa Kubernetes en su centro de datos y quiere portabilidad. ¿ECS o EKS?**
   Respuesta: EKS: es Kubernetes estándar, así que reutilizan manifiestos, Helm y herramientas, aunque pagan el plano de control y deben gestionar las actualizaciones de versión.

CAPÍTULO 8

Kubernetes y Amazon EKS

Del modelo declarativo de Kubernetes a un clúster gestionado en AWS: pods, servicios, escalado, redes, almacenamiento e identidades.

EN ESTE CAPÍTULO APRENDERÁS

1. Qué problema resuelve Kubernetes y su modelo declarativo
2. La arquitectura de un clúster: control plane y nodos
3. Pods, Deployments, Services y el resto de objetos esenciales
4. Qué gestiona AWS en EKS y cómo elegir el data plane
5. Redes, balanceadores y almacenamiento en EKS
6. IAM en EKS: acceso al clúster y permisos de los pods

SERVICIOS PROTAGONISTAS

Amazon EKS · EKS Auto Mode · AWS Fargate · Amazon EC2 · Elastic Load Balancing · Amazon EBS · Amazon EFS

8.1 ¿Qué problema resuelve Kubernetes?

En el capítulo de Contenedores viste cómo ejecutar aplicaciones en contenedores con ECS y Fargate, y cuándo conviene EKS. Cuando tienes cientos de contenedores repartidos entre decenas de máquinas surgen preguntas difíciles: ¿dónde coloco cada uno?, ¿quién lo reinicia si se cae?, ¿cómo actualizo sin cortar el servicio?, ¿cómo se encuentran entre sí si sus IP cambian? Responderlas es **orquestar**, y **Kubernetes** («timonel» en griego, abreviado **K8s**) es el orquestador de código abierto más extendido: nació en Google, es abierto desde 2014, lo custodia la Cloud Native Computing Foundation (CNCF) y funciona igual en tu portátil, en tu centro de datos o en cualquier nube. Se encarga de:

- **Planificar**: decide en qué máquina (**nodo**) corre cada contenedor, según los recursos que pide.
- **Autorreparar**: reinicia lo que falla y recrea en otro nodo lo que corría en un nodo caído.
- **Escalar y desplegar sin corte**, con marcha atrás si algo sale mal.
- **Descubrir y balancear**: nombres DNS estables delante de contenedores efímeros.

El modelo declarativo y el bucle de reconciliación

Kubernetes es **declarativo**: no le das órdenes («arranca tres contenedores»), sino que describes el resultado («que siempre haya tres copias de esta imagen») en un fichero YAML llamado **manifiesto**. Kubernetes guarda ese **estado deseado** y unos procesos llamados **controladores** lo comparan sin descanso con el **estado actual**; si difieren, actúan para igualarlos. Es el **bucle de reconciliación**.

> **ANALOGÍA**
>
> Un termostato. Fijas 21 °C (estado deseado); él mide la temperatura (estado actual) y enciende o apaga la calefacción hasta igualarlas, también si alguien abre una ventana. No le dices «enciende la caldera diez minutos», sino el resultado. Si un nodo muere con un contenedor dentro, Kubernetes «nota el frío» y lo recrea en otro nodo.



Figura 50 · El bucle de reconciliación: el controlador compara sin descanso el estado deseado con el actual y actúa hasta igualarlos.

8.2 Arquitectura de un clúster

Un **clúster** tiene dos partes: el **control plane** (plano de control), el cerebro que guarda el estado y toma las decisiones, y los **nodos** (*worker nodes*), las máquinas (en AWS, instancias EC2) donde corren los contenedores. Kubernetes no coloca contenedores sueltos, sino **pods**: grupos de uno o varios contenedores que se despliegan juntos y comparten IP (los verás en detalle en la sección siguiente). Y tú hablas con el clúster mediante **kubectl**, su herramienta de línea de comandos. Estos son los componentes (figura siguiente):

kube-apiserver — la puerta de entrada; expone la API y autentica, autoriza y valida cada petición.
etcd — base de datos clave-valor distribuida con todo el estado del clúster; solo el API server accede a ella.
kube-scheduler — elige en qué nodo se ejecuta cada pod nuevo, según los recursos que pide, sus restricciones y el hueco libre.

kube-controller-manager — ejecuta los controladores integrados (réplicas, nodos, Jobs…).
cloud-controller-manager — la integración con la nube (nodos que desaparecen, balanceadores…).
kubelet — el agente de cada nodo; arranca los contenedores de sus pods y vigila su salud.
kube-proxy — programa en cada nodo las reglas de red que hacen funcionar los Services (la IP estable delante de un grupo de pods).
Container runtime — ejecuta los contenedores; normalmente **containerd**, a través de la interfaz CRI.

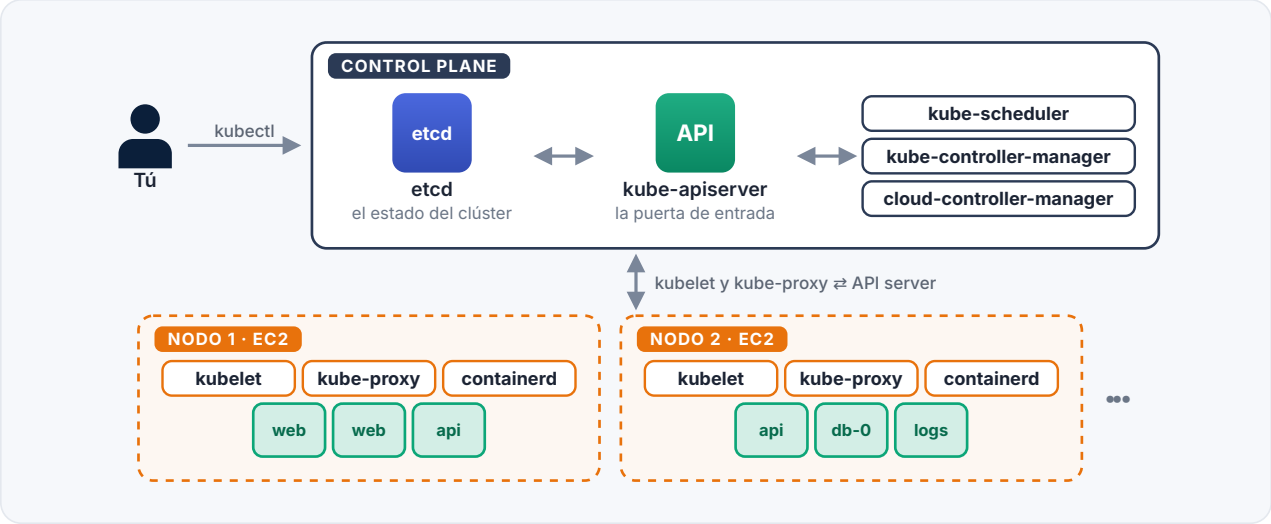


Figura 51 · Arquitectura de un clúster. Todos los componentes se comunican a través del API server, y solo él lee y escribe en etcd.

8.3 Los objetos fundamentales

Todo en Kubernetes es un **objeto** de la API descrito en YAML, con un tipo (`kind`), unos metadatos (nombre, namespace, labels) y una especificación (`spec`) con el estado deseado. La figura resume cómo se conectan los principales.

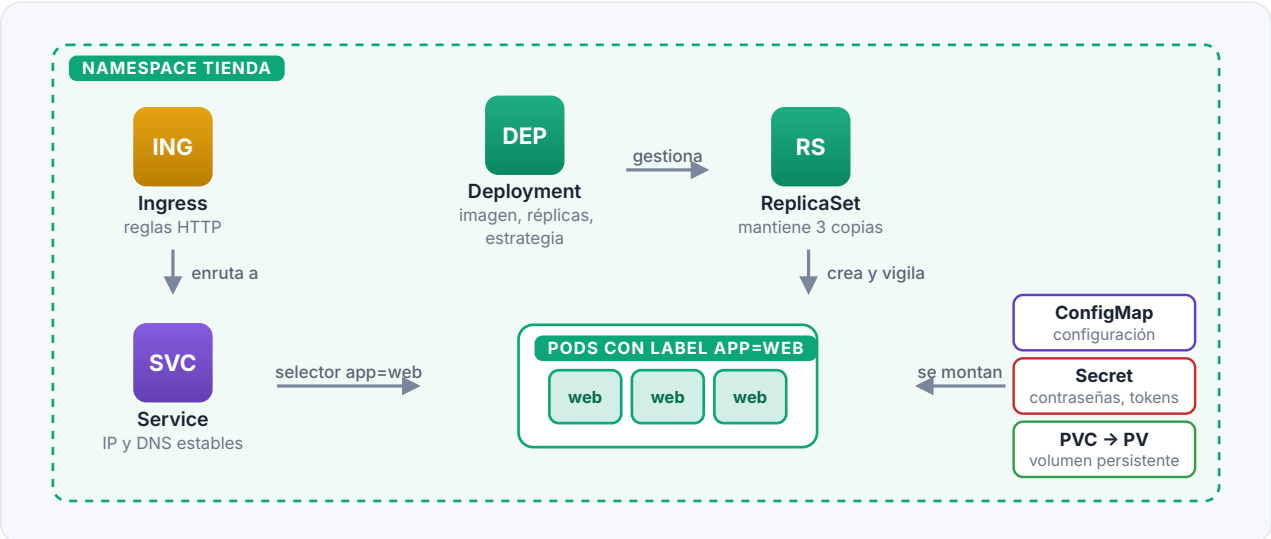


Figura 52 · Cómo se relacionan los objetos: el Deployment gestiona un ReplicaSet, que mantiene los pods; el Service los encuentra por su label y el Ingress le envía el tráfico HTTP; la configuración y los volúmenes se montan en los pods.

Pods y controladores de carga de trabajo

Un **Pod** es la unidad mínima de despliegue: uno o varios contenedores que comparten IP y volúmenes y corren juntos en un nodo (normalmente uno principal y, a veces, un **sidecar** auxiliar). Los pods son efímeros: si mueren, un controlador crea otros, con otra IP. Por eso se gestionan con estos objetos:

Objeto	Qué garantiza (y ejemplo típico)
Deployment	Aplicaciones sin estado, como una API web. Escala y hace **rolling updates**: cambia pods viejos por nuevos poco a poco, sin corte, al ritmo de `maxSurge` y `maxUnavailable` (25 % por defecto), con vuelta atrás. Por debajo gestiona **ReplicaSets**, que mantienen *N* copias idénticas.
StatefulSet	Pods con identidad estable (`db-0` , `db-1` ...), DNS propio y un volumen persistente cada uno, creados en orden: bases de datos, Kafka.
DaemonSet	Un pod en cada nodo (o en los elegidos): agentes de logs y de métricas, el plugin de red.
Job / **CronJob**	Pods que se ejecutan hasta terminar con éxito, con reintentos; el CronJob los lanza con horario cron: migraciones, informes.

Services, DNS e Ingress

Un **Service** da a un grupo de pods, elegidos por su **selector** de labels, una IP virtual y un nombre DNS estables, y reparte el tráfico entre los que están listos. Sus tres tipos principales se apilan, cada uno sobre el anterior:

ClusterIP
Por defecto · solo interno

IP alcanzable solo dentro del clúster, para que los microservicios se hablen.

NodePort
Un puerto en cada nodo

Abre un puerto (30000–32767 por defecto) en todos los nodos. Rara vez se usa solo.

LoadBalancer
Balanceador de la nube

Pide a la nube un balanceador externo; en EKS, normalmente un Network Load Balancer.

CoreDNS, el DNS interno, registra cada Service como `<servicio>.<namespace>.svc.cluster.local`: desde el namespace `tienda` basta con llamar a `http://pagos`. Un **Ingress** publica varios Services tras un único balanceador con reglas HTTP por host y ruta, que aplica un **Ingress controller** (en EKS, creando un ALB). Su sucesora es la **Gateway API**, más expresiva y orientada a roles (`Gateway` para la infraestructura, `HTTPRoute` para las rutas de cada equipo): la API de Ingress está congelada y en noviembre de 2025 se anunció la retirada del popular controlador ingress-nginx, así que en diseños nuevos considera primero Gateway API, que el AWS Load Balancer Controller admite en sus versiones recientes.

Configuración, organización y recursos

ConfigMap y Secret — configuración no sensible y sensible (contraseñas, tokens), fuera de la imagen; llegan al pod como variables de entorno o como ficheros.

> **CUIDADO**
>
> Un Secret solo va codificado en Base64, que **no** es cifrado: quien pueda leerlo con la API lo ve en claro. Limita su lectura con RBAC y guarda lo crítico en AWS Secrets Manager, que llega al pod con el Secrets Store CSI Driver o External Secrets Operator.

Namespace — partición lógica para equipos o entornos, con sus propios permisos (RBAC) y cuotas. No aísla la red por sí solo.
Labels y selectors — etiquetas clave-valor (`app: web`) y consultas sobre ellas, el pegamento de Kubernetes: Services y Deployments encuentran sus pods por selector, nunca por nombre.
Requests y limits — las requests son lo reservado (el scheduler busca un nodo con ese hueco); los limits, el máximo: por encima del de CPU se frena al contenedor (*throttling*) y por encima del de memoria se le mata (`OOMKilled`). `250m` es un cuarto de vCPU.
Las **probes** son las comprobaciones de salud que kubelet hace a cada contenedor (por HTTP, TCP, gRPC o con un comando):

Probe	Pregunta	Si falla…
startup	¿Ha terminado de arrancar?	Las otras dos esperan a que pase: protege a las aplicaciones lentas en arrancar.
liveness	¿Sigue vivo o se ha colgado?	kubelet lo reinicia. Que mire solo el propio proceso: si comprueba la base de datos y esta cae, reiniciará todos los pods en bucle.
readiness	¿Puede atender tráfico ahora?	Se le retira de los Services, sin reiniciarlo, hasta que se recupere.

PersistentVolumeClaim (PVC) — para los datos que deben sobrevivir al pod, la aplicación pide almacenamiento: «20 GiB en modo `ReadWriteOnce`» (un nodo; `ReadWriteMany` si son varios).
PersistentVolume (PV) — el volumen real que la satisface; en AWS, un volumen EBS o un sistema de ficheros EFS.
StorageClass — la receta para crear PV bajo demanda (*aprovisionamiento dinámico*): qué driver y con qué parámetros.

8.4 Manos a la obra: YAML, kubectl y Helm

Así se describe una aplicación web: un Deployment con tres réplicas y un Service que las agrupa, conectados por sus labels.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata: { name: web, namespace: tienda }
spec:
  replicas: 3                       # estado deseado: tres pods
  selector:
    matchLabels: { app: web }       # qué pods gestiona
  template:                         # plantilla de cada pod
    metadata:
      labels: { app: web }          # debe coincidir con el selector
    spec:
      containers:
        - name: web
          image: 123456789012.dkr.ecr.eu-west-1.amazonaws.com/web:1.4.2
          resources:
            requests: { cpu: 250m, memory: 256Mi }   # lo que se reserva
            limits: { memory: 512Mi }                # el máximo
          readinessProbe: { httpGet: { path: /ready, port: 8080 } }
          livenessProbe: { httpGet: { path: /healthz, port: 8080 } }
---
apiVersion: v1
kind: Service
metadata: { name: web, namespace: tienda }
spec:                               # type: ClusterIP, el valor por defecto
  selector: { app: web }            # envía el tráfico a los pods app=web
  ports: [{ port: 80, targetPort: 8080 }]
```

Lo aplicas con `kubectl apply -f web.yaml`. Si cambias la imagen a `web:1.5.0` y vuelves a aplicar, el Deployment crea un ReplicaSet nuevo y hace un rolling update: arranca pods nuevos, espera a que pasen su readiness probe y retira los viejos; `kubectl rollout undo` vuelve atrás. Usa etiquetas de imagen fijas, nunca `latest`. Los comandos que más usarás:

```bash
kubectl get pods -n tienda -o wide        # pods con su nodo y su IP
kubectl describe pod web-7d4b9            # detalle y eventos (¿por qué Pending?)
kubectl logs web-7d4b9 -f                 # logs (--previous: del que se cayó)
kubectl exec -it web-7d4b9 -- sh          # shell dentro del contenedor
kubectl port-forward svc/web 8080:80      # prueba un Service desde tu equipo
kubectl auth can-i delete pods -n tienda  # ¿me lo permite RBAC?
```

Helm es el gestor de paquetes de Kubernetes: un **chart** empaqueta plantillas YAML y un `values.yaml` con sus parámetros; cada instalación es una **release** versionada que puedes actualizar o revertir (`helm upgrade`, `helm rollback`). Así se instala el controlador de balanceadores de la sección de redes:

```bash
helm repo add eks https://aws.github.io/eks-charts
helm install aws-load-balancer-controller eks/aws-load-balancer-controller \
  -n kube-system --set clusterName=prod
```

Completan el kit **Kustomize** (`kubectl apply -k`) y **GitOps**: Argo CD o Flux reconcilian el clúster con un repositorio Git.

8.5 Escalado: pods y nodos

Kubernetes escala en dos niveles: más pods cuando sube la carga y más nodos cuando los pods ya no caben.

HorizontalPodAutoscaler (HPA) — ajusta las réplicas para mantener una métrica objetivo, como la CPU respecto a las requests (requiere Metrics Server):
`kubectl autoscale deployment web --cpu-percent=60 --min=3 --max=20`
VerticalPodAutoscaler (VPA) — recalcula las requests de cada pod según su consumo real; no lo combines con un HPA sobre la misma métrica.
KEDA — escala por eventos, como los mensajes pendientes en una cola SQS, incluso hasta cero.
Los pods que no caben en ningún nodo quedan en `Pending`: el HPA decide **cuántos** pods; el autoscaler de nodos, **dónde** caben.

Cluster Autoscaler
El clásico, basado en grupos de nodos

Sube o baja la capacidad deseada del Auto Scaling group de cada node group. Cada grupo usa tipos de instancia similares: si un pod necesita GPU, debe existir un grupo para ello.

Karpenter
Nodos a la medida · open source, creado por AWS

Lanza directamente, sin Auto Scaling groups, las instancias EC2 (On-Demand o Spot) que mejor encajan con los pods pendientes y **consolida** nodos para ahorrar. Es el motor de EKS Auto Mode y lo recomendable en clústeres nuevos.

8.6 Amazon EKS: Kubernetes gestionado

Operar Kubernetes por tu cuenta obliga a cuidar etcd, la alta disponibilidad del API server, los certificados y las actualizaciones. **Amazon Elastic Kubernetes Service (EKS)** te da un control plane estándar, certificado por la CNCF, que AWS ejecuta en una VPC propia con al menos dos instancias del API server y tres de etcd en tres AZ. Se conecta con tus nodos mediante interfaces de red (ENI) en tus subredes, y su endpoint puede ser público (restringible por CIDR), privado o ambos.

FICHA DEL SERVICIO
Amazon EKS
Kubernetes gestionado: AWS opera el control plane y tú decides dónde corren los pods

ÁMBITO	Regional; control plane repartido en tres zonas de disponibilidad
AWS GESTIONA	El control plane (disponibilidad, escalado y parches), versiones compatibles de los add-ons y, con Fargate o Auto Mode, los nodos
TÚ GESTIONAS	Cuándo actualizar; los nodos EC2 (salvo en Auto Mode); aplicaciones, acceso, IAM de los pods, red y costes
FUERA DE AWS	EKS Anywhere y EKS Hybrid Nodes (nodos propios unidos a un clúster EKS)

COSTES

Orientativamente, 0,10 USD por clúster y hora en soporte estándar (unos 73 USD al mes) y 0,60 USD/h en soporte extendido; consulta los precios de tu región. Súmale el cómputo (EC2; en Fargate, vCPU y memoria por pod; en Auto Mode, EC2 más un recargo por instancia), balanceadores, volúmenes EBS, NAT Gateway, IPv4 públicas y tráfico entre AZ. Por esa tarifa fija, en entornos pequeños suele salir más barato compartir un clúster con namespaces que tener muchos.

¿Dónde se ejecutan tus pods? Opciones de data plane

Opción	Quién opera los nodos	Escalado	DaemonSets	Pagas	Encaja si…
Managed node groups	AWS gestiona el Auto Scaling group y renueva los nodos cuando se lo pides	Cluster Autoscaler	✔	EC2	Quieres EC2 con poco esfuerzo
Nodos autogestionados	Tú: grupo, AMI, arranque y parches	Cluster Autoscaler	✔	EC2	Necesitas algo muy a medida
Fargate	AWS: cada pod en su propia micro-VM; no ves nodos	Implícito, pod a pod	✘	vCPU y memoria por pod	Cargas pequeñas o esporádicas
Karpenter (lo instalas tú)	Tú operas Karpenter; él lanza y retira instancias EC2	A la medida de los pods pendientes	✔	EC2 (On-Demand o Spot)	Cargas variadas, Spot
EKS Auto Mode	AWS: nodos, sistema operativo, parches y add-ons clave	Karpenter gestionado	✔	EC2 + recargo	EC2 sin operar nodos

FARGATE EN EKS: LO QUE NO PUEDES HACER

Los **perfiles de Fargate** (por namespace y labels) eligen qué pods van a Fargate. Cada pod tiene su propia micro-VM, así que no hay DaemonSets (los agentes van como sidecar; para logs hay un router integrado basado en Fluent Bit), ni contenedores privilegiados, GPU, `hostNetwork`, volúmenes EBS (sí EFS) o Pod Identity (usa IRSA). Solo admite subredes privadas, y los balanceadores deben apuntar a la IP del pod.

EKS Auto Mode (2024) es lo más «llave en mano» sin renunciar a EC2: un Karpenter gestionado aprovisiona nodos con AMI inmutables basadas en Bottlerocket (sin SSH ni SSM), renovados como máximo cada 21 días, y AWS se ocupa de la red de pods, el DNS, el almacenamiento en bloque y los balanceadores. Las opciones se combinan (por ejemplo, Fargate para un solo namespace).

Crear tu primer clúster

La vía más rápida es **eksctl**, la CLI oficial de EKS: a partir de un fichero declarativo crea, con CloudFormation por debajo, la VPC, el clúster, los add-ons y los nodos. En producción se suele usar Terraform o AWS CDK (capítulo de Infraestructura como código y DevOps), pero la idea es la misma: describir el clúster como código. Para un clúster solo con Auto Mode, el fichero llevaría `autoModeConfig: { enabled: true }` en lugar de `managedNodeGroups` y de los add-ons básicos, que Auto Mode ya incluye.

```yaml
# cluster.yaml
apiVersion: eksctl.io/v1alpha5
kind: ClusterConfig
metadata: { name: prod, region: eu-west-1, version: "1.35" }
accessConfig: { authenticationMode: API }   # access entries, sin aws-auth
addons: [{ name: vpc-cni }, { name: coredns }, { name: kube-proxy },
         { name: eks-pod-identity-agent }]  # add-ons gestionados por EKS
managedNodeGroups:                          # nodos en subredes privadas
  - { name: general, instanceTypes: ["m7i.large", "m6i.large"],
      minSize: 2, desiredCapacity: 3, maxSize: 6, privateNetworking: true }
```

```bash
eksctl create cluster -f cluster.yaml     # VPC, clúster, add-ons y nodos
aws eks update-kubeconfig --region eu-west-1 --name prod  # kubectl con tu IAM
kubectl get nodes                         # tres nodos en estado Ready
```

8.7 Redes en EKS

Amazon VPC CNI: cada pod, una IP de tu VPC

En Kubernetes, un plugin de red (**CNI**, Container Network Interface) da una IP a cada pod. En EKS se usa **Amazon VPC CNI** (el DaemonSet `aws-node`): cada pod recibe una IP **real** de la subred de su nodo, tomada de las IP secundarias de las ENI de la instancia. Sin redes superpuestas (*overlay*) ni encapsulación, los pods son ciudadanos de primera de la VPC: aparecen en los VPC Flow Logs y un ALB o una base de datos los alcanzan directamente.

> **EL GRAN RIESGO: QUEDARTE SIN IP**
>
> Cada pod consume una IP de la subred, y cada nodo reserva algunas más por adelantado. Una subred /24 (251 IP utilizables) se agota enseguida y los pods nuevos se atascan en `ContainerCreating`. Además, las ENI e IP que admite cada tipo de instancia limitan los pods por nodo (29 en un `m5.large`). Planifica subredes grandes desde el principio.

Prefix delegation — asigna prefijos /28 (16 IP) en vez de IP sueltas: muchos más pods por nodo en instancias Nitro, pero exige bloques /28 libres y contiguos en la subred.
Custom networking — los pods toman IP de otras subredes, normalmente de un CIDR secundario como `100.64.0.0/16`, sin gastar direcciones enrutables de tu red (se configura por AZ con `ENIConfig`).
IPv6 — un clúster IPv6 elimina el problema de raíz, si todo tu entorno lo soporta.
Con **Security Groups for Pods** puedes asignar Security Groups a pods concretos (por ejemplo, que solo los de `pagos` lleguen a RDS). Y como por defecto todos los pods hablan con todos, define **network policies**: firewalls a nivel de pod que fijan, por labels, namespaces o rangos de IP, qué tráfico se acepta. La VPC CNI las aplica de forma nativa si lo activas en el add-on (Calico y Cilium son alternativas); empieza denegándolo todo y abre solo lo necesario.

AWS Load Balancer Controller

El **AWS Load Balancer Controller** es otro controlador (otro bucle de reconciliación) que convierte objetos de Kubernetes en balanceadores (capítulo de Balanceo de carga y Auto Scaling): un **Ingress** de clase `alb`, en un **Application Load Balancer** con reglas por host y ruta (varios Ingress pueden compartirlo con una **IngressGroup**), y un **Service** `LoadBalancer`, en un **Network Load Balancer**. En modo de destino `ip` el balanceador envía el tráfico directamente a la IP de cada pod, gracias a la VPC CNI (obligatorio en Fargate); en modo `instance`, al NodePort de los nodos, con un salto extra. Necesita permisos IAM (Pod Identity o IRSA) y localiza las subredes por sus etiquetas: `kubernetes.io/role/elb` en las públicas y `kubernetes.io/role/internal-elb` en las privadas. En Auto Mode viene integrado; sin él, los Services `LoadBalancer` los atiende el controlador heredado de Kubernetes, que crea Classic Load Balancers.

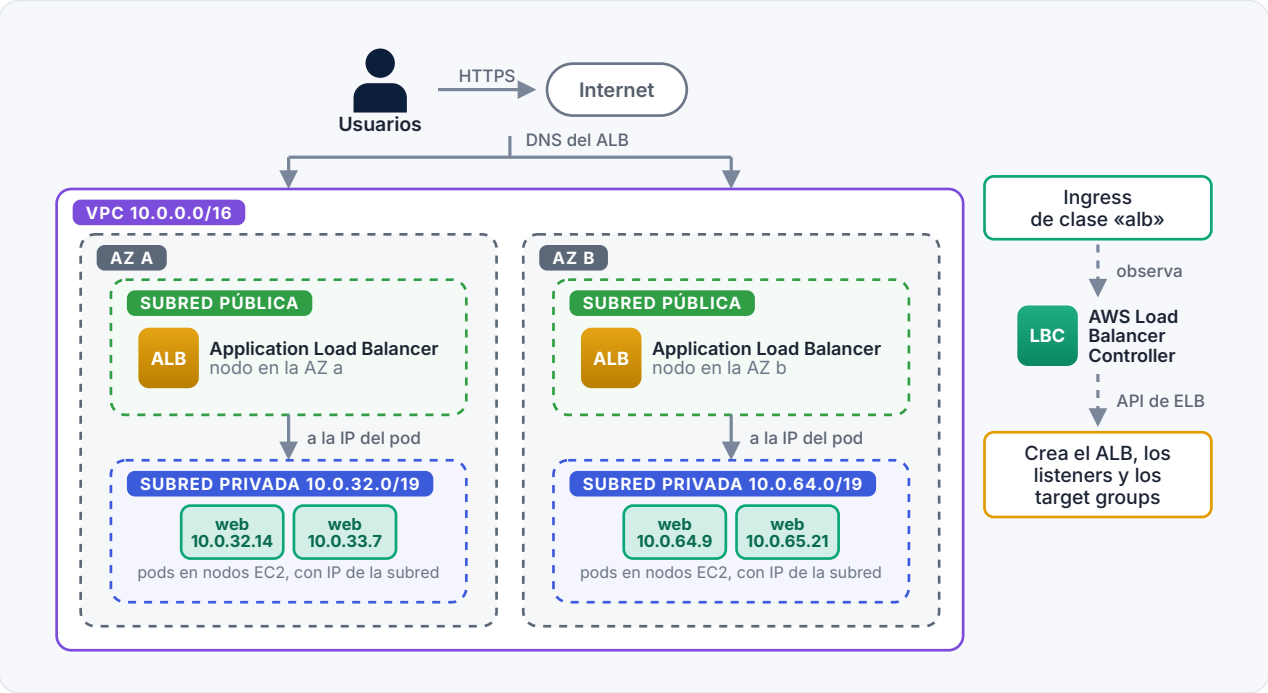


Figura 53 · Tráfico de entrada en EKS: el ALB, con nodos en las subredes públicas de dos AZ, envía cada petición directamente a la IP de un pod (modo IP) en las subredes privadas. A la derecha, cómo el controlador crea y mantiene el ALB a partir del Ingress.

> **CUIDADO**
>
> Antes de borrar un clúster, elimina sus Ingress y Services de tipo LoadBalancer. Esos balanceadores los creó el controlador, no tu plantilla: si el clúster desaparece primero, quedan huérfanos, siguen facturando e impiden borrar la VPC.

8.8 Almacenamiento en EKS

Los PVC se convierten en almacenamiento de AWS gracias a los drivers **CSI** (Container Storage Interface), que se instalan como add-ons y necesitan permisos IAM:

Driver CSI	Servicio	Acceso	Ámbito	Úsalo para
EBS CSI driver	Amazon EBS (`gp3`, `io2`…)	`ReadWriteOnce`: un nodo	Una AZ	Bases de datos y otros StatefulSets
EFS CSI driver	Amazon EFS	`ReadWriteMany`: muchos pods y nodos	Regional (varias AZ)	Ficheros compartidos; persistencia en Fargate
Mountpoint for S3 CSI driver	Amazon S3	Lectura masiva y escritura secuencial	Regional	Datasets de ML, contenido de solo lectura

> **CUIDADO**
>
> Un volumen EBS vive en una sola AZ: si su pod se reprograma en otra, queda en `Pending`. Usa `volumeBindingMode: WaitForFirstConsumer` en la StorageClass (crea el disco en la AZ del pod) y, para alta disponibilidad, replica en la aplicación o usa EFS. Ojo con `Retain`: conserva el disco (y su factura) al borrar el PVC.

8.9 IAM y EKS: dos direcciones que no debes confundir

En EKS conviven dos sistemas de permisos: **IAM**, que protege las API de AWS, y el **RBAC** de Kubernetes, que protege la API del clúster. Se cruzan en dos direcciones opuestas, y casi todas las dudas vienen de mezclarlas:

	A · De fuera hacia dentro	B · De dentro hacia fuera
Pregunta	¿Quién puede usar el clúster y qué puede hacer en él?	¿Qué puede hacer un pod en los servicios de AWS?
Quién actúa, y contra qué API	Personas y pipelines con una identidad IAM (un usuario o un rol, incluido el que crea un permission set), contra la API de Kubernetes (`kubectl` , Helm…)	El código de un pod, identificado por su **ServiceAccount** (su identidad en Kubernetes), contra las API de AWS (S3, SQS…)
Quién decide	IAM autentica; RBAC o una access policy de EKS autoriza	La política IAM del rol asociado a la ServiceAccount
Cómo se configura	Access entry + RoleBinding (o access policy)	Asociación de EKS Pod Identity (o IRSA)
Si falla	`Unauthorized` o `Forbidden` en `kubectl`	`AccessDenied` en la llamada a AWS

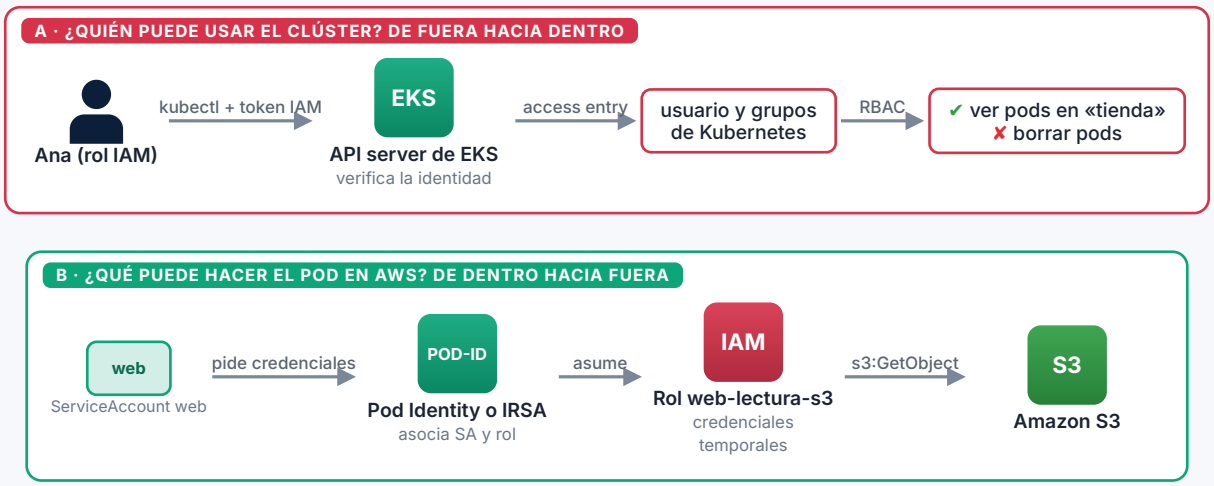


Figura 54 · Las dos direcciones de IAM en EKS. Arriba, una persona entra al clúster: IAM la identifica y RBAC decide qué puede hacer. Abajo, un pod actúa en AWS con el rol IAM asociado a su ServiceAccount.

RBAC: los permisos dentro de Kubernetes

Kubernetes autoriza cada petición a su API con **RBAC** (Role-Based Access Control) y cuatro objetos:

	En un namespace	En todo el clúster
Qué se permite: reglas con verbos (`get` , `list` , `create` , `delete` …) sobre recursos	`Role`	`ClusterRole`
A quién: usuarios, grupos o ServiceAccounts	`RoleBinding`	`ClusterRoleBinding`

Un RoleBinding puede apuntar a un ClusterRole para concederlo en un solo namespace (así se reutilizan los roles integrados `view` , `edit` y `admin`). RBAC es solo aditivo, sin las denegaciones de IAM (capítulo «IAM (II)»), y no tiene objetos «usuario»: confía en que alguien externo autentique a quien llama. En EKS, ese alguien es IAM.

Dirección A: quién puede usar el clúster

Al ejecutar `kubectl`, tu kubeconfig obtiene con `aws eks get-token` un token firmado con tus credenciales de IAM. EKS verifica tu identidad y busca la **access entry** (entrada de acceso) de tu ARN, que fija con qué usuario y grupos de Kubernetes entras. Los permisos llegan por grupos vinculados con tus RoleBinding o por **access policies** de EKS (`AmazonEKSClusterAdminPolicy`, `AmazonEKSAdminPolicy`, `AmazonEKSEditPolicy`, `AmazonEKSViewPolicy`) asociadas al clúster o a ciertos namespaces con `aws eks associate-access-policy`. Con IAM Identity Center, la entrada apunta al rol que crea el permission set (capítulo «IAM (I)»).

```bash
# 1) El rol IAM entra al clúster con el grupo de Kubernetes "desarrolladores"
aws eks create-access-entry --cluster-name prod \
  --principal-arn arn:aws:iam::123456789012:role/Desarrolladores \
  --kubernetes-groups desarrolladores
# 2) RBAC: ese grupo puede ver (rol integrado "view") el namespace tienda
kubectl create rolebinding devs-ven-tienda -n tienda \
  --clusterrole=view --group=desarrolladores
```

Las access entries (2023) sustituyen al antiguo ConfigMap `aws-auth` de `kube-system`, que se editaba a mano y en el que un error de formato podía dejar a todos fuera; ahora el acceso se gestiona con la API de EKS o con tu IaC y queda en CloudTrail. El clúster admite los modos de autenticación `CONFIG_MAP`, `API_AND_CONFIG_MAP` (para migrar) y `API`, el recomendado. La identidad que crea el clúster recibe acceso de administrador por defecto: no dependas de ella.

> ★ **CLAVE PARA LA CERTIFICACIÓN**
>
> **Unauthorized** en `kubectl` significa que EKS no te reconoce (falta tu access entry); **Forbidden**, que te reconoce pero RBAC no permite la acción. Los permisos IAM sobre la API de EKS (como `eks:DescribeCluster`) no dan acceso a los objetos de Kubernetes.

Dirección B: qué pueden hacer los pods en AWS

Una aplicación que usa S3 o DynamoDB nunca debe tirar de claves de acceso en un Secret ni del rol IAM del nodo, que heredan todos sus pods (un contenedor comprometido tendría los permisos de todos). Lo correcto es asociar un rol IAM a la ServiceAccount del pod, de una de estas dos formas:

	IRSA (IAM Roles for Service Accounts)	EKS Pod Identity
Cómo funciona	Proveedor OIDC del clúster en IAM; el SDK canjea el token del pod con `AssumeRoleWithWebIdentity`	Un agente en cada nodo (add-on) entrega al pod credenciales del rol asociado
En IAM	Proveedor OIDC por clúster; la trust policy nombra clúster, namespace y ServiceAccount	Trust policy estándar para `pods.eks.amazonaws.com`, igual en todos los clústeres
En Kubernetes	Anotación `eks.amazonaws.com/role-arn` en la ServiceAccount	Nada: una **asociación** en la API de EKS
Reutilizar el rol en otro clúster	Hay que editar la trust policy	Directo
Etiquetas de sesión (ABAC)	✘	✔
Fargate y Kubernetes fuera de EKS	✔	✘

> **BUENA PRÁCTICA**
>
> Usa EKS Pod Identity por defecto en EC2 (IRSA en Fargate), con un rol por aplicación que asocias a su ServiceAccount con `aws eks create-pod-identity-association`. Y bloquea el IMDS (capítulo de EC2): con IMDSv2 y un *hop limit* de 1 en los nodos, ningún contenedor obtiene las credenciales del rol del nodo.

No confundas estas dos direcciones con los dos roles de servicio que pide EKS: el **rol del clúster**, con el que el control plane gestiona recursos de tu cuenta (interfaces de red, balanceadores...), y el **rol de los nodos**, con el que cada nodo se une al clúster y descarga imágenes de ECR. Son de la infraestructura: no les añadas permisos para tus aplicaciones.

8.10 Operación del clúster

Add-ons gestionados. EKS ofrece como add-ons el software que el clúster necesita además de tus aplicaciones, en versiones probadas, compatibles y parcheadas por AWS que tú decides cuándo instalar o actualizar. Los imprescindibles son **VPC CNI**, **CoreDNS** y **kube-proxy**; les siguen **EKS Pod Identity Agent**, los drivers CSI (**EBS**, **EFS**, **Mountpoint for S3**) y **CloudWatch Observability**.

Versiones y actualizaciones. Kubernetes publica unas tres versiones menores al año. EKS da a cada una algo más de un año de **soporte estándar** y, después, alrededor de otro año de **soporte extendido**, más caro; al agotarse, EKS actualiza automáticamente el control plane (con la **política de actualización** en `STANDARD`, lo hace al acabar el estándar). Mejor adelantarse:

1. **Prepara**: notas de la versión y **EKS Cluster Insights**, que detecta, entre otras cosas, el uso de API que desaparecen.
2. **Control plane**: una versión menor cada vez (de 1.34 a 1.35, nunca directamente a 1.36), sin vuelta atrás.
3. **Add-ons**: a versiones compatibles con la nueva.
4. **Nodos**: AMI nueva en los managed node groups; Karpenter reemplaza los desfasados (*drift*); en Fargate, reinicia los pods; Auto Mode lo hace solo. Los nodos pueden ir por detrás del control plane, nunca por delante.

Observabilidad (capítulo de Observabilidad). **Container Insights** recoge métricas y logs de clúster, nodos y pods, y **Amazon Managed Service for Prometheus** y **Amazon Managed Grafana** cubren el ecosistema de Prometheus sin servidores que operar. Los **logs del control plane** (API server, auditoría, autenticador, scheduler, controller manager) van a CloudWatch Logs, pero vienen desactivados: activa al menos el de auditoría.

Cifrado de los Secrets con KMS. Desde marzo de 2025, EKS cifra por defecto todos los datos de la API de Kubernetes, Secrets incluidos (clústeres 1.28 o posteriores), con cifrado envolvente y una clave de KMS propiedad de AWS, sin coste. Para controlar tú esa clave (política, rotación, auditoría), usa una **customer managed key**; KMS y el cifrado envolvente se explican en el capítulo de Seguridad.

8.11 Buenas prácticas y errores comunes

- **Fiabilidad**: dos o más réplicas repartidas entre AZ, un `PodDisruptionBudget` para que renovar nodos no tumbe la aplicación, y requests, limits y probes en todo contenedor.
- **Seguridad**: access entries y RBAC por namespace, un rol IAM por aplicación (Pod Identity), IMDS bloqueado, Pod Security Standards, network policies y endpoint privado o restringido.
- **Coste y operación**: Karpenter o Auto Mode con Spot y consolidación, requests ajustadas, ALB compartidos, IaC, GitOps y versiones en soporte estándar.

Síntoma	Causa habitual	Por dónde empezar
`Pending`	Ningún nodo tiene hueco para sus requests o cumple sus afinidades, o su volumen EBS está en otra AZ	`kubectl describe pod` y el autoscaler de nodos
`ContainerCreating` atascado	Subred sin IP libres o volumen que no se monta	Eventos del pod; logs de `aws-node`
`ImagePullBackOff`	Imagen inexistente, sin permiso en ECR o sin ruta a ECR (sin NAT ni VPC endpoints)	`kubectl describe pod`
`CrashLoopBackOff`	La aplicación termina al arrancar o la liveness probe es demasiado agresiva	`kubectl logs --previous`
Service sin respuesta	El selector no coincide con las labels o la readiness probe falla	`kubectl get endpointslices`

RESUMEN DEL CAPÍTULO

- Kubernetes es **declarativo**: describes el estado deseado y los controladores lo reconcilian.
- El **control plane** (API server, etcd, scheduler, controller manager, cloud controller manager) decide; los **nodos** (kubelet, kube-proxy, runtime) ejecutan. Solo el API server habla con etcd.
- **Pods** gestionados por Deployment, StatefulSet, DaemonSet o Job; **Services** con IP y DNS estables por selector; Ingress y Gateway API para HTTP; requests, limits y probes; PVC, PV y StorageClass para los datos. El HPA escala pods; Karpenter, nodos.
- **EKS** opera un control plane Multi-AZ por una tarifa por clúster y hora (más cara en soporte extendido); los pods corren en managed node groups, nodos propios, Fargate, Karpenter o **Auto Mode**.
- La **VPC CNI** da a cada pod una IP de la VPC (vigila el agotamiento) y el **AWS Load Balancer Controller** crea ALB y NLB, mejor en modo IP.
- IAM va en dos direcciones: **access entries** + RBAC para entrar al clúster; **Pod Identity** o IRSA para que los pods usen AWS, nunca el rol del nodo. Y actualiza una versión menor cada vez.

Preguntas de repaso

1. **Un Deployment tiene `replicas: 3` y se cae el nodo donde corrían dos de sus pods. ¿Qué ocurre y qué principio lo explica?**
   Respuesta: El modelo declarativo: el estado deseado sigue siendo 3, así que el ReplicaSet crea dos pods que el scheduler coloca donde haya hueco (si no lo hay, quedan en `Pending` hasta que el autoscaler añada nodos).
2. **Los pods nuevos se quedan en `ContainerCreating` por falta de IP, y las subredes son /24. ¿Qué haces?**
   Respuesta: Es agotamiento de IP por la VPC CNI: prefix delegation, subredes más grandes, custom networking con un CIDR secundario o un clúster IPv6.
3. **Un compañero propone añadir `s3:GetObject` al rol IAM de los nodos para que una aplicación lea de un bucket. ¿Qué propones?**
   Respuesta: Un rol de mínimo privilegio asociado a la ServiceAccount de esa aplicación con Pod Identity (IRSA en Fargate): el rol del nodo lo heredarían todos los pods. Y bloquear el IMDS (hop limit de 1).
4. **Una desarrolladora con un rol IAM válido recibe Unauthorized con `kubectl get pods`. ¿Qué falta?**
   Respuesta: Su access entry: sin ella, EKS no la reconoce. Después, si no tiene permisos de RBAC o una access policy, el error pasará a ser **Forbidden**.
5. **Quieres mover a Fargate un agente de logs que hoy se despliega como DaemonSet. ¿Es posible?**
   Respuesta: No como DaemonSet: Fargate ejecuta cada pod en su propia micro-VM y no los admite. Usa su router de logs integrado o el agente como sidecar.
6. **Tu clúster está en la versión 1.33 y debes llegar a la 1.35. ¿Cómo lo planificas?**
   Respuesta: En dos saltos (1.33 → 1.34 → 1.35), porque EKS sube una versión menor cada vez. En cada uno: Cluster Insights, control plane, add-ons y, por último, nodos.

CAPÍTULO 9

Serverless: Lambda, API Gateway y Step Functions

Ejecuta código, publica tus API y orquesta procesos sin aprovisionar ni administrar un solo servidor.

EN ESTE CAPÍTULO APRENDERÁS

1. Qué significa realmente *serverless* y cuándo encaja
2. Cómo ejecuta Lambda tu código: entorno, *cold starts* y concurrencia
3. Los modos de invocación y cómo se gestionan los errores
4. Permisos, VPC, secretos, versiones y *aliases*
5. Publicar API con API Gateway: tipos, autorización y límites
6. Orquestar flujos con Step Functions y desplegar con SAM

SERVICIOS PROTAGONISTAS

AWS Lambda · Amazon API Gateway · AWS Step Functions · AWS SAM

9.1 ¿Qué significa serverless?

Serverless («sin servidor») no significa que no haya servidores: significa que **no son asunto tuyo**. AWS los aprovisiona, parchea, escala y reemplaza; tú entregas código o configuración y pagas por el uso real. Un servicio es serverless cuando reúne cuatro rasgos:

Sin servidores	Escala sola	Pago por uso	Alta disponibilidad
Ni sistema operativo que parchear ni capacidad que dimensionar.	De cero a miles de peticiones por segundo, y de vuelta a cero.	Por petición, por milisegundo o por GB, no por horas encendido.	Repartido entre varias AZ de serie, sin configurar nada.

El ecosistema va más allá de Lambda: API Gateway, Step Functions, S3, DynamoDB, SQS, SNS, EventBridge, Fargate o Aurora Serverless v2 comparten esta filosofía, y combinándolos construyes aplicaciones completas sin una sola instancia EC2. A cambio aceptas límites (duración, tamaño de los mensajes), latencias de arranque ocasionales, una arquitectura distribuida más difícil de depurar y un coste que, con carga alta y constante, puede superar al de instancias o contenedores.

≈ **ANALOGÍA**

Tener servidores es como tener coche: pagas compra, seguro y garaje aunque lo uses una hora al día. Serverless es un taxi: pagas por trayecto, no te ocupas del mantenimiento y, si mañana necesitas diez, aparecen diez; eso sí, no eliges el motor.

9.2 AWS Lambda: funciones bajo demanda

Lambda ejecuta tu código en respuesta a eventos —una petición HTTP, un fichero que llega a S3, un mensaje en una cola, una hora programada—. Subes una **función** (código y configuración), defines qué la dispara y Lambda crea entornos de ejecución aislados, escala con la demanda y te factura por milisegundo.

FICHA DEL SERVICIO

AWS Lambda

Cómputo serverless dirigido por eventos

TIPO	Serverless; pago por petición y por duración (GB-segundo, al milisegundo)
ÁMBITO	Regional; Lambda reparte los entornos entre varias AZ
IDEAL PARA	API, procesamiento de eventos y ficheros, tareas programadas, «pegamento» entre servicios
LÍMITES CLAVE	15 min por invocación; de 128 MB a 10 240 MB de memoria; *payload* síncrono de 6 MB; 1000 ejecuciones concurrentes por región por defecto (ampliable)

El modelo de ejecución: eventos y handler

Toda función tiene un **handler**, el punto de entrada que Lambda llama en cada invocación con dos argumentos: el **evento** (un JSON cuya forma depende del origen) y el **contexto** (metadatos como el identificador de la petición o el tiempo restante). Lo que devuelve el handler es la respuesta.

```python
import json, os
from decimal import Decimal
import boto3

# Fase Init: se ejecuta una vez por entorno y se reutiliza después
tabla = boto3.resource("dynamodb").Table(os.environ["TABLA_PEDIDOS"])

def lambda_handler(event, context):
    """Crea un pedido a partir de una petición de API Gateway."""
    pedido = json.loads(event["body"], parse_float=Decimal)
    tabla.put_item(Item={"id": pedido["id"], "importe": pedido["importe"]})
    return {"statusCode": 201,
            "headers": {"Content-Type": "application/json"},
            "body": json.dumps({"id": pedido["id"]})}
```

El cliente de DynamoDB se crea **fuera** del handler: ese código se ejecuta una sola vez, al preparar el entorno de ejecución (lo verás enseguida), y se reutiliza en las invocaciones siguientes. Es la optimización más sencilla y rentable de Lambda.

Runtimes, paquetes y recursos

Hay **runtimes gestionados** para Node.js, Python, Java, .NET y Ruby, y **runtimes de solo sistema operativo** (`provided.al2023`) para Go, Rust o runtimes propios. AWS los parchea y los retira según un calendario publicado. El código se empaqueta como **.zip** (hasta 50 MB comprimido en subida directa y 250 MB descomprimido) o como **imagen de contenedor** de hasta 10 GB alojada en ECR. Puedes elegir `x86_64` o `arm64` (Graviton), con un precio por GB-segundo en torno a un 20 % menor.

Lambda tiene un solo mando de potencia: la **memoria**, de 128 MB a 10 240 MB. La CPU y la red crecen **en proporción**: con 1769 MB tienes el equivalente a una vCPU y con el máximo, 6 vCPU. Por eso subir la memoria acelera a menudo funciones que ni usan esa RAM y, como terminan antes, pueden salir igual de baratas (la herramienta de código abierto **Lambda Power Tuning** te dibuja la curva coste-velocidad).

Timeout — de 1 s a 900 s; 3 s por defecto. Ajústalo a lo esperado más un margen, no al máximo.
`/tmp` — almacenamiento efímero de 512 MB (incluido) a 10 240 MB. Sobrevive entre invocaciones **del mismo entorno** —sirve de caché—, nunca entre entornos.

Ciclo de vida del entorno de ejecución

Cada invocación se ejecuta en un **entorno de ejecución**: una microVM Firecracker aislada con tu runtime y tu código.



Figura 55 · Vida de un entorno de ejecución: una fase Init, muchas invocaciones y un apagado final.

Init — descarga el código, arranca extensiones y runtime y ejecuta tu inicialización (lo que está fuera del handler). En funciones bajo demanda dispone de hasta 10 s.
Invoke — ejecuta el handler; al terminar, el entorno se **congela** y queda listo para otra invocación (*warm start*).
Shutdown — tras un tiempo sin uso, o periódicamente para aplicar actualizaciones, Lambda destruye el entorno.
Un **cold start** es la latencia extra de una invocación que necesita un entorno nuevo: la primera, cada invocación **concurrente** adicional al escalar y las que llegan tras un periodo de inactividad. Va de

decenas de milisegundos en funciones pequeñas de Python o Node.js a varios segundos en Java o imágenes grandes; según AWS, en cargas reales afecta a menos del 1 % de las invocaciones. Y no solo cuesta tiempo: desde agosto de 2025, AWS factura la fase Init en todas las configuraciones (antes era gratuita en las funciones .zip con runtimes gestionados).

SnapStart
Java, Python y .NET

Al **publicar una versión**, Lambda ejecuta Init y guarda una instantánea cifrada de la microVM; los entornos nuevos se **restauran** desde ella. Gratis en Java; en Python y .NET se cobran caché y restauraciones. Ojo: lo que debería ser único (aleatorios, conexiones) se repite en cada entorno restaurado; regenéralo con *runtime hooks*.

Concurrencia aprovisionada
Cualquier runtime

Mantiene N entornos **ya inicializados**: cero cold starts dentro de ese margen. Se configura sobre una versión o un alias y se paga mientras está activa, haya tráfico o no. Se puede escalar con Application Auto Scaling, por ejemplo con una programación para las horas punta.

Concurrencia: cuánto escala una función

La **concurrencia** es el número de invocaciones simultáneas, es decir, de entornos ocupados: aproximadamente peticiones por segundo × duración media. Así, 200 peticiones/s de 0,5 s necesitan unos 100 entornos.

Cuota regional — 1000 ejecuciones concurrentes por cuenta y región, compartidas por **todas** las funciones; es ampliable (y algunas cuentas nuevas empiezan con menos). Al agotarse hay *throttling* (Lambda rechaza las invocaciones que no caben): error 429 en las síncronas; las asíncronas y los event source mappings reintentan. Cada función puede sumar hasta 1000 entornos cada 10 segundos.
Concurrencia reservada — aparta parte de la cuota para una función. Le **garantiza** capacidad y a la vez la **limita** (útil para proteger una base de datos). Sin coste. Reservar 0 detiene la función: un interruptor de emergencia.

9.3 Cómo se invoca una función

Hay tres formas de llegar a una función, y en cada una cambia quién espera la respuesta y quién reintenta si algo falla.

Síncrona — el llamante espera (`aws lambda invoke` lo hace por defecto); los errores le llegan a él y es él quien reintenta.
Asíncrona — Lambda encola el evento, responde al instante y lo procesa después; así invocan S3, SNS, EventBridge o EventBridge Scheduler, que lanza funciones a horas fijas (capítulo de Mensajería e integración). Si la función falla, Lambda reintenta dos veces más con esperas crecientes (configurable) y descarta el evento cuando supera su antigüedad máxima (hasta 6 h). Para no perderlo, configura un **destino** de fallo —SQS, SNS, EventBridge, S3 u otra función—, que recibe el evento y el error, o una **dead-letter queue** (DLQ) clásica: una cola SQS o un topic SNS que aparta los eventos fallidos, sin el error. Como la entrega es «al menos una vez», haz tus funciones **idempotentes**: que procesar dos veces el mismo evento no cambie el resultado.
Event source mapping — con SQS, Kinesis, DynamoDB Streams, Kafka o Amazon MQ, Lambda lee lotes y llama a tu función de forma síncrona. La función necesita permiso para leer del origen, y puedes **filtrar** eventos para no pagar invocaciones inútiles.

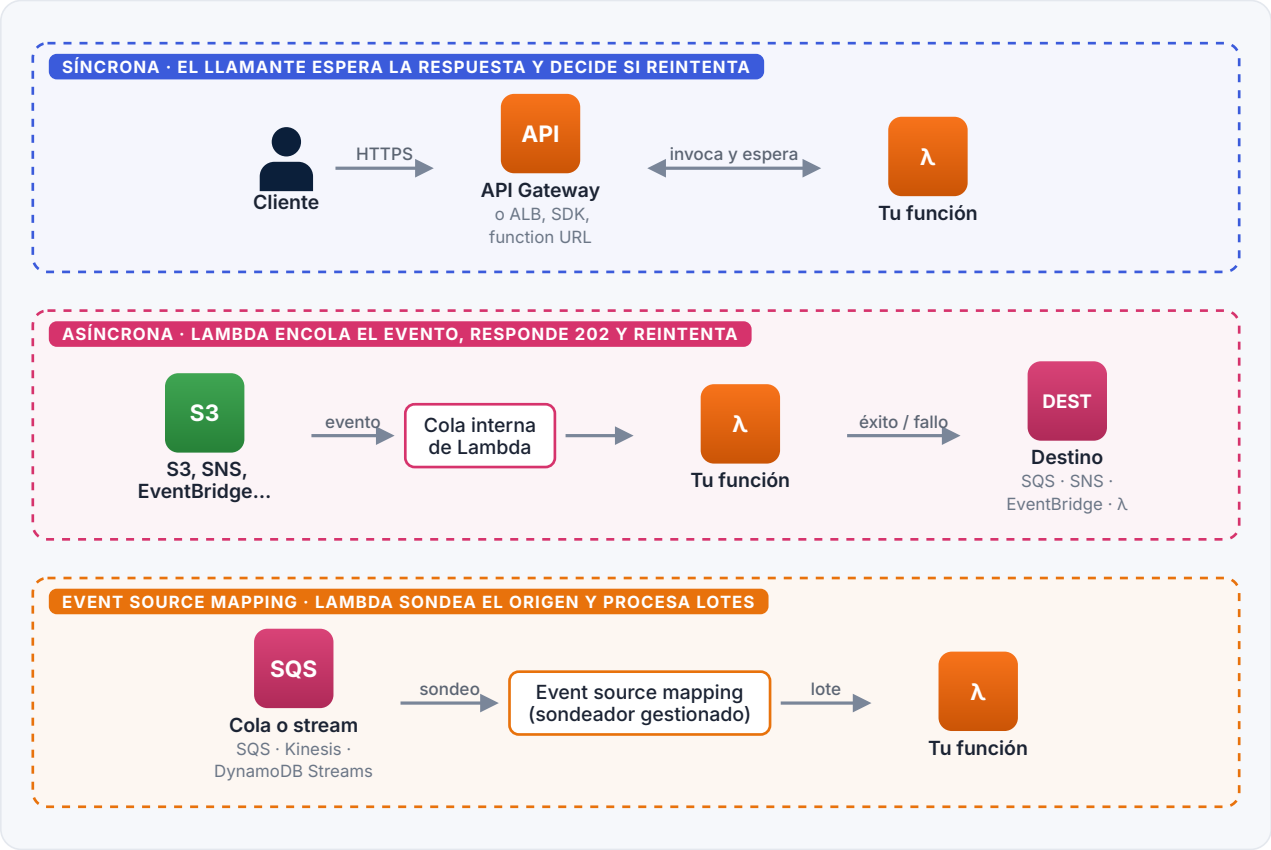


Figura 56 · Los tres modos de invocación de Lambda.

El comportamiento depende del origen (colas y *streams* se explican a fondo en el capítulo de Mensajería e integración). Con **SQS**, Lambda añade sondeadores a medida que crece la cola y, si un lote falla, sus mensajes vuelven a la cola y se reintentan; tras varios fallos, acaban en su DLQ. Un stream de **Kinesis** o de **DynamoDB Streams** se divide en *shards* (particiones ordenadas): Lambda procesa un lote por shard a la vez (hasta 10 con el *parallelization factor*) y respeta el orden, así que un lote que falla **bloquea el shard** hasta que tiene éxito o caduca. Para evitarlo, puedes partir en dos el lote que falla y enviar los registros fallidos a un destino.

9.4 Permisos, red y configuración

Los permisos de una función van en dos direcciones, y confundirlas es un error clásico:

Execution role
¿Qué puede hacer la función?

Rol IAM que Lambda asume al ejecutar tu código: escribir logs (`AWSLambdaBasicExecutionRole`) y lo que haga tu código, como leer de DynamoDB o de una cola.

Resource-based policy
¿Quién puede invocarla?

La política de recurso de la función: permite invocarla a servicios (S3, SNS, API Gateway...) o a otras cuentas. La consola la crea al añadir un disparador.

```bash
aws lambda add-permission --function-name procesar-foto \
  --statement-id s3-invoca --action lambda:InvokeFunction \
  --principal s3.amazonaws.com --source-arn arn:aws:s3:::mis-fotos \
  --source-account 123456789012   # restringe el origen: evita el confused deputy
```

Cómo se combinan estas políticas con las de identidad lo explica el capítulo «IAM (II): permisos — políticas y evaluación».

Lambda en una VPC. Por defecto, una función llega a Internet y a los endpoints públicos de AWS, pero no a recursos privados como una base de datos RDS. Para alcanzarlos la asocias a subredes y Security Groups; Lambda crea interfaces de red (**Hyperplane ENIs**) compartidas por las funciones con la misma combinación, de modo que conectarla a una VPC ya no alarga apreciablemente los cold starts.

> **CUIDADO**
>
> Una función conectada a una VPC **pierde el acceso a Internet**, incluso en una subred pública: sus ENI nunca tienen IP pública. Necesitas subredes privadas con ruta a un NAT Gateway o, para S3, DynamoDB o Secrets Manager, VPC endpoints (capítulo de Redes).

Configuración y secretos. Las **variables de entorno** (hasta 4 KB) separan configuración y código; se cifran con KMS, pero quien lea la configuración las ve en claro, así que **no son sitio para secretos**: usa Secrets Manager o Parameter Store (capítulo «Seguridad: cifrado con KMS, secretos y protección») con la extensión **AWS Parameters and Secrets Lambda Extension**, que los cachea.

Layers. Un *layer* es un .zip con bibliotecas, un runtime o datos que se monta en `/opt` y se comparte entre funciones; cada función admite hasta 5, dentro de los 250 MB descomprimidos (con imágenes de contenedor todo va en la imagen).

9.5 Versiones, aliases y function URLs

Mientras desarrollas editas `$LATEST`, la versión mutable. Al **publicar una versión**, Lambda congela código y configuración en una versión numerada e inmutable (`:5`, `:6`…) con su propio ARN. Un **alias** (`prod`, `staging`) es un puntero a una versión: los clientes apuntan al alias, y promocionar o volver atrás es mover el puntero. Un alias también puede repartir el tráfico entre **dos versiones** con pesos: la base de los despliegues **canary**.

```bash
aws lambda update-alias --function-name crear-pedido --name prod \
  --function-version 5 --routing-config 'AdditionalVersionWeights={"6"=0.1}'
```

En la práctica, SAM o CodeDeploy desplazan el tráfico gradualmente (`Canary10Percent5Minutes`, `Linear10PercentEvery1Minute`…) y vuelven atrás solos si salta una alarma.



Figura 57 · Despliegue canary con un alias ponderado: el 10 % del tráfico prueba la versión nueva.

Para exponer una función por HTTPS sin más, una **function URL** da un endpoint dedicado (`https://<id>.lambda-url.<región>.on.aws`) con autenticación `AWS_IAM` o `NONE`, CORS y *response streaming*. No ofrece throttling por cliente, API keys, caché ni dominio propio: para eso, CloudFront delante (que firma las peticiones con OAC) o API Gateway.

9.6 Buenas prácticas con Lambda

- **Inicializa fuera del handler** clientes, conexiones y configuración; con bases de datos relacionales, usa RDS Proxy para no agotar conexiones al escalar.
- **Hazla idempotente**: un identificador por operación y una escritura condicional (por ejemplo, en DynamoDB) evitan cobrar dos veces un pedido.

- **Una función, una responsabilidad**, con pocas dependencias: paquetes pequeños, cold starts cortos.
- **Evita bucles recursivos** (escribir en el bucket o la cola que te dispara); Lambda detecta y corta algunos, no todos.
- **No orquestes desde el código** encadenando funciones que se esperan: para eso está Step Functions (o, desde finales de 2025, las **durable functions** de Lambda, que guardan su progreso en *checkpoints* y esperan sin consumir cómputo).
- **Observa**: Lambda envía la salida estándar a CloudWatch Logs, con formato JSON y niveles nativos (capítulo de Observabilidad).
- **Apóyate en Powertools for AWS Lambda**, biblioteca de código abierto de AWS (Python, TypeScript, Java y .NET) que aplica estas prácticas en pocas líneas: logs JSON, trazas de X-Ray, métricas, idempotencia sobre DynamoDB, respuestas parciales de lote y secretos con caché.

> **$ COSTES**
>
> Pagas por petición (del orden de 0,20 USD por millón) y por GB-segundo, más extras como la concurrencia aprovisionada; la capa gratuita incluye un millón de peticiones al mes. Lambda es imbatible con tráfico bajo o irregular; con carga alta y constante, compara con Fargate o EC2 (los Compute Savings Plans también cubren Lambda) o con **Lambda Managed Instances** (finales de 2025), que ejecuta tus funciones sobre instancias EC2 gestionadas por Lambda.

9.7 Amazon API Gateway

API Gateway es la puerta de entrada gestionada de tus API: recibe peticiones HTTP o WebSocket, las autentica, aplica límites y las enruta a un *backend* —Lambda, un servicio HTTP u otro servicio de AWS—. Escala solo y se paga por petición.

FICHA DEL SERVICIO

Amazon API Gateway

API REST, HTTP y WebSocket totalmente gestionadas

TIPO	Serverless; pago por millón de peticiones (mensajes y minutos de conexión en WebSocket)
ÁMBITO	Regional; las REST API también pueden ser *edge-optimized* (vía CloudFront) o privadas
LÍMITES CLAVE	Payload de 10 MB; 10 000 peticiones/s por cuenta y región (ampliable); el backend debe responder en 29 s (REST; ampliable en API regionales y privadas desde 2024) o 30 s (HTTP API): para trabajos largos, responde un 202 y procesa de forma asíncrona

Característica	REST API	HTTP API	WebSocket API
Modelo y precio	Petición-respuesta; la más cara	Petición-respuesta; más barata y rápida	Conexión persistente bidireccional
Endpoints	Edge-optimized, regional, privado	Regional	Regional
Autorización	IAM, Cognito, Lambda, políticas de recurso	IAM, JWT (OIDC/OAuth 2.0), Lambda	IAM, Lambda en `$connect`
Caché y AWS WAF	✔	✘	✘

Empieza por **HTTP API**, la opción por defecto si solo enrutas a Lambda o a un servicio HTTP con JWT o IAM. Elige **REST API** si necesitas API keys, caché, WAF, validación, endpoints privados o transformar peticiones, y **WebSocket** para chats, notificaciones o paneles en tiempo real.

Stages. Una API se compone de **rutas** (`POST /pedidos`), cada una con su integración, y se despliega en **stages** (`dev`, `prod`) con su propia URL, límites, logs y **stage variables** (por ejemplo, a qué alias de Lambda apuntar). En REST cada cambio exige un despliegue explícito y los stages admiten canary; las HTTP API pueden desplegarse solas. Un **dominio personalizado** con certificado de ACM publica `api.miempresa.com`.

Integraciones — **Lambda proxy** (pasa la petición completa como evento y la función devuelve `statusCode`, `headers` y `body`; la más común), **HTTP** (cualquier endpoint, y servicios privados de tu VPC a través de un **VPC Link**), **servicios AWS** (llama directamente a `SendMessage` de SQS, `PutItem` de DynamoDB o `StartExecution` de Step Functions, sin Lambda intermedia) y **mock**.
Autorización — **IAM** (peticiones firmadas con SigV4, ideal entre servicios); **Cognito** (REST), cuyos *user pools* registran e identifican a los usuarios de tu aplicación (capítulo «IAM (I): identidades — usuarios, grupos, roles y permission sets»), o **JWT authorizer** (HTTP API), que valida los *JSON Web Tokens* de cualquier proveedor OIDC (Cognito, Auth0, Okta, Entra ID...); **Lambda authorizer**, tu función decide y su resultado se cachea (300 s por defecto, hasta 1 h); y **políticas de recurso** (REST) por cuenta, IP o VPC endpoint. La combinación típica (Cognito, JWT, Lambda y DynamoDB) se monta paso a paso en el capítulo de Arquitecturas de referencia.
Throttling — *token bucket* de 10 000 peticiones/s con ráfagas de 5000 por cuenta y región, más límites por stage y método; el exceso recibe **429 Too Many Requests**. En REST, los **usage plans** asocian **API keys** a límites y cuotas por cliente (por ejemplo, 1000 peticiones al día en el plan gratuito).
Caché (REST) — por stage, de 0,5 GB a 237 GB, TTL de 300 s por defecto (máx. 3600 s); se paga por hora, se use o no.
CORS — si `app.ejemplo.com` llama a una API de otro dominio, el navegador exige cabeceras `Access-Control-Allow-*` y a menudo un `OPTIONS` previo. Las HTTP API traen CORS integrado; en REST con proxy, **tu función** debe devolver las cabeceras.

> **CUIDADO**
>
> Las API keys **identifican** a un cliente para aplicarle un plan, pero **no autentican**: cualquiera que vea la clave puede usarla. Combínalas con IAM, Cognito o un Lambda authorizer.

9.8 AWS Step Functions

Encadenar funciones que se llaman entre sí produce código frágil: ¿quién reintenta?, ¿qué pasa si falla el tercer paso?, ¿dónde se ve el estado? **Step Functions** lo resuelve con **máquinas de estado**: defines el flujo de forma declarativa en **Amazon States Language (ASL)**, un JSON —o lo dibujas en **Workflow Studio**—, y el servicio lo ejecuta, guarda el estado de cada ejecución y gestiona reintentos y errores.

FICHA DEL SERVICIO

AWS Step Functions

Orquestación serverless de flujos de trabajo

TIPO	Serverless; Standard por transición de estado, Express por ejecución y duración
IDEAL PARA	Procesos de negocio, ETL, microservicios, aprobaciones humanas, proceso masivo en paralelo

	Standard	Express
Duración máxima	1 año	5 minutos
Semántica	Exactamente una vez	Al menos una vez (asíncrona) o como máximo una vez (síncrona)
Precio e historial	Por transición; historial de 90 días en la consola	Por ejecución, duración y memoria; historial en CloudWatch Logs
Esperar trabajos y *callbacks*	✔	✘
Casos típicos	Procesos largos, aprobaciones, ETL	Eventos de alto volumen (más de 100 000 ejecuciones/s)

Task — hace el trabajo: invoca una Lambda, llama a un servicio de AWS o a una API HTTPS, o espera un *callback*.
Choice, Parallel, Map — bifurcan según los datos, ejecutan ramas en paralelo o recorren una lista; el Map **distribuido** procesa millones de objetos de S3 con hasta 10 000 ejecuciones hijas a la vez.
Wait, Pass, Succeed, Fail — esperan (sin pagar cómputo), transforman datos o terminan.
Cada Task, Parallel o Map puede declarar **Retry** (qué errores reintentar, cuántas veces y con qué *backoff*) y **Catch** (a qué estado saltar al agotar los reintentos). Los errores se identifican por nombre: los tuyos o predefinidos como `States.Timeout`, `States.TaskFailed` o el comodín `States.ALL`.

```json
"Cobrar": {
  "Type": "Task",
  "Resource": "arn:aws:states:::lambda:invoke",
  "Parameters": {
    "FunctionName": "cobrar-pedido",
    "Payload.$": "$"
  },
  "OutputPath": "$.Payload",
  "Retry": [{
    "ErrorEquals": ["States.TaskFailed"],
    "IntervalSeconds": 2,
    "MaxAttempts": 3,
    "BackoffRate": 2
  }],
  "Catch": [{
    "ErrorEquals": ["States.ALL"],
    "ResultPath": "$.error",
    "Next": "NotificarFallo"
  }],
  "Next": "EnviarPedido"
}
```

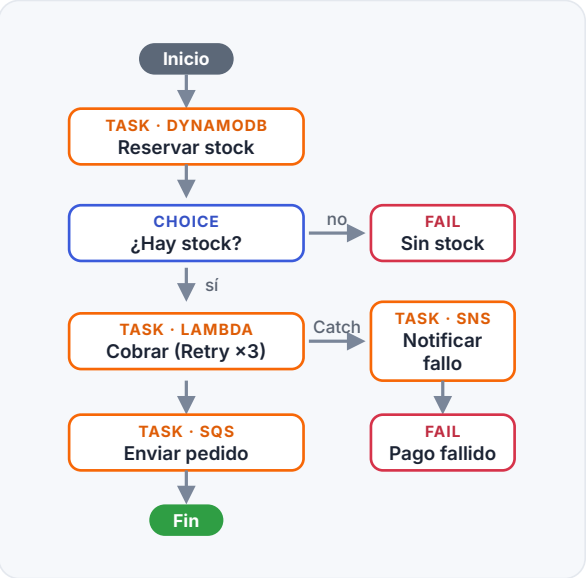


Figura 58 · Flujo de un pedido; a la izquierda, el estado «Cobrar» en ASL.

Step Functions no necesita Lambda para todo: con las **integraciones optimizadas** (DynamoDB, SQS, SNS, ECS/Fargate, Batch, Glue, EventBridge, Bedrock…) y las del **AWS SDK** (`arn:aws:states:::aws-sdk:servicio:acción`, más de 200 servicios) llama a las API directamente, de tres maneras: **Request Response** sigue en cuanto el servicio responde; **Run a Job** (`.sync`) espera a que termine el trabajo, como una tarea de ECS o un *job* de Glue, y **Wait for Callback** (`.waitForTaskToken`) se pausa hasta que alguien devuelve un *token*, por ejemplo tras una aprobación humana. Además transforma datos con JSONata y permite relanzar (*redrive*) una ejecución desde el paso que falló.

9.9 Desplegar con AWS SAM

AWS SAM (*Serverless Application Model*) es una extensión de AWS CloudFormation, el servicio de infraestructura como código de AWS, que verás junto a CDK y Terraform en el capítulo de Infraestructura como código. Con recursos abreviados describes funciones, API y eventos, y su CLI

construye, prueba en local y despliega (`sam build` , `sam local invoke` , `sam deploy --guided`). Estas líneas crean función, rol, HTTP API, tabla, alias y despliegue canary:

```yaml
Transform: AWS::Serverless-2016-10-31
Resources:
  CrearPedido:
    Type: AWS::Serverless::Function
    Properties:
      CodeUri: src/
      Handler: app.lambda_handler
      Runtime: python3.13
      Architectures: [arm64]
      Environment: { Variables: { TABLA_PEDIDOS: !Ref Pedidos } }
      Policies: [{ DynamoDBCrudPolicy: { TableName: !Ref Pedidos } }]
      AutoPublishAlias: prod
      DeploymentPreference: { Type: Canary10Percent5Minutes }
      Events:
        Api: { Type: HttpApi, Properties: { Path: /pedidos, Method: post } }
  Pedidos:
    Type: AWS::Serverless::SimpleTable
```

RESUMEN DEL CAPÍTULO

- Serverless: nada que administrar, escala sola (incluso a cero), pago por uso y alta disponibilidad.
- Lambda: un **handler** por evento, hasta 15 min y 10 240 MB de memoria, con CPU proporcional.
- El entorno pasa por **Init → Invoke (reutilizable) → Shutdown**; los **cold starts** se mitigan con SnapStart o concurrencia aprovisionada.
- La concurrencia regional (1000 por defecto) es compartida; la **reservada** garantiza y limita, la **aprovisionada** preinicializa.
- Invocación **síncrona** (reintenta el cliente), **asíncrona** (reintenta Lambda; destinos y DLQ) y **event source mapping** (Lambda sondea SQS, Kinesis o DynamoDB Streams).
- El **execution role** define qué hace la función; la **resource-based policy**, quién puede invocarla.
- Versiones inmutables y **aliases** ponderados permiten despliegues canary con *rollback* automático.
- API Gateway: REST (más funciones), HTTP (más barata) y WebSocket; autorización IAM, JWT/Cognito o Lambda; throttling con 429.
- Step Functions orquesta pasos con Retry y Catch: Standard (flujos largos) o Express (alto volumen).

Preguntas de repaso

1. **¿Por qué conviene crear los clientes del SDK fuera del handler?**
   Respuesta: Porque ese código se ejecuta una vez, en la fase Init, y se reutiliza en todas las invocaciones del mismo entorno.
2. **Una API en Java sufre cold starts en horas punta. ¿Qué opciones tienes?**
   Respuesta: SnapStart (sin coste extra en Java) o concurrencia aprovisionada sobre un alias, escalada con una programación.
3. **¿Qué ocurre si una invocación asíncrona falla repetidamente?**
   Respuesta: Lambda la reintenta (2 veces por defecto) y, si sigue fallando o caduca, la envía al destino de fallo o a la DLQ; sin ellos, se pierde.
4. **Una función en una VPC no puede llamar a una API externa. ¿Por qué?**
   Respuesta: Sus ENI no tienen IP pública: necesita una subred privada con ruta a un NAT Gateway.
5. **Necesitas API keys con cuotas por cliente y caché. ¿REST o HTTP API?**
   Respuesta: REST API: las HTTP API no tienen *usage plans*, API keys ni caché.
6. **¿Cuándo elegirías Express en lugar de Standard?**
   Respuesta: Para ejecuciones cortas (menos de 5 minutos) y de muy alto volumen, donde «al menos una vez» es aceptable.

CAPÍTULO 10

Almacenamiento: S3 y compañía

Guarda cualquier cantidad de datos de forma duradera, segura y barata: objetos, ficheros, archivo y copias de seguridad.

EN ESTE CAPÍTULO APRENDERÁS

1. Diferenciar almacenamiento de bloque, de ficheros y de objetos
2. Los conceptos de S3: buckets, objetos, claves y durabilidad
3. Elegir clase de almacenamiento y automatizar con *lifecycle*
4. Proteger datos: versionado, Object Lock, replicación y cifrado
5. Controlar el acceso: *bucket policies*, URL prefirmadas y endpoints
6. Conocer EFS, FSx, Storage Gateway, DataSync y AWS Backup

SERVICIOS PROTAGONISTAS

Amazon S3 · Amazon S3 Glacier · Amazon EFS · Amazon FSx · AWS DataSync · AWS Backup

10.1 Bloque, ficheros y objetos

Antes de elegir servicio conviene distinguir tres formas de guardar datos, porque responden a necesidades distintas:

Bloque · Amazon EBS
Un disco duro virtual

El sistema operativo ve un volumen de bloques y lo formatea. Baja latencia; se conecta a una instancia en una sola AZ. Para sistemas operativos y bases de datos (lo viste en el capítulo de EC2).

Ficheros · EFS y FSx
Una carpeta compartida

Directorios accesibles por NFS o SMB desde muchas máquinas a la vez, con permisos y bloqueos. Para contenido compartido, directorios de usuario y aplicaciones heredadas.

Objetos · Amazon S3
Un almacén accesible por API

Cada objeto (datos y metadatos) se guarda y lee entero con una API HTTP y una clave única. Escala sin límite práctico y es el más barato. Para backups, *data lakes*, web y logs.

≈ **ANALOGÍA**

EBS es el disco de tu portátil; EFS, la unidad compartida de la oficina; S3, un guardarropa con resguardo: entregas el abrigo (el objeto), te dan un número (la clave) y lo recuperas entero, sin poder cambiarle un botón en el sitio.

10.2 Amazon S3: conceptos esenciales

Amazon S3 (*Simple Storage Service*) es probablemente el servicio más usado de AWS: almacena desde las imágenes de una web hasta data lakes de exabytes, y casi todos los demás servicios lo usan para guardar logs, copias o artefactos.

FICHA DEL SERVICIO

Amazon S3

Almacenamiento de objetos ilimitado, duradero y económico

TIPO	Serverless; pago por GB-mes almacenado, por petición y por transferencia de salida
ÁMBITO	Buckets regionales, con nombres únicos en todo AWS
DURABILIDAD	Diseñado para el 99,999999999 % (11 nueves): datos repartidos en al menos 3 AZ, salvo las clases One Zone
LÍMITES CLAVE	Objetos de hasta 50 TB; un único PUT sube hasta 5 GB (por encima, *multipart upload*); 3500 escrituras y 5500 lecturas por segundo por prefijo

Bucket — contenedor de objetos que creas en una región; los datos no salen de ella salvo que los copies o repliques. Su nombre (3–63 caracteres: minúsculas, números, guiones y puntos) es **único en todo AWS**: si alguien tiene `fotos`, tú no.
Objeto — los datos —desde 0 bytes hasta 50 TB (desde diciembre de 2025; antes, 5 TB)— más metadatos (tipo de contenido, metadatos propios, hasta 10 etiquetas) y, si hay versionado, un identificador de versión. Es inmutable: para cambiarlo se sube entero de nuevo.
Clave — el nombre completo del objeto en el bucket, como `facturas/2026/03/f-0042.pdf`.
Prefijo — S3 no tiene carpetas, su espacio de nombres es plano. La consola las simula con prefijos (`facturas/2026/`) y el delimitador `/`, que también sirven para filtrar listados, reglas y permisos.

BASH

```
aws s3 mb s3://facturas-acme-prod --region eu-west-1          # crear bucket
aws s3 cp f-0042.pdf s3://facturas-acme-prod/facturas/2026/03/
aws s3 ls s3://facturas-acme-prod/facturas/2026/ --recursive
aws s3 sync ./web s3://web-acme --delete                       # sincronizar carpeta
```

Con una durabilidad de 11 nueves, si guardas diez millones de objetos puedes esperar perder uno cada 10 000 años de media. Desde diciembre de 2020, S3 ofrece además **consistencia fuerte**: tras escribir o borrar un objeto, cualquier lectura o listado posterior ve el cambio. Si dos clientes escriben a la vez la misma clave gana el último, salvo que uses **escrituras condicionales** (`If-None-Match`, `If-Match`, disponibles desde 2024).

> **NOTA**
>
> Además de los buckets de propósito general existen **directory buckets** (para S3 Express One Zone), **table buckets** (S3 Tables, tablas Apache Iceberg gestionadas) y **vector buckets** (S3 Vectors, para búsquedas por similitud en aplicaciones de IA).

10.3 Clases de almacenamiento

Cada objeto tiene una **clase de almacenamiento** que fija su precio por GB, su disponibilidad y cuánto cuesta —en dinero y en tiempo— leerlo. La durabilidad es la misma en todas; lo que cambia es el equilibrio entre almacenar barato y leer barato.

Clase	Uso típico	AZ	Disponib.	Mínimo	Recuperación
Standard	Datos de uso frecuente	≥ 3	99,99 %	—	Milisegundos
Intelligent-Tiering	Patrón de acceso desconocido o cambiante	≥ 3	99,9 %	—	Milisegundos (capas de archivo opcionales: horas)
Standard-IA	Acceso infrecuente pero inmediato	≥ 3	99,9 %	30 días	Milisegundos, con cargo por GB
One Zone-IA	Copias que puedes regenerar	1	99,5 %	30 días	Milisegundos, con cargo por GB
Glacier Instant Retrieval	Archivo consultado unas pocas veces al año	≥ 3	99,9 %	90 días	Milisegundos, con cargo por GB
Glacier Flexible Retrieval	Archivo y copias de seguridad	≥ 3	99,99 %	90 días	De minutos a 12 horas
Glacier Deep Archive	Archivo a largo plazo y normativo	≥ 3	99,99 %	180 días	De 12 a 48 horas
Express One Zone	Latencia mínima (ML, analítica)	1	99,95 %	—	Un dígito de milisegundos

- **Intelligent-Tiering** es la opción por defecto cuando no conoces el patrón de acceso: por una pequeña cuota de monitorización por objeto, mueve cada objeto a una capa infrecuente tras 30 días sin accesos y a una de archivo instantáneo tras 90, y lo devuelve a la frecuente en cuanto se lee, sin cargos de recuperación. Los objetos de menos de 128 KB no se mueven.
- **Glacier Flexible Retrieval** y **Deep Archive** no se leen directamente: pides una **restauración** que crea una copia temporal. Flexible ofrece tres velocidades —*Expedited* (1–5 min), *Standard* (3–5 h) y *Bulk* (5–12 h, gratuita)— y Deep Archive, Standard (hasta 12 h) y Bulk (hasta 48 h).
- **Express One Zone** usa directory buckets en la AZ que elijas, junto a tu cómputo, para entrenamiento de ML o analítica interactiva.

> **CLAVE PARA LA CERTIFICACIÓN**
>
> Las clases IA y Glacier Instant Retrieval facturan como mínimo 128 KB por objeto, y todas las que tienen duración mínima cobran los días restantes si borras o mueves antes el objeto. Glacier Flexible y Deep Archive añaden unos 40 KB de metadatos por objeto: archivar millones de ficheros diminutos sale caro, mejor agruparlos antes.

10.4 Lifecycle: mover y borrar automáticamente

Las **reglas de lifecycle** aplican a los objetos que cumplen un filtro (prefijo, etiquetas o tamaño) dos tipos de acciones: **transiciones** a clases más baratas y **expiraciones** (borrado). Siguen una cascada: solo se baja de clase, nunca se sube.

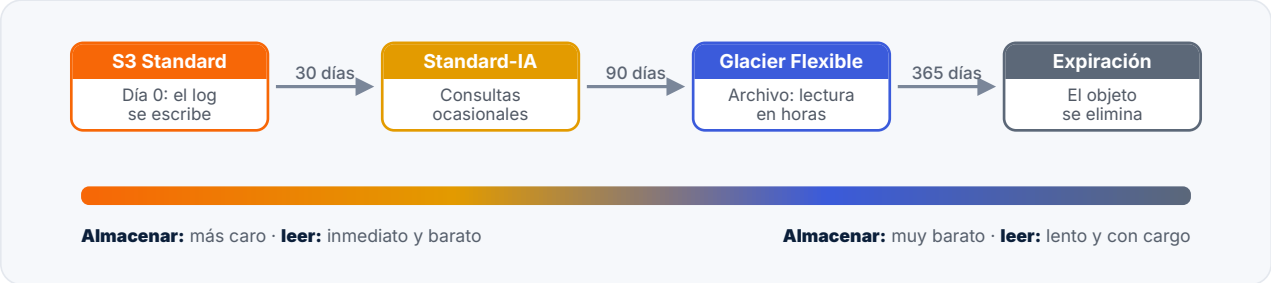


Figura 59 · Ciclo de vida de los logs con la regla del ejemplo: cada transición abarata el almacenamiento y encarece la lectura.

```json
{
  "Rules": [{
    "ID": "logs-a-archivo",
    "Filter": { "Prefix": "logs/" },
    "Status": "Enabled",
    "Transitions": [
      { "Days": 30, "StorageClass": "STANDARD_IA" },
      { "Days": 90, "StorageClass": "GLACIER" }
    ],
    "Expiration": { "Days": 365 },
    "NoncurrentVersionExpiration": { "NoncurrentDays": 30 },
    "AbortIncompleteMultipartUpload": { "DaysAfterInitiation": 7 }
  }]
}
```

En la API, `GLACIER` es Glacier Flexible Retrieval, `GLACIER_IR` Instant Retrieval y `DEEP_ARCHIVE` Deep Archive. La regla se aplica así:

```bash
aws s3api put-bucket-lifecycle-configuration --bucket logs-acme \
  --lifecycle-configuration file://lifecycle.json
```

> **CUIDADO**
>
> S3 exige 30 días en Standard antes de pasar a Standard-IA o One Zone-IA y, desde 2024, por defecto no transiciona objetos de menos de 128 KB. Cada transición se cobra como una petición: mover millones de objetos pequeños puede costar más de lo que ahorras.

10.5 Versionado, MFA Delete y Object Lock

Con el **versionado** activado, cada escritura crea una versión nueva y un borrado no elimina nada: añade un **delete marker** que oculta el objeto (y que puedes quitar para «resucitarlo»). Un bucket puede estar sin versionar, con versionado activado o suspendido; una vez activado, nunca vuelve al estado inicial. Cada versión se factura como un objeto completo: combínalo con `NoncurrentVersionExpiration`. **MFA Delete** exige además un código MFA para borrar versiones definitivamente o cambiar el estado del versionado; solo el usuario root puede activarlo, y únicamente por CLI o API.

Object Lock aplica el modelo **WORM** (*write once, read many*): impide borrar o sobrescribir una versión durante un **periodo de retención** o mientras tenga una **retención legal** (*legal hold*, sin fecha de fin). Requiere versionado, puede fijar una retención por defecto en el bucket y tiene dos modos:

Governance
Protección con válvula de escape

Los usuarios con el permiso `s3:BypassGovernanceRetention` pueden saltarse la retención o acortarla. Útil contra errores humanos y para probar antes de pasar a Compliance.

Compliance
Inmutable de verdad

Nadie, ni siquiera el usuario root, puede borrar la versión ni acortar la retención hasta que expire. Para normativas financieras o sanitarias y protección frente a *ransomware*.

10.6 Replicación entre buckets

La **replicación** copia objetos de forma asíncrona a otro bucket, en otra región (**CRR**, *Cross-Region Replication*: recuperación ante desastres, latencia, requisitos geográficos) o en la misma (**SRR**: agregar logs, copiar entre cuentas de producción y pruebas).

- Exige versionado en origen y destino y un rol IAM que S3 asume para copiar; el destino puede estar en otra cuenta y usar otra clase de almacenamiento.
- Solo replica objetos nuevos; para los ya existentes se usa **S3 Batch Replication**.
- Los delete markers se replican solo si lo indicas, y los borrados de versiones concretas nunca, lo que protege frente a borrados maliciosos. Tampoco es transitiva: con A → B → C, los objetos de A no llegan a C.
- **Replication Time Control (RTC)** garantiza por SLA que el 99,99 % de los objetos se replica en 15 minutos.



Figura 60 · Replicación entre regiones (CRR) hacia una cuenta de recuperación ante desastres.

10.7 Seguridad en S3

La mayoría de las fugas de datos en S3 de la última década se debieron a buckets públicos por error. Hoy los valores por defecto son seguros, pero conviene entender las capas:

Block Public Access — cuatro ajustes, a nivel de cuenta, bucket o access point, que impiden ACL y políticas públicas aunque alguien las escriba. Activado por defecto en los buckets nuevos desde 2023; actívalo también a nivel de cuenta.
Object Ownership — con **Bucket owner enforced**, valor por defecto desde 2023, las ACL quedan deshabilitadas y el dueño del bucket posee todos los objetos: el acceso se controla solo con políticas.
Políticas — las **políticas IAM** dicen qué puede hacer una identidad; la **bucket policy**, quién puede acceder al bucket, incluidas otras cuentas y servicios. Un `Deny` explícito en cualquiera de ellas gana siempre. Cómo se evalúan, con una bucket policy de ejemplo, lo viste en «IAM (II): permisos — políticas y evaluación».

Cifrado

En tránsito, S3 admite HTTPS, y puedes exigirlo con una bucket policy que deniegue las peticiones con `aws:SecureTransport` igual a `false`. En reposo, **todos los objetos nuevos se cifran por defecto** desde 2023 con **SSE-S3**: S3 gestiona las claves, sin coste ni configuración. Por bucket u objeto puedes elegir otro método:

SSE-KMS — con una clave de AWS KMS, controlas con políticas quién puede usarla y cada uso queda auditado en CloudTrail. Cada operación llama a KMS (con coste): activa las **S3 Bucket Keys**, que reducen esas llamadas hasta un 99 %.
DSSE-KMS — dos capas de cifrado con KMS, para normativas que exigen doble cifrado.
SSE-C — envías tu propia clave en cada petición (siempre por HTTPS) y S3 no la guarda.
Cómo funcionan las claves de KMS y qué permisos necesitas para leer un objeto cifrado se explica en el capítulo «Seguridad: cifrado con KMS, secretos y protección».

URL prefirmadas, Access Points y VPC endpoints

Una **URL prefirmada** concede acceso temporal a un objeto concreto con los permisos de quien la firma, sin abrir el bucket. Es el patrón estándar para que un navegador suba o descargue directamente de S3 sin pasar el fichero por tu servidor. Con credenciales de un usuario IAM puede durar hasta 7 días; firmada con credenciales temporales de un rol, caduca como muy tarde con ellas.

```python
import boto3

s3 = boto3.client("s3")
url = s3.generate_presigned_url(
    "put_object",
    Params={"Bucket": "subidas-app", "Key": "fotos/u-42/perfil.jpg",
            "ContentType": "image/jpeg"},
    ExpiresIn=900,  # 15 minutos
)
# El navegador hace PUT de la foto directamente a esa URL
```

Para descargar basta la CLI: `aws s3 presign s3://informes/2026/q3.pdf --expires-in 3600`.

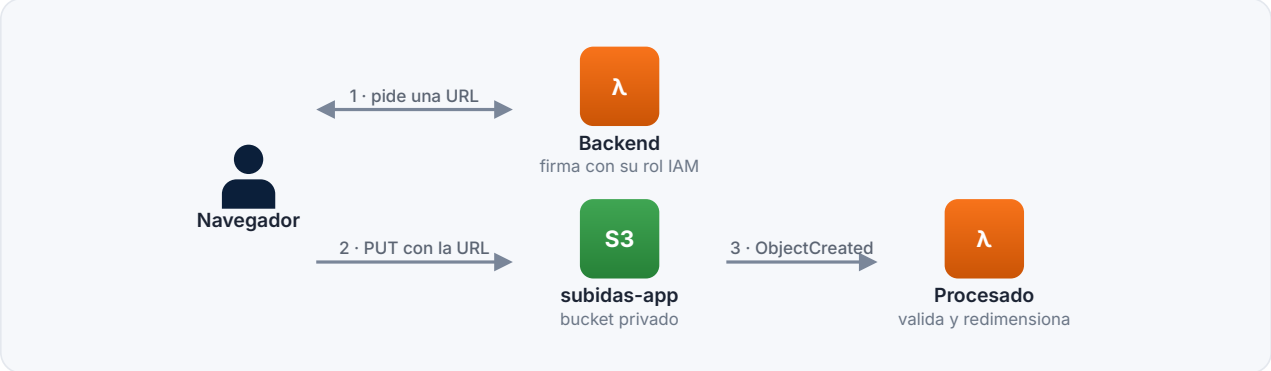


Figura 61 · Subida directa a S3 con una URL prefirmada: el fichero nunca pasa por tu backend.

- Un **Access Point** es un punto de acceso con nombre, su propia política y, si quieres, restringido a una VPC. Simplifica buckets compartidos por muchos equipos: una política por consumidor en lugar de una bucket policy gigante. Un **Multi-Region Access Point** ofrece un único endpoint global que enruta cada petición al bucket replicado más cercano.
- El **acceso entre cuentas** exige dos permisos: que la bucket policy permita a la otra cuenta (o a un rol suyo) y que una política IAM de esa cuenta conceda la acción a quien accede. Alternativa: que la otra cuenta asuma un rol de la tuya.
- Desde tu VPC, un **gateway endpoint** para S3 (gratuito, mediante una ruta) evita pasar por Internet o por un NAT Gateway; un **interface endpoint** (de pago) además es accesible desde tu red local. La bucket policy puede exigir `aws:SourceVpce` para aceptar solo tráfico de tu endpoint (capítulo de Redes).

BUENA PRÁCTICA

Activa Block Public Access a nivel de cuenta, exige TLS, usa SSE-KMS con Bucket Keys para datos sensibles, revisa los hallazgos de IAM Access Analyzer y, para auditar accesos, activa los eventos de datos de CloudTrail o los *server access logs*. Amazon Macie detecta datos personales en tus buckets.

10.8 Rendimiento

Peticiones por prefijo — al menos 3500 escrituras y 5500 lecturas por segundo **por prefijo**, sin límite de prefijos: reparte las claves y paraleliza. Mientras S3 escala puede responder `503 Slow Down`; los SDK reintentan con *backoff* exponencial.
Multipart upload — divide el objeto en partes (hasta 10 000) que se suben en paralelo y se reintentan por separado. Recomendado desde unos 100 MB y obligatorio por encima de 5 GB; `aws s3 cp` lo usa automáticamente. Las subidas que nunca se completan dejan partes invisibles que se **facturan**: de ahí la regla `AbortIncompleteMultipartUpload` del ejemplo de lifecycle.
Byte-range fetches — la cabecera `Range` descarga solo una parte del objeto, para paralelizar descargas o leer la cabecera de un fichero grande.
Transfer Acceleration — enruta las transferencias por las ubicaciones *edge* de CloudFront y la red de AWS (`bucket.s3-accelerate.amazonaws.com`); útil desde lejos, con un cargo por GB acelerado.

10.9 Eventos de S3

S3 puede notificar cuando se crea, borra, restaura, replica o transiciona un objeto (`s3:ObjectCreated:*`, `s3:ObjectRemoved:*`…), filtrando por prefijo y sufijo. Los destinos directos son Lambda, colas SQS Standard y topics SNS Standard, y el permiso se concede en el destino (política de la cola o del topic, o permiso de invocación). Si activas **EventBridge** en el bucket, todos los eventos llegan también al bus por defecto, con filtros más ricos, varios destinos por regla, archivo y repetición.

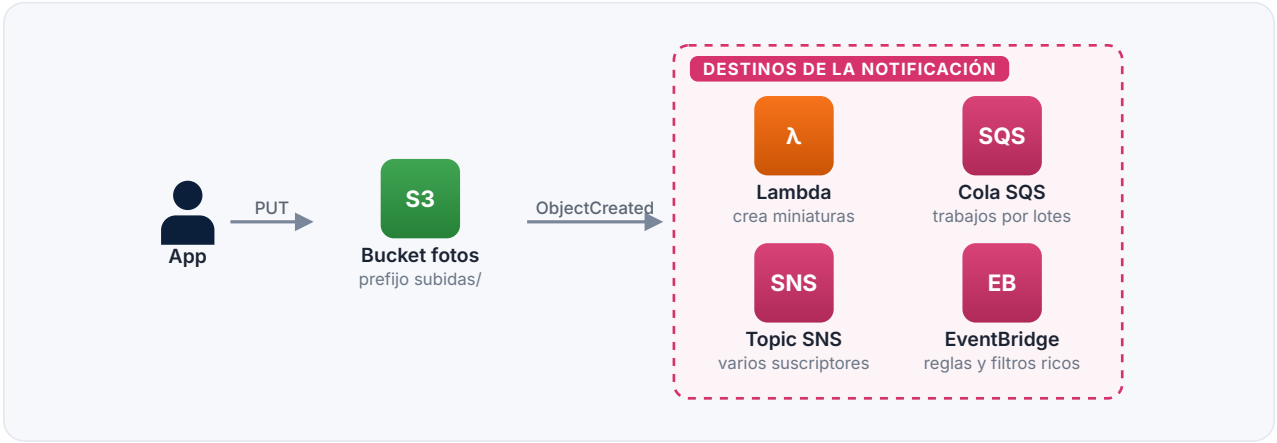


Figura 62 · Un objeto nuevo dispara una notificación hacia los destinos configurados.

CUIDADO

Las notificaciones suelen llegar en segundos, pero **al menos una vez** y sin orden garantizado: el consumidor debe ser idempotente. Y no escribas el resultado en el mismo bucket y prefijo que dispara la función, o crearás un bucle infinito.

10.10 Alojamiento web estático

S3 puede servir una web estática (HTML, CSS, JavaScript, imágenes) desde el **website endpoint** del bucket, con documento de índice, página de error y reglas de redirección. Pero ese endpoint solo habla HTTP y exige un bucket público. En producción se pone **CloudFront** delante de un bucket **privado**: obtienes HTTPS con tu dominio y caché en el borde, y solo la distribución puede leer el bucket, gracias a **Origin Access Control** (OAC). Además, la transferencia de S3 a CloudFront no se cobra. La arquitectura completa, con su bucket policy, está en el capítulo «CloudFront, Route 53 y el borde de la red».

10.11 Consultar datos y controlar costes

S3 Select, que filtraba con SQL el contenido de un objeto CSV o JSON, no está disponible para clientes nuevos desde julio de 2024. Para consultar con SQL los datos de tus buckets se usa **Amazon Athena**, que verás en el capítulo de Bases de datos. La factura de S3 tiene cuatro componentes:

Almacenamiento	Peticiones	Transferencia	Gestión
GB-mes según la clase: del orden de 2 céntimos de dólar por GB en Standard, mucho menos en Glacier.	Escribir y listar cuesta unas diez veces más que leer; IA y Glacier suman la recuperación por GB.	Entrar es gratis; salir a Internet o a otra región se paga por GB (incluida la replicación).	Monitorización de Intelligent-Tiering, Inventory, Storage Lens avanzado, RTC...

> **$ COSTES**
>
> Trampas típicas: versiones antiguas que nadie expira, subidas multipart incompletas, millones de objetos diminutos en clases con mínimo de 128 KB o con transiciones de lifecycle, y descargas masivas a Internet sin CloudFront. Con un bucket **Requester Pays**, quien descarga paga peticiones y transferencia. **S3 Storage Lens** te muestra dónde se va el dinero.

Para gestionar miles de millones de objetos, **S3 Inventory** genera listados diarios o semanales de tus objetos y sus propiedades (clase, cifrado, réplica...), y **S3 Batch Operations** aplica una acción a todos los de una lista: copiar, etiquetar, restaurar desde Glacier o invocar una Lambda por objeto.

10.12 Más allá de S3

Amazon EFS: ficheros compartidos para Linux

Amazon EFS, que ya presentamos en el capítulo de EC2, es el sistema de ficheros NFS compartido de AWS: elástico (pagas por GB usado), replicado en varias AZ (o en una sola, más barato) y con clases Infrequent Access y Archive para lo que no se lee. Lo montan a la vez miles de instancias EC2, contenedores de ECS, EKS o Fargate y funciones Lambda, pero solo clientes Linux: para Windows está FSx. No lo confundas con **Mountpoint for Amazon S3**, un cliente que monta un bucket en Linux como si fuera un directorio: rinde muy bien leyendo ficheros grandes y escribiendo ficheros nuevos, pero no es un sistema de ficheros POSIX completo (no permite, por ejemplo, editar en el sitio un fichero existente).

Amazon FSx: sistemas de ficheros conocidos, gestionados

Servicio	Protocolos	Para qué
FSx for Windows File Server	SMB	Recursos compartidos de Windows integrados con Active Directory (permisos NTFS, DFS)
FSx for Lustre	Lustre (POSIX)	HPC, ML y renderizado: *throughput* muy alto y latencia inferior al milisegundo; se enlaza con un bucket S3
FSx for NetApp ONTAP	NFS, SMB, iSCSI	Migrar cabinas NetApp: multiprotocolo, snapshots, SnapMirror, deduplicación y compresión
FSx for OpenZFS	NFS	Migrar servidores ZFS o Linux: snapshots y clones instantáneos, baja latencia

Híbrido, migración y copias de seguridad

AWS Storage Gateway — *appliance* virtual en tu centro de datos (o en EC2) que expone almacenamiento de AWS con protocolos locales y una caché: **S3 File Gateway** (NFS/SMB sobre objetos de S3), **Volume Gateway** (volúmenes iSCSI con snapshots en AWS) y **Tape Gateway** (cintas virtuales para tu software de backup, archivadas en Glacier).

AWS DataSync — transferencias en línea para migrar o sincronizar periódicamente: un agente lee de NFS, SMB, HDFS u otros almacenes de objetos (incluidas otras nubes) y copia a S3, EFS o FSx con cifrado, verificación de integridad, programación y límite de ancho de banda.

Snow Family — dispositivos físicos (Snowball Edge) para mover decenas o cientos de terabytes sin red suficiente o para cómputo en el edge. AWS ha retirado parte de la familia (Snowmobile, Snowcone): consulta la disponibilidad actual y prefiere DataSync o Direct Connect cuando la red lo permita.

AWS Backup — copias centralizadas por políticas para EBS, EC2, RDS y Aurora, DynamoDB, EFS, FSx, S3 y más. Los **backup plans** (frecuencia, retención, paso a almacenamiento frío) se asignan por etiquetas; admite copias entre regiones y cuentas, **Vault Lock** (WORM), políticas de backup de AWS Organizations y pruebas de restauración automatizadas.

> **BUENA PRÁCTICA**
>
> Aplica la regla 3-2-1 en la nube: varias copias, en al menos dos ubicaciones (otra región) y una inmutable y aislada (otra cuenta, con Vault Lock u Object Lock en modo Compliance). Y prueba las restauraciones: una copia que nunca has restaurado es solo una esperanza.

¿Qué servicio de almacenamiento elijo?

Si necesitas…	Usa
Disco para el sistema operativo o una base de datos en EC2	**Amazon EBS**
Guardar ficheros de cualquier tamaño a escala: backups, data lake, web, logs	**Amazon S3**
Archivar durante años al mínimo coste	**S3 Glacier Deep Archive**
Una carpeta compartida por muchas instancias Linux, contenedores o Lambda	**Amazon EFS**
Recursos compartidos de Windows con Active Directory	**FSx for Windows File Server**
Rendimiento extremo para HPC o entrenamiento de ML	**FSx for Lustre**
Ampliar el almacenamiento de tu centro de datos con la nube	**AWS Storage Gateway**
Migrar o sincronizar terabytes por la red	**AWS DataSync**
Políticas de copia centralizadas para muchos servicios y cuentas	**AWS Backup**

RESUMEN DEL CAPÍTULO

- **Bloque** (EBS) para discos de una instancia, **ficheros** (EFS, FSx) para carpetas compartidas, **objetos** (S3) para todo lo demás a escala.
- S3 guarda objetos inmutables con una clave en buckets regionales de nombre global; 11 nueves de durabilidad y consistencia fuerte.
- Las clases de almacenamiento equilibran precio por GB y coste de lectura; Intelligent-Tiering es la opción por defecto si no conoces el patrón.
- Las reglas de lifecycle transicionan y expiran objetos automáticamente; vigila duraciones mínimas, el mínimo de 128 KB y las multipart incompletas.
- Versionado, MFA Delete y Object Lock (Governance o Compliance) protegen contra borrados; la replicación CRR/SRR copia a otro bucket.
- Seguridad por capas: Block Public Access, ACL deshabilitadas, bucket policies, cifrado por defecto (SSE-S3, SSE-KMS, DSSE-KMS, SSE-C) y URL prefirmadas.
- Rendimiento: paraleliza por prefijos, usa multipart y byte-range; los eventos de S3 alimentan Lambda, SQS, SNS y EventBridge.
- Para webs estáticas, CloudFront con OAC delante de un bucket privado.
- EFS (NFS para Linux), FSx (Windows, Lustre, ONTAP, OpenZFS), Storage Gateway, DataSync y AWS Backup completan el catálogo.

Preguntas de repaso

1. **¿Qué hace falta para que un objeto en Glacier Deep Archive sea legible?**
   Respuesta: Solicitar una restauración (Standard, hasta 12 h; Bulk, hasta 48 h), que crea una copia temporal durante los días indicados.
2. **Unos logs se consultan mucho la primera semana y casi nunca después; deben conservarse un año. ¿Qué harías?**
   Respuesta: Una regla de lifecycle que los pase a Standard-IA o Glacier Instant Retrieval tras 30 días, a Glacier Flexible o Deep Archive después y los expire a los 365 días.
3. **Un auditor exige que nadie, ni siquiera root, pueda borrar ciertos documentos durante 7 años. ¿Qué configuras?**
   Respuesta: Object Lock en modo Compliance con una retención de 7 años (requiere versionado).
4. **¿Cómo permites que un navegador suba un fichero a un bucket privado sin pasar por tu servidor?**
   Respuesta: Generando en el backend una URL prefirmada para `PutObject` con caducidad corta.
5. **¿Por qué no basta el website endpoint de S3 para una web en producción?**
   Respuesta: Solo admite HTTP y exige un bucket público; con CloudFront y OAC obtienes HTTPS, dominio propio, caché y un bucket privado.
6. **Varios cientos de servidores Linux necesitan leer y escribir los mismos ficheros. ¿S3, EBS o EFS?**
   Respuesta: EFS: un sistema de ficheros NFS compartido que montan muchas instancias a la vez en varias AZ.
7. **¿Qué aporta SSE-KMS frente a SSE-S3?**
   Respuesta: Controlas con políticas quién puede usar la clave y cada uso queda auditado en CloudTrail; a cambio, cada operación llama a KMS (con coste), algo que las S3 Bucket Keys reducen.
8. **Unas instancias en subredes privadas leen terabytes de S3 a través de un NAT Gateway y la factura se dispara. ¿Qué cambias?**
   Respuesta: Creas un gateway endpoint para S3 (gratuito) asociado a sus tablas de rutas: el tráfico deja de pasar por el NAT.

CAPÍTULO 11

Bases de datos

Relacionales, NoSQL, en memoria y analíticas: cómo elegir la base de datos adecuada para cada problema y operarla en AWS sin sobresaltos.

EN ESTE CAPÍTULO APRENDERÁS

1. Elegir el tipo de base de datos según el patrón de acceso
2. Qué te quita de encima una base de datos gestionada
3. Alta disponibilidad en RDS: Multi-AZ frente a réplicas de lectura
4. La arquitectura de Aurora: almacenamiento compartido y endpoints
5. Modelar en DynamoDB: claves, índices, capacidad y consistencia
6. Cachés con ElastiCache, analítica con Redshift y migraciones con DMS

SERVICIOS PROTAGONISTAS

Amazon RDS · Amazon Aurora · Amazon DynamoDB · Amazon ElastiCache · Amazon Redshift · AWS DMS

11.1 Una base de datos para cada problema

Durante décadas, la respuesta automática a «¿dónde guardo los datos?» fue una base de datos relacional. Sigue siendo la mejor opción en muchísimos casos, pero no en todos: un carrito con millones de usuarios simultáneos, un ranking que cambia cada segundo o un informe que recorre cinco años de ventas tienen necesidades muy distintas. AWS defiende las **bases de datos de propósito específico** (*purpose-built*): eliges el motor que encaja con tu **patrón de acceso** y, si hace falta, combinas varios.

Para decidir, pregúntate cómo vas a leer y escribir (por clave, con SQL y *joins*, recorriendo relaciones, buscando texto, agregando millones de filas), qué latencia y escala necesitas, qué garantías transaccionales exiges y cuánta infraestructura quieres operar. Y distingue dos tipos de carga: **OLTP** (*Online Transaction Processing*), muchas operaciones pequeñas y concurrentes, como crear un pedido, y **OLAP** (*Online Analytical Processing*), pocas consultas enormes que agregan millones de filas, como un informe de ventas.

Tipo	Pensado para (ejemplos)	Servicio de AWS
Relacional (OLTP)	SQL, joins y transacciones ACID (ERP, tiendas, CRM)	Amazon RDS, Amazon Aurora
Clave-valor	Acceso por clave a cualquier escala en milisegundos (carritos, perfiles)	Amazon DynamoDB
Documental	Documentos JSON flexibles (catálogos, contenidos)	Amazon DocumentDB (API de MongoDB)
En memoria	Microsegundos y estructuras de datos (cachés, sesiones, rankings)	Amazon ElastiCache, Amazon MemoryDB
Columnar ancha	Escrituras masivas con el modelo de Cassandra (telemetría)	Amazon Keyspaces
Grafos	Relaciones entre entidades (redes sociales, fraude)	Amazon Neptune
Series temporales	Mediciones con marca de tiempo (IoT, métricas)	Amazon Timestream
Ledger	Historial inmutable y verificable (registros contables)	Amazon QLDB cerró en julio de 2025; AWS propone Aurora PostgreSQL
Búsqueda	Texto completo y relevancia (buscadores, logs)	Amazon OpenSearch Service
Data warehouse (OLAP)	Consultas analíticas sobre grandes volúmenes (BI, informes)	Amazon Redshift; Athena sobre S3

11.2 Gestionada frente a autogestionada

Nada te impide lanzar una instancia EC2, instalar PostgreSQL y administrarlo tú. Pero una base de datos en producción exige mucho más que instalarla: parches, copias y pruebas de restauración, réplicas, failover, monitorización, ampliar el disco a las tres de la mañana... Con un servicio gestionado, AWS asume esas tareas y tú te concentras en el modelo de datos y las consultas.

> ≈ **ANALOGÍA**
>
> EC2 es comprar una casa: decides cada detalle, pero las goteras las arreglas tú. RDS es un piso con conserje: amueblas a tu gusto y el mantenimiento lo hace otro. DynamoDB es un hotel: pides habitación y la usas.

	Base de datos en EC2	Amazon RDS / Aurora	Amazon DynamoDB
Esquema, consultas y usuarios	Tú	Tú	Tú
Capacidad y escalado	Tú	Compartido	AWS
Alta disponibilidad y failover	Tú	Compartido	AWS
Copias de seguridad	Tú	Compartido	Compartido
Instalación y parches del motor	Tú	AWS	AWS
Sistema operativo	Tú	AWS	AWS
Hardware, red y centro de datos	AWS	AWS	AWS

Compartido: tú decides y configuras (tamaño, Multi-AZ, retención) y AWS lo ejecuta.

Figura 63 · Quién se ocupa de cada tarea según el modelo de base de datos.

Con RDS o Aurora **sigue siendo tuyo** el esquema, los índices, las consultas, los usuarios del motor, el tamaño de la instancia, la red (subredes y Security Groups) y la retención de copias. ¿Cuándo tiene sentido EC2? Cuando necesitas un motor o versión que RDS no ofrece, acceso de administrador al sistema operativo o extensiones no soportadas. Entre ambos extremos está **RDS Custom** (Oracle y SQL Server): automatiza copias y monitorización, pero te deja entrar en el sistema operativo.

11.3 Amazon RDS

FICHA DEL SERVICIO

Amazon RDS

Bases de datos relacionales gestionadas

MOTORES	MySQL, PostgreSQL, MariaDB, Oracle, SQL Server e IBM Db2 (Aurora es una familia aparte; la vemos en la sección siguiente)
ÁMBITO	Regional; cada instancia vive en una AZ y, con Multi-AZ, su standby en otra
FACTURACIÓN	Horas de instancia, GB de almacenamiento (e IOPS según el tipo), copias por encima del tamaño aprovisionado y transferencia de datos
IDEAL PARA	Aplicaciones transaccionales con SQL que no quieren administrar servidores

Motores, instancias y almacenamiento

Una **instancia de base de datos** (*DB instance*) es un entorno aislado con su propia capacidad de cómputo —la **clase de instancia**— y su almacenamiento en EBS. Te conectas a ella con un nombre DNS del estilo `pedidos.abc123xyz.eu-west-1.rds.amazonaws.com`, nunca con una IP fija.

- **Clases de instancia**: `db.t4g` y `db.t3` (con ráfagas de CPU, para desarrollo), `db.m*` (uso general) y `db.r*` o `db.x*` (optimizadas en memoria). La `g` indica Graviton, con mejor relación precio-rendimiento. Cambiar de clase implica un reinicio; con Multi-AZ, el corte se reduce a un failover.
- **Almacenamiento**: `gp3` (uso general, IOPS configurables) o `io1` / `io2` Block Express (IOPS aprovisionadas y latencia muy estable). **Storage Auto Scaling** amplía el disco cuando se llena, hasta el máximo que fijes.

```bash
aws rds create-db-instance \
  --db-instance-identifier pedidos-prod \
  --engine postgres --db-instance-class db.r7g.large \
  --storage-type gp3 --allocated-storage 100 --max-allocated-storage 500 \
  --master-username appadmin --manage-master-user-password \
  --multi-az --storage-encrypted --backup-retention-period 14 \
  --db-subnet-group-name subredes-privadas-bd \
  --vpc-security-group-ids sg-0a1b2c3d4e5f67890 --no-publicly-accessible
```

`--max-allocated-storage` activa Storage Auto Scaling y `--manage-master-user-password` guarda la contraseña maestra en Secrets Manager, que además la rota por ti.

Alta disponibilidad: Multi-AZ y réplicas de lectura

RDS ofrece dos mecanismos que a menudo se confunden, porque los dos «copian la base de datos», pero responden a preguntas distintas: **Multi-AZ**, a «¿qué pasa si se cae?» (disponibilidad), y las **réplicas de lectura**, a «¿cómo atiendo más consultas?» (rendimiento). Piensa en Multi-AZ como la rueda de repuesto, que no usas hasta que pinchas, y en las réplicas como ventanillas extra, que atienden a más gente pero no sustituyen solas a la principal si cierra.

- **Multi-AZ** mantiene un **standby** en otra AZ que recibe cada cambio de forma **síncrona**: una escritura no se confirma hasta que está en las dos AZ, así que al conmutar no se pierde nada. El standby **no admite conexiones**, solo espera. Si la primaria o su AZ fallan, RDS hace el *failover* (la conmutación) solo y apunta al standby el mismo nombre DNS: la aplicación únicamente tiene que reconectar. En el **despliegue de clúster** (MySQL y PostgreSQL) hay un writer y **dos standbys legibles** en tres AZ, con un endpoint de escritura, otro de lectura y failovers más rápidos.
- Las **réplicas de lectura** (*read replicas*) reciben los cambios de forma **asíncrona**, así que pueden ir unos segundos por detrás (*replica lag*). Cada una tiene su propio endpoint, y es tu aplicación la que les envía las lecturas (MySQL, MariaDB y PostgreSQL admiten hasta 15). Pueden estar en otra región y, ante un desastre, se **promocionan** a base de datos independiente, pero a mano.

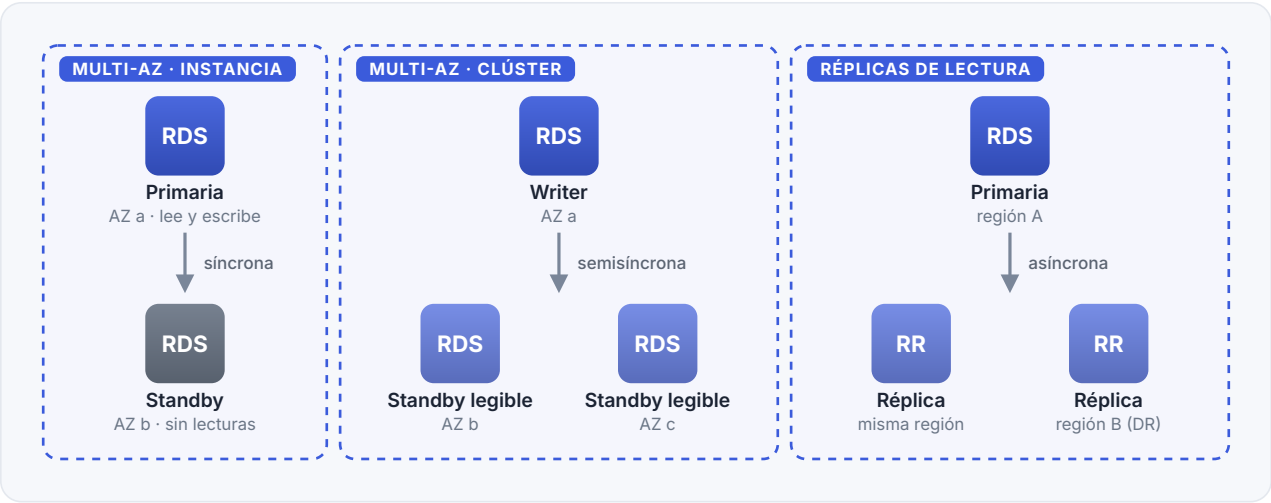


Figura 64 · Las tres formas de replicar en RDS: Multi-AZ de instancia, Multi-AZ de clúster y réplicas de lectura.

	Multi-AZ (instancia)	Multi-AZ (clúster)	Réplica de lectura
Objetivo	Disponibilidad	Disponibilidad y lecturas	Escalar lecturas y DR
Replicación	Síncrona	Semisíncrona (confirma un standby)	Asíncrona
¿Sirve lecturas?	✘ El standby solo espera	✔ Los dos standbys	✔
Ubicación	Otra AZ	Tres AZ	Cualquier AZ u otra región
Failover	Automático, 60–120 s	Automático, menos de 35 s	Manual: se *promociona*

> **CLAVE PARA LA CERTIFICACIÓN**
>
> Multi-AZ = disponibilidad; réplica de lectura = rendimiento de lectura (y DR entre regiones). Una réplica puede ser a su vez Multi-AZ, y replicar dentro de la región no genera cargos de transferencia; entre regiones, sí.

Copias de seguridad y restauración

RDS hace dos tipos de copias:

- **Backups automáticos**: un snapshot diario en la **ventana de backup** más los **logs de transacciones**, que se suben a S3 cada cinco minutos. Permiten una **restauración a un punto en el tiempo** (PITR) a cualquier segundo del periodo de retención —de 1 a 35 días—, normalmente hasta unos cinco minutos antes del presente.
- **Snapshots manuales**: se conservan hasta que los borras, aunque elimines la instancia. Puedes copiarlos a otra región y compartirlos con otras cuentas (si están cifrados, con una clave KMS gestionada por el cliente, no con `aws/rds`).
- **Coste**: las copias no se cobran hasta el 100 % del almacenamiento aprovisionado en la región; el exceso (snapshots acumulados, retenciones largas) se paga por GB-mes.

> **CUIDADO**
>
> Restaurar —un snapshot o un PITR— **siempre crea una instancia nueva**, con otro endpoint: no «rebobina» la existente, así que planifica cómo apuntará la aplicación a ella. Al borrar una instancia, los backups automáticos desaparecen salvo que elijas conservarlos: haz siempre un snapshot final. Y si detienes una instancia para ahorrar, vuelve a arrancar sola a los siete días.

Seguridad: red, cifrado y credenciales

- **Red**: subredes privadas (un *DB subnet group* con al menos dos AZ), sin acceso público, y el puerto del motor abierto solo al Security Group de la aplicación (capítulo de Redes).
- **Cifrado en reposo** con KMS: cubre almacenamiento, backups, snapshots, réplicas y logs. Se elige **al crear** la instancia; para cifrar una existente, copia un snapshot activando el cifrado y restaura desde él (detalles en «Seguridad: cifrado con KMS, secretos y protección»).
- **Cifrado en tránsito** con TLS, obligatorio si lo fijas por parámetro (`rds.force_ssl` en PostgreSQL, `require_secure_transport` en MySQL).
- **Credenciales**: contraseña maestra gestionada en **Secrets Manager**, o **autenticación IAM** (MySQL, MariaDB y PostgreSQL): la aplicación obtiene con su rol un **token** válido 15 minutos (`aws rds generate-db-auth-token`) y lo usa como contraseña, por TLS. El rol necesita el permiso `rds-db:connect` sobre ese usuario de la base de datos y, en PostgreSQL, el usuario debe tener el rol `rds_iam` (`GRANT rds_iam TO app_lectura;`).

RDS Proxy

Cada conexión a una base de datos relacional consume memoria y CPU del servidor, y una flota de funciones Lambda que escala a cientos de copias en segundos puede agotar `max_connections`. **RDS Proxy** es un *pool* de conexiones gestionado entre la aplicación y la base de datos: mantiene pocas conexiones abiertas y las reutiliza entre muchos clientes.



Figura 65 · RDS Proxy absorbe las conexiones cortas de cientos de funciones Lambda y las multiplexa sobre unas pocas.

Durante un failover, RDS Proxy mantiene las conexiones de los clientes y las redirige a la nueva primaria, así que la aplicación apenas lo nota. Lee las credenciales de Secrets Manager, puede exigir autenticación IAM, es compatible con MySQL, PostgreSQL, MariaDB y SQL Server (y con Aurora), solo es accesible desde la VPC y se paga por vCPU (o ACU) de la base de datos a la que sirve.

Operación: configuración, mantenimiento y actualizaciones

Parameter group — configuración del motor (`max_connections`, `work_mem`...). El grupo por defecto no se edita: crea uno propio. Los parámetros **dinámicos** se aplican al momento; los **estáticos** dejan la instancia en `pending-reboot` hasta reiniciarla. Los **option groups** añaden funciones extra de algunos motores, sobre todo Oracle y SQL Server.

Ventana de mantenimiento — franja semanal para parches del sistema operativo y, si activas *auto minor version upgrade*, versiones menores del motor. Con Multi-AZ se parchea primero el standby y se hace failover. Las versiones **mayores** las decides tú.

Monitorización — además de las métricas de CloudWatch (`CPUUtilization`, `FreeStorageSpace`, `ReplicaLag`...), **Enhanced Monitoring** muestra el sistema operativo y los procesos, y **CloudWatch Database Insights** (desde finales de 2024, sucesor de Performance Insights) desglosa la carga por consulta SQL y evento de espera: *¿qué consulta satura la base de datos?* Su modo Standard es gratuito; el Advanced, de pago, guarda 15 meses y añade la vista de flota y los planes de ejecución.

Blue/Green deployments — para cambios con riesgo (versión mayor, parámetros, clase de instancia), RDS crea un entorno **green**, copia del de producción (**blue**) sincronizada por replicación lógica. Lo modificas y lo pruebas; en el **switchover**, RDS bloquea las escrituras, espera a que green esté al día e intercambia los endpoints, normalmente en menos de un minuto y sin cambiar la cadena de conexión. Existe para RDS MySQL, MariaDB y PostgreSQL y para Aurora MySQL y PostgreSQL.

> **COSTES**
>
> Si una versión mayor llega al final de su soporte estándar y no actualizas, pasa a **RDS Extended Support**, que se factura aparte por vCPU y hora.

11.4 Amazon Aurora

Aurora es el motor relacional que AWS diseñó para la nube: compatible con MySQL y PostgreSQL —tus drivers y herramientas funcionan igual—, pero con un almacenamiento completamente distinto.

FICHA DEL SERVICIO

Amazon Aurora

Base de datos relacional nativa de la nube, compatible con MySQL y PostgreSQL

ALMACENAMIENTO	Volumen distribuido y compartido, con 6 copias en 3 AZ, que crece solo
RÉPLICAS	Hasta 15 réplicas de Aurora, con un retardo típico inferior a 100 ms
FACTURACIÓN	Instancias (o ACU), GB-mes almacenados, operaciones de E/S (salvo con I/O-Optimized), copias y transferencia

Cómputo y almacenamiento separados

En RDS, cada instancia tiene su propio volumen EBS. En Aurora, todas las instancias del **clúster** —un **writer** y hasta 15 **réplicas**— trabajan sobre el mismo **volumen de clúster**: una capa distribuida que divide los datos en segmentos y guarda **seis copias de cada uno en tres AZ**. El writer solo envía registros del log de *redo*, y una escritura se confirma cuando 4 de las 6 copias la han recibido.



Figura 66 · Arquitectura de Aurora: las instancias comparten un único volumen distribuido con seis copias de los datos en tres AZ.

Consecuencias: el clúster sigue escribiendo aunque pierda dos copias y leyendo aunque pierda tres (sobrevive a la caída de una AZ), y las copias dañadas se reparan solas; las réplicas no necesitan copiar datos, así que se crean enseguida y apenas se retrasan; y no dimensionas disco: el volumen crece (y se reduce) solo y pagas por lo que ocupas.

Endpoints, réplicas y failover

En Aurora te conectas a **endpoints** que AWS mantiene actualizados aunque cambien los papeles de las instancias:

Endpoint	Apunta a	Úsalo para
Cluster (writer)	La instancia writer actual (`app.cluster-abc123…`); tras un failover, la nueva	Escrituras y lecturas que deben ver el último dato
Reader	Reparte las conexiones entre las réplicas (`app.cluster-ro-abc123…`)	Lecturas escalables
Custom	El subconjunto de instancias que tú elijas	Separar cargas: informes en réplicas más grandes, por ejemplo
Instance	Una instancia concreta	Diagnóstico y casos muy específicos

Si el writer falla, Aurora **promociona una réplica** —según su prioridad (*tier* 0 a 15) y, a igualdad, la de mayor tamaño— y repunta el endpoint del clúster, normalmente en unos 30 segundos. Sin réplicas, tiene que recrear la instancia y tarda bastante más: en producción, mantén al menos una réplica en otra AZ. **Aurora Auto Scaling** añade o quita réplicas según la CPU o las conexiones. Ojo: el reader endpoint reparte **conexiones** (por DNS), no consultas; un pool que nunca renueva sus conexiones puede desequilibrar la carga.

Aurora Serverless v2

Con **Aurora Serverless v2**, la capacidad se mide en **ACU** (*Aurora Capacity Units*; cada una equivale a unos 2 GiB de memoria con su CPU y red) y se ajusta sola, en incrementos de 0,5 ACU, entre el mínimo

y el máximo que definas: desde 0 ACU —la instancia se pausa sin conexiones y tarda unos segundos en reanudarse— hasta 256 ACU. Pagas por ACU-hora. Encaja con cargas variables, entornos de desarrollo y aplicaciones con muchos inquilinos, y puede convivir con instancias aprovisionadas en el mismo clúster.

Global Database, backtrack y clonado

Global Database
Multirregión

Replica el volumen a otras regiones con retardo típico inferior a un segundo; las secundarias sirven lecturas locales. Ante un desastre regional, promocionas una: RPO de segundos, RTO del orden de un minuto.

Backtrack
Solo Aurora MySQL

«Rebobina» el clúster en el sitio hasta 72 horas atrás sin restaurar ningún backup: ideal para deshacer un `DELETE` sin `WHERE`. Se activa al crear el clúster.

Clonado
Copy-on-write

Crea en minutos un clúster que comparte las páginas del original y solo copia las que cambian. Pruebas con datos reales sin tocar producción.

AURORA DSQL

Disponible desde 2025, **Aurora DSQL** es una base de datos SQL distribuida y *serverless*, compatible con PostgreSQL, que funciona en modo activo-activo en varias regiones con consistencia fuerte y sin instancias. No es «Aurora con otro nombre»: usa control de concurrencia optimista y no admite todas las funciones de PostgreSQL (por ejemplo, las claves foráneas).

COSTES

Con **Aurora Standard** pagas las operaciones de E/S por millón; con **Aurora I/O-Optimized** no pagas E/S, a cambio de instancias y almacenamiento más caros. Regla práctica de AWS: si la E/S supera aproximadamente el 25 % del gasto en Aurora, I/O-Optimized suele salir más barato.

11.5 Amazon DynamoDB

DynamoDB es la base de datos NoSQL serverless de AWS: sin instancias, parches ni ventanas de mantenimiento. Creas una tabla, eliges cómo pagar la capacidad y lees y escribes en milisegundos de un dígito, tengas cien ítems o cien mil millones. A cambio, te pide **diseñar la tabla a partir de tus patrones de acceso**.

FICHA DEL SERVICIO

Amazon DynamoDB

Base de datos NoSQL clave-valor y documental, totalmente serverless

ÁMBITO	Regional; los datos se replican en 3 AZ (global tables para multirregión)
LÍMITES CLAVE	Ítem de hasta 400 KB; Query y Scan devuelven como mucho 1 MB por página
FACTURACIÓN	Por petición (on-demand) o por capacidad aprovisionada, más GB almacenados y extras (backups, streams, réplicas)

Modelo de datos

Tabla, ítem y atributo — una tabla es una colección de **ítems** (el equivalente a filas, de hasta 400 KB) y cada ítem es un conjunto de **atributos** nombre-valor: escalares, documentos (listas y mapas anidados) y conjuntos. Sin esquema fijo salvo la clave.
Partition key — clave de partición (*hash key*). Su hash decide en qué partición física se guarda el ítem.

Sort key — clave de ordenación (*range key*), opcional. Ordena los ítems que comparten partition key —una **colección de ítems**— y permite consultas por rango. La **clave primaria** es la partition key sola o junto con la sort key, y es única: no puede haber dos ítems con la misma.

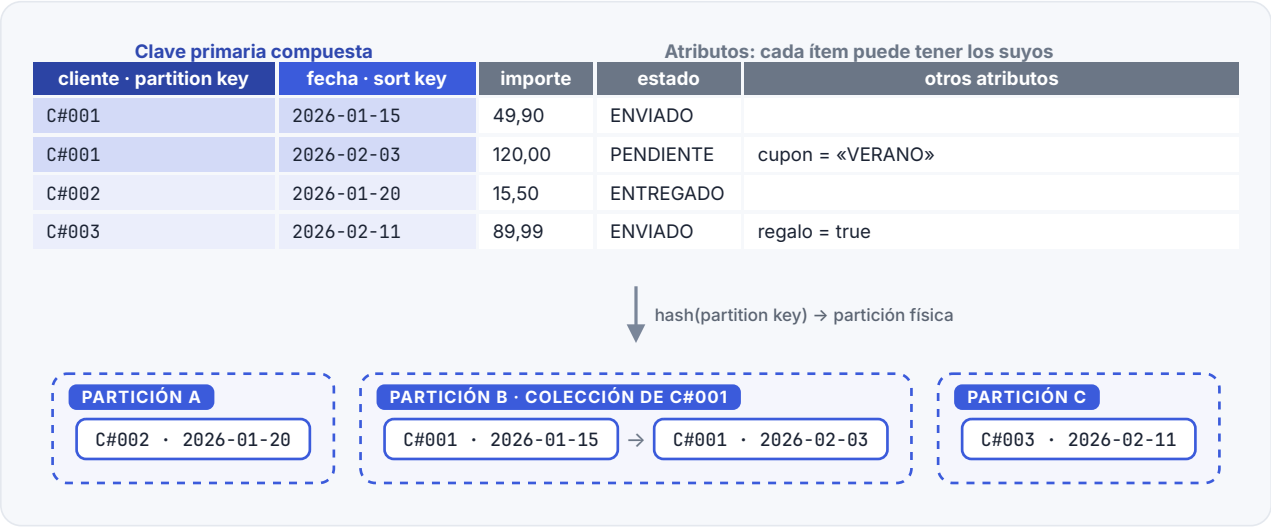


Figura 67 · Tabla `Pedidos` con clave primaria compuesta. DynamoDB reparte los ítems entre particiones según el hash de la partition key y, dentro de cada una, los ordena por la sort key.

Particiones y claves calientes

Cada partición física guarda en torno a 10 GB y admite hasta **3000 RCU y 1000 WCU** (unidades de lectura y escritura que verás en el apartado siguiente). DynamoDB divide particiones por ti cuando crecen los datos o el tráfico, pero no puede repartir lo que se concentra en **un único valor** de partition key. Si todo va a la misma clave —la fecha de hoy, un producto viral, un estado como `PENDIENTE`— aparece una **partición caliente**: *throttling* aunque la tabla tenga capacidad de sobra. La **capacidad adaptativa** mitiga desequilibrios moderados, pero el remedio está en el diseño.

> **BUENA PRÁCTICA**
>
> Elige una partition key con **muchos valores distintos** y accesos repartidos (id de usuario, de pedido). Si un valor es inevitablemente popular, reparte sus escrituras con un sufijo (`2026-03-14#0` … `#9`); para lecturas repetidas de pocas claves, pon una caché delante.

Capacidad y consistencia

Cada lectura y escritura consume **unidades de capacidad**. **1 RCU** (*read capacity unit*) es una lectura fuertemente consistente por segundo de hasta **4 KB**, o dos eventualmente consistentes; **1 WCU** (*write capacity unit*), una escritura por segundo de hasta **1 KB**. Los tamaños se **redondean hacia arriba** (6 KB cuentan como 8 KB al leer; 2,5 KB, como 3 KB al escribir) y las operaciones transaccionales cuestan el doble.

¿Eventual o fuerte? Por defecto, las lecturas son **eventualmente consistentes**: pueden no reflejar una escritura de hace un instante (normalmente todo converge en un segundo). Con `ConsistentRead=True` obtienes una lectura **fuertemente consistente**, que devuelve el último valor confirmado, a cambio del doble de capacidad:

Necesidad	Cálculo	Capacidad
80 lecturas/s de 6 KB, fuertemente consistentes	6 KB → 2 bloques de 4 KB = 2 RCU × 80	**160 RCU**
Las mismas lecturas, eventualmente consistentes	La mitad: 160 / 2	**80 RCU**
40 escrituras/s de 2,5 KB	2,5 KB → 3 KB = 3 WCU × 40	**120 WCU**
Las mismas escrituras, dentro de transacciones	El doble: 120 × 2	**240 WCU**

Esa capacidad se paga de una de estas dos maneras, y puedes cambiar de modo más adelante:

On-demand
Pagas por petición

Sin planificar: absorbe al instante hasta el doble del pico de tráfico anterior. Ideal para cargas nuevas, impredecibles o con picos; tras la rebaja de precios de finales de 2024, AWS lo recomienda por defecto para la mayoría de tablas.

Aprovisionada
Reservas RCU y WCU

Fijas las unidades por segundo y pagas por hora, las uses o no; **Auto Scaling** las ajusta y la capacidad reservada abarata más. Lo más económico con tráfico estable; si te quedas corto, hay throttling.

Query frente a Scan

Para leer un ítem concreto basta `GetItem` con su clave primaria completa: es la operación más rápida y barata. Para leer varios ítems hay dos operaciones muy distintas:

Query
Eficiente: lee solo lo que pides

- Exige la partition key (igualdad).
- Condición opcional sobre la sort key: `=`, `<`, `>`, `BETWEEN`, `begins_with`.
- Devuelve los ítems ordenados por la sort key.
- Consume capacidad solo por lo que lee.

Scan
Caro: recorre toda la tabla

- Lee todos los ítems de todas las particiones.
- Consume capacidad por **todo** lo leído.
- Un `FilterExpression` se aplica **después** de leer: no abarata nada.
- Úsalo para tablas pequeñas o procesos puntuales; para analítica, exporta a S3.

PYTHON

```python
import boto3
from boto3.dynamodb.conditions import Key

pedidos = boto3.resource("dynamodb").Table("Pedidos")
# Pedidos de 2026 del cliente C#001, del más reciente al más antiguo
resp = pedidos.query(
    KeyConditionExpression=Key("cliente").eq("C#001")
        & Key("fecha").begins_with("2026-"),
    ScanIndexForward=False,
)
for p in resp["Items"]:      # más de 1 MB → paginar con LastEvaluatedKey
    print(p["fecha"], p["importe"])
```

Índices secundarios: LSI y GSI

Una Query solo busca por la clave primaria. Para consultar por otros atributos —«el pedido P81», sin saber de qué cliente es— creas **índices secundarios**: copias de la tabla organizadas con otra clave, que DynamoDB mantiene al día por ti. Decides qué atributos se **proyectan** (se copian) en el índice (`KEYS_ONLY`, `INCLUDE` o `ALL`), y como el índice solo contiene los ítems que tienen su clave, permite **índices dispersos** muy baratos.

Característica	LSI (índice secundario local)	GSI (índice secundario global)
Clave del índice	Misma partition key, **otra sort key**	**Otra partition key** (y sort key opcional)
Cuándo se crea	Solo al crear la tabla	En cualquier momento
Máximo por tabla	5	20 (cuota ajustable)
Capacidad	Comparte la de la tabla	Propia (en modo aprovisionado)
Lecturas fuertemente consistentes	✔	✘
Restricción de tamaño	10 GB por valor de partition key	Ninguna específica
Alcance de la consulta	Dentro de una colección de ítems	Toda la tabla

En la tabla `Pedidos` del ejemplo (clave `cliente` + `fecha`):

- Un **GSI** con partition key `num_pedido` responde a «dame el pedido P81» con una Query, sea del cliente que sea. Como se crea en cualquier momento, es la solución habitual cuando aparece un patrón de acceso nuevo.
- Un **LSI** con sort key `importe` daría «los pedidos de C#001, ordenados por importe», pero tendría que haberse definido al crear la tabla. A cambio, admite lecturas fuertemente consistentes: el GSI se actualiza de forma asíncrona.

TTL, Streams y transacciones

TTL
Caducidad automática

Indicas un atributo con una fecha en *epoch* (segundos) y DynamoDB borra los ítems caducados en segundo plano, sin consumir WCU. No es inmediato —puede tardar días—, así que filtra los caducados al leer. Ideal para sesiones, cachés y datos temporales.

Streams
Registro de cambios

Cada alta, cambio o borrado genera un registro, ordenado por ítem y retenido 24 horas, con la imagen nueva, la antigua o ambas. Una función Lambda puede procesarlos para auditar, notificar, agregar o replicar.

Transacciones
ACID entre ítems

`TransactWriteItems` y `TransactGetItems` agrupan hasta 100 acciones (máximo 4 MB) sobre una o varias tablas de la misma cuenta y región: o se aplican todas o ninguna. Cuestan el doble de capacidad.

Operaciones condicionales

Las escrituras aceptan una **expresión de condición**: si no se cumple, fallan con `ConditionalCheckFailedException` sin cambiar nada. Con `attribute_not_exists(PK)` evitas sobrescribir un ítem existente, y con un número de versión implementas **bloqueo optimista**:

```python
# Solo si nadie ha cambiado la revisión 7 mientras tanto
pedidos.update_item(
    Key={"cliente": "C#001", "fecha": "2026-02-03"},
    UpdateExpression="SET estado = :e, revision = revision + :uno",
    ConditionExpression="revision = :r",
    ExpressionAttributeValues={":e": "ENVIADO", ":uno": 1, ":r": 7},
)
```

Global tables, DAX y copias de seguridad

- **Global tables**: réplicas **activas** en varias regiones (todas aceptan lecturas y escrituras); los cambios se propagan normalmente en torno a un segundo y los conflictos se resuelven con «gana la última escritura». Desde 2025 hay también un modo de consistencia fuerte multirregión.

- **DAX** (*DynamoDB Accelerator*): caché en memoria compatible con la API de DynamoDB que baja las lecturas a **microsegundos**; cambias el cliente, no la lógica. Es un clúster dentro de tu VPC y solo acelera lecturas eventualmente consistentes.
- **Copias**: PITR restaura a cualquier segundo de los últimos 35 días (periodo configurable); los backups bajo demanda duran hasta que los borras y AWS Backup los copia a otras regiones o cuentas. Restaurar crea una **tabla nueva**, y exportar a S3 desde PITR no consume capacidad.

Introducción al single-table design

DynamoDB no tiene joins: enumeras tus **patrones de acceso** y diseñas las claves para resolver cada uno con una sola Query. El **single-table design** lleva la idea al extremo: varias entidades en **una misma tabla**, con claves genéricas (`PK`, `SK`) y prefijos de tipo, para que los datos relacionados se lean juntos.

PK	SK	Atributos	Entidad
`CLIENTE#ana`	`PERFIL`	nombre, email, fecha de alta	Cliente
`CLIENTE#ana`	`PEDIDO#2026-02-03#P81`	importe: 120, estado: PENDIENTE	Pedido
`CLIENTE#ana`	`PEDIDO#2026-01-15#P42`	importe: 49,90, estado: ENVIADO	Pedido
`PEDIDO#P81`	`LINEA#01`	sku: TAZA-AZUL, cantidad: 2	Línea
`PEDIDO#P81`	`LINEA#02`	sku: CAFE-1KG, cantidad: 1	Línea

- Perfil y pedidos de Ana en una sola petición: Query con `PK = "CLIENTE#ana"`.
- Solo sus pedidos de 2026: añade `begins_with(SK, "PEDIDO#2026")`.
- Líneas de un pedido: Query con `PK = "PEDIDO#P81"`.
- Pedidos pendientes de todos los clientes: un GSI disperso, cuya clave (estado y fecha) solo llevan los pedidos pendientes.

No es obligatorio: exige conocer los patrones de acceso de antemano y complica la analítica y la evolución del modelo. Varias tablas bien diseñadas también son una opción válida.

11.6 Amazon ElastiCache y MemoryDB

Muchas aplicaciones leen los mismos datos una y otra vez (la ficha de un producto, la sesión del usuario). Guardarlos en memoria baja la latencia a microsegundos o pocos milisegundos y descarga a la base de datos.

FICHA DEL SERVICIO

Amazon ElastiCache

Caché en memoria gestionada: Valkey, Redis OSS y Memcached

MODALIDADES	Clústeres de nodos (eliges tipo y número) o ElastiCache Serverless
DESPLIEGUE	Dentro de tu VPC; accesible solo desde la red privada
IDEAL PARA	Cachear consultas y objetos, sesiones, rankings, contadores y limitación de peticiones

Valkey, el *fork* abierto de Redis nacido en 2024 (Linux Foundation) tras su cambio de licencia, tiene la misma API que Redis OSS y un precio inferior en ElastiCache: es la opción recomendada para proyectos nuevos. Memcached, más simple, solo sirve como caché volátil que escala en horizontal.

	Valkey / Redis OSS	Memcached
Estructuras de datos (hashes, listas, sets ordenados, streams...)	✔	✘
Réplicas, Multi-AZ con failover y snapshots	✔	✘
Particionado (*sharding*)	✔ Modo clúster	✔ Desde el cliente
Pub/Sub, scripts Lua y transacciones	✔	✘

Estrategias de caché

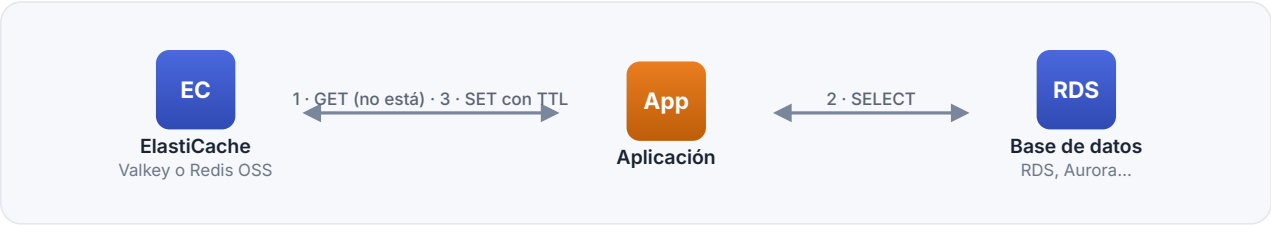


Figura 68 · Patrón *lazy loading*: la aplicación consulta primero la caché y solo va a la base de datos cuando no encuentra el dato.

Lazy loading (cache-aside)

Se carga al leer

Solo se cachea lo que alguien pide y una caída de la caché no rompe nada. A cambio, cada fallo cuesta tres viajes y el dato puede quedar **obsoleto** hasta que caduque su TTL.

Write-through

Se actualiza al escribir

Cada escritura actualiza también la caché: datos siempre frescos, a cambio de escrituras más lentas y de cachear datos que quizá nadie lea. Combina ambos y pon siempre un TTL.

Amazon MemoryDB va un paso más allá: es una **base de datos** en memoria, compatible con Valkey y Redis OSS, que guarda cada escritura en un log transaccional distribuido en varias AZ. No pierde datos si cae un nodo, así que puede ser la base de datos principal, no solo una caché.

> ★ **CLAVE PARA LA CERTIFICACIÓN**
>
> «Caché delante de una base de datos» → ElastiCache; «caché para DynamoDB sin tocar el código» → DAX; «base de datos principal en memoria y duradera» → MemoryDB.

11.7 Otras bases de datos de propósito específico

Servicio	Modelo	Lo esencial
Amazon DocumentDB	Documental (API de MongoDB)	Documentos JSON con índices y consultas ricas; almacenamiento al estilo Aurora (6 copias en 3 AZ) y *elastic clusters* para repartir datos.
Amazon Keyspaces	Columnar ancha (Cassandra)	Compatible con CQL y los drivers de Cassandra, pero serverless: on-demand o aprovisionado, sin nodos.
Amazon Neptune	Grafos	Grafos de propiedades (Gremlin, openCypher) y RDF (SPARQL): fraude, recomendaciones, relaciones de varios saltos.
Amazon Timestream	Series temporales	Para proyectos nuevos, **Timestream for InfluxDB**; Timestream for LiveAnalytics dejó de aceptar clientes nuevos en 2025.
Amazon OpenSearch Service	Búsqueda, logs y vectores	OpenSearch gestionado (antes Amazon Elasticsearch Service), en clústeres o Serverless: texto completo, logs y búsqueda vectorial.

Analítica: Redshift y Athena

Amazon Redshift es el *data warehouse* de AWS. Guarda los datos por **columnas** —sumar importes solo lee esa columna, muy comprimida— y reparte el trabajo con una arquitectura **MPP** (*massively parallel processing*): un nodo líder planifica y los nodos de cómputo ejecutan en paralelo. Los nodos RA3 separan cómputo y almacenamiento, **Redshift Spectrum** consulta S3 sin cargar los datos y **Redshift Serverless** elimina el clúster (pagas por RPU-hora). Las integraciones **zero-ETL** traen datos de Aurora, RDS y DynamoDB casi en tiempo real.

Amazon Athena es un motor SQL serverless que consulta ficheros en S3 (CSV, JSON, Parquet, ORC…) usando el catálogo de AWS Glue. No hay nada que aprovisionar y se paga por datos escaneados: perfecto para consultas ad hoc sobre un *data lake* o sobre logs, como los de los balanceadores:

```sql
SELECT elb_status_code, count(*) AS peticiones
FROM alb_logs
WHERE day = '2026/03/14' AND elb_status_code >= 500
GROUP BY elb_status_code
ORDER BY peticiones DESC;
```

COSTES

En Athena pagas por TB escaneado: usa formato columnar (Parquet u ORC), comprime y **particiona** (por fecha, por ejemplo) para que el `WHERE` descarte carpetas enteras. La misma consulta puede costar decenas de veces menos.

11.8 Migraciones: AWS DMS y SCT

AWS Database Migration Service (DMS) copia datos de un origen a un destino mientras el origen **sigue funcionando**: primero una **carga completa** y después los cambios en curso (**CDC**, *change data capture*) hasta el corte, así que la parada se reduce a minutos. Admite migraciones **homogéneas** (MySQL → Aurora MySQL) y **heterogéneas** (Oracle → Aurora PostgreSQL), desde bases de datos en tu centro de datos, en EC2 o en RDS hacia destinos como RDS, Aurora, S3, DynamoDB o Redshift, con una instancia de replicación o con **DMS Serverless**.

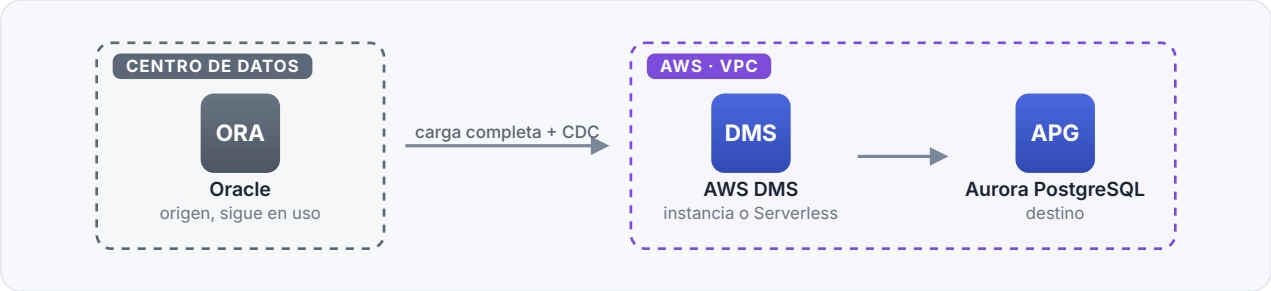


Figura 69 · Migración heterogénea de Oracle a Aurora PostgreSQL: SCT convierte el esquema y DMS copia los datos y replica los cambios hasta el corte.

DMS mueve **datos**, no traduce esquemas: en una migración heterogénea, primero conviertes tablas, vistas y procedimientos (PL/SQL → PL/pgSQL, por ejemplo) con **AWS Schema Conversion Tool (SCT)**, una aplicación de escritorio, o con **DMS Schema Conversion**, integrada en la consola; ambas informan de lo que exige revisión manual. El plan típico: evaluar, convertir el esquema, carga completa más CDC, validar los datos y cambiar la aplicación al destino.

RESUMEN DEL CAPÍTULO

- Elige por **patrón de acceso**: relacional para SQL y transacciones, DynamoDB para clave-valor a escala, ElastiCache para microsegundos, Redshift y Athena para analítica. Lo gestionado te quita parches, backups y failover.
- RDS: **Multi-AZ** da disponibilidad (standby síncrono; la variante de clúster tiene dos standbys legibles) y las **réplicas de lectura** escalan lecturas de forma asíncrona, incluso entre regiones.
- PITR de hasta 35 días (restaurar crea una instancia nueva); RDS Proxy agrupa conexiones y Blue/Green reduce el riesgo de actualizar.
- Aurora separa cómputo y almacenamiento: 6 copias en 3 AZ, hasta 15 réplicas con poco retardo, endpoints de escritura y lectura, Serverless v2 desde 0 ACU y Global Database.
- DynamoDB se diseña desde las claves: partition key con muchos valores, Query en lugar de Scan, GSI para accesos nuevos y RCU/WCU por bloques de 4 KB y 1 KB; TTL, Streams, transacciones, global tables y DAX lo completan.
- ElastiCache (Valkey, Redis OSS, Memcached) acelera lecturas con lazy loading o write-through; MemoryDB es una base de datos en memoria duradera.
- DMS migra con parada mínima (carga completa + CDC); SCT convierte el esquema en migraciones heterogéneas.

Preguntas de repaso

1. **Tu aplicación en RDS necesita sobrevivir a la caída de una AZ y, además, descargar los informes de la primaria. ¿Qué configuras?**
   Respuesta: Multi-AZ para la disponibilidad y réplicas de lectura para los informes. Alternativa en MySQL o PostgreSQL: un despliegue Multi-AZ de clúster, cuyos dos standbys sí admiten lecturas.
2. **Alguien borró una tabla a las 10:42. ¿Cómo recuperas los datos en RDS y qué debes tener en cuenta?**
   Respuesta: Con una restauración a un punto en el tiempo (PITR) a las 10:41, dentro del periodo de retención. Se crea una instancia nueva con otro endpoint; luego copias los datos o apuntas la aplicación a ella. En Aurora MySQL con backtrack activado podrías rebobinar el clúster en el sitio.
3. **Cientos de funciones Lambda agotan las conexiones de tu base de datos Aurora. ¿Qué servicio lo resuelve?**
   Respuesta: RDS Proxy, que mantiene un pool de conexiones y las reutiliza entre las invocaciones; además, acelera los failovers y admite autenticación IAM.
4. **Necesitas 100 lecturas por segundo, eventualmente consistentes, de ítems de 10 KB. ¿Cuántas RCU aprovisionas?**
   Respuesta: 10 KB se redondean a 12 KB, es decir, 3 RCU por lectura fuertemente consistente; eventual es la mitad, 1,5 RCU. En total, 100 × 1,5 = 150 RCU.
5. **Quieres consultar una tabla de DynamoDB existente por un atributo que no forma parte de la clave. ¿LSI o GSI?**
   Respuesta: Un GSI: se puede crear en cualquier momento y admite otra partition key. Los LSI solo se definen al crear la tabla y comparten la partition key de la tabla.
6. **Vas a migrar de Oracle a Aurora PostgreSQL con la mínima parada. ¿Qué herramientas usas?**
   Respuesta: SCT (o DMS Schema Conversion) para convertir el esquema y el código, y DMS con carga completa más CDC para copiar los datos y mantenerlos sincronizados hasta el corte.

CAPÍTULO 12

Mensajería e integración: SQS, SNS, EventBridge y Kinesis

Conecta componentes que no se esperan unos a otros: colas, publicación-suscripción, buses de eventos y streaming de datos.

EN ESTE CAPÍTULO APRENDERÁS

1. Por qué desacoplar y qué cuesta hacerlo
2. SQS a fondo: Standard y FIFO, visibilidad, DLQ y Lambda
3. Publicar y suscribir con SNS y el patrón *fan-out*
4. Enrutar eventos con EventBridge: reglas, Pipes y Scheduler
5. Streaming con Kinesis, Firehose, Flink y MSK
6. Elegir servicio y aplicar patrones: saga, *outbox*, idempotencia

SERVICIOS PROTAGONISTAS

Amazon SQS · Amazon SNS · Amazon EventBridge · Amazon Kinesis Data Streams · Amazon Data Firehose · Amazon MSK · Amazon MQ

12.1 Acoplamiento y desacoplamiento

Imagina una tienda online: al confirmar un pedido hay que cobrar, enviar un correo y avisar al almacén. Si el servicio de pedidos llama a los demás de forma **síncrona**, el cliente espera la suma de todos los tiempos y, si el servicio de correo se cae, falla el pedido entero. Ese es el problema del **acoplamiento**: cada componente depende de que los demás estén disponibles, sean rápidos y aguanten su ritmo.

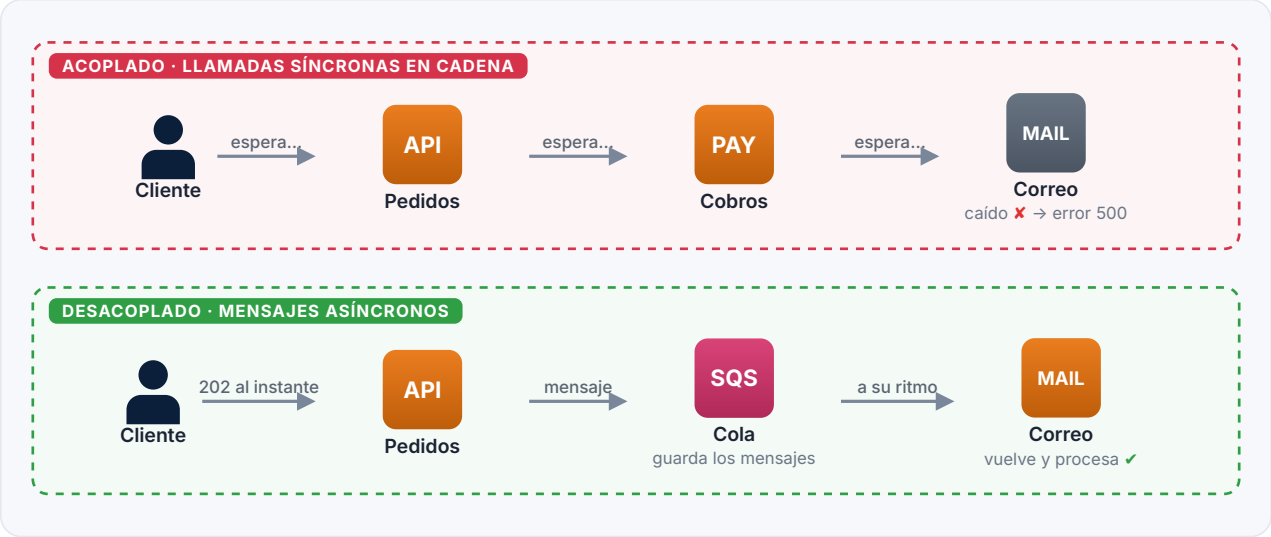


Figura 70 · Arriba, una cadena síncrona: un fallo al final tumba la petición. Abajo, la cola absorbe el fallo y los picos.

Con mensajería **asíncrona**, el productor deja un mensaje y sigue; los consumidores lo procesan a su ritmo. Ganas **resiliencia** (si un consumidor cae, los mensajes esperan), **absorción de picos** (la cola actúa de amortiguador), **escalado independiente** de cada parte y **evolución** (añadir un consumidor no obliga a tocar al productor). A cambio aceptas consistencia eventual, mensajes duplicados o desordenados y una depuración más difícil, que exige trazas distribuidas (capítulo de Observabilidad).

> ≈ **ANALOGÍA**
>
> Llamar por teléfono es síncrono: si no contestan, no avanzas. Dejar una carta en un buzón es asíncrono: el cartero la guarda hasta que el destinatario la recoge, aunque esté de vacaciones.

12.2 Amazon SQS: colas de mensajes

Amazon SQS (*Simple Queue Service*) ofrece colas totalmente gestionadas: los **productores** envían mensajes y los **consumidores** los reciben, los procesan y los borran. Es un modelo **pull**: el consumidor pregunta; la cola no empuja.

FICHA DEL SERVICIO

Amazon SQS

Colas de mensajes totalmente gestionadas

TIPO	Serverless; pago por petición, sin coste por cola
ÁMBITO	Regional; los mensajes se guardan de forma redundante en varias AZ
IDEAL PARA	Desacoplar productores y consumidores, colas de trabajo, absorber picos
LÍMITES CLAVE	Mensaje de hasta 1 MiB; retención de 1 minuto a 14 días (4 por defecto); *visibility timeout* de hasta 12 horas

Ciclo de vida de un mensaje

Leer un mensaje no lo borra: así, si el consumidor falla a medias, el mensaje no se pierde. El diagrama muestra el recorrido completo:

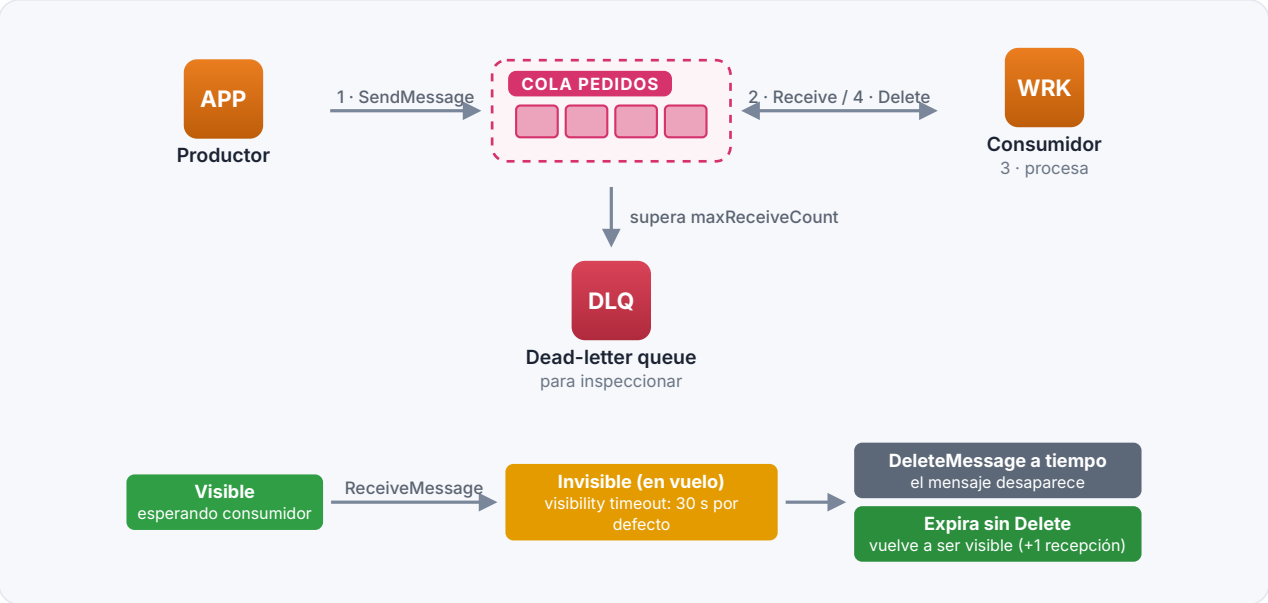


Figura 71 · Un mensaje recibido queda invisible durante el visibility timeout; si no se borra, reaparece y, tras varios intentos, va a la DLQ.

Visibility timeout — SQS no borra un mensaje al entregarlo: lo oculta al resto de consumidores durante este tiempo (30 s por defecto, hasta 12 h) y espera a que el consumidor lo borre con `DeleteMessage` al terminar. Si no lo hace a tiempo —porque falló o tardó demasiado—, el mensaje reaparece y otro consumidor lo procesará. Ajústalo por encima del tiempo de proceso y amplíalo sobre la marcha con `ChangeMessageVisibility` si una tarea se alarga.

Long polling — con `WaitTimeSeconds` de 1 a 20 s, `ReceiveMessage` espera a que lleguen mensajes antes de responder. El **short polling** responde al instante consultando solo parte de los servidores, así que puede devolver vacío aunque haya mensajes. Usa siempre long polling (o fíjalo en la cola con `ReceiveMessageWaitTimeSeconds`): menos respuestas vacías y menos coste.

Retención — de 1 minuto a 14 días (4 días por defecto); después, el mensaje se borra aunque nadie lo haya procesado.

Tamaño — hasta 1 MiB por mensaje (256 KB hasta agosto de 2025). Para más, la **Extended Client Library** (Java y Python) guarda el cuerpo en S3 y envía solo una referencia, hasta 2 GB: el patrón **claim check**.

Delay queues — retrasan la entrega de los mensajes nuevos de 0 a 15 minutos; en colas Standard, también por mensaje (*message timers*).

Dead-letter queues y redrive

Un mensaje que falla una y otra vez —un **poison pill**: datos corruptos, un bug— se reintentaría eternamente. La **redrive policy** fija un `maxReceiveCount`: cuando un mensaje se recibe más veces sin borrarse, SQS lo mueve a una **dead-letter queue (DLQ)**, una cola normal del mismo tipo, cuenta y región. Allí lo inspeccionas, alertas con una alarma de CloudWatch y, cuando corrijas el fallo, lo devuelves a la cola original con un **redrive** desde la consola o la API (`StartMessageMoveTask`).

```python
import json
import boto3

sqs = boto3.client("sqs")
dlq_arn = "arn:aws:sqs:eu-west-1:123456789012:pedidos-dlq"
sqs.create_queue(
    QueueName="pedidos",
    Attributes={
        "VisibilityTimeout": "60",              # segundos
        "ReceiveMessageWaitTimeSeconds": "20",  # long polling
        "MessageRetentionPeriod": "345600",     # 4 días
        "RedrivePolicy": json.dumps({"deadLetterTargetArn": dlq_arn,
                                     "maxReceiveCount": "5"}),
    },
)
```

CUIDADO

Da a la DLQ una retención mayor que la de la cola de origen: en las colas Standard la caducidad se cuenta desde que el mensaje entró en la cola original, así que un mensaje veterano podría expirar nada más llegar a la DLQ.

Standard frente a FIFO

Al crear la cola eliges su tipo, y no se puede cambiar después:

Standard

Rendimiento prácticamente ilimitado

- Entrega **al menos una vez**: en raras ocasiones, un mensaje llega dos veces.
- Orden **aproximado** (*best effort*): «pedido pagado» podría llegar antes que «pedido creado».
- Casi sin límite de mensajes por segundo.

FIFO

Orden estricto y sin duplicados

- Orden estricto **dentro de cada message group**.
- Procesamiento **exactamente una vez** (deduplicación en una ventana de 5 min).
- 300 operaciones/s por acción (3000 mensajes/s con lotes); mucho más en modo de alto rendimiento. El nombre acaba en `.fifo`.

En una cola FIFO, cada mensaje lleva un **MessageGroupId**: los mensajes de un mismo grupo (un pedido, un cliente) se entregan en orden y de uno en uno, mientras que grupos distintos se procesan en paralelo; por eso el paralelismo lo marca el número de grupos. La **deduplicación** descarta repetidos durante 5 minutos, según un **MessageDeduplicationId** explícito o un hash del cuerpo (*content-based deduplication*).

Lotes, costes y escalado de consumidores

`SendMessageBatch`, `DeleteMessageBatch` y `ChangeMessageVisibilityBatch` agrupan hasta 10 mensajes, y `ReceiveMessage` devuelve hasta 10 por llamada: menos peticiones, menos coste. Para escalar consumidores en EC2 o ECS, usa como métrica el **backlog por consumidor** (mensajes visibles ÷ consumidores activos) con una política de *target tracking*: si cada consumidor procesa 10 mensajes/s y aceptas 30 s de retraso, el objetivo es 300 mensajes por consumidor. Vigila con una alarma `ApproximateAgeOfOldestMessage`: si crece, los consumidores no dan abasto.

COSTES

SQS cobra por petición (del orden de 0,40 USD por millón en Standard y algo más en FIFO; el primer millón al mes es gratis), y cada fragmento de 64 KB de *payload* cuenta como una petición: un mensaje de 1 MiB equivale a 16. Los consumidores con short polling disparan la factura con miles de respuestas vacías.

SQS como origen de eventos de Lambda

Con un event source mapping, Lambda sondea la cola con long polling, invoca tu función con lotes y borra los mensajes si termina bien. Si falla, **todo el lote** vuelve a la cola… salvo que actives `ReportBatchItemFailures` y devuelvas solo los fallidos:

PYTHON

```python
import json

def handler(event, context):
    fallidos = []
    for record in event["Records"]:
        try:
            procesar(json.loads(record["body"]))
        except Exception:
            # Solo este mensaje volverá a la cola (y, tras varios intentos, a la DLQ)
            fallidos.append({"itemIdentifier": record["messageId"]})
    return {"batchItemFailures": fallidos}
```

BUENA PRÁCTICA

Fija el visibility timeout de la cola en al menos **seis veces** el timeout de la función, configura siempre una DLQ y limita la **concurrencia máxima** del event source mapping para no saturar una base de datos. En colas FIFO, si un mensaje falla, devuelve también como fallidos los siguientes del mismo lote para no romper el orden.

12.3 Amazon SNS: publicar y suscribir

SNS (*Simple Notification Service*) implementa **publicación-suscripción**: un productor publica un mensaje en un **topic** y SNS lo **empuja** (modelo *push*) a todos sus suscriptores a la vez. El productor no sabe quién escucha ni cuántos son.

FICHA DEL SERVICIO

Amazon SNS

Mensajería pub/sub totalmente gestionada

TIPO	Serverless; pago por millón de publicaciones; las entregas a SQS y Lambda no se cobran, las de HTTP, email o SMS sí
SUSCRIPTORES	Colas SQS, funciones Lambda, endpoints HTTP/HTTPS, Data Firehose, email, SMS y notificaciones push móviles
IDEAL PARA	Difundir un evento a varios sistemas (*fan-out*) y notificar a personas

Filtros — cada suscripción puede tener una **filter policy** sobre los atributos del mensaje o sobre su cuerpo JSON, para recibir solo lo que le interesa. Esta solo deja pasar pedidos creados o cancelados, de fuera de EE. UU. y de al menos 1000 euros:

JSON

```json
{
  "tipo": ["pedido_creado", "pedido_cancelado"],
  "pais": [{ "anything-but": ["US"] }],
  "importe": [{ "numeric": [">=", 1000] }]
}
```

Fiabilidad — SNS reintenta las entregas fallidas —con mucha más insistencia hacia SQS y Lambda que hacia HTTP, cuya política puedes ajustar— y cada suscripción puede tener su propia DLQ.
Raw message delivery — hacia SQS o HTTP, entrega el mensaje tal cual, sin el sobre JSON de SNS.
Topics FIFO — orden estricto y deduplicación, normalmente con colas SQS FIFO como suscriptores; permiten además archivar mensajes y reproducirlos.

12.4 El patrón fan-out: SNS + SQS

El patrón más conocido de la integración en AWS combina ambos servicios: el productor publica **una vez** en un topic y cada sistema interesado tiene **su propia cola SQS** suscrita, con su propio filtro. Cada consumidor procesa a su ritmo, con sus reintentos y su DLQ, y un fallo en uno no afecta a los demás. Añadir un consumidor nuevo es crear otra cola y otra suscripción, sin tocar al productor.

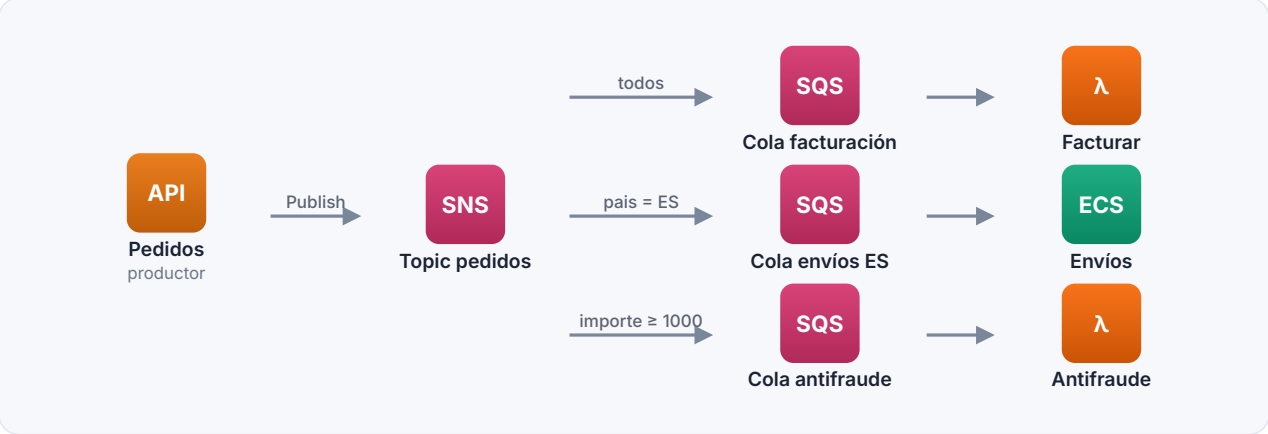


Figura 72 · Fan-out: una publicación, tres colas independientes, cada una con su filtro y su consumidor.

> **CUIDADO**
>
> Cada cola necesita una **política de acceso** que permita `sqs:SendMessage` a `sns.amazonaws.com` con la condición `aws:SourceArn` igual al ARN del topic; si la cola se cifra con una clave KMS propia, SNS necesita además permiso para usarla. Es el fallo de configuración más habitual del fan-out.

12.5 Amazon EventBridge: el bus de eventos

Mientras SNS difunde mensajes a suscriptores, **EventBridge** enruta **eventos** —avisos de que algo ha pasado— según su **contenido**. Es el pegamento de las arquitecturas dirigidas por eventos en AWS: recibe eventos de tus aplicaciones, de cientos de servicios de AWS y de aplicaciones SaaS, y los entrega a los destinos cuyas reglas coinciden.

FICHA DEL SERVICIO

Amazon EventBridge

Bus de eventos serverless con enrutamiento por contenido

TIPO	Serverless; pago por millón de eventos publicados (los de servicios de AWS en el bus por defecto son gratuitos)
ÁMBITO	Regional, con envío de eventos a otras cuentas y regiones
IDEAL PARA	Arquitecturas dirigidas por eventos, reaccionar a cambios en AWS, integrar SaaS

Event bus — canal que recibe eventos. El **bus por defecto** recibe los de los servicios de AWS (una instancia que se detiene, un *job* fallido…); creas **buses personalizados** para tus aplicaciones, y los **buses de socios** traen eventos de SaaS como Zendesk, Datadog o Shopify.
Regla — un **patrón** JSON que selecciona eventos por cualquier campo, con hasta 5 **destinos**: Lambda, Step Functions, SQS, SNS, Kinesis, Firehose, tareas ECS, otro bus (de otra cuenta o región) o **API destinations** (cualquier API HTTP externa, con autenticación y límite de ritmo). Un **input transformer** adapta el evento al formato del destino.

Entrega — al menos una vez y sin orden garantizado; si un destino falla, EventBridge reintenta hasta 24 horas y puedes asignar una DLQ a cada destino.



Figura 73 · EventBridge recibe eventos de varias fuentes y cada regla los enruta, según su contenido, a sus destinos.

Un evento tiene campos fijos (`source`, `detail-type`, `account`, `region`, `time`…) y un `detail` libre. Este patrón selecciona pedidos de España o México de más de 1000 euros que no sean de clientes internos (tus aplicaciones publican los eventos con `PutEvents`):

```json
{
  "source": ["com.acme.pedidos"],
  "detail-type": ["PedidoCreado"],
  "detail": {
    "pais": ["ES", "MX"],
    "importe": [{ "numeric": [">", 1000] }],
    "cliente": { "tipo": [{ "anything-but": "interno" }] }
  }
}
```

Schema registry — EventBridge puede **descubrir** la estructura de los eventos de un bus, guardarla como esquema y generar tipos para tu código (Java, Python, TypeScript, Go).
Archivo y replay — archiva los eventos de un bus (todos o los que cumplen un patrón) y los reproduce más tarde, para recuperarte de un fallo o probar un consumidor nuevo.
Pipes — integraciones **punto a punto** de un origen (SQS, Kinesis, DynamoDB Streams, MSK, Amazon MQ) a un destino, con filtrado, enriquecimiento opcional (con una Lambda, por ejemplo) y transformación, sin código de «pegamento».
Scheduler — el «cron» serverless: programaciones puntuales (`at(...)`), periódicas (`rate(5 minutes)`) o `cron(...)` con zona horaria que invocan Lambda, Step Functions, SQS o casi cualquier API de AWS, con reintentos y DLQ. Escala a millones (un recordatorio por usuario, por ejemplo) y sustituye a las antiguas reglas programadas. Esta lanza a diario, a las 7:00 de Madrid, una función de informes:

```bash
aws scheduler create-schedule --name informe-diario \
  --schedule-expression "cron(0 7 * * ? *)" \
  --schedule-expression-timezone "Europe/Madrid" \
  --flexible-time-window '{"Mode": "OFF"}' \
  --target '{"Arn": "arn:aws:lambda:eu-west-1:123456789012:function:informe",
             "RoleArn": "arn:aws:iam::123456789012:role/scheduler-invoca"}'
```

12.6 Amazon Kinesis Data Streams: streaming en tiempo real

Una cola entrega cada mensaje a **un** consumidor y lo borra. Un **stream** es distinto: un registro ordenado e inmutable (un *log*) que **varios** consumidores leen a la vez, cada uno a su ritmo, y que se conserva aunque ya se haya leído. Es la base de la analítica en tiempo real: clics, telemetría IoT, logs, transacciones.

FICHA DEL SERVICIO

Amazon Kinesis Data Streams

Streaming de datos en tiempo real, escalable y duradero

TIPO	Gestionado y regional (datos en tres AZ); *on-demand* (por GB y hora de stream) o provisionado (por shard y hora)
LÍMITES CLAVE	Por shard: escritura de 1 MB/s o 1000 registros/s y lectura de 2 MB/s; retención de 24 h (por defecto) a 365 días

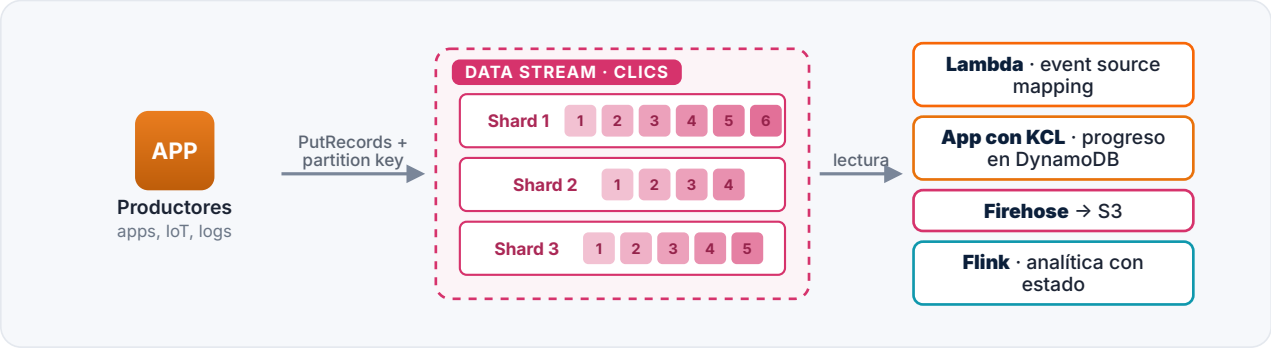


Figura 74 · La partition key decide el shard; dentro de cada shard los registros están ordenados y varios consumidores los leen.

Shard — la unidad de capacidad y de orden. La capacidad del stream es la suma de la de sus shards (los límites de cada uno están en la ficha).
Partition key — la elige el productor (un ID de usuario o de dispositivo) y su hash decide el shard. Los registros con la misma clave van al mismo shard y se leen **en orden**; una clave muy repetida crea un **shard caliente** (los errores `ProvisionedThroughputExceeded` delatan ese reparto o shards insuficientes).
Retención — 24 h por defecto, ampliable hasta 365 días; puedes **releer** el pasado para reprocesar tras un bug o alimentar un consumidor nuevo.
Consumidores — Lambda, aplicaciones con la **Kinesis Client Library** (KCL, que reparte shards entre instancias y guarda el progreso en DynamoDB), Firehose o Flink. Por defecto comparten los 2 MB/s de lectura de cada shard; con **enhanced fan-out**, cada consumidor registrado recibe sus propios 2 MB/s por shard, empujados por HTTP/2 y con menor latencia.
Capacidad — en modo **on-demand** escala solo y pagas por GB escrito y leído, más una tarifa por hora de stream (ideal si el tráfico es impredecible); en modo **provisionado** fijas los shards (y los divides o fusionas para reajustar) y pagas por shard y hora.

12.7 Firehose, Flink, MSK y Amazon MQ

Amazon Data Firehose

Antes Kinesis Data Firehose

Entrega datos en streaming **sin gestionar capacidad** a S3, Redshift, OpenSearch, Splunk, Snowflake, tablas Apache Iceberg o endpoints HTTP. Agrupa por tamaño o tiempo (casi tiempo real), transforma con Lambda, convierte a Parquet y comprime; se paga por GB. La forma más sencilla de llevar logs y eventos a un *data lake*.

Managed Service for Apache Flink

Antes Kinesis Data Analytics

Ejecuta aplicaciones Apache Flink (Java, Python o SQL) que procesan streams **con estado**: ventanas temporales, agregaciones, *joins* y detección de anomalías sobre datos de Kinesis o MSK.

Amazon MSK
Apache Kafka gestionado

Clústeres Kafka provisionados (con *brokers* Standard o Express) o MSK Serverless, y MSK Connect para conectores. Elígelo si ya usas Kafka o necesitas su ecosistema, retenciones largas y relectura por *offset*.

Amazon MQ
ActiveMQ y RabbitMQ gestionados

Brokers con protocolos estándar (JMS, AMQP, MQTT, STOMP, OpenWire) para **migrar** aplicaciones que ya los usan sin reescribirlas. Para desarrollos nuevos, SQS y SNS escalan mejor y no hay brokers que dimensionar.

12.8 ¿Qué servicio elijo?

Servicio	Modelo	¿Quién recibe cada mensaje?	Orden	Retención y relectura	Ideal para
SQS	Cola (pull)	Un consumidor	Solo FIFO	Hasta 14 días; sin relectura	Colas de trabajo, amortiguar picos
SNS	Pub/sub (push)	Todos los suscriptores	Solo FIFO	No guarda (salvo archivo en FIFO)	Fan-out, notificaciones
EventBridge	Bus con reglas (push)	Los destinos de cada regla que coincide	No	Archivo y *replay* opcionales	Integrar servicios, SaaS y eventos de AWS
Kinesis Data Streams	Stream (pull)	Todos, cada uno a su ritmo	Por shard	De 24 h a 365 días; relectura	Telemetría, clics, analítica en tiempo real
Amazon MSK	Stream Kafka	Todos los grupos de consumidores	Por partición	Configurable; relectura por offset	Plataformas y ecosistema Kafka

12.9 Patrones de integración

Ya conoces dos patrones básicos: la **cola de trabajo**, en la que varios consumidores idénticos compiten por los mensajes de una cola (*competing consumers*) y escalas añadiendo consumidores, y el fan-out. Sobre ellos se construyen los demás. El primer dilema es quién dirige el flujo:

Coreografía
EventBridge, SNS

Cada servicio reacciona a eventos y publica los suyos; nadie dirige. Muy desacoplado y fácil de ampliar, pero el flujo completo es difícil de ver y depurar.

Orquestación
Step Functions

Un coordinador central llama a cada paso y decide el siguiente, con reintentos y compensaciones explícitos. El flujo es visible y auditable, a costa de que el orquestador conozca a todos.

En la práctica se combinan: orquestación **dentro** de un dominio (el proceso de un pedido) y coreografía **entre** dominios (Pedidos publica `PedidoConfirmado` y Envíos reacciona).

Saga — una transacción de negocio repartida entre servicios sin transacción distribuida: cada paso tiene una **acción compensatoria** (reembolsar el cobro, liberar el stock) que se ejecuta si un paso posterior falla. Step Functions, con `Catch`, es la forma natural de orquestarla.
Transactional outbox — si un servicio guarda en su base de datos **y** publica un evento, un fallo entre ambas operaciones pierde o inventa eventos. La solución: escribir el evento en una tabla *outbox* en la **misma transacción** y que otro proceso lo publique. En AWS es habitual usar DynamoDB Streams (o la captura de cambios de la base de datos) con EventBridge Pipes como relé.
Idempotencia — casi toda la mensajería de AWS entrega **al menos una vez**, así que los consumidores deben tolerar duplicados: registra una **clave de idempotencia** (el ID del mensaje o del pedido) con una escritura condicional y descarta lo ya procesado. La deduplicación de SQS FIFO solo cubre 5 minutos; Powertools for AWS Lambda lo resuelve con DynamoDB.

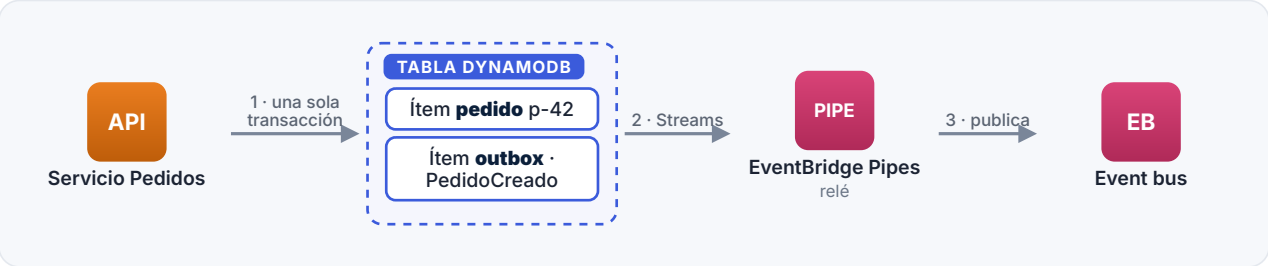


Figura 75 · Outbox con DynamoDB: el pedido y su evento se guardan juntos; el stream de la tabla y Pipes publican el evento.

RESUMEN DEL CAPÍTULO

- Desacoplar con mensajería asíncrona da resiliencia, absorbe picos y permite escalar y evolucionar cada parte; a cambio exige tolerar duplicados y consistencia eventual.
- **SQS** es una cola pull: Standard (al menos una vez, rendimiento casi ilimitado) o FIFO (orden por *message group* y exactamente una vez).
- Un mensaje recibido queda invisible durante el **visibility timeout**; si no se borra, reaparece y, tras `maxReceiveCount`, va a la **DLQ**.
- Usa long polling, lotes y, con Lambda, respuestas parciales de lote; el tamaño máximo de mensaje es 1 MiB.
- **SNS** empuja cada publicación a todos sus suscriptores, con filtros por suscripción; **SNS + SQS** es el patrón fan-out.
- **EventBridge** enruta eventos por contenido con reglas y patrones JSON; añade schema registry, archivo y replay, Pipes y Scheduler.
- **Kinesis Data Streams** es un log ordenado por shard que leen varios consumidores, con relectura de hasta 365 días; Firehose entrega a destinos, Flink procesa con estado, y MSK (Kafka) y Amazon MQ (ActiveMQ, RabbitMQ) cubren ecosistemas y migraciones existentes.
- Patrones clave: cola de trabajo, fan-out, coreografía frente a orquestación, saga, outbox e idempotencia.

Preguntas de repaso

1. **Un consumidor tarda unos 2 minutos por mensaje y los mensajes se procesan dos veces. ¿Qué ajustas?**
   Respuesta: El visibility timeout, por encima de 2 minutos (o ampliado con `ChangeMessageVisibility` durante el proceso); además, consumidor idempotente.
2. **Necesitas procesar en orden los eventos de cada cliente, pero en paralelo entre clientes. ¿Cómo?**
   Respuesta: Cola SQS FIFO con el ID de cliente como `MessageGroupId`.
3. **¿Qué ventaja tiene SNS + SQS frente a suscribir varias Lambda directamente al topic?**
   Respuesta: Cada consumidor tiene su cola: absorbe picos, procesa a su ritmo, reintenta y tiene su DLQ sin afectar a los demás.
4. **Quieres reaccionar cuando cualquier instancia EC2 de la cuenta se detenga. ¿Qué servicio usas?**
   Respuesta: Una regla de EventBridge en el bus por defecto con un patrón sobre el evento de cambio de estado de EC2.
5. **Una función que lee de SQS falla con un mensaje de cada lote de 10 y se reprocesan los 10. ¿Solución?**
   Respuesta: Activar `ReportBatchItemFailures` y devolver en `batchItemFailures` solo los mensajes fallidos.
6. **Tres equipos deben leer el mismo flujo de clics y poder reprocesar la última semana. ¿SQS o Kinesis?**
   Respuesta: Kinesis Data Streams con retención de al menos 7 días: varios consumidores leen los mismos datos y pueden releerlos.
7. **¿Qué problema resuelve el patrón *transactional outbox*?**
   Respuesta: La inconsistencia entre guardar en la base de datos y publicar el evento: ambos se escriben en la misma transacción y un relé publica después.

CAPÍTULO 13

CloudFront, Route 53 y el borde de la red

Acerca tu contenido a usuarios de todo el mundo: caché global, seguridad en el borde, código en las edge locations y un DNS gestionado y programable.

EN ESTE CAPÍTULO APRENDERÁS

1. Entender qué aporta una CDN y cómo funciona la caché de CloudFront
2. Configurar orígenes, cache behaviors y políticas de caché
3. Servir un bucket S3 privado con Origin Access Control y HTTPS
4. Elegir entre CloudFront Functions y Lambda@Edge
5. Gestionar zonas, registros y políticas de enrutamiento en Route 53
6. Distinguir cuándo usar CloudFront y cuándo Global Accelerator

SERVICIOS PROTAGONISTAS

Amazon CloudFront · Amazon Route 53 · AWS Certificate Manager · AWS WAF · AWS Shield · CloudFront Functions · Lambda@Edge · AWS Global Accelerator

13.1 ¿Por qué una CDN?

Si tu aplicación vive en `eu-west-1` (Irlanda), un usuario de Madrid la tiene cerca, pero uno de Sídney o de São Paulo paga cientos de milisegundos de latencia en cada ida y vuelta, multiplicados por cada imagen, script o llamada a la API. Una **CDN** (*Content Delivery Network*) resuelve el problema guardando copias del contenido en servidores repartidos por el mundo, cerca de los usuarios. **Amazon CloudFront** es la CDN de AWS y aporta tres cosas:

Menos latencia
El contenido, cerca del usuario

Responde el punto de presencia más cercano; el TLS termina allí y el resto del camino va por la red troncal de AWS.

Menos carga en el origen
Cada acierto, una petición menos

Cada acierto de caché (*cache hit*) es una petición que tu servidor no procesa: menos coste y más resistencia a los picos.

Más seguridad
El borde como escudo

Absorbe ataques DDoS (Shield Standard incluido), filtra con AWS WAF y oculta el origen.

La red de CloudFront tiene más de 700 puntos de presencia en más de 50 países. La mayoría son **edge locations**, que atienden a los usuarios; por detrás hay una capa de **Regional Edge Caches** (REC), cachés más grandes compartidas por varias edge locations. Si un objeto cacheable no está en la edge location (*miss*), CloudFront lo busca en su REC y solo si tampoco está allí acude al origen; así, incluso el contenido poco popular se beneficia de la caché. Opcionalmente, **Origin Shield** añade una capa más, centralizada en una región, para reducir todavía más las peticiones al origen.

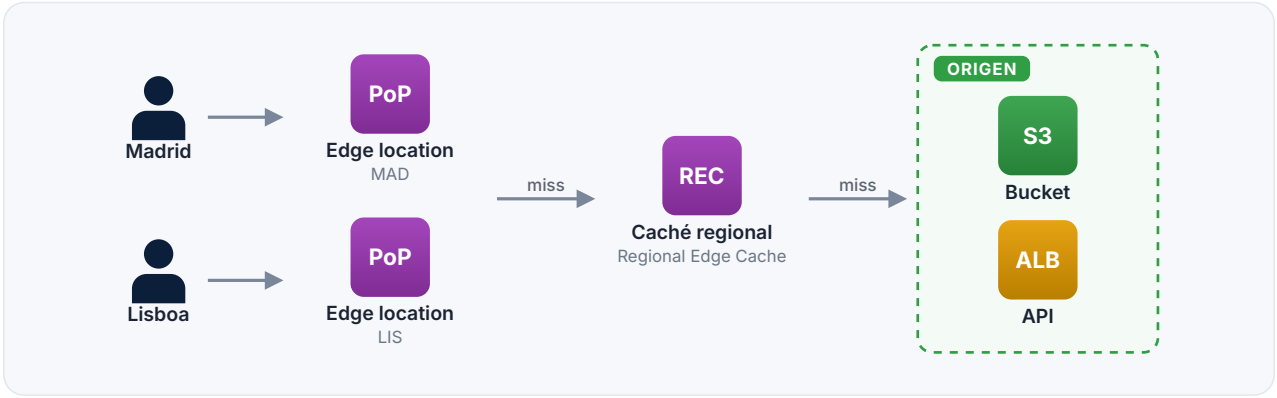


Figura 76 · Una petición a través de CloudFront: si la edge location tiene el objeto, responde al instante; si no, lo pide a la Regional Edge Cache y, en último término, al origen.

13.2 Distribuciones, orígenes y cache behaviors

FICHA DEL SERVICIO

Amazon CloudFront

Red de distribución de contenidos (CDN) global de AWS

ÁMBITO	Global: una distribución se despliega en toda la red (o en parte, según la *price class*)
PROTOCOLOS	HTTP/1.1, HTTP/2 y HTTP/3; IPv4 e IPv6; WebSocket
CONTENIDO	Estático (imágenes, JS, vídeo) y dinámico (API, HTML personalizado)
FACTURACIÓN	Por GB servido a los usuarios (varía por zona geográfica) y por peticiones; la transferencia desde orígenes de AWS hasta CloudFront es gratuita

La unidad de configuración es la **distribución**. Cada una recibe un nombre de CloudFront, como `d111111abcdef8.cloudfront.net`, y admite tus propios dominios (*alternate domain names*, como

`www.ejemplo.com`) con su certificado TLS. Una distribución tiene uno o varios **orígenes**, de donde CloudFront obtiene el contenido:

- Un **bucket S3** (endpoint REST), protegido con Origin Access Control; o el endpoint de **sitio web estático** de S3, que se trata como origen HTTP personalizado.
- Un **Application Load Balancer**, instancias EC2 o cualquier servidor HTTP (*custom origin*). Con los **VPC origins** (2024), el ALB, el NLB o las instancias pueden estar en subredes privadas, accesibles solo desde CloudFront.
- **API Gateway** y **Lambda function URLs** (estas, también con OAC).

Un **origin group** combina un origen primario y uno secundario: si el primario devuelve ciertos códigos de error (por ejemplo, 500, 502, 503 o 504) o no responde, CloudFront reintenta la petición en el secundario. Funciona con peticiones GET, HEAD y OPTIONS.

Los **cache behaviors** deciden qué hacer con cada petición. Cada uno asocia un patrón de ruta (*path pattern*, como `/api/*` o `*.jpg`) con un origen y sus ajustes: protocolos y métodos HTTP permitidos, políticas de caché, compresión, funciones de borde o la obligación de usar URL firmadas. CloudFront los evalúa en orden y aplica el primero que coincide, así que los patrones más específicos van primero; el comportamiento por defecto (`*`) recoge el resto. En la figura, la API no se cachea y las imágenes y el sitio sí: `CachingDisabled` y `CachingOptimized` son políticas de caché gestionadas, que verás a continuación.

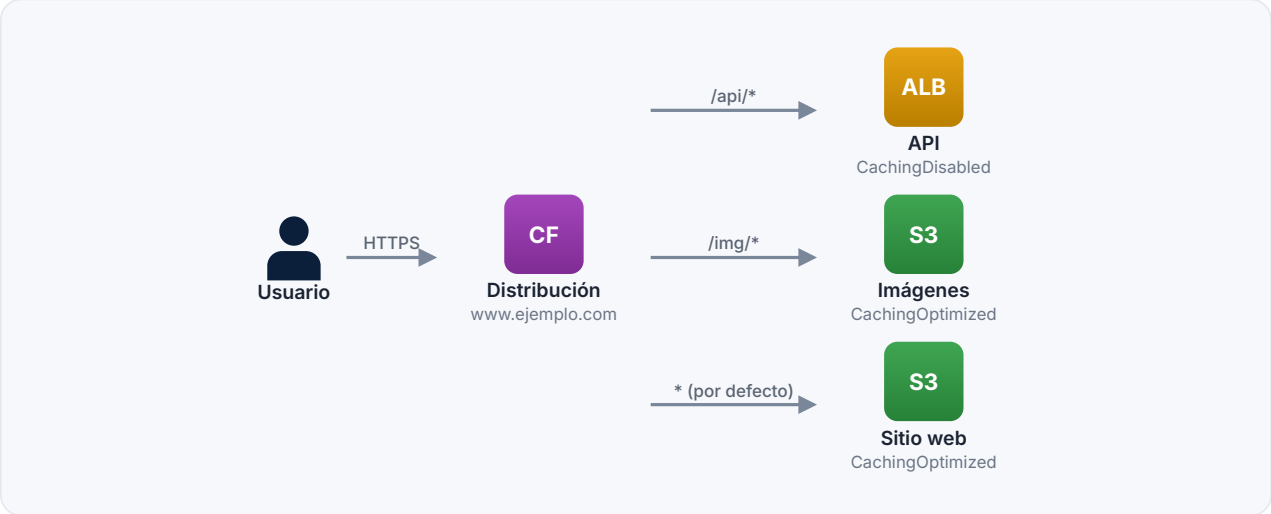


Figura 77 · Una distribución con tres cache behaviors: cada patrón de ruta va a su origen, con su propia política de caché.

13.3 La caché: cache key, políticas y TTL

Cuando llega una petición, CloudFront calcula su **cache key** (clave de caché), el nombre con el que guarda y busca cada respuesta. Por defecto, la clave es el dominio más la ruta; puedes añadir cadenas de consulta (*query strings*), cabeceras y cookies. La regla de oro: incluye solo lo que cambia la respuesta.

Por ejemplo, `/producto?id=42&utm_source=correo` y `/producto?id=42&utm_source=web` devuelven la misma página. Si la clave incluye todas las query strings, CloudFront guarda dos copias y va dos veces al origen; si solo incluye `id`, la segunda petición es un acierto. Cada valor de más fragmenta la caché —una cookie de sesión en la clave significa una copia por usuario— y hunde la tasa de aciertos (*cache hit ratio*). La configuración se reparte en tres políticas reutilizables:

Cache policy
Qué se cachea y cuánto tiempo

Los valores que forman la cache key y los TTL mínimo, máximo y por defecto. Gestionadas: `CachingOptimized`, `CachingDisabled`…

Origin request policy
Qué llega al origen

Lo que está en la clave llega siempre al origen; esta política **reenvía** además valores que no deben fragmentar la caché: cabeceras como *CloudFront-Viewer-Country*, cookies o query strings.

Response headers policy
Qué cabeceras añadir

Cabeceras que CloudFront añade o quita en la respuesta: de seguridad (HSTS, *X-Frame-Options*, CSP…), CORS o propias.

El tiempo que un objeto pasa en la caché (**TTL**) lo propone el origen con sus cabeceras y lo acota la cache policy:

- Si el origen envía `Cache-Control: max-age=N` (o `s-maxage=N`, que tiene prioridad en cachés compartidas como CloudFront), el objeto se guarda *N* segundos, siempre dentro de los TTL mínimo y máximo de la política.
- Si no envía cabeceras de caché, se aplica el TTL por defecto (24 horas en `CachingOptimized`).
- Con `no-cache`, `no-store` o `private`, CloudFront no cachea… siempre que el TTL mínimo sea 0. Si es mayor, cachea durante ese mínimo igualmente: es una sorpresa habitual.

Las cabeceras de la respuesta a `GET /assets/app.3f9a2c.js` te dicen qué ha pasado:

```http
HTTP/2 200
content-type: application/javascript
cache-control: public, max-age=31536000, immutable
age: 5233
x-cache: Hit from cloudfront
via: 1.1 7c5e1b0a2f9d3e4c.cloudfront.net (CloudFront)
x-amz-cf-pop: MAD53-P1
```

`x-cache` indica si fue un acierto (`Hit`), un fallo (`Miss`) o un objeto caducado que se revalidó con el origen (`RefreshHit`); `age`, cuántos segundos lleva el objeto en caché, y `x-amz-cf-pop`, qué edge location respondió.

Invalidaciones frente a versionado

Invalidación
Borrar de la caché

Pides a CloudFront que elimine rutas de todas las edge locations (`/index.html`, `/css/*`). No es instantánea; los primeros 1000 paths de cada mes son gratuitos y después se pagan (un comodín cuenta como un path).

Versionado
Nombres nuevos para contenido nuevo

Incluyes una huella del contenido en el nombre (`app.3f9a2c.js`) y lo cacheas un año con `immutable`; el HTML, con TTL corto, apunta al fichero nuevo. Nada que invalidar y sin mezclas de versiones.

```bash
aws cloudfront create-invalidation --distribution-id E1A2B3C4D5E6F7 \
  --paths "/index.html" "/css/*"
```

COSTES

Pagas por GB servido a los usuarios (más caro en zonas como Sudamérica u Oceanía), por cada 10 000 peticiones y por extras (invalidaciones por encima de las gratuitas, funciones, logs en tiempo real, Origin Shield…). La **price class** (100, 200 o todas las ubicaciones) excluye las zonas más caras a cambio de algo más de latencia allí. Desde noviembre de 2025 existen además planes de **tarifa plana** mensual que agrupan CloudFront, WAF, protección DDoS, DNS de Route 53 y otros servicios, sin cargos por exceso.

13.4 Seguridad en el borde

Origin Access Control: un bucket S3 privado detrás de CloudFront

El patrón más común: el bucket sigue **privado** (Block Public Access activado) y solo CloudFront puede leerlo. Hacen falta dos piezas:

- En la distribución, un **Origin Access Control** (OAC) hace que CloudFront firme cada petición al bucket con SigV4, el mismo mecanismo de firma que usan la CLI y los SDK.
- En el bucket, la bucket policy da permiso de lectura (`s3:GetObject`) al servicio CloudFront, cuyo principal es `cloudfront.amazonaws.com`, pero solo si la petición procede de **tu** distribución (condición `AWS:SourceArn`). Sin esa condición, cualquier distribución de cualquier cuenta podría leer tu bucket.

OAC sustituye al antiguo *Origin Access Identity* (OAI): admite objetos cifrados con SSE-KMS (si la key policy de la clave lo permite a CloudFront), todas las regiones y peticiones de escritura.

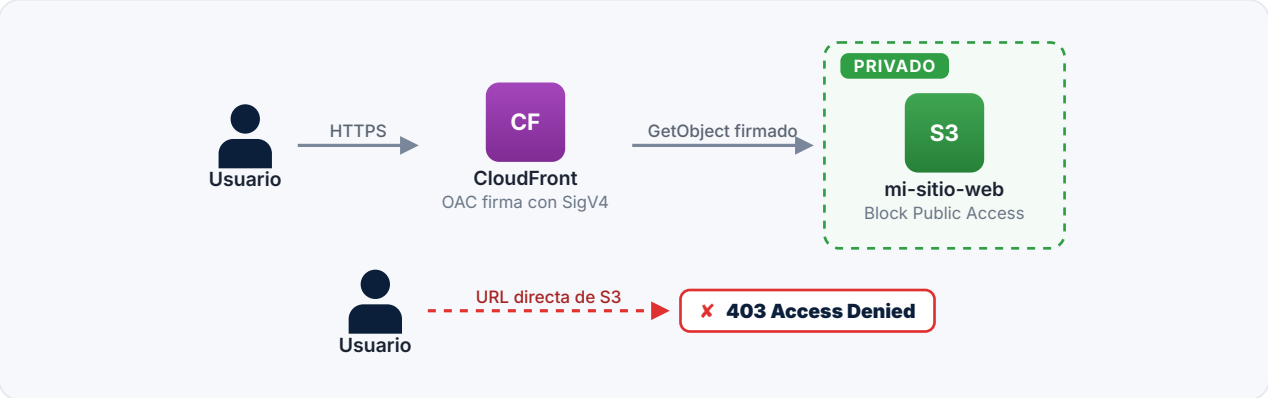


Figura 78 · Origin Access Control: el bucket solo acepta peticiones firmadas por tu distribución; el acceso directo a S3 se rechaza.

```json
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "SoloDesdeMiDistribucion",
    "Effect": "Allow",
    "Principal": { "Service": "cloudfront.amazonaws.com" },
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::mi-sitio-web/*",
    "Condition": {
      "StringEquals": {
        "AWS:SourceArn":
          "arn:aws:cloudfront::123456789012:distribution/E1A2B3C4D5E6F7"
      }
    }
  }]
}
```

HTTPS: certificados y protocolos

Una petición a CloudFront recorre dos tramos cifrados: del usuario a la edge location y de la edge location al origen.

- **Certificado de tu dominio**: emitido por ACM (gratuito y con renovación automática) y obligatoriamente en `us-east-1` (Norte de Virginia), aunque tu aplicación viva en otra región.
- **Viewer protocol policy**: *HTTP and HTTPS*, *Redirect HTTP to HTTPS* (lo habitual) o *HTTPS only*. Una **security policy** fija la versión mínima de TLS.

- **Origin protocol policy** (orígenes personalizados): *HTTP only*, *HTTPS only* o *Match viewer*. Con HTTPS, el certificado del origen debe ser válido para su nombre: nada de certificados autofirmados.
- **SNI** (*Server Name Indication*) sirve muchos certificados desde las mismas IP; es la opción por defecto y gratuita. Las IP dedicadas, para clientes antiguos sin SNI, se pagan aparte y son caras.

> ★ **CLAVE PARA LA CERTIFICACIÓN**
>
> «Certificado para CloudFront» = ACM en `us-east-1`. En cambio, el certificado de un ALB debe estar en la región del propio ALB.

Signed URLs y signed cookies

Para contenido privado o de pago (cursos, descargas, vídeo), CloudFront puede exigir que cada petición vaya **firmada**. Tu aplicación autentica al usuario y genera la firma con una clave privada cuya clave pública has registrado en un **trusted key group** de la distribución (las antiguas *CloudFront key pairs* exigían el usuario root: evítalas).

	Signed URL	Signed cookies
Alcance	Un fichero por URL	Muchos ficheros (por ejemplo, `/premium/*`)
Cambia las URL	✔ Añade parámetros de firma	✘
Ideal para	Una descarga concreta; clientes que no admiten cookies	Vídeo HLS o DASH con cientos de segmentos; zonas de suscriptores
Política	*Canned* (solo caducidad) o *custom* (comodines, fecha de inicio, rango de IP)	Las mismas opciones

Una signed URL es la URL normal con tres parámetros añadidos: `Expires` (o `Policy`), `Signature` y `Key-Pair-Id`, el identificador de la clave pública.

> ! **CUIDADO**
>
> No confundas las *signed URLs* de CloudFront con las *presigned URLs* de S3: estas dan acceso temporal directo a un objeto de S3, con los permisos de quien las genera, y no pasan por la CDN.

Geo-restricción, WAF y Shield

Con la **geo-restricción** defines una lista de países permitidos o bloqueados para toda la distribución, según la geolocalización de la IP del usuario; los bloqueados reciben un 403. Para reglas más finas (por ruta, por IP, contra bots o contra el OWASP Top 10), asocia a la distribución una *web ACL* de **AWS WAF** de ámbito CloudFront, creada en `us-east-1`. Además, todas las distribuciones tienen gratis **AWS Shield Standard** frente a ataques DDoS de capas 3 y 4 (**Shield Advanced**, de pago, añade más). Ambos servicios se explican en el capítulo de Seguridad.

> ◆ **CIERRA LA PUERTA TRASERA DEL ORIGEN**
>
> Si tu origen es un ALB público, impide que se pueda saltar CloudFront: en su Security Group permite solo la *managed prefix list* `com.amazonaws.global.cloudfront.origin-facing` y haz que CloudFront añada una cabecera secreta que el ALB compruebe en sus reglas. Mejor aún: usa VPC origins y deja el ALB en subredes privadas.

13.5 Código en el borde: CloudFront Functions y Lambda@Edge

CloudFront puede ejecutar tu código en cuatro momentos del ciclo de una petición, con dos tecnologías distintas:

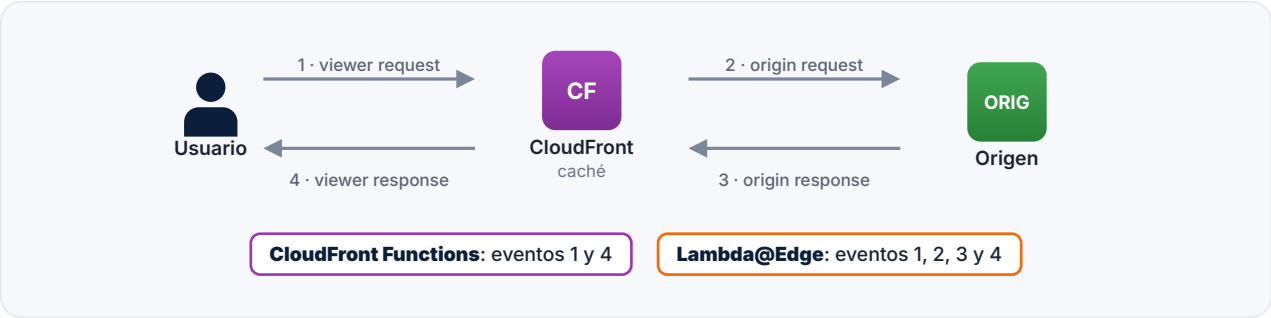


Figura 79 · Los cuatro eventos en los que puede ejecutarse código. Los eventos de origen solo se disparan cuando hay un fallo de caché.

	CloudFront Functions	Lambda@Edge
Lenguaje	JavaScript (entorno ligero)	Node.js y Python
Dónde se ejecuta	En las edge locations	En las Regional Edge Caches
Eventos	*Viewer request* y *viewer response*	Los cuatro
Tiempo máximo	Menos de 1 ms	5 s (viewer) y 30 s (origin)
Memoria	2 MB	128 MB (viewer); hasta 10 240 MB (origin)
Tamaño del código	10 KB	1 MB (viewer) y 50 MB (origin)
Red, ficheros, cuerpo de la petición	✘	✔
Escala y precio	Millones de peticiones por segundo; en torno a una sexta parte del precio	Miles por segundo y región; más caro
Usos típicos	Redirecciones, reescritura de URL, cabeceras, normalizar la cache key, validar tokens sencillos	Autenticación contra servicios externos, generar respuestas, elegir el origen dinámicamente

Las funciones de Lambda@Edge se crean en `us-east-1`, y CloudFront replica una versión numerada por todo el mundo. Las CloudFront Functions pueden leer datos de **CloudFront KeyValueStore**, un almacén clave-valor global muy útil para tablas de redirecciones. Este ejemplo, asociado al evento viewer request, añade `index.html` a las rutas de directorio:

```js
// CloudFront Function (viewer request): /docs/ → /docs/index.html
function handler(event) {
  var request = event.request;
  var uri = request.uri;
  if (uri.endsWith('/')) {
    request.uri += 'index.html';
  } else if (!uri.includes('.')) {
    request.uri += '/index.html';
  }
  return request;
}
```

13.6 Logs y métricas

CloudFront publica en CloudWatch (región `us-east-1`) métricas como peticiones, bytes y tasas de error 4xx y 5xx; otras, como la tasa de aciertos de caché, tienen coste adicional. Para el detalle de cada

petición están los **standard logs** (IP, ruta, código de estado, edge location, resultado de caché…), que se entregan en S3, CloudWatch Logs o Data Firehose, y los **real-time logs**, que llegan en segundos a Kinesis Data Streams. Más en el capítulo de Observabilidad.

13.7 Route 53: el DNS gestionado de AWS

Amazon Route 53 (el nombre viene del puerto 53, el del DNS) hace tres cosas: registra dominios, responde a las consultas DNS de tus zonas y comprueba la salud de tus endpoints para enrutar el tráfico en función de ella.

FICHA DEL SERVICIO

Amazon Route 53

DNS gestionado, registro de dominios y health checks

ÁMBITO	Global
ZONAS	Públicas (Internet) y privadas (asociadas a una o varias VPC)
FACTURACIÓN	Cuota mensual por zona alojada y por millón de consultas (las consultas alias a recursos de AWS son gratuitas), por health check y, si registras dominios, por dominio y año
SLA	100 % de disponibilidad del servicio de DNS

Zonas alojadas y tipos de registro

Una **hosted zone** contiene los registros de un dominio (`ejemplo.com`) y sus subdominios. Una zona **pública** responde a todo Internet; al crearla, Route 53 le asigna cuatro servidores de nombres (registro NS) que debes configurar en el registrador del dominio. Una zona **privada** solo responde dentro de las VPC asociadas (lo viste en el capítulo de Redes); si usas el mismo nombre en ambas (*split-horizon*), dentro de esas VPC prevalece la privada.

Tipo	Para qué sirve	Ejemplo
A	Nombre → dirección IPv4	`api.ejemplo.com → 203.0.113.10`
AAAA	Nombre → dirección IPv6	`api.ejemplo.com → 2001:db8::10`
CNAME	Nombre → otro nombre (nunca en el apex)	`blog.ejemplo.com → mi-blog.example.net`
Alias	Extensión de Route 53: nombre → recurso de AWS, también en el apex	`ejemplo.com → d111111abcdef8.cloudfront.net`
MX	Servidores de correo, con prioridad	`10 mail.ejemplo.com`
TXT	Texto libre: SPF, verificación de dominio…	`"v=spf1 include:amazonses.com ~all"`
NS	Servidores de nombres de la zona o de un subdominio delegado	`ns-123.awsdns-15.com`

Registros alias y el apex

El estándar DNS prohíbe un CNAME en el **apex** (la raíz de la zona, `ejemplo.com`), porque allí ya existen los registros SOA y NS y un CNAME no puede convivir con otros registros del mismo nombre. Los registros **alias** resuelven el problema: Route 53 responde directamente con las IP (A o AAAA) del recurso de destino —una distribución de CloudFront, un balanceador, un bucket configurado como sitio web, un API Gateway, un endpoint de VPC u otro registro de la zona—, sigue sus cambios automáticamente y no cobra las consultas cuando el destino es un recurso de AWS. El TTL lo marca el destino.

> **CLAVE PARA LA CERTIFICACIÓN**
>
> «Apuntar `ejemplo.com` (sin www) a un ALB o a CloudFront» → registro **alias**. CNAME solo para subdominios, y nunca hacia una IP.

El **TTL** indica a los resolutores cuántos segundos pueden cachear una respuesta: un TTL alto reduce consultas (y coste), pero hace lentos los cambios. Antes de una migración, bájalo con antelación (al menos un TTL antiguo antes), haz el cambio y vuelve a subirlo.

Políticas de enrutamiento

Política	Cómo decide	Caso de uso
Simple	Devuelve los valores del registro, sin health checks	Un único recurso
Weighted	Reparte según pesos relativos (por ejemplo, 90/10)	Despliegues *canary*, pruebas A/B
Latency	La región de AWS con menor latencia para el usuario	Aplicaciones multirregión
Failover	El primario mientras esté sano; si no, el secundario	Activo-pasivo, recuperación ante desastres
Geolocation	El país o continente del usuario (con un registro por defecto)	Contenido localizado, requisitos legales
Geoproximity	La distancia geográfica, ajustable con un *bias*	Desplazar tráfico entre regiones
Multivalue answer	Hasta 8 registros sanos, elegidos al azar	Reparto sencillo con health checks
IP-based	Rangos CIDR del cliente (*CIDR collections*)	Enrutar según el proveedor o la red de origen

Health checks y failover

Los **health checks** comprueban un endpoint por HTTP, HTTPS o TCP desde ubicaciones repartidas por el mundo, cada 30 segundos (o cada 10, con coste extra); se considera sano si más del 18 % de los comprobadores lo ven sano. Hay también health checks **calculados**, que combinan otros, y basados en **alarmas de CloudWatch**, para recursos privados que los comprobadores no alcanzan. Con la política *failover* dan una recuperación ante desastres sencilla:

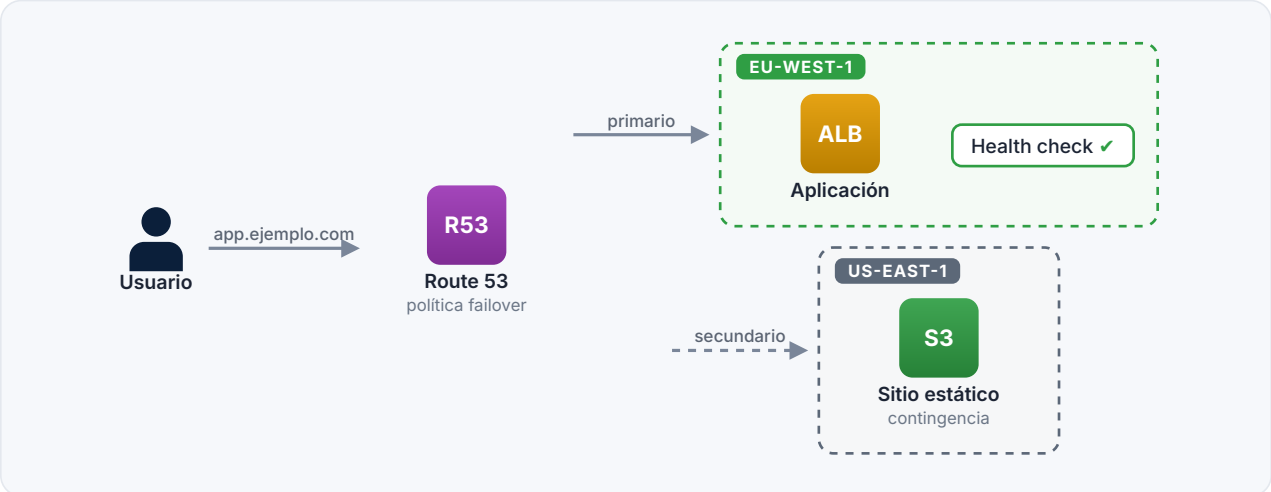


Figura 80 · Failover con Route 53: mientras el health check del primario sea correcto, las respuestas apuntan a eu-west-1; si falla, a una página de contingencia en otra región.

Registro de dominios

Route 53 también es **registrador**: compra o transfiere dominios, con renovación automática y protección de los datos de contacto; al registrarlo, crea automáticamente su zona pública. Si el dominio está

en otro registrador, basta con apuntar sus servidores de nombres (NS) a los de tu zona. Para más seguridad, activa **DNSSEC**.

13.8 Patrón: sitio web estático con S3, CloudFront, Route 53 y ACM

Con todo lo visto ya puedes montar la arquitectura más rentable para publicar una web estática o una SPA (aplicación de una sola página) con dominio propio y HTTPS: sin servidores que mantener y con un coste casi nulo cuando hay poco tráfico. Route 53 resuelve tu dominio hacia la distribución, CloudFront sirve por HTTPS con un certificado de ACM y lee el bucket privado mediante OAC (el diagrama de referencia completo, con un origen de API añadido, está en el capítulo de Arquitecturas de referencia):

1. Crea un bucket privado y sube el sitio: `aws s3 sync ./dist s3://mi-sitio-web --delete`.
2. Solicita en ACM, en `us-east-1`, un certificado para `www.ejemplo.com` con validación DNS (con Route 53, el registro de validación se crea con un clic).
3. Crea la distribución: origen S3 con OAC, *default root object* `index.html`, redirección de HTTP a HTTPS, nombre alternativo y certificado. Añade al bucket la política que te propone CloudFront (la del apartado de OAC).
4. En Route 53, crea registros alias A y AAAA de `www.ejemplo.com` hacia la distribución.

> **CUIDADO**
>
> El default root object solo se aplica a la raíz (`/`): `/blog/` no se traduce a `/blog/index.html`. Resuélvelo con una CloudFront Function como la del apartado de código en el borde. En una SPA, configura respuestas de error personalizadas que devuelvan `/index.html` con código 200 ante un 403 o un 404.

13.9 Global Accelerator frente a CloudFront

AWS Global Accelerator también usa la red global de AWS, pero no es una CDN: te da dos direcciones IP estáticas *anycast*, anunciadas desde las edge locations. El usuario entra por la más cercana y su tráfico TCP o UDP viaja por la red troncal de AWS hasta el endpoint regional sano más próximo (ALB, NLB, instancias EC2 o Elastic IP).

	CloudFront	Global Accelerator
Qué es	CDN con caché	Acelerador de red (capa 4), sin caché
Protocolos	HTTP y HTTPS (también WebSocket)	TCP y UDP
Punto de entrada	Un nombre DNS (las IP cambian)	Dos IP estáticas anycast
Failover entre regiones	Origin groups, petición a petición	Automático en segundos, sin depender de cachés DNS
Casos típicos	Webs, API, vídeo, descargas	Juegos, VoIP, IoT; clientes que necesitan IP fijas en listas de permitidos
Facturación	Por GB y por peticiones	Cuota por hora + recargo por GB transferido

CLAVE PARA LA CERTIFICACIÓN

IP fijas, UDP o failover regional en segundos, sin caché → Global Accelerator. Contenido HTTP cacheable, funciones en el borde o WAF → CloudFront.

RESUMEN DEL CAPÍTULO

- **CloudFront** es la CDN de AWS: más de 700 puntos de presencia y una capa de Regional Edge Caches para reducir la latencia, descargar el origen y proteger la aplicación.
- Una **distribución** combina orígenes (S3, ALB, servidores propios, API Gateway, Lambda function URLs, VPC origins) y **cache behaviors** por patrón de ruta; los **origin groups** aportan failover.
- La **cache key** debe incluir solo lo que cambia la respuesta: **cache policy** (clave y TTL), **origin request policy** (qué se reenvía) y **response headers policy** (cabeceras añadidas).
- El TTL lo propone el origen con `Cache-Control`, dentro de los límites de la política. Versiona los ficheros y reserva las invalidaciones para el HTML.
- **OAC** mantiene privado el bucket S3: la bucket policy autoriza a `cloudfront.amazonaws.com` con la condición `AWS:SourceArn` de tu distribución.
- El certificado de CloudFront va en **ACM de** `us-east-1`; redirige HTTP a HTTPS y usa SNI. Las **signed URLs** protegen ficheros concretos; las **signed cookies**, conjuntos de ficheros.
- **CloudFront Functions** (eventos viewer, submilisegundo, baratas) frente a **Lambda@Edge** (cuatro eventos, acceso a red, más potente).
- **Route 53**: zonas públicas y privadas, registros **alias** en el apex, TTL, ocho políticas de enrutamiento y health checks para el failover.
- **Global Accelerator**: dos IP anycast estáticas, TCP y UDP y failover rápido, pero sin caché.

Preguntas de repaso

1. **Tu certificado de ACM no aparece al configurar la distribución. ¿Por qué?**
   Respuesta: Porque no está en `us-east-1`: CloudFront solo usa certificados de ACM de esa región.
2. **¿Cómo impides que se acceda a tu bucket S3 sin pasar por CloudFront?**
   Respuesta: Con el bucket privado (Block Public Access), OAC en la distribución y una bucket policy que autorice a `cloudfront.amazonaws.com` con la condición `AWS:SourceArn` de tu distribución.
3. **Tras un despliegue, los usuarios siguen viendo el CSS antiguo. ¿Qué puedes hacer?**
   Respuesta: Invalidar `/css/*` o, mejor, versionar los nombres de fichero para que el HTML apunte a los nuevos.
4. **¿Por qué no puedes crear un CNAME para `ejemplo.com` hacia tu distribución?**
   Respuesta: Porque el DNS no permite un CNAME en el apex de la zona; usa registros alias A y AAAA.
5. **Quieres enviar el 10 % del tráfico a la nueva versión de tu aplicación. ¿Qué política de Route 53 usas?**
   Respuesta: Weighted (ponderada), con pesos 90 y 10.
6. **Una función debe consultar un servicio externo de autorización antes de ir al origen. ¿CloudFront Functions o Lambda@Edge?**
   Respuesta: Lambda@Edge: las CloudFront Functions no tienen acceso a red.
7. **Un juego multijugador por UDP necesita IP fijas y un failover rápido entre regiones. ¿CloudFront o Global Accelerator?**
   Respuesta: Global Accelerator: ofrece dos IP anycast estáticas, admite UDP y conmuta entre regiones en segundos; CloudFront solo sirve HTTP y HTTPS.

CAPÍTULO 14

Observabilidad: logging, métricas y auditoría

Saber qué está pasando en tus sistemas —y quién hizo qué— antes de que te lo cuenten tus usuarios.

EN ESTE CAPÍTULO APRENDERÁS

1. Los tres pilares: métricas, logs y trazas
2. Métricas y alarmas de CloudWatch
3. Centralizar y consultar logs con CloudWatch Logs
4. Seguir una petición entre servicios con X-Ray
5. Auditar la cuenta con CloudTrail y AWS Config
6. Diseñar un logging centralizado multicuenta

SERVICIOS PROTAGONISTAS

Amazon CloudWatch · AWS X-Ray · AWS CloudTrail · AWS Config · Amazon EventBridge · Amazon Managed Grafana

14.1 Por qué observar: los tres pilares

Monitorizar es vigilar lo que ya sabes que puede fallar: la CPU, el disco, un proceso. **Observar** va más allá: poder responder preguntas nuevas —«¿por qué solo fallan los pagos con tarjeta de este país?»— a partir de las señales que el sistema ya emite, sin desplegar código nuevo. En AWS esas señales son de tres tipos:

Métricas
¿Va algo mal?

Series numéricas en el tiempo: latencia, errores, CPU. Baratas de guardar y consultar; la base de alarmas y tendencias. → **CloudWatch Metrics**.

Logs
¿Qué ha pasado exactamente?

Registros de eventos con detalle: un error con su traza de pila, una petición concreta. Más voluminosos: se filtran y se buscan. → **CloudWatch Logs**.

Trazas
¿Dónde se va el tiempo?

El recorrido de una petición por varios servicios, con la duración de cada tramo. → **AWS X-Ray** y **OpenTelemetry**.

A estas señales de tus aplicaciones se suma la **auditoría** de la propia cuenta: quién llamó a qué API (**CloudTrail**) y cómo estaba configurado cada recurso (**AWS Config**).

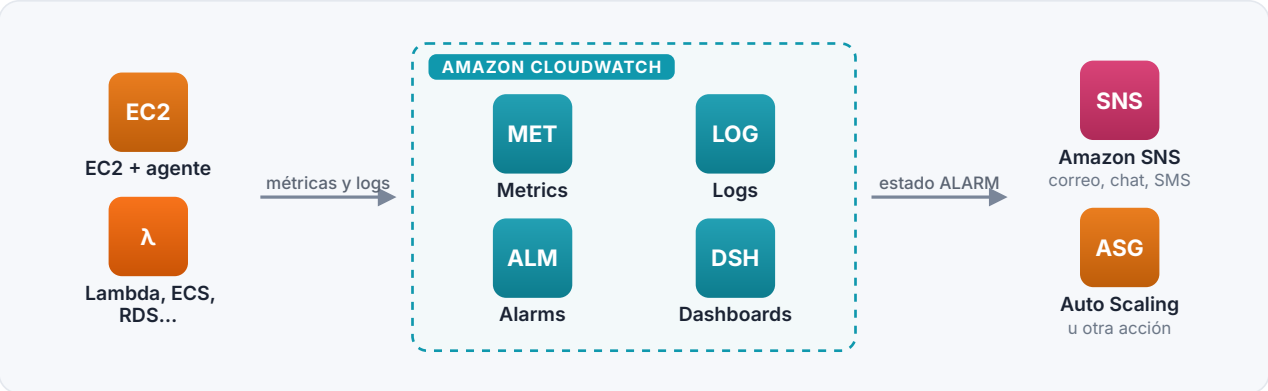


Figura 81 · Flujo básico: los recursos publican métricas y logs en CloudWatch y las alarmas disparan avisos o acciones automáticas.

14.2 CloudWatch Metrics

FICHA DEL SERVICIO

Amazon CloudWatch

Métricas, logs, alarmas y paneles para AWS y tus aplicaciones

ÁMBITO	Regional; los dashboards y la observabilidad entre cuentas reúnen varias regiones y cuentas
INCLUYE	Metrics, Alarms, Logs, Dashboards, Synthetics, RUM, Application Signals y varios «Insights»
FACTURACIÓN	Métricas personalizadas, peticiones a la API, alarmas, GB de logs ingeridos, almacenados y analizados, dashboards..., con una capa gratuita

Namespace — contenedor de métricas: `AWS/EC2`, `AWS/Lambda`... o uno tuyo, como `TiendaOnline`.
Métrica — una serie temporal con nombre (`CPUUtilization`) formada por *datapoints*: marca de tiempo, valor y unidad.
Dimensión — par nombre-valor que identifica la serie (`InstanceId=i-0abc…`, `Entorno=prod`). Cada combinación distinta es una métrica distinta; admite hasta 30 dimensiones.
Estadística y periodo — los datapoints se agregan por periodos (1 minuto, 5 minutos...) con `Average`, `Sum`, `Minimum`, `Maximum`, `SampleCount` o percentiles (`p99`).

Resolución — **estándar** (granularidad de 1 minuto) o **alta** (hasta 1 segundo, solo para métricas personalizadas).
CloudWatch conserva las métricas 15 meses, agregándolas a medida que envejecen:

3 h	15 días	63 días	15 meses
datapoints de 1 segundo	datapoints de 1 minuto	datapoints de 5 minutos	datapoints de 1 hora

Sin instalar nada, EC2 publica cada 5 minutos (cada minuto con *detailed monitoring*, de pago) la CPU, el tráfico de red, las operaciones de disco, los *status checks* y, en las instancias T, los créditos de CPU. Igual hacen Lambda (invocaciones, errores, duración, *throttles*), RDS, ELB, SQS y casi todos los servicios.

> **CUIDADO**
>
> EC2 **no** publica memoria ni espacio en disco: el hipervisor ve la CPU y la red de la instancia, pero no lo que ocurre dentro del sistema operativo. Para medir memoria (`mem_used_percent`), disco (`disk_used_percent`) o procesos, instala el **agente unificado de CloudWatch**, que publica en el namespace `CWAgent` .

Métricas personalizadas

Puedes publicar tus propias métricas de negocio o de aplicación con la API `PutMetricData` :

```bash
aws cloudwatch put-metric-data --namespace TiendaOnline \
  --metric-name PedidosFallidos --unit Count --value 1 \
  --dimensions Servicio=checkout,Entorno=prod
```

En Lambda y contenedores es más eficiente el **Embedded Metric Format** (EMF): escribes una línea de log JSON con un bloque `_aws` y CloudWatch extrae la métrica de forma asíncrona, sin llamadas a la API. El resto de campos (`pedidoId`) quedan en el log como contexto:

```json
{
  "_aws": {
    "Timestamp": 1773484867000,
    "CloudWatchMetrics": [{
      "Namespace": "TiendaOnline", "Dimensions": [["Servicio"]],
      "Metrics": [{ "Name": "LatenciaCheckout", "Unit": "Milliseconds" }]
    }]
  },
  "Servicio": "checkout", "LatenciaCheckout": 182, "pedidoId": "P-81"
}
```

> **MEDIA FRENTE A PERCENTILES**
>
> Si 98 peticiones tardan 100 ms y dos tardan 10 s, la media ronda los 300 ms y parece aceptable, pero el percentil 99 (`p99`) es de 10 s: hay usuarios esperando diez segundos. Para latencias, vigila `p90` y `p99` , no la media.

14.3 CloudWatch Alarms

Una **alarma** vigila una métrica (o una expresión matemática que combine varias) y compara cada periodo con un umbral. Tiene tres estados: `OK` , `ALARM` e `INSUFFICIENT_DATA` (no hay datos suficientes: la métrica acaba de empezar, la instancia está parada...), y cada cambio de estado puede disparar acciones.

Para no despertar a nadie por un pico aislado, decides cuántos periodos evaluar (**N**, `EvaluationPeriods`) y cuántos de ellos deben superar el umbral (**M**, `DatapointsToAlarm`): con «3 de

5», la alarma salta cuando 3 de los últimos 5 periodos lo superan, aunque no sean consecutivos. También eliges cómo tratar los datos que faltan: como ausentes, como dentro del umbral (`notBreaching`), como fuera (`breaching`) o ignorándolos.

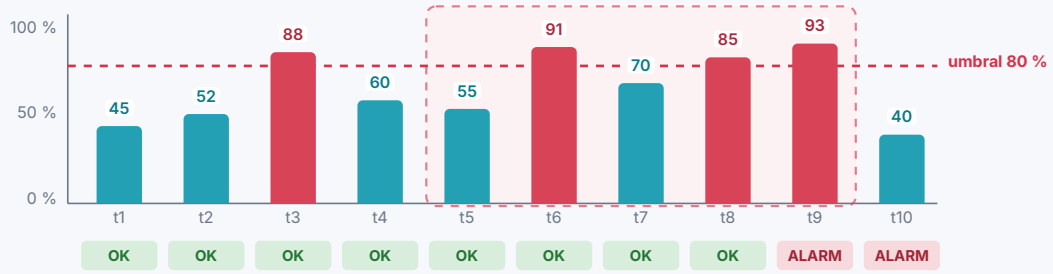


Figura 82 · Alarma «3 de 5» sobre la CPU media, con umbral del 80 % y periodos de 1 minuto. El pico aislado de t3 no la dispara; en t9, tres de los últimos cinco periodos superan el umbral y pasa a `ALARM`.

```bash
aws cloudwatch put-metric-alarm --alarm-name cpu-alta-web \
  --namespace AWS/EC2 --metric-name CPUUtilization \
  --dimensions Name=AutoScalingGroupName,Value=web-asg \
  --statistic Average --period 60 --threshold 80 \
  --comparison-operator GreaterThanThreshold \
  --evaluation-periods 5 --datapoints-to-alarm 3 \
  --treat-missing-data notBreaching \
  --alarm-actions arn:aws:sns:eu-west-1:123456789012:avisos-ops
```

- **Acciones**: notificar a un tema de **SNS** (correo, SMS, chat, webhooks), ejecutar una política de **Auto Scaling**, detener, terminar, reiniciar o recuperar una **instancia EC2**, invocar una función **Lambda** o crear un OpsItem o un incidente en **Systems Manager**.
- **Alarmas compuestas**: combinan otras alarmas con `AND`, `OR` y `NOT` («latencia alta **y** errores altos») para avisar solo cuando importa y reducir el ruido.
- **Detección de anomalías**: en lugar de un umbral fijo, CloudWatch aprende el comportamiento habitual de la métrica, con sus ciclos diarios y semanales, y dibuja una banda de valores esperados; la alarma salta cuando el valor sale de ella.

14.4 CloudWatch Logs

Evento de log — una línea con marca de tiempo y mensaje (texto o JSON).
Log stream — secuencia de eventos de una misma fuente: una instancia, un contenedor, un entorno de ejecución de Lambda.
Log group — agrupa streams del mismo tipo (`/aws/lambda/pedidos`, `/ecs/web`). En él se configuran la **retención**, el cifrado con KMS, los permisos y los filtros.
Lambda, ECS (con el driver `awslogs`), API Gateway y muchos otros servicios envían sus logs sin agente. Desde EC2 o servidores on-premises, el **agente unificado de CloudWatch** envía ficheros de log y métricas del sistema; su configuración JSON se guarda en Parameter Store y se distribuye con Systems Manager. Al crear un log group eliges su **clase**: **Standard**, con todas las funciones, o **Infrequent Access**, con una ingesta más barata a cambio de menos funciones (por ejemplo, sin metric filters ni Live Tail), aunque se puede consultar con Logs Insights. La clase no se puede cambiar después.

COSTES

Por defecto, un log group **nunca caduca**: pagas cada mes por todo lo acumulado desde el primer día. Fija una retención (de 1 día a 10 años) en cada log group —mejor desde IaC— y archiva en S3 lo que debas guardar años. La ingesta suele ser el mayor coste: no dejes el nivel DEBUG activado en producción.

Del log a la acción: filtros y exportación

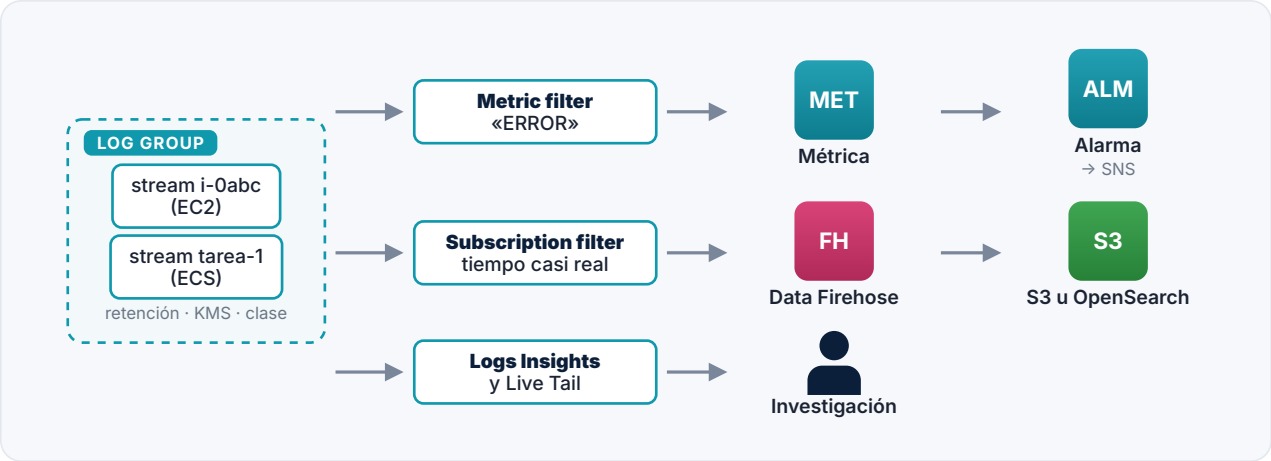


Figura 83 · Qué puedes hacer con los eventos de un log group: convertirlos en métricas, enviarlos a otros destinos o consultarlos.

- **Metric filters**: convierten en métrica los eventos que cumplen un patrón —contar `ERROR`, extraer la latencia de un campo JSON— para crear alarmas. Solo procesan eventos nuevos, no los anteriores a su creación.
- **Subscription filters**: envían en tiempo casi real los eventos que cumplen un patrón a Kinesis Data Streams, Amazon Data Firehose, Lambda u OpenSearch Service, incluso en otra cuenta. Admiten hasta dos por log group.
- **Exportación a S3**: una tarea por lotes (`create-export-task`), no en tiempo real: los datos pueden tardar hasta 12 horas en estar disponibles para exportar. Para un flujo continuo hacia S3, usa un subscription filter con Firehose.

Logs Insights y Live Tail

Logs Insights consulta uno o varios log groups con un lenguaje de tuberías (también admite SQL y PPL de OpenSearch) y se paga por GB analizado. Descubre automáticamente los campos de los logs JSON y los de servicios como Lambda (`@duration`, `@maxMemoryUsed`…) o los VPC Flow Logs (`srcAddr`, `action`…) del capítulo de Redes:

LOGS

```
# Los 20 errores más recientes
fields @timestamp, @logStream, @message
| filter @message like /ERROR/
| sort @timestamp desc
| limit 20

# Lambda: duración media, p99 y memoria máxima cada 5 minutos
filter @type = "REPORT"
| stats avg(@duration), pct(@duration, 99), max(@maxMemoryUsed) by bin(5m)

# VPC Flow Logs: quién acumula más conexiones rechazadas
filter action = "REJECT"
| stats count(*) as rechazos by srcAddr, dstPort
| sort rechazos desc
| limit 10
```

Live Tail, el `tail -f` de la nube, muestra en tiempo real y con filtros los eventos que van llegando; se factura por minuto de sesión.

14.5 Dashboards y monitorización de aplicaciones

Herramienta	Qué hace	Úsala para
Dashboards	Paneles con gráficos de métricas, alarmas y consultas de logs, incluso de varias regiones y cuentas	La vista de un servicio o del negocio
Synthetics	*Canaries*: scripts programados (Node.js o Python) que recorren tu web o API como un usuario, con capturas de pantalla	Detectar caídas antes que los usuarios, aunque no haya tráfico
RUM	Un fragmento JavaScript mide la experiencia real en el navegador: tiempos de carga, errores, Core Web Vitals	Saber cómo va la web para usuarios reales
Application Signals	Monitorización del rendimiento de aplicaciones (APM) con instrumentación automática (OpenTelemetry): latencia, errores y disponibilidad por servicio y operación, y mapa de servicios	Salud de microservicios y objetivos de nivel de servicio (SLO)
Container, Lambda y Database Insights	Métricas detalladas por clúster, servicio, tarea o pod (ECS, EKS); por función (memoria, CPU, arranques en frío); y carga de la base de datos por consulta SQL	Contenedores, funciones Lambda, RDS y Aurora (capítulos de Kubernetes y de Bases de datos)

14.6 Trazas distribuidas con AWS X-Ray

En una arquitectura de microservicios, una petición puede pasar por API Gateway, varias funciones Lambda, una cola y una tabla de DynamoDB. Si tarda dos segundos, ¿dónde se fueron? **AWS X-Ray** sigue cada petición de extremo a extremo y lo muestra así:

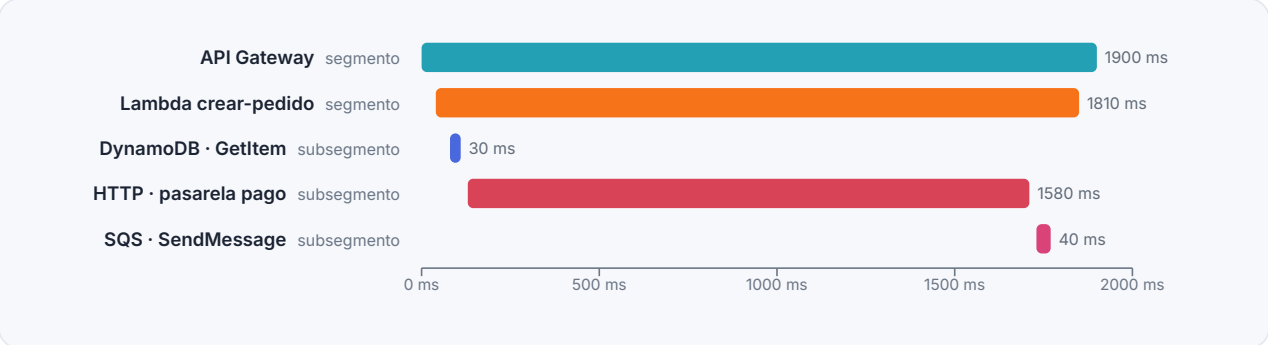


Figura 84 · Traza de una petición lenta: el subsegmento de la llamada HTTP a la pasarela de pago consume más del 80 % de los 1,9 s.

Traza — el recorrido completo de una petición, identificado por un *trace ID* que viaja en la cabecera `X-Amzn-Trace-Id`.
Segmento y subsegmento — el **segmento** es el trabajo de un servicio dentro de la traza (la función Lambda); los **subsegmentos** lo detallan, típicamente cada llamada a otro servicio (DynamoDB, HTTP, SQL).
Anotaciones y metadatos — pares clave-valor que añades tú; las **anotaciones** se indexan y permiten filtrar trazas (`cliente = "premium"`), los metadatos no.
Muestreo — no se traza todo; por defecto, la primera petición de cada segundo y el 5 % de las demás.
Mapa de servicios — grafo generado automáticamente con los servicios, sus conexiones, latencias y tasas de error.
En Lambda y API Gateway, activar el *tracing* activo es una casilla; en tu código instrumentas con los SDK de X-Ray o, cada vez más, con **OpenTelemetry**: AWS mantiene su propia distribución, **AWS Distro for OpenTelemetry (ADOT)**, y recomienda OpenTelemetry para las instrumentaciones nuevas.

14.7 Auditoría con AWS CloudTrail

FICHA DEL SERVICIO

AWS CloudTrail

Registro de auditoría de las llamadas a la API de tus cuentas

QUÉ REGISTRA	Quién hizo qué, cuándo, desde dónde y con qué resultado, ya sea desde la consola, la CLI, un SDK o un servicio de AWS
SIN CONFIGURAR	*Event history*: 90 días de eventos de gestión, gratis, consultables por región
FACTURACIÓN	La primera copia de los eventos de gestión en un trail es gratuita; se pagan los eventos de datos, Insights, las copias adicionales y CloudTrail Lake

Tipo de evento	Qué incluye	Ejemplos	Por defecto
De gestión	Operaciones del plano de control	`CreateBucket`, `RunInstances`, `AttachRolePolicy`, `ConsoleLogin`	✔ Sí
De datos	Operaciones sobre los datos, de gran volumen	`GetObject` / `PutObject` en S3, `Invoke` en Lambda, ítems de DynamoDB	✘ De pago
Insights	Actividad inusual: picos de llamadas o de errores	Ráfaga de borrados, aumento de `AccessDenied`	✘ De pago
De actividad de red	Llamadas a través de VPC endpoints, incluidas las denegadas	Accesos desde la VPC a S3 o KMS	✘ De pago

Para conservar más de 90 días o registrar eventos de datos, crea un **trail**: entrega los eventos a un bucket de S3 —con unos minutos de retraso— y, si quieres, a CloudWatch Logs para crear alarmas. Hazlo **multirregión** (también registra las regiones que no usas) y, con Organizations, crea un **trail de organización**, que cubre todas las cuentas sin que estas puedan modificarlo. La **validación de integridad** entrega cada hora un *digest* firmado con los hashes SHA-256 de los logs: `aws cloudtrail validate-logs` demuestra que nadie los ha alterado. **CloudTrail Lake** guarda los eventos en un almacén gestionado consultable con SQL, hasta 10 años.

Así es un evento (resumido): quién (`userIdentity`), qué (`eventName`), cuándo, desde dónde y sobre qué recurso.

JSON

```
{
  "userIdentity": {
    "type": "AssumedRole", "accountId": "123456789012",
    "arn": "arn:aws:sts::123456789012:assumed-role/Admin/ana"
  },
  "eventTime": "2026-03-14T09:21:07Z",
  "eventSource": "s3.amazonaws.com", "eventName": "DeleteBucket",
  "awsRegion": "eu-west-1", "sourceIPAddress": "203.0.113.25",
  "requestParameters": { "bucketName": "facturas-2025" }
}
```

14.8 AWS Config: configuración y cumplimiento

AWS Config inventaría tus recursos y registra cada cambio de configuración como un *configuration item*, junto con sus relaciones (esta instancia usa este Security Group, en esta subred). Así obtienes la **línea temporal de configuración** de cada recurso —qué cambió y cuándo, con enlace al evento de CloudTrail que dice quién— y puedes evaluar el cumplimiento:

- **Reglas**: comprueban la configuración deseada. Hay cientos de reglas gestionadas (`s3-bucket-public-read-prohibited`, `encrypted-volumes`, `restricted-ssh`…) y puedes escribir las tuyas con Lambda o con Guard. Se evalúan al cambiar un recurso o periódicamente.
- **Remediación**: asocia a una regla un documento de **Systems Manager Automation** que corrige el problema, a mano o automáticamente (por ejemplo, bloquear el acceso público de un bucket).
- **Conformance packs**: paquetes de reglas y remediaciones que se despliegan como una unidad, incluso en toda la organización, con plantillas para marcos como CIS o PCI DSS.
- **Agregadores**: reúnen el cumplimiento de varias cuentas y regiones en una sola vista.

> **CUIDADO**
>
> AWS Config es **detectivo**: avisa (y puede corregir) después del cambio, pero no lo impide. Para impedirlo usa controles **preventivos**, como las SCP o las políticas IAM (capítulo «IAM (II): permisos — políticas y evaluación»). Además, se factura por *configuration item* registrado y por evaluación de reglas: en cuentas con muchos cambios, limita los tipos de recurso que registra.

14.9 Reaccionar a eventos y otros logs

Muchos servicios emiten eventos a **Amazon EventBridge** (capítulo de Mensajería): cambios de estado de EC2, hallazgos de GuardDuty, avisos de AWS Health, llamadas a la API registradas por CloudTrail… Una **regla** con un patrón los filtra y los envía a un destino (Lambda, SNS, SQS, una automatización de Systems Manager): así pasas de observar a **reaccionar**. Este patrón avisa cada vez que alguien borra un bucket o cambia su política:

```json
{
  "source": ["aws.s3"],
  "detail-type": ["AWS API Call via CloudTrail"],
  "detail": {
    "eventSource": ["s3.amazonaws.com"],
    "eventName": ["DeleteBucket", "PutBucketPolicy"]
  }
}
```

Otros servicios generan sus propios logs, que conviene activar y llevar a la cuenta central: los **VPC Flow Logs** (capítulo de Redes), los logs de acceso de los balanceadores y de CloudFront (en sus capítulos) y los *server access logs* de S3. Estos últimos se entregan «best effort»: para auditar con garantías quién lee o escribe objetos, usa los eventos de datos de CloudTrail.

14.10 Logging centralizado multicuenta

En una organización con decenas de cuentas no quieres iniciar sesión en cada una para investigar un incidente. El patrón habitual —el que despliega AWS Control Tower— lleva cada tipo de dato a una cuenta especializada:

- El **trail de organización** deja los eventos de todas las cuentas en un bucket de **Log Archive** con acceso mínimo, Object Lock contra borrados y ciclo de vida hacia clases más baratas; se consulta con Athena.
- En la cuenta de auditoría, el **agregador** de AWS Config, Security Hub y GuardDuty dan la vista de cumplimiento y de seguridad de la organización (capítulo de Seguridad).
- La **observabilidad entre cuentas** de CloudWatch deja que la cuenta de monitorización vea métricas, logs y trazas de las demás sin copiarlos. Si tus equipos usan herramientas de código abierto, **Amazon Managed Grafana** crea paneles sobre CloudWatch, X-Ray o **Amazon Managed Service for Prometheus**, que guarda métricas de Prometheus (muy habitual en Kubernetes) y se consulta con PromQL.

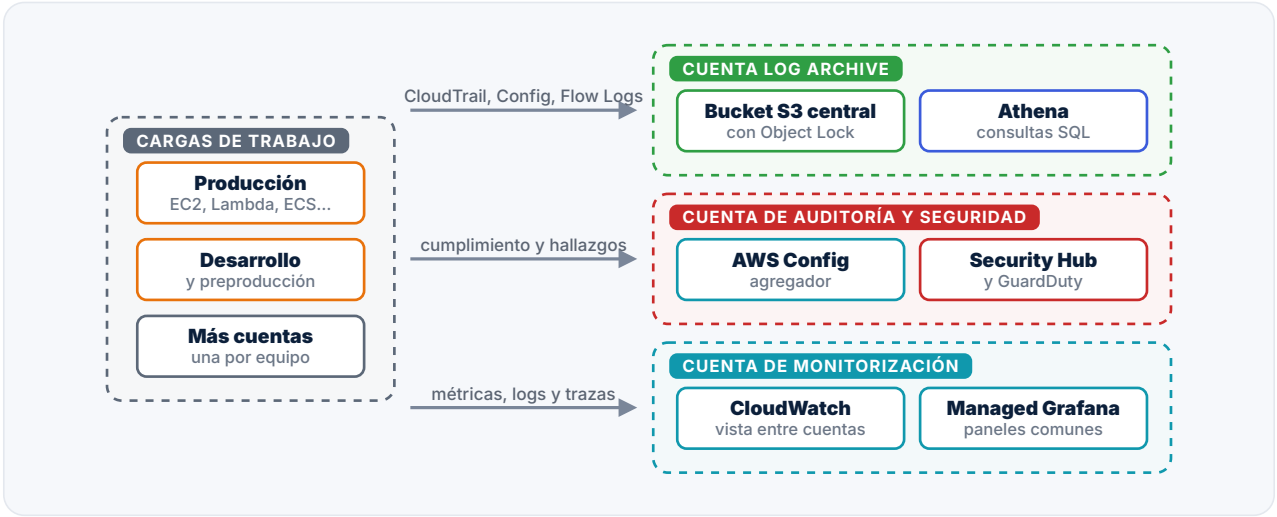


Figura 85 · Logging centralizado en una organización: los datos de las cuentas de trabajo terminan en cuentas dedicadas a archivo, auditoría y monitorización.

14.11 ¿Qué herramienta para qué pregunta?

> ★ **CLAVE PARA LA CERTIFICACIÓN**
>
> CloudTrail responde a **quién** hizo **qué** llamada a la API y **cuándo**; AWS Config, a **cómo estaba configurado** un recurso y si **cumple** tus reglas; CloudWatch, a **cómo está funcionando**. Los eventos de datos de CloudTrail (`GetObject` , `Invoke`) están desactivados por defecto.

Pregunta	Herramienta
¿Quién borró este bucket o cambió este Security Group?	CloudTrail
¿Cómo estaba configurado este recurso el martes? ¿Están cifrados todos los volúmenes?	AWS Config
¿Por qué va lenta la API? ¿En qué servicio se va el tiempo?	X-Ray, Application Signals
¿Cuántos errores 5xx hay? ¿Está la CPU al límite?	CloudWatch Metrics y Alarms
¿Qué excepción lanzó la aplicación a las 03:12?	CloudWatch Logs Insights
¿Qué IP intenta conectar al puerto 22 y es rechazada?	VPC Flow Logs
¿Funciona el login desde fuera, aunque ahora nadie lo use?	CloudWatch Synthetics
¿Cuánto tarda en cargar la web para los usuarios reales?	CloudWatch RUM
¿Qué consulta SQL satura la base de datos?	Database Insights
¿Hay un pico anómalo de llamadas a la API en la cuenta?	CloudTrail Insights

14.12 Buenas prácticas

- **Logs estructurados en JSON**: campos consultables (`nivel` , `servicio` , `ruta` , `latencia_ms`) en lugar de texto libre; Logs Insights los descubre solos.
- **Correlación**: incluye en cada línea el *request id* o el *trace id* y propágalo entre servicios para reconstruir una petición completa.
- **Alarmas sobre síntomas**: alerta por lo que nota el usuario (tasa de errores, latencia `p99` , disponibilidad), no por cada causa posible (CPU al 85 %); las causas, a los paneles. Y que cada alarma sea accionable, con un destinatario claro y un *runbook* (los pasos escritos para responder).

- **Nada de secretos ni datos personales en los logs**: las *data protection policies* de CloudWatch Logs pueden detectarlos y enmascararlos.
- **Observabilidad como código**: alarmas, dashboards y retenciones en CloudFormation, CDK o Terraform (capítulo de Infraestructura como código).
- En Lambda, **Powertools for AWS Lambda** (capítulo de Serverless) genera por ti logs JSON con el *request id*, métricas EMF y trazas.

RESUMEN DEL CAPÍTULO

- Observar se apoya en **métricas** (¿va algo mal?), **logs** (¿qué pasó?) y **trazas** (¿dónde se va el tiempo?), más la **auditoría** de la cuenta.
- CloudWatch Metrics organiza las series por namespace y dimensiones; EC2 no publica memoria ni disco sin el agente unificado. Vigila percentiles, no medias.
- Las alarmas se evalúan con «M de N» y avisan por SNS o actúan; las compuestas y la detección de anomalías reducen el ruido.
- CloudWatch Logs: retención indefinida por defecto (¡fíjala!), metric y subscription filters, Logs Insights y Live Tail.
- Synthetics, RUM, Application Signals y los «Insights» cubren usuarios y aplicaciones; X-Ray (u OpenTelemetry con ADOT) traza cada petición.
- CloudTrail registra las llamadas a la API (90 días de eventos de gestión gratis); AWS Config, la configuración en el tiempo, y es detectivo, no preventivo.
- EventBridge convierte eventos en acciones, y el modelo multicuenta centraliza archivo, auditoría y monitorización.

Preguntas de repaso

1. **No ves la memoria de tus instancias EC2 en CloudWatch. ¿Por qué y cómo lo solucionas?**
   Respuesta: El hipervisor no ve el interior del sistema operativo, así que EC2 no publica memoria ni disco. Instala el agente unificado de CloudWatch, que publica `mem_used_percent` y `disk_used_percent` en el namespace `CWAgent`.
2. **Una alarma tiene periodos de 1 minuto, `EvaluationPeriods = 5` y `DatapointsToAlarm = 3`. ¿Cuándo pasa a `ALARM`?**
   Respuesta: Cuando 3 de los últimos 5 datapoints de un minuto superan el umbral, aunque no sean consecutivos. Un pico aislado no la dispara.
3. **Tu factura de CloudWatch Logs crece cada mes aunque el tráfico es estable. ¿Causa probable y solución?**
   Respuesta: Los log groups tienen retención indefinida por defecto y acumulan datos. Fija una retención en cada uno, archiva en S3 lo que deba conservarse y reduce la verbosidad (o usa la clase Infrequent Access donde encaje).
4. **Alguien eliminó una tabla de DynamoDB hace tres semanas. ¿Dónde averiguas quién fue?**
   Respuesta: En CloudTrail: el Event history guarda 90 días de eventos de gestión, y `DeleteTable` lo es. Verás la identidad, la IP de origen, la hora y los parámetros.
5. **¿Cómo llevarías en tiempo casi real los logs de aplicación de todas las cuentas a un bucket central?**
   Respuesta: Con subscription filters en cada log group hacia un destino entre cuentas que alimente Amazon Data Firehose, que entrega en S3 de la cuenta Log Archive. Para CloudTrail, un trail de organización.
6. **Una API de microservicios tarda 3 segundos de vez en cuando. ¿Qué herramienta te dice en qué servicio se va el tiempo?**
   Respuesta: AWS X-Ray (o Application Signals): las trazas muestran la duración de cada segmento y subsegmento, y el mapa de servicios señala dónde se acumulan la latencia y los errores.

CAPÍTULO 15

Seguridad: cifrado con KMS, secretos y protección

Cifra tus datos con claves que controlas, saca los secretos del código y levanta defensas en cada capa: del borde de la red a la respuesta ante incidentes.

EN ESTE CAPÍTULO APRENDERÁS

1. Cifrado en reposo y en tránsito; simétrico y asimétrico
2. Tipos de clave de KMS y cómo funcionan las key policies
3. Cifrado envolvente, paso a paso
4. Rotación, alias, claves multirregión y borrado
5. Secretos y certificados: Secrets Manager, Parameter Store y ACM
6. WAF, Shield, GuardDuty, Security Hub y respuesta a incidentes

SERVICIOS PROTAGONISTAS

AWS KMS · AWS CloudHSM · AWS Secrets Manager · Parameter Store · AWS Certificate Manager · AWS WAF · AWS Shield · Amazon GuardDuty · AWS Security Hub · Amazon Inspector · Amazon Macie · Amazon Detective

15.1 Por qué cifrar

Imagina que alguien consigue una copia de tus datos: un snapshot compartido por error, una copia de seguridad olvidada, un disco retirado. Contra una copia que ya está fuera, las políticas de IAM no sirven de nada; el cifrado sí: sin la clave, el atacante se lleva ruido. Además, muchas normativas (RGPD, PCI DSS, HIPAA, ENS...) exigen cifrar y demostrar quién controla las claves. AWS cifra mucho por defecto, pero decidir qué clave protege qué datos y quién puede usarla es tu parte del modelo de responsabilidad compartida (capítulo de Fundamentos).

La primera parte del capítulo explica el cifrado y **AWS KMS** desde cero; la segunda, secretos, certificados, protección perimetral, detección y respuesta a incidentes. IAM, redes y observabilidad tienen sus propios capítulos: aquí solo los citamos.

Cifrado en reposo
at rest: datos almacenados

Volúmenes EBS, objetos de S3, bases de datos, snapshots, copias y logs. En AWS suele bastar un ajuste (o viene activado), con claves gestionadas por **KMS**: SSE-KMS en S3, cifrado de EBS y de RDS...

Cifrado en tránsito
in transit: datos que viajan por la red

Navegador → balanceador, aplicación → base de datos... Se consigue con **TLS** (HTTPS) y certificados de **ACM**. Puedes exigirlo: `aws:SecureTransport` en una bucket policy, `rds.force_ssl` en PostgreSQL.

Simétrico y asimétrico

El cifrado **simétrico** usa la misma clave para cifrar y descifrar: es rápido y sirve para cualquier volumen de datos, pero hay que hacer llegar la clave, con seguridad, a quien descifra (el estándar es AES-256). El **asimétrico** usa un par de claves: lo que cifra la **pública**, que se reparte sin miedo, solo lo descifra la **privada**, que no sale de su dueño. Es mucho más lento, así que se usa para intercambiar claves y para **firmas digitales** (RSA, curvas elípticas o ECC). TLS combina ambos: acuerda con criptografía asimétrica una clave simétrica de sesión y cifra el tráfico con ella.

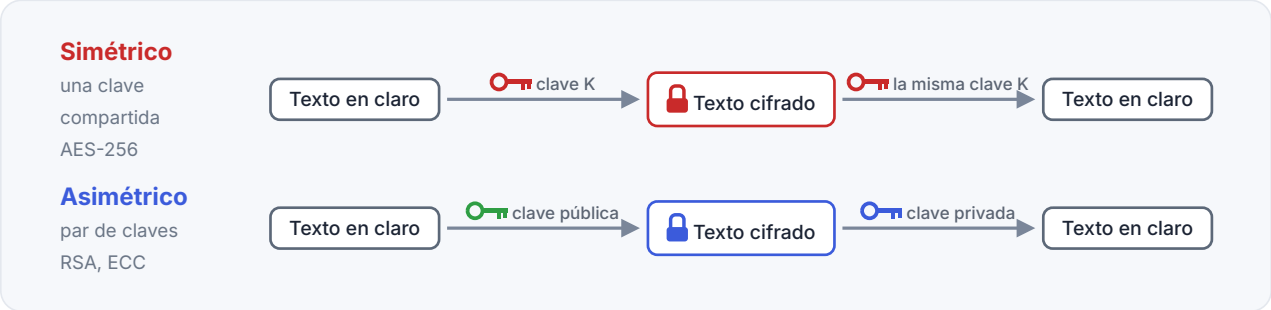


Figura 86 · Cifrado simétrico (una clave compartida) frente a asimétrico (un par de claves pública y privada).

Clave — un secreto —256 bits en AES-256— que, junto con un algoritmo público, cifra y descifra. La seguridad depende de que la clave sea secreta, no el algoritmo.
HSM (Hardware Security Module) — dispositivo físico resistente a manipulaciones que genera y guarda claves y opera con ellas por dentro, como una cámara acorazada donde, además de guardar las combinaciones, se abren las cajas: no le pides «dame la clave», sino «cifra esto con la clave». KMS y CloudHSM usan HSM validados según FIPS 140 de nivel 3.

15.2 AWS KMS y sus tipos de clave

AWS Key Management Service (KMS) crea y guarda claves criptográficas —las **claves KMS**— en HSM gestionados por AWS. Nunca ves el material de la clave: llamas a la API (`Encrypt`, `Decrypt`, `GenerateDataKey`, `Sign`...) y KMS opera dentro del HSM y te devuelve el resultado. Cada clave vive

en una región y se usa a través del endpoint de KMS de esa región. (En exámenes y textos antiguos verás **CMK**, *customer master key*: hoy se dice **clave KMS**.)

FICHA DEL SERVICIO

AWS KMS

Key Management Service: crea, guarda y usa claves criptográficas en HSM gestionados por AWS

TIPO	Servicio gestionado y regional; pago por clave al mes y por petición
INTEGRACIÓN	Más de cien servicios: S3, EBS, RDS, DynamoDB, Secrets Manager, Lambda, SQS…
CONTROL DE ACCESO	Key policy (obligatoria) + políticas IAM + grants
AUDITORÍA	Cada uso de una clave queda registrado en CloudTrail

¿De quién es la clave?

La primera clasificación responde a quién crea y controla la clave; DynamoDB, por ejemplo, te deja elegir entre las tres.

	AWS owned	AWS managed	Customer managed
Qué es	Del servicio, compartidas entre cuentas (p. ej., DynamoDB por defecto)	Las crea un servicio en tu cuenta: `aws/s3`, `aws/ebs`…	Las creas y administras tú
Visible y auditada en tu cuenta (CloudTrail)	✘	✔	✔
¿Editas su key policy?	✘	✘ (solo lectura)	✔
¿La usas desde tu código?	✘	✘ (solo vía el servicio)	✔
Rotación	La decide AWS	Automática y obligatoria, cada año	Opcional: automática (365 días por defecto, de 90 a 2560) o bajo demanda
¿Compartir con otra cuenta?	✘	✘	✔
Coste	Gratis	Sin cuota mensual; las peticiones pueden cobrarse	Cuota mensual por clave + peticiones

★ **CLAVE PARA LA CERTIFICACIÓN**

Si el enunciado pide editar la key policy, rotar a voluntad, compartir la clave con otra cuenta o poder borrarla, la respuesta es una **customer managed**.

¿Qué criptografía? Simétricas, asimétricas y HMAC

El **uso** de una clave (cifrar, firmar, generar MAC…) se fija al crearla y no se puede cambiar.

Simétricas

la opción por defecto

AES-256-GCM (`SYMMETRIC_DEFAULT`). La usan **todos** los servicios integrados; admite contexto de cifrado, rotación automática, material importado y multirregión.

Asimétricas

RSA y ECC

La privada no sale de KMS; la pública se descarga (`GetPublicKey`). RSA cifra o firma; ECC firma o acuerda secretos. Para firmas digitales y terceros sin credenciales de AWS.

HMAC

integridad y autenticidad

Genera y verifica códigos HMAC (`GenerateMac`, `VerifyMac`) sin que la clave salga de KMS: prueba que un mensaje o un token no se ha alterado.

15.3 Quién puede usar una clave: key policies, IAM y grants

En casi todo AWS, una política IAM basta para acceder a un recurso de tu cuenta (capítulo «IAM (II): permisos — políticas y evaluación»). KMS es la excepción: **la key policy manda**. Nadie —ni siquiera el usuario root— puede usar ni administrar una clave si su key policy no lo permite, directamente o delegando en IAM. Hay tres mecanismos: la **key policy** (obligatoria: toda clave tiene **exactamente una**), las **políticas IAM** (solo cuentan si la key policy delega en la cuenta) y los **grants** (permisos puntuales que se crean y revocan por API).

Una key policy, comentada

Si creas una clave sin indicar política, KMS le asigna la **política por defecto**, cuya sentencia esencial es la primera de este ejemplo, pensado para una aplicación de pagos:

JSON

```
{
  "Version": "2012-10-17",
  "Statement": [
    { "Sid": "Delegar en IAM", "Effect": "Allow",
      "Principal": { "AWS": "arn:aws:iam::111122223333:root" },
      "Action": "kms:*", "Resource": "*" },
    { "Sid": "Administradores", "Effect": "Allow",
      "Principal": { "AWS": "arn:aws:iam::111122223333:role/AdminClaves" },
      "Action": [ "kms:Create*", "kms:Describe*", "kms:Enable*", "kms:List*",
        "kms:Put*", "kms:Update*", "kms:Revoke*", "kms:Disable*", "kms:Get*",
        "kms:Delete*", "kms:ScheduleKeyDeletion", "kms:CancelKeyDeletion" ],
      "Resource": "*" },
    { "Sid": "Uso por la app solo via S3", "Effect": "Allow",
      "Principal": { "AWS": "arn:aws:iam::111122223333:role/AppPagos" },
      "Action": [ "kms:GenerateDataKey", "kms:Decrypt" ], "Resource": "*",
      "Condition": { "StringEquals": {
        "kms:ViaService": "s3.eu-west-1.amazonaws.com" } } },
    { "Sid": "Otra cuenta puede descifrar", "Effect": "Allow",
      "Principal": { "AWS": "arn:aws:iam::444455556666:root" },
      "Action": [ "kms:Decrypt", "kms:DescribeKey" ], "Resource": "*" }
  ]
}
```

1. **Delegar en IAM.** El principal `…:root` no es el usuario root, sino la **cuenta entera**: «la cuenta decide, con IAM, quién usa esta clave». Sin esta sentencia, las políticas IAM no tienen efecto sobre la clave. Aquí `"Resource": "*"` significa «esta clave».
2. **Administradores.** `AdminClaves` gestiona la clave (política, activación, borrado), pero esta key policy no le deja cifrar ni descifrar: **separación de funciones**.
3. **Uso.** `AppPagos` genera claves de datos y descifra, pero solo a través de S3 en eu-west-1 (`kms:ViaService`), no con llamadas directas a KMS.
4. **Otra cuenta.** La cuenta 444455556666 puede descifrar… si, además, su administrador lo concede con IAM, como verás enseguida.

> **!** **CUIDADO**
>
> Una key policy que deja fuera a todo el mundo vuelve la clave **inmanejable**: solo AWS Support podrá ayudarte. Por eso KMS rechaza una política que impediría a quien la envía volver a cambiarla, salvo que fuerces `BypassPolicyLockoutSafetyCheck`.

Cómo se evalúa una petición

KMS combina los tres mecanismos con la lógica que conoces de IAM (II): un Deny explícito gana siempre y, para permitir, basta uno de los tres caminos. Por encima siguen aplicando las SCP y RCP de AWS Organizations y las políticas de los endpoints de VPC.

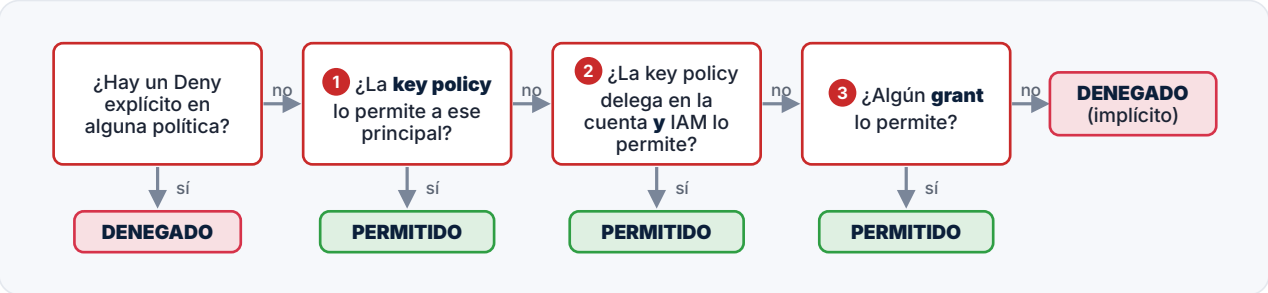


Figura 87 · Cómo decide KMS una petición (por ejemplo, AppPagos llama a `kms:Decrypt`). Un Deny explícito deniega; para permitir basta uno de los tres caminos.

Un **grant** da a un principal concreto ciertas operaciones sobre una clave, sin tocar la key policy, y se revoca al instante. Los servicios los usan a diario: al adjuntar un volumen EBS cifrado, EC2 crea un grant para descifrar la clave de datos del volumen mientras lo usa.

Acceso entre cuentas

Para que la cuenta B use una clave de la cuenta A hace falta permiso **en los dos lados**: la key policy (en A) debe permitir a la cuenta B o a un rol suyo, y una política IAM (en B) debe conceder a ese rol las acciones sobre el ARN de la clave. Si falta uno, se deniega. Desde B, la clave se indica por su **ARN completo** (el alias corto solo funciona en su cuenta), y solo se comparten operaciones de uso, nunca las de administración.

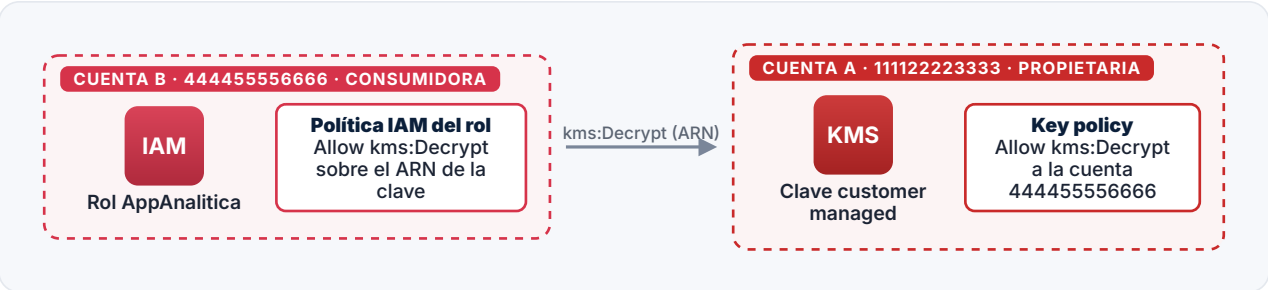


Figura 88 · Acceso entre cuentas: la propietaria lo permite en la key policy y la consumidora lo concede con IAM. Hacen falta las dos.

> ★ **CLAVE PARA LA CERTIFICACIÓN**
>
> Las claves AWS managed (`aws/rds`, `aws/ebs` …) **no se pueden compartir**: no puedes editar su key policy. Si un snapshot de EBS o RDS está cifrado con una de ellas, cópialo cifrándolo con una customer managed cuya key policy permita a la cuenta destino y comparte la copia.

15.4 Cifrado envolvente (envelope encryption)

Si KMS nunca deja salir sus claves, ¿cómo cifra S3 un objeto de 5 GB? No enviándolo a KMS: `Encrypt` solo acepta hasta 4 KB, y mandar gigabytes a un servicio compartido sería lento, caro y chocaría con las cuotas. La solución es el **cifrado envolvente**: los datos se cifran en local con una **clave de datos** (*data key*), y con la clave KMS solo se cifra esa clave de datos.

> ≈ **ANALOGÍA**
>
> Cierras tus documentos en una caja con una llave cualquiera (la clave de datos), metes la llave en un sobre lacrado que solo el banco sabe abrir (KMS) y pegas el sobre a la caja. Quien robe la caja se lleva el sobre, pero sin el banco no lo abre. Y el banco nunca ve tus documentos: solo abre sobres.

1. **Pedir una clave de datos:** la aplicación (o S3, EBS…) llama a `GenerateDataKey` indicando la clave KMS, y KMS genera una clave de datos aleatoria, normalmente AES-256.
2. **Recibir dos copias:** la clave de datos **en claro** y otra **cifrada** con la clave KMS, que no sale del HSM.
3. **Cifrar en local** los datos con la clave en claro (AES-GCM, rapidísimo), sean 4 KB o 40 GB.
4. **Descartar la clave en claro** de la memoria en cuanto termina. Nunca se escribe en disco.
5. **Guardar juntas** la clave cifrada y los datos cifrados (en los metadatos del objeto, en la cabecera del fichero…). La clave cifrada no es secreta: sin KMS no sirve de nada.
6. **Para leer**, enviar la clave cifrada a `Decrypt`. Si hay permiso, KMS devuelve la clave en claro; se descifran los datos y se vuelve a descartar.

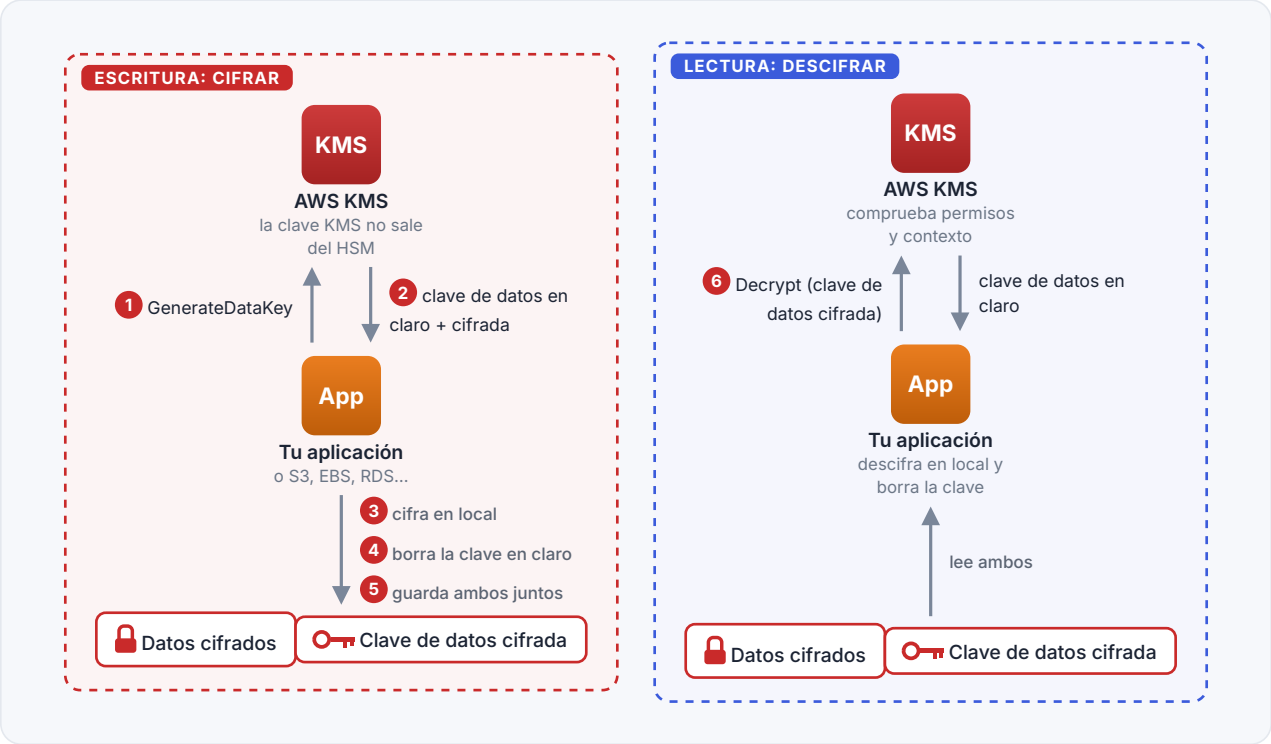


Figura 89 · Cifrado envolvente. La clave KMS no sale de KMS: lo que viaja y se guarda junto a los datos es la clave de datos cifrada.

Así se resuelven a la vez el **tamaño** (no hay límite de 4 KB), el **rendimiento** (los datos no viajan a KMS), el **coste y las cuotas** (una petición por objeto, o por lote si reutilizas la clave de datos, y no una por bloque) y la **seguridad** (sin `kms:Decrypt`, quien copie datos y clave cifrada no abre el sobre). Los servicios integrados lo hacen por ti; en tu código, usa el AWS Encryption SDK. Y si un componente solo prepara claves que otro usará después, `GenerateDataKeyWithoutPlaintext` devuelve únicamente la copia cifrada.

```bash
aws kms generate-data-key --key-id alias/app-pagos --key-spec AES_256
# "CiphertextBlob": "AQIDAHh...",  <- clave cifrada: guárdala con los datos
# "Plaintext":      "c2VjcmV0...",  <- clave en claro (Base64): úsala y bórrala
```

15.5 Contexto de cifrado, alias, rotación y ciclo de vida

Contexto de cifrado (encryption context). Pares clave-valor **no secretos** que acompañan al cifrado, como `{"app": "pagos"}`. KMS los vincula criptográficamente al texto cifrado: para descifrar hay que presentar el mismo contexto. Impide reutilizar un texto cifrado fuera de su sitio, permite condiciones

en las políticas (`kms:EncryptionContext:app`) y aparece en claro en CloudTrail, así que nunca metas en él datos sensibles. S3, por ejemplo, usa el ARN del objeto o del bucket.

Alias. Un nombre amigable (`alias/app-pagos`) que apunta a una clave: tu código usa el alias y, si cambias de clave, basta con reasignarlo (`update-alias`). Una clave puede tener varios; los que empiezan por `alias/aws/` son de las AWS managed. Como las claves, son regionales.

Rotación de claves

Rotar es generar material criptográfico nuevo. Las claves **AWS managed** rotan solas cada año, de forma obligatoria. En las **customer managed** simétricas, la rotación automática es opcional y está desactivada por defecto; al activarla rota cada 365 días, o con el periodo que elijas entre 90 y 2560, y además puedes forzar una **rotación bajo demanda**. La clave conserva su **ID y su ARN**: aplicaciones, alias y políticas no cambian. KMS guarda todas las versiones anteriores del material para descifrar lo que se cifró con ellas y usa la más reciente para cifrar, así que los datos existentes no se recifran: no hace falta. La rotación automática solo existe en claves simétricas cuyo material genera KMS; las asimétricas y HMAC se rotan a mano, creando una clave nueva y reasignando el alias.

Claves multirregión. Claves en varias regiones con el mismo ID (empieza por `mrk-`) y el mismo material: lo cifrado en una región se descifra con la réplica de otra sin llamadas entre regiones. Sirven para recuperación ante desastres, aplicaciones activo-activo o firmas verificables en varias regiones. Cada réplica tiene su propia key policy y se factura como una clave más; AWS recomienda claves de una sola región salvo que las necesites.

Deshabilitar o borrar (las AWS managed y AWS owned no se pueden borrar):

Deshabilitar
reversible

La clave deja de poder usarse al instante (todo `Decrypt` falla) y vuelve a funcionar al habilitarla. Úsalo para comprobar si algo depende aún de ella, con una alarma sobre los intentos de uso que registra CloudTrail.

Programar el borrado
irreversible al acabar la espera

`ScheduleKeyDeletion` con una espera de **7 a 30 días** (30 por defecto), durante la que puedes cancelar. Después, la clave se destruye y lo cifrado con ella es **irrecuperable**.

15.6 KMS en los servicios de AWS

Los servicios integrados aplican el cifrado envolvente por ti: tú eliges la clave y la key policy decide si el servicio puede usarla en tu nombre.

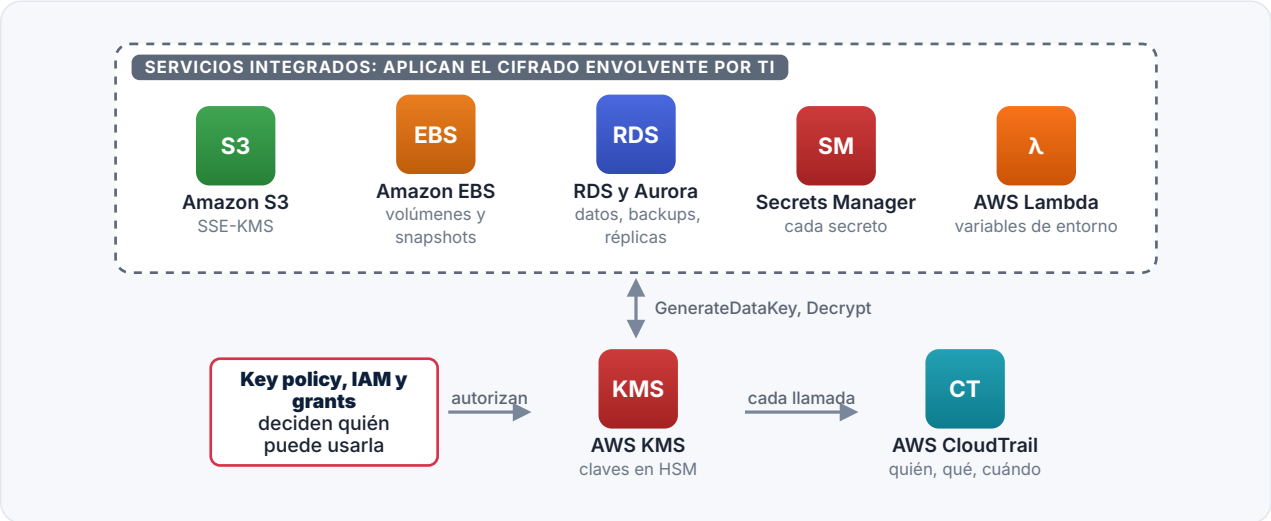


Figura 90 · Los servicios integrados piden a KMS claves de datos en tu nombre; la key policy, IAM y los grants deciden si pueden, y CloudTrail registra cada llamada.

Servicio	Qué se cifra	Por defecto	Lo que debes saber
S3	Objetos	SSE-S3 en todos los objetos nuevos (desde 2023)	SSE-KMS para controlar y auditar quién descifra; activa S3 Bucket Keys
EBS	Volúmenes, snapshots y tráfico instancia–volumen	Sin cifrar, salvo que actives el **cifrado por defecto** (por cuenta y región)	Con el ajuste activo, todo volumen nuevo se cifra con `aws/ebs` o con la clave que elijas
RDS / Aurora	Almacenamiento, backups, snapshots, réplicas y logs	Se elige al crear la base de datos	No se cifra en caliente una instancia existente: snapshot → copia cifrada → restaurar
Secrets Manager	Cada secreto	Siempre, con `aws/secretsmanager`	Para compartir secretos entre cuentas necesitas una customer managed
CloudTrail	Ficheros de log en S3	SSE-S3, si no eliges otra cosa	SSE-KMS con clave propia: separa quién lee los logs de quién administra el bucket
Lambda	Variables de entorno	Clave AWS managed (`aws/lambda`)	Customer managed si quieres decidir quién puede descifrarlas

S3. Con SSE-S3 (el cifrado por defecto) S3 gestiona las claves sin coste, pero no controlas ni auditas su uso; con SSE-KMS hay doble llave (permiso de S3 **y** de KMS) y cada uso queda en CloudTrail (los demás métodos, DSSE-KMS y SSE-C, están en el capítulo de Almacenamiento). El problema de SSE-KMS es el volumen: una petición a KMS por objeto escrito y otra por lectura. Con **S3 Bucket Keys**, S3 obtiene de KMS una clave de vida corta a nivel de bucket y deriva de ella las de muchos objetos: las llamadas a KMS caen en picado (AWS habla de hasta un 99 % menos de coste de peticiones).

> **CUIDADO**
>
> Con SSE-KMS, `s3:GetObject` no basta: quien lee necesita `kms:Decrypt` sobre la clave, y quien sube, `kms:GenerateDataKey` (y `kms:Decrypt` en las subidas multipart). Es la causa número uno de *Access Denied* «misteriosos» con objetos cifrados.

EBS y RDS. Con el **cifrado por defecto de EBS** activado en la región, todo volumen nuevo y toda copia de snapshot se cifran sin que nadie tenga que acordarse; un volumen existente se cifra creando otro desde su snapshot. En RDS, el cifrado se decide al crear la instancia y **no se activa en caliente**: crea un snapshot manual, **cópialo activando el cifrado** con la clave KMS elegida, restaura desde la copia una instancia nueva y apunta a ella la aplicación (si la parada es inaceptable, replica antes los datos, por ejemplo con AWS DMS).

15.7 BYOK, CloudHSM, cifrado en cliente y costes

Con KMS, AWS genera y custodia el material de tus claves. Si una normativa exige más control, hay tres escalones:

Material importado (BYOK, *bring your own key*) — creas una clave sin material e importas el tuyo, envuelto con una clave pública que te da KMS. Puedes fijarle caducidad o borrarlo al instante (la clave queda inutilizable hasta reimportarlo), pero conservar una copia es cosa tuya.
Custom key stores — el material vive en tu propio clúster de **CloudHSM** o, con un **external key store**, en un gestor de claves fuera de AWS al que KMS llama mediante un proxy (si cortas el acceso, AWS no puede descifrar). Solo admiten claves simétricas y añaden coste y complejidad: úsalos si una normativa lo exige.

AWS CloudHSM — HSM dedicados (*single-tenant*) en tu VPC, que usas directamente. A diferencia de KMS, gestionas tú los usuarios y las claves del HSM: AWS opera el hardware, pero no tiene acceso a tus claves.

	AWS KMS	AWS CloudHSM
Modelo	Servicio multiinquilino totalmente gestionado; alta disponibilidad incluida	HSM dedicados en tu VPC; la alta disponibilidad la diseñas tú (varios HSM en varias AZ)
Quién controla las claves	Tú, con políticas; AWS opera los HSM	Solo tú; AWS no tiene acceso
Integración con servicios	✔ Nativa con la mayoría	◐ Indirecta, como custom key store de KMS
Interfaces	API de AWS (SDK, CLI) y permisos de IAM	Estándares: PKCS#11, JCE, OpenSSL, CNG/KSP
Precio	Por clave y por petición: poco dinero al mes	Por HSM y hora: del orden de mil dólares al mes por HSM
Casos de uso	Cifrar datos en los servicios de AWS y en tus aplicaciones	Normativa de HSM dedicado, descarga de TLS, TDE en EC2, CA propia

Cifrado del lado del cliente. Si quieres que AWS solo vea texto cifrado, cifra antes de enviar. El **AWS Encryption SDK** (para varios lenguajes y CLI) implementa el cifrado envolvente con buenas prácticas: claves de datos de KMS, la clave cifrada junto a los datos en un formato portable y caché de claves de datos. Hay clientes específicos para S3 y para bases de datos.

COSTES

Importes orientativos, que varían por región:

- **Customer managed:** del orden de un dólar al mes por clave, prorrateado por horas; las primeras rotaciones lo incrementan, con un tope. Cada réplica multirregión cuenta como clave.
- **Peticiones:** del orden de céntimos por cada 10 000 simétricas (más las asimétricas), con capa gratuita; las de claves AWS managed también pueden cobrarse.
- La factura grande suele venir de las peticiones, no de las claves.

Además de costar, las peticiones tienen cuota: KMS limita las que admite por segundo en cada cuenta y región. Las operaciones criptográficas con claves simétricas comparten una cuota (de miles a decenas de miles por segundo, según la región), mucho menor con asimétricas, y las peticiones de los servicios en tu nombre también cuentan. Si la superas recibes `ThrottlingException`: reintenta con backoff exponencial (los SDK ya lo hacen), usa S3 Bucket Keys o caché de claves de datos, o pide un aumento en Service Quotas.

KMS desde la línea de comandos

BASH

```bash
# Crear una clave simétrica customer managed, guardar su ID y ponerle alias
KEY_ID=$(aws kms create-key --description "Clave de la app de pagos" \
           --query KeyMetadata.KeyId --output text)
aws kms create-alias --alias-name alias/app-pagos --target-key-id "$KEY_ID"
aws kms enable-key-rotation --key-id "$KEY_ID" --rotation-period-in-days 180

# Cifrar un dato pequeño (máx. 4 KB) con contexto de cifrado...
aws kms encrypt --key-id alias/app-pagos --plaintext fileb://token.txt \
    --encryption-context app=pagos \
    --query CiphertextBlob --output text | base64 --decode > token.enc
# ...y descifrarlo presentando el mismo contexto
aws kms decrypt --key-id alias/app-pagos --ciphertext-blob fileb://token.enc \
    --encryption-context app=pagos --query Plaintext --output text | base64 --decode
```

15.8 Secretos: Secrets Manager y Parameter Store

Una contraseña escrita en el código, en un fichero de configuración o en una variable de entorno en texto plano acaba, tarde o temprano, en un repositorio, un log o una captura de pantalla. **AWS Secrets Manager** y el **Parameter Store** de AWS Systems Manager la sacan del código: ambos cifran con KMS, controlan el acceso con IAM y registran cada lectura en CloudTrail.

FICHA DEL SERVICIO

AWS Secrets Manager

Almacén de secretos con rotación automática

QUÉ GUARDA	Texto o JSON de hasta 64 KB (credenciales, claves de API, tokens), cifrado siempre con KMS
ROTACIÓN	Plantillas para RDS, Aurora, Redshift y DocumentDB, rotación gestionada por algunos servicios y funciones Lambda propias para el resto
EXTRAS	Versiones, réplicas en otras regiones y políticas de recurso para el acceso entre cuentas

	Secrets Manager	Parameter Store
Pensado para	Secretos con ciclo de vida: credenciales que rotan	Configuración y secretos sencillos (`String`, `StringList` y `SecureString`, cifrado con KMS)
Rotación automática	✔ Integrada	✘ Tendrías que montarla (EventBridge + Lambda)
Tamaño máximo	64 KB	4 KB (Standard) · 8 KB (Advanced)
Coste	Por secreto (menos de un dólar al mes) + llamadas	Standard: gratis · Advanced: céntimos por parámetro al mes
Jerarquías	◐ Nombres con `/` y filtros por prefijo	✔ Rutas (`/prod/pagos/db-url`) y lectura por ruta (`GetParametersByPath`)
Entre cuentas y regiones	✔ Política de recurso + clave customer managed; réplicas en otras regiones	◐ Parámetros Advanced compartidos con AWS RAM; sin réplicas
Úsalo para	Credenciales de bases de datos, claves de API que rotan, acceso entre cuentas o regiones	URL, *feature flags*, IDs de AMI, configuración por entorno, secretos que no rotan

Leer un secreto desde Python es una sola llamada, y programar su rotación desde la CLI, otra:

PYTHON

```python
import json, boto3

sm = boto3.client("secretsmanager", region_name="eu-west-1")
resp = sm.get_secret_value(SecretId="prod/pagos/db")  # versión AWSCURRENT
cred = json.loads(resp["SecretString"])               # usuario, contraseña, host
```

BASH

```bash
aws secretsmanager rotate-secret --secret-id prod/pagos/db \
    --rotation-lambda-arn arn:aws:lambda:eu-west-1:111122223333:function:rotar \
    --rotation-rules AutomaticallyAfterDays=30
```

Rotación. Según la programación (cada N días o una expresión `rate`/`cron`), Secrets Manager invoca la función de rotación, que sigue cuatro pasos: `createSecret` genera la contraseña nueva como versión `AWSPENDING`, `setSecret` la cambia en la base de datos, `testSecret` comprueba que funciona y `finishSecret` le mueve la etiqueta `AWSCURRENT` (la anterior pasa a `AWSPREVIOUS`). Quien lee `AWSCURRENT` obtiene la nueva sin redesplegar. Cachea el secreto (en Lambda, la extensión *AWS*

Parameters and Secrets Lambda Extension lo hace por ti), pero reléelo si la conexión falla por credenciales.

> ★ **CLAVE PARA LA CERTIFICACIÓN**
>
> ¿Credenciales que deben rotar solas? **Secrets Manager**. ¿Configuración o un secreto sin rotación al menor coste? **Parameter Store** con `SecureString`. Ambos se referencian desde CloudFormation (`{{resolve:secretsmanager:...}}`, `{{resolve:ssm-secure:...}}`, capítulo de Infraestructura como código) y desde las tareas de ECS.

15.9 Certificados TLS con ACM

AWS Certificate Manager (ACM) emite, guarda y renueva certificados TLS para tus dominios. Los públicos son gratuitos con los servicios integrados: ACM los instala, protege la clave privada y los renueva antes de que caduquen.

Públicos confiables para los navegadores	Privados con AWS Private CA	Importados de otra CA
Emitidos por la CA de Amazon tras validar que controlas el dominio. Gratis con los servicios integrados; renovación automática.	Para servicios internos, mTLS o dispositivos, emitidos por tu propia CA privada (de pago). Los navegadores no confían en ellos.	Los subes a ACM para usarlos en servicios integrados. **ACM no los renueva**: avisa antes de que caduquen y tú los reimportas.

Validación por DNS — añades a tu zona un CNAME que te da ACM (con Route 53, en un clic). Mientras exista, la renovación es automática y sin intervención. Es la opción recomendada.
Validación por email — los contactos del dominio (`admin@`, `webmaster@`...) aprueban la solicitud por correo, y las renovaciones pueden volver a requerirlo.
ACM se integra con ELB (ALB y NLB), CloudFront, API Gateway, App Runner y otros servicios. Los certificados son **regionales**: el de un ALB debe estar en la región del ALB.

> ! **CUIDADO**
>
> **CloudFront solo acepta certificados de ACM de us-east-1** (Norte de Virginia), porque es un servicio global, aunque tu origen esté en otra región. Lo mismo ocurre con las API *edge-optimized* de API Gateway. Si no ves tu certificado en la distribución, casi seguro lo creaste en otra región.

¿Y en mi propia EC2? Un certificado público estándar de ACM no se puede instalar en tu servidor: ACM no deja exportar su clave privada, solo usarlo a través de los servicios integrados. Lo habitual es poner delante un ALB o CloudFront, que terminan TLS con él, o usar *ACM for Nitro Enclaves*. Desde 2025 hay también certificados públicos **exportables**, instalables donde quieras, pero tienen coste y hay que redesplegarlos tras cada renovación.

15.10 Protección perimetral: WAF, Shield y firewalls

Los Security Groups y las NACL (capítulo de Redes) filtran por IP, puerto y protocolo, pero no entienden lo que viaja dentro: una inyección SQL entra por el puerto 443 como cualquier otra petición. Para eso están estos servicios.

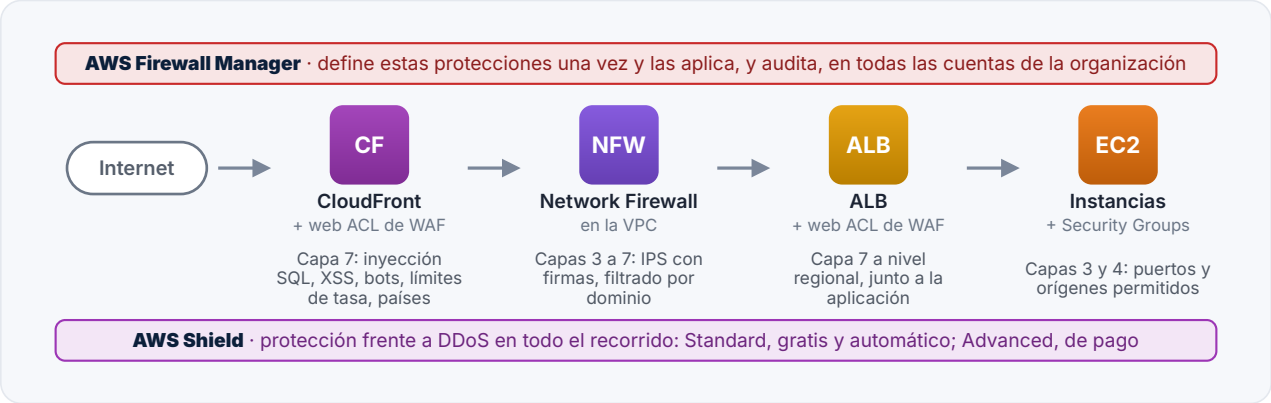


Figura 91 · Dónde actúa cada servicio de protección perimetral en el camino de una petición. Firewall Manager gobierna las políticas en todas las cuentas.

AWS WAF

FICHA DEL SERVICIO

AWS WAF

Firewall de aplicaciones web: filtra peticiones HTTP(S) antes de que lleguen a tu aplicación

SE ASOCIA A	CloudFront (web ACL global, creada en us-east-1), ALB, API Gateway (REST), AppSync, Cognito, App Runner y Verified Access. No a un NLB ni a una EC2
WEB ACL	Reglas por prioridad sobre IP, país, cabeceras, URI o cuerpo (inyección SQL, XSS, regex...), con acciones Allow, Block, Count (probar sin bloquear), CAPTCHA o Challenge
REGLAS GESTIONADAS	Grupos de AWS (riesgos como los del OWASP Top 10, reputación de IP...) o de Marketplace. Bot Control (de pago) identifica bots; hay grupos contra el robo de cuentas
RATE-BASED	Bloquean a quien supere N peticiones en una ventana de minutos, mientras la supere, agrupando por IP u otra clave: fuerza bruta y DDoS de capa 7
FACTURACIÓN	Por web ACL y regla al mes + por millón de peticiones; Bot Control y similares, aparte

AWS Shield

Shield protege frente a ataques de denegación de servicio distribuida (DDoS). Todo cliente tiene **Shield Standard** sin hacer nada; **Shield Advanced** es una suscripción para aplicaciones críticas.

	Shield Standard	Shield Advanced
Precio	Incluido, sin coste y automático	Del orden de miles de dólares al mes por organización, con compromiso de un año, más transferencia de datos
Qué cubre	Ataques comunes de capas 3 y 4 (SYN floods, reflexión UDP...) en toda la infraestructura de AWS	Detección y mitigación avanzadas, también de capa 7 (con WAF), para CloudFront, Route 53, Global Accelerator, ELB y Elastic IP
Equipo de respuesta y protección de costes	✘	✔ Equipo de respuesta 24/7 (SRT, con soporte Business o superior) y créditos por el escalado que provoque un ataque
AWS WAF	Se paga aparte	Incluido para los recursos protegidos (con excepciones)

AWS Firewall Manager centraliza WAF, Shield Advanced, Security Groups, Network Firewall y el firewall DNS de Route 53 Resolver en todas las cuentas de una organización: defines una política

(«todo ALB de producción lleva esta web ACL») y la aplica, también a cuentas y recursos futuros. Requiere AWS Organizations y AWS Config.

AWS Network Firewall es un firewall gestionado y con estado para tus VPC: filtra de la capa 3 a la 7, con reglas compatibles con Suricata (IPS), listas de dominios permitidos para la salida e inspección de TLS. Se despliega en subredes dedicadas a las que el tráfico llega por las tablas de rutas (con Transit Gateway, una VPC central inspecciona muchas) y se paga por hora de endpoint y por GB procesado.

15.11 Detección y respuesta: GuardDuty, Security Hub y más

Prevenir no basta: algo acabará escapándose. Estos servicios vigilan cuentas, cargas y datos, y generan **hallazgos** (*findings*) cuando detectan algo sospechoso. Son regionales, se activan con pocos clics y se gestionan para toda la organización desde una cuenta de **administrador delegado**.

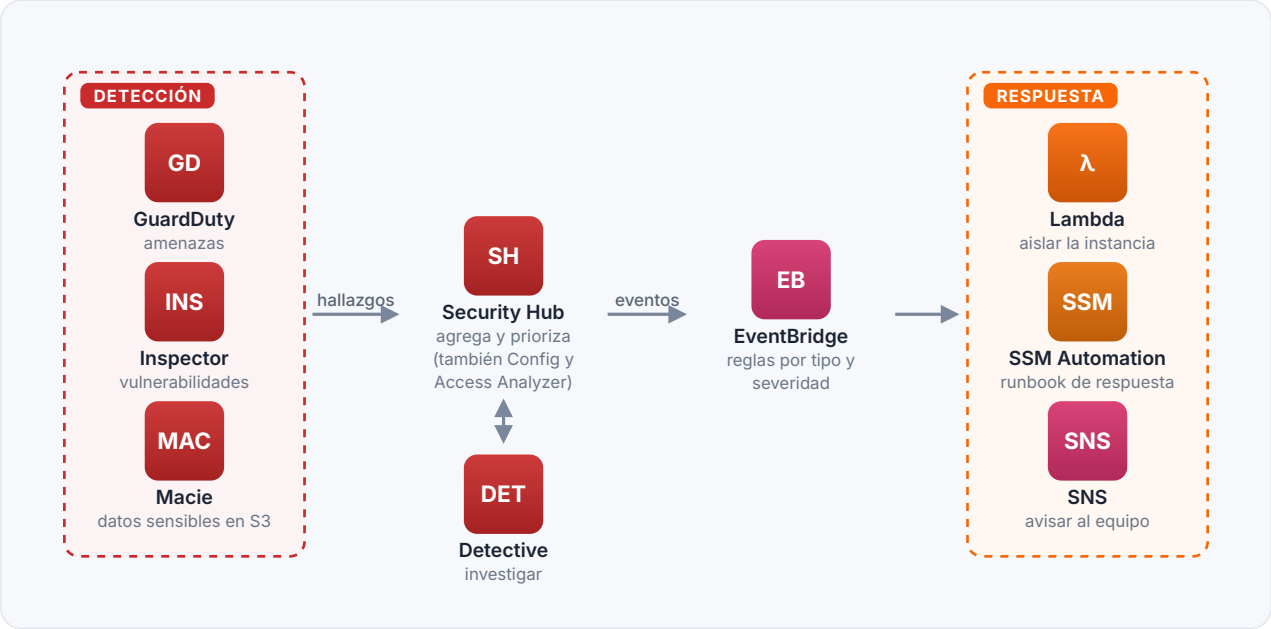


Figura 92 · Detección y respuesta automatizada: los hallazgos convergen en Security Hub, EventBridge los enruta y Lambda, SSM o SNS responden. Detective ayuda a investigar.

Servicio	Qué hace	Qué datos analiza	Ejemplo de hallazgo o uso
GuardDuty	Detecta amenazas con inteligencia de amenazas, aprendizaje automático y anomalías	CloudTrail, VPC Flow Logs y DNS (sin que tengas que activarlos); opcionalmente S3, EKS, RDS, Lambda, ejecución y malware	Credenciales de una instancia usadas desde fuera de AWS; minería de criptomonedas
Security Hub	Agrega y prioriza hallazgos; comprueba la postura frente a estándares	Hallazgos de los demás servicios y de terceros; controles basados en AWS Config	Puntuación frente a AWS Foundational Security Best Practices, CIS o PCI DSS
Inspector	Gestiona vulnerabilidades de forma continua	Instancias EC2, imágenes de contenedor en ECR y funciones Lambda	CVE crítica en un paquete de la AMI; puerto expuesto a Internet
Macie	Descubre datos sensibles y riesgos en S3	Objetos y configuración de los buckets	CSV con números de tarjeta en un bucket público
Detective	Ayuda a investigar el origen y el alcance	CloudTrail, Flow Logs, hallazgos de GuardDuty, logs de EKS → grafo de comportamiento	Todas las llamadas e IP de un rol sospechoso en las últimas horas

En 2025 AWS renombró las comprobaciones de postura de Security Hub como **Security Hub CSPM** y amplió Security Hub para correlacionar hallazgos y priorizar riesgos, pero el patrón no cambia: los

detectores envían hallazgos a un punto central, de ahí pasan a **EventBridge** (capítulo de Mensajería) y una regla dispara la respuesta automática, como resume la figura. Por ejemplo, una regla cuyo patrón seleccione los eventos `GuardDuty Finding` con `severity` de 7 o más (alta o crítica) puede invocar una función que aísle la instancia.

> **BUENA PRÁCTICA**
>
> Activa GuardDuty y Security Hub en **todas** las cuentas y regiones, también las que no usas: con credenciales robadas, un atacante suele lanzar instancias justo donde nadie mira. Con un administrador delegado en AWS Organizations, las cuentas nuevas quedan cubiertas solas.

15.12 Defensa en profundidad

Ningún control es perfecto, así que se apilan: si una capa falla, la siguiente frena el ataque. La figura resume en qué capa actúa cada servicio de este capítulo y de los anteriores.

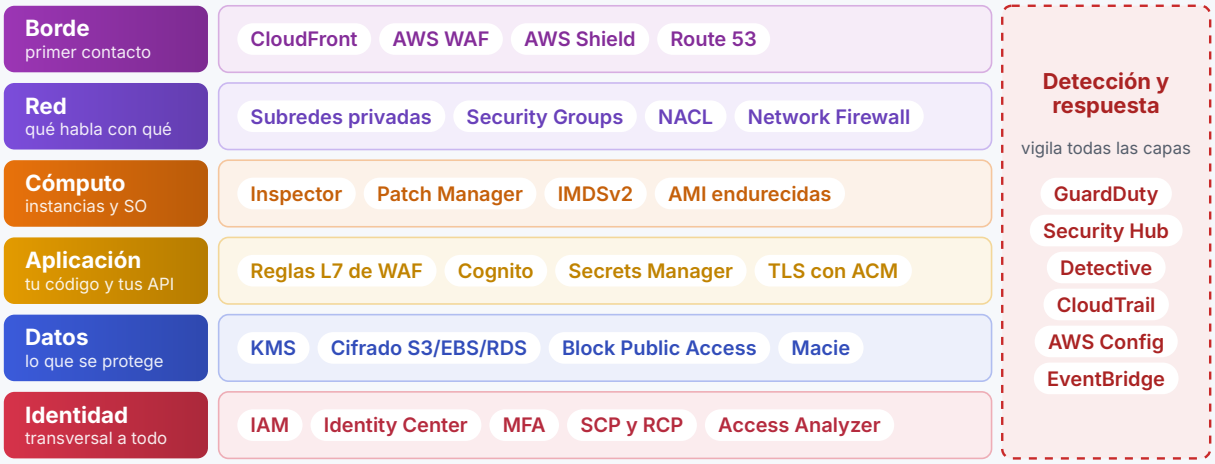


Figura 93 · Defensa en profundidad: cada capa frena lo que la anterior dejó pasar; la identidad es transversal y la detección vigila todas las capas.

15.13 Respuesta a incidentes

Ante una alarma seria —una instancia hablando con un servidor de mando y control, unas claves usadas desde un país inesperado—, improvisar es lo peor. Prepárate antes: GuardDuty y Security Hub activos, un *trail* de organización en una cuenta de logs protegida, roles de respuesta listos y *runbooks* ensayados. El esqueleto básico es este:

1. **Aislar, no terminar.** Cambia el Security Group de la instancia por uno sin reglas y sácala del Auto Scaling group y del balanceador; terminarla destruiría la memoria y los discos efímeros que necesitas para investigar. Desactiva las claves de acceso comprometidas y revoca las sesiones activas del rol.
2. **Preservar evidencias:** snapshots de los volúmenes EBS, captura de memoria si procede, etiquetas con el número de incidente y copia de los logs en un bucket con S3 Object Lock de otra cuenta.
3. **Rotar credenciales:** claves de acceso, secretos, contraseñas y tokens a los que pudo llegar el atacante.
4. **Investigar con CloudTrail** (y Detective): qué llamadas hizo la identidad comprometida, desde qué IP y qué creó o modificó (usuarios, roles, instancias en otras regiones…).

5 **Recuperar y aprender:** reconstruye desde una AMI o una plantilla de IaC limpias, corrige la causa raíz, escribe un post-mortem sin culpables y automatiza la respuesta.

RESUMEN DEL CAPÍTULO

- Cifra en reposo (KMS) y en tránsito (TLS con ACM). KMS guarda las claves en HSM, nunca las deja salir y se convierte en un segundo punto de control de acceso. Hay claves **AWS owned**, **AWS managed** (`aws/servicio` , rotación anual, key policy de solo lectura) y **customer managed** (control total).
- Toda clave tiene exactamente una **key policy**; sin la sentencia que delega en la cuenta, IAM no basta. Entre cuentas hace falta permiso en la key policy **y** en IAM de la cuenta consumidora.
- **Cifrado envolvente:** `GenerateDataKey` → cifrar en local → descartar la clave en claro → guardar la clave cifrada con los datos → `Decrypt` para leer.
- La rotación mantiene el ID y el ARN y conserva el material antiguo. Borrar una clave exige una espera de 7 a 30 días y es irreversible.
- S3 Bucket Keys reduce las peticiones a KMS; RDS no se cifra en caliente (snapshot → copia cifrada → restaurar). **Secrets Manager** para secretos que rotan; **Parameter Store** para configuración. ACM renueva certificados gratis; para CloudFront, en us-east-1.
- WAF filtra en la capa 7, Shield frena DDoS, Firewall Manager gobierna las políticas y Network Firewall inspecciona las VPC.
- GuardDuty, Inspector y Macie detectan; Security Hub agrega; EventBridge automatiza la respuesta. Ante un incidente, aísla y preserva antes de borrar nada.

Preguntas de repaso

1 **Un rol con `s3:GetObject` recibe *Access Denied* al descargar objetos cifrados con SSE-KMS y una clave customer managed. ¿Qué falta?**
Respuesta: `kms:Decrypt` sobre la clave: en la key policy, o en la política IAM del rol si la key policy delega en la cuenta.

2 **¿Por qué S3 no envía tus objetos a KMS para cifrarlos?**
Respuesta: Porque usa cifrado envolvente: `Encrypt` admite como máximo 4 KB y enviar los datos sería lento y caro. KMS solo genera y descifra la clave de datos; los datos se cifran en local.

3 **Debes compartir con la cuenta de auditoría un snapshot de RDS cifrado con `aws/rds` . ¿Cómo?**
Respuesta: Las AWS managed no se comparten: copia el snapshot cifrándolo con una customer managed, permite a la otra cuenta en su key policy y comparte el snapshot (en destino, IAM debe conceder el uso de la clave).

4 **En una key policy ves `"Principal": {"AWS": "arn:aws:iam::111122223333:root"}` con `kms:*` . ¿Da acceso total al usuario root?**
Respuesta: No: ese principal es la cuenta entera. La sentencia delega en IAM, de modo que las políticas IAM de la cuenta deciden quién usa la clave; sin ella, esas políticas no tendrían ningún efecto sobre la clave.

5 **Activas la rotación automática de una clave. ¿Qué pasa con los datos ya cifrados?**
Respuesta: Nada: ID, ARN y alias no cambian, y KMS conserva el material antiguo para descifrarlos. Lo nuevo se cifra con el material actual.

6 **Tu web recibe intentos de inyección SQL por el puerto 443. ¿Por qué no basta un Security Group y qué usas?**
Respuesta: El Security Group filtra por IP, puerto y protocolo, y el ataque llega por el 443 como cualquier petición legítima. Hace falta AWS WAF (capa 7) en CloudFront o en el ALB, con reglas gestionadas frente a inyecciones y límites de tasa.

7 **GuardDuty detecta una instancia que consulta dominios de minería de criptomonedas. ¿Qué automatizas y qué evitas?**
Respuesta: Una regla de EventBridge que invoque una Lambda para aislar la instancia, hacer snapshots y avisar al equipo; luego rotar credenciales e investigar con CloudTrail y Detective. No la termines antes de preservar las evidencias.

CAPÍTULO 16

Infraestructura como código y DevOps

Describe tu infraestructura en ficheros versionados y despliégala con pipelines automáticos, revisables y repetibles.

EN ESTE CAPÍTULO APRENDERÁS

1. Por qué usar IaC y qué es el drift
2. Plantillas y funciones de CloudFormation
3. Change sets, rollback y políticas de borrado
4. CDK, SAM y Terraform: cuándo usar cada uno
5. CodePipeline, CodeBuild y CodeDeploy
6. Rolling, blue/green, canary y feature flags

SERVICIOS PROTAGONISTAS

AWS CloudFormation · AWS CDK · AWS SAM · Terraform · AWS CodePipeline · AWS CodeBuild · AWS CodeDeploy · AWS AppConfig

16.1 Por qué infraestructura como código

Crear recursos haciendo clic en la consola —la llamada *ClickOps*— sirve para aprender, pero no escala: nadie recuerda qué casillas marcó hace seis meses, replicar un entorno en otra región es un proyecto y los cambios se aplican sin revisión. Con **infraestructura como código** (IaC) describes la infraestructura en ficheros de texto, los guardas en Git junto a la aplicación y una herramienta crea, modifica o borra recursos hasta que la cuenta coincide con lo escrito.

Así la infraestructura se vuelve **reproducible** (dev, staging y producción salen de la misma plantilla), **revisable** (antes de aplicar ves qué se creará, cambiará o borrará) y **trazable** (Git guarda quién cambió qué, cuándo y por qué, y permite volver atrás). Y tiene una única fuente de verdad, que destapa el **drift** (deriva): la diferencia entre lo que declara tu código y lo que existe de verdad en la cuenta. Suele nacer de un cambio manual «urgente» que nadie llevó al código, y el siguiente despliegue puede deshacerlo sin avisar.

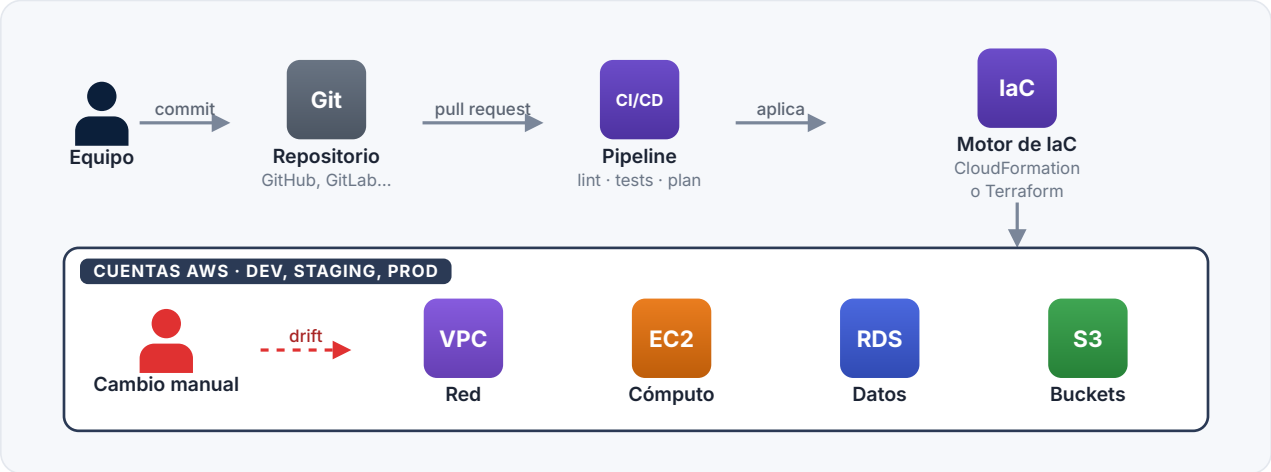


Figura 94 · Flujo de trabajo con IaC: el repositorio es la fuente de verdad y solo el pipeline modifica las cuentas. Un cambio manual en la consola aparece como *drift*.

Declarativo
describes el estado deseado

«Quiero un bucket versionado». La herramienta calcula qué falta y en qué orden, y es **idempotente**: aplicar dos veces lo mismo no crea nada nuevo. CloudFormation, Terraform y CDK (que solo **genera** una plantilla declarativa).

Imperativo
describes los pasos

«Crea el bucket, luego activa el versionado...». Tú gestionas orden, errores y estado previo. Scripts con AWS CLI o un SDK: útiles para tareas puntuales, frágiles para toda una infraestructura.

16.2 AWS CloudFormation

CloudFormation, el servicio de IaC nativo de AWS, convierte una **plantilla** YAML o JSON en un **stack**: recursos que se crean, actualizan y borran como una unidad, en paralelo cuando se puede y deshaciendo los cambios si algo falla.

FICHA DEL SERVICIO

AWS CloudFormation

Infraestructura declarativa nativa de AWS

UNIDAD	**Stack**: recursos con un ciclo de vida común, en una cuenta y una región
FACTURACIÓN	Gratuito para los recursos de AWS: pagas lo que crea (las extensiones de terceros, por operación)
EXTRAS	Change sets, drift, StackSets e *IaC generator* (plantillas a partir de lo ya desplegado)

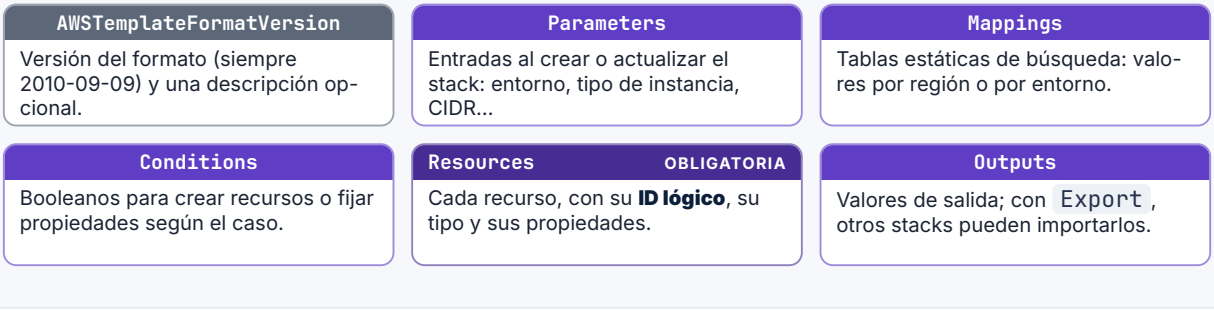


Figura 95 · Secciones de una plantilla de CloudFormation. Solo `Resources` es obligatoria; existen también `Metadata`, `Rules` y `Transform` (macros, como la de SAM).

Esta plantilla crea un bucket y una función Lambda que procesa sus objetos. Es pequeña, pero usa todas las secciones:

YAML

```yaml
AWSTemplateFormatVersion: "2010-09-09"
Parameters:
  Entorno: { Type: String, AllowedValues: [dev, prod], Default: dev }
Mappings:
  PorEntorno:
    dev:  { Memoria: 256 }
    prod: { Memoria: 1024 }
Conditions:
  EsProd: !Equals [!Ref Entorno, prod]
Resources:
  Subidas:
    Type: AWS::S3::Bucket
    DeletionPolicy: Retain                  # sobrevive al borrado del stack
    UpdateReplacePolicy: Retain
    Properties:
      VersioningConfiguration:
        Status: !If [EsProd, Enabled, Suspended]
  Procesador:
    Type: AWS::Lambda::Function
    Properties:
      FunctionName: !Sub "procesador-${Entorno}-${AWS::Region}"
      Runtime: python3.13
      Handler: index.handler
      MemorySize: !FindInMap [PorEntorno, !Ref Entorno, Memoria]
      Role: !ImportValue base-RolLambdaArn  # exportado por otro stack
      Code:
        ZipFile: "def handler(event, context): print(event)"
Outputs:
  ArnBucket:
    Value: !GetAtt Subidas.Arn
    Export: { Name: !Sub "${AWS::StackName}-ArnBucket" }
```

Las **funciones intrínsecas** calculan valores al desplegar (en YAML, con la forma corta `!Funcion`); los **pseudoparámetros** como `AWS::Region`, `AWS::AccountId` o `AWS::StackName` los rellena CloudFormation y evitan escribir regiones o cuentas a mano:

Función	Qué devuelve	Ejemplo
`!Ref`	Valor de un parámetro o ID físico de un recurso (nombre del bucket, ID de instancia).	`!Ref Entorno`
`!GetAtt`	Un atributo del recurso: ARN, nombre DNS, endpoint…	`!GetAtt Subidas.Arn`
`!Sub`	Una cadena con las variables `${...}` sustituidas.	`!Sub "app-${Entorno}"`
`!ImportValue`	Un valor exportado por otro stack (misma cuenta y región).	`!ImportValue red-VpcId`
`!FindInMap`, `!If`	Un valor de `Mappings`; un valor condicional.	`!If [EsProd, Enabled, Suspended]`

Change sets, drift y rollback

En producción no actualices un stack a ciegas: crea antes un **change set**, una vista previa de cada recurso que se añadirá, modificará o eliminará y de si la modificación exige **reemplazarlo** (`Replacement: True`): crear uno nuevo y borrar el antiguo, con el riesgo de perder datos. Pasa, por ejemplo, al cambiar el nombre de un bucket.

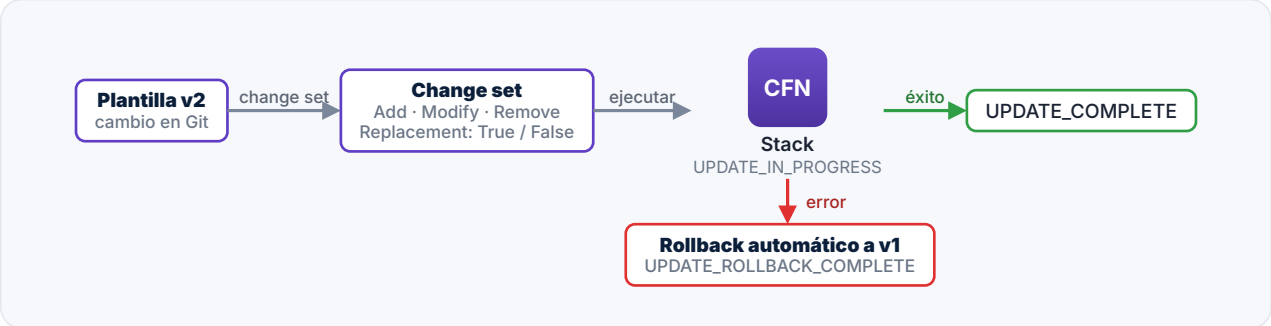


Figura 96 · Actualizar un stack con un *change set*: primero ves el impacto y luego decides si lo ejecutas. Si algo falla, CloudFormation vuelve al estado anterior.

Si falla la creación de un stack, CloudFormation borra lo que había creado; si falla una actualización, vuelve a la configuración anterior. Los **rollback triggers** revierten además el despliegue si salta una alarma de CloudWatch. La **detección de drift** compara cada recurso con la plantilla y marca el stack como `IN_SYNC` o `DRIFTED`, con el detalle de cada propiedad cambiada.

Políticas de borrado y reemplazo

`DeletionPolicy` decide qué pasa con un recurso al borrar el stack (o quitarlo de la plantilla); `UpdateReplacePolicy`, qué pasa con el recurso **antiguo** cuando una actualización lo reemplaza:

Valor	Qué hace	Úsalo en
`Delete`	Borra el recurso. Es el valor por defecto en casi todos los tipos.	Funciones, roles, Security Groups
`Retain`	Conserva el recurso, que deja de estar gestionado por el stack.	Buckets, tablas DynamoDB, claves KMS
`Snapshot`	Crea un snapshot final y después borra el recurso.	RDS, Aurora, EBS, ElastiCache
`RetainExceptOnCreate`	Como `Retain`, salvo si falla la creación inicial: entonces borra y no deja restos.	Recursos con datos en stacks nuevos

> **CUIDADO**
>
> Hay excepciones: en `AWS::RDS::DBCluster` (y en las instancias de RDS que no pertenecen a un clúster) el valor por defecto es `Snapshot`, y un bucket S3 solo se borra si está vacío. En producción, activa además la **protección contra terminación** del stack.

Reutilizar y compartir: nested stacks, exports y StackSets

Mecanismo	Qué es	Cuándo usarlo
Nested stacks	Un stack padre incluye plantillas hijas (recurso `AWS::CloudFormation::Stack`); se despliegan juntos, como una unidad.	Reutilizar componentes estándar (un ALB, una VPC tipo) dentro de una aplicación.
Exports + `!ImportValue`	Un stack exporta salidas y otros las importan; no puedes borrar ni cambiar un export mientras alguien lo importe.	Compartir valores (VPC, subredes) entre stacks de la misma cuenta y región. Menos acoplado: Parameter Store y `{{resolve:ssm:...}}`.
StackSets	Despliega el mismo stack en muchas cuentas y regiones; integrado con Organizations, cubre automáticamente las cuentas nuevas de una OU.	Línea base de seguridad en toda la organización: roles de auditoría, reglas de Config, alarmas.

16.3 AWS CDK y AWS SAM

El **AWS Cloud Development Kit** (CDK) define la infraestructura con TypeScript, JavaScript, Python, Java, C# o Go: bucles, funciones, clases, tipos, autocompletado y tests. Por debajo no hay magia: tu programa (una **App** con uno o más **Stacks**) **sintetiza** plantillas de CloudFormation y es CloudFormation quien las despliega, así que heredas change sets, rollback y drift. Es gratuito y de código abierto. Las piezas se llaman **constructs** y tienen tres niveles:

L1 (`Cfn*`) — equivalen 1:1 a los recursos de CloudFormation (`CfnBucket`, `CfnFunction`...): control total, sin valores por defecto.
L2 (curados) — los más usados (`s3.Bucket`, `lambda.Function`...), con valores por defecto sensatos y ayudas como `grantRead()`.
L3 (patrones) — arquitecturas completas; el módulo `ecs_patterns`, por ejemplo, crea un servicio Fargate tras un ALB con clúster, roles y logs.

```typescript
import { App, Stack, Duration, RemovalPolicy } from 'aws-cdk-lib';
import * as s3 from 'aws-cdk-lib/aws-s3';
import * as lambda from 'aws-cdk-lib/aws-lambda';

const app = new App();
const stack = new Stack(app, 'Subidas-dev', { env: { region: 'eu-west-1' } });

const bucket = new s3.Bucket(stack, 'Subidas', {
  versioned: true, enforceSSL: true, removalPolicy: RemovalPolicy.RETAIN,
});
const fn = new lambda.Function(stack, 'Procesador', {
  runtime: lambda.Runtime.PYTHON_3_13, handler: 'index.handler',
  code: lambda.Code.fromAsset('lambda'),     // carpeta local → asset en S3
  timeout: Duration.seconds(30),
});
bucket.grantRead(fn);                        // política IAM de mínimo privilegio
```

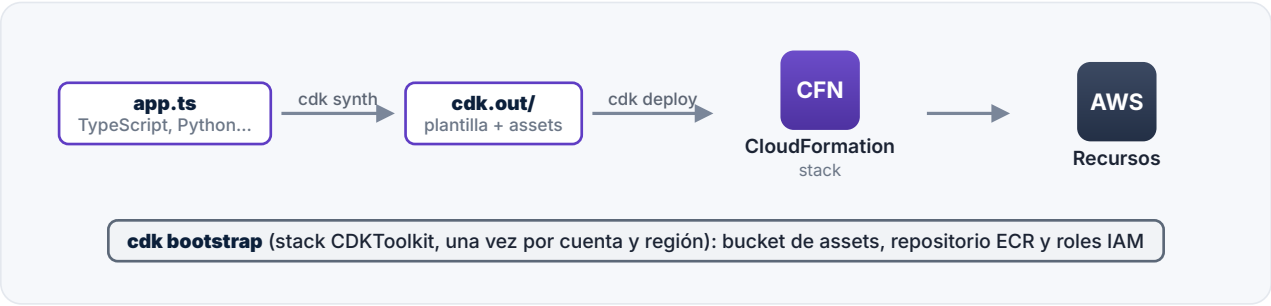


Figura 97 · Del código al despliegue con CDK. `cdk synth` genera plantilla y *assets*, `cdk diff` compara con lo desplegado y `cdk deploy` sube los *assets* a los recursos creados por `cdk bootstrap` (una vez por cuenta y región) y lanza el stack.

AWS SAM, presentado en el capítulo de Serverless, es la otra gran extensión de CloudFormation: una macro (`Transform: AWS::Serverless-2016-10-31`) que expande tipos abreviados como `AWS::Serverless::Function` en función, rol, permisos, API y eventos, con su propia CLI (`sam build`, `sam local invoke`, `sam deploy`).

16.4 Terraform

Terraform, de HashiCorp, es la herramienta de IaC más extendida fuera de AWS: con su lenguaje declarativo, **HCL**, y sus **proveedores** gestiona AWS, otras nubes y cientos de servicios con el mismo flujo: `terraform init`, `plan` (qué cambiará) y `apply`.

A diferencia de CloudFormation, Terraform guarda un **fichero de estado** que relaciona cada recurso con su identificador real. Es crítico y puede contener secretos: guárdalo en un **backend remoto** (un bucket S3 versionado, cifrado y restringido) con **bloqueo**, para que dos `apply` simultáneos no lo corrompan. Desde Terraform 1.11 el backend S3 bloquea de forma nativa con `use_lockfile = true`; la tabla de DynamoDB que se usaba antes está marcada como obsoleta.

```hcl
terraform {
  required_providers { aws = { source = "hashicorp/aws", version = "~> 6.0" } }
  backend "s3" {
    bucket       = "miempresa-tfstate"
    key          = "subidas/prod/terraform.tfstate"
    region       = "eu-west-1"
    encrypt      = true
    use_lockfile = true            # bloqueo nativo en S3
  }
}
provider "aws" {
  region = "eu-west-1"
  default_tags { tags = { Proyecto = "subidas" } }   # etiquetas en todo
}
resource "aws_s3_bucket" "subidas" { bucket = "miempresa-subidas-prod" }
```

	CloudFormation	AWS CDK	Terraform
Lenguaje	YAML o JSON	TypeScript, Python, Java, C#, Go	HCL
Estado	Lo guarda AWS (el stack)	Lo guarda AWS (vía CloudFormation)	Fichero de estado que gestionas tú
Vista previa	Change sets	`cdk diff`	`terraform plan`
Rollback automático	✔	✔ vía CloudFormation	✘ se corrige y se vuelve a aplicar
Elígelo si...	Quieres plantillas sin dependencias externas	Todo está en AWS y tu equipo programa	Usas varios proveedores (o **OpenTofu**, su fork abierto desde 2023)

16.5 CI/CD en AWS

La **integración continua** (CI) compila y prueba cada cambio en cuanto llega al repositorio; la **entrega** o **despliegue continuo** (CD) lleva ese artefacto ya probado hasta los entornos sin pasos manuales. AWS ofrece tres piezas gestionadas:

CodePipeline
el orquestador

Etapas (Source → Build → Deploy), aprobaciones manuales y despliegues con CloudFormation, ECS, CodeDeploy o S3.

CodeBuild
compilar y probar

Contenedores efímeros que ejecutan tu `buildspec.yml` y publican artefactos e informes de tests. Pagas por tiempo de compilación.

CodeDeploy
desplegar con seguridad

EC2 u on-premises (*in-place* o blue/green), Lambda y ECS (canary, lineal), con rollback por alarmas.

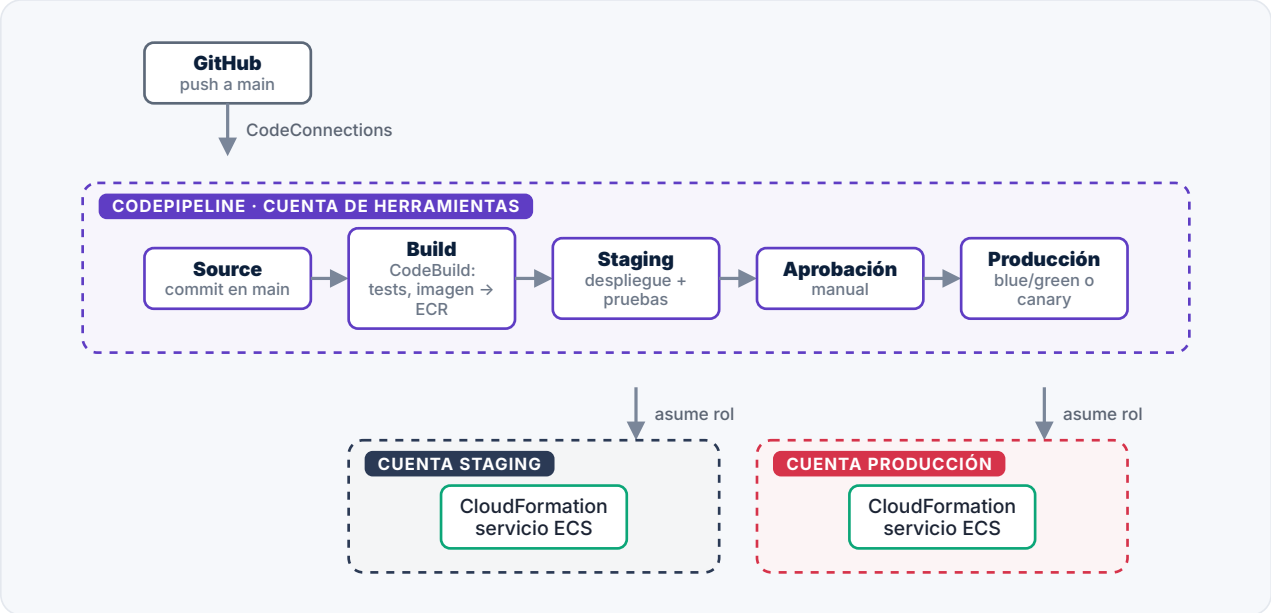


Figura 98 · Pipeline típico: el código vive en GitHub, CodePipeline orquesta las etapas desde una cuenta de herramientas y despliega en las cuentas de staging y producción asumiendo un rol de despliegue en cada una.

> **¿DÓNDE VIVE EL CÓDIGO?**
>
> **AWS CodeCommit**, el servicio de repositorios Git de AWS, dejó de aceptar clientes nuevos en julio de 2024 y volvió a abrirse a ellos el 25 de noviembre de 2025. Aun así, lo habitual es usar GitHub, GitLab o Bitbucket conectados a CodePipeline con **AWS CodeConnections** (antes CodeStar Connections), o el CI del propio proveedor. Si usas GitHub Actions, no guardes claves de acceso en el repositorio: federación OIDC con un rol que solo acepte tu repositorio y rama (capítulo «IAM (I): identidades — usuarios, grupos, roles y permission sets»).

Este `buildspec.yml` ejecuta los tests, construye la imagen, la sube a ECR y genera el `imagedefinitions.json` que consume la acción de despliegue en ECS:

```yaml
version: 0.2
env:
  variables:
    REPO: 111111111111.dkr.ecr.eu-west-1.amazonaws.com/api
phases:
  pre_build:
    commands:
      - npm ci && npm test
      - aws ecr get-login-password | docker login -u AWS --password-stdin ${REPO%/*}
      - TAG=$(echo $CODEBUILD_RESOLVED_SOURCE_VERSION | cut -c1-7)
  build:
    commands:
      - docker build -t $REPO:$TAG . && docker push $REPO:$TAG
  post_build:
    commands:
      - printf '[{"name":"api","imageUri":"%s"}]' $REPO:$TAG > imagedefinitions.json
artifacts:
  files: [imagedefinitions.json]
```

16.6 Estrategias de despliegue

Desplegar es decidir **cómo** pasa el tráfico de la versión actual (v1) a la nueva (v2) y cómo vuelves atrás si algo va mal.

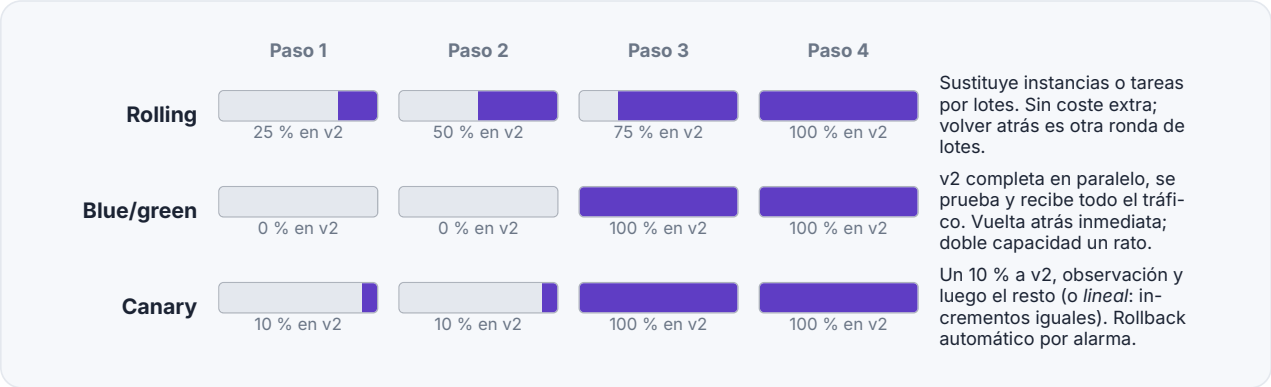


Figura 99 · Reparto del tráfico entre v1 (gris) y v2 (color) a lo largo de un despliegue, con la vuelta atrás y el coste de cada estrategia.

Desplegar **todo a la vez** es lo más simple, pero implica corte y una vuelta atrás lenta; las demás estrategias las ofrecen CodeDeploy, ECS (de forma nativa desde 2025), los alias ponderados de Lambda y los target groups del ALB. Los **feature flags** separan **desplegar** de **lanzar**: el código llega desactivado y lo enciendes por configuración para un porcentaje de usuarios. **AWS AppConfig** los gestiona: valida cada cambio, lo propaga gradualmente con un periodo de observación (*bake time*) y lo revierte si salta una alarma.

16.7 Entornos y buenas prácticas

- **Una cuenta por entorno** (dev, staging, prod), con el pipeline en una cuenta de herramientas que asume roles de despliegue (en CDK, `cdk bootstrap --trust`); la estructura completa, en el capítulo de Arquitecturas de referencia.
- **El mismo artefacto y la misma plantilla en todos los entornos**: solo cambian los parámetros, y los secretos se leen de Secrets Manager o Parameter Store.
- Consola de **solo lectura** en producción, stacks **pequeños y por ciclo de vida**, y el change set o el `plan` revisados en la *pull request*.
- Mínimo privilegio para el pipeline, OIDC en lugar de claves y etiquetas desde el código (lo agradecerá el capítulo de Costes).

RESUMEN DEL CAPÍTULO

- La IaC hace la infraestructura **reproducible, revisable y trazable** y destapa el **drift** de los cambios manuales.
- Una plantilla de CloudFormation tiene `Parameters`, `Mappings`, `Conditions`, `Resources` (la única obligatoria) y `Outputs`; `!Ref`, `!GetAtt`, `!Sub` e `!ImportValue` conectan las piezas.
- Los **change sets** muestran el impacto antes de ejecutar y, si algo falla, hay **rollback**; `DeletionPolicy` y `UpdateReplacePolicy` protegen los datos. **Nested stacks**, **exports** y **StackSets** reutilizan, comparten y llevan stacks a muchas cuentas.
- CDK genera CloudFormation desde código (constructs L1, L2, L3); Terraform gestiona varios proveedores con un **fichero de estado** en S3 con bloqueo (`use_lockfile`).
- CodePipeline orquesta, CodeBuild prueba y CodeDeploy o ECS despliegan (rolling, blue/green, canary); AppConfig da feature flags.
- Una cuenta por entorno, el mismo artefacto en todos, el pipeline asumiendo roles de despliegue y OIDC en lugar de claves de acceso.

Preguntas de repaso

1. **¿Qué es el drift y cómo lo detectas?**
   Respuesta: La diferencia entre el código y la realidad, casi siempre por cambios manuales. Con la detección de drift de CloudFormation o con `terraform plan`.
2. **¿Qué diferencia hay entre `!Ref` y `!GetAtt`?**
   Respuesta: `!Ref` devuelve un parámetro o el ID físico del recurso (el nombre de un bucket); `!GetAtt`, un atributo como el ARN.
3. **¿Cómo evitas perder una base de datos RDS al borrar el stack o al reemplazarla?**
   Respuesta: `DeletionPolicy` y `UpdateReplacePolicy` con `Snapshot` o `Retain`, protección contra terminación y revisar `Replacement` en el change set.
4. **¿Por qué hay que bloquear el estado de Terraform?**
   Respuesta: Para que dos ejecuciones simultáneas no lo corrompan; en AWS, backend S3 con `use_lockfile = true`.
5. **¿En qué se diferencian blue/green y canary?**
   Respuesta: Blue/green cambia todo el tráfico de una vez a una versión nueva completa (vuelta atrás inmediata); canary envía antes un pequeño porcentaje y observa las alarmas.

CAPÍTULO 17

Costes y optimización

Entiende cómo se factura AWS, dónde se va el dinero y qué palancas tienes para pagar solo por lo que aporta valor.

EN ESTE CAPÍTULO APRENDERÁS

1. El modelo de pago por uso
2. Qué tráfico de datos se paga y cuál no
3. Cost Explorer, Budgets y compañía
4. Etiquetas de asignación de costes
5. A qué servicios llega cada descuento por compromiso
6. Optimizar y evitar las trampas típicas

SERVICIOS PROTAGONISTAS

AWS Cost Explorer · AWS Budgets · AWS Data Exports · AWS Cost Anomaly Detection · AWS Compute Optimizer · Savings Plans

17.1 El modelo de pago por uso

En AWS no compras servidores: pagas por segundo, por hora, por GB o por petición, sin inversión inicial, y dejas de pagar cuando apagas el recurso. Ojo con el matiz: pagas por lo que **aprovisionas**, no por lo que usas. Una instancia al 3 % de CPU o un volumen sin adjuntar se facturan enteros. El precio por unidad baja con el volumen (tramos) y con el **compromiso** de uso a uno o tres años, y varía por región. La capa gratuita, que cambió en julio de 2025, se resume en el capítulo de Fundamentos.

0
coste de los datos que **entran** desde Internet

100 GB
de salida a Internet gratis al mes (agregado)

72 %
descuento máximo con compromiso a 3 años

90 %
descuento máximo con instancias Spot

17.2 Los grandes conductores de coste

Cómputo
suele ser la mayor partida

EC2 y RDS por tiempo de instancia; Lambda por petición y GB-segundo; Fargate por vCPU y GB.

Almacenamiento
crece en silencio

GB-mes de S3 (y peticiones), EBS aprovisionado (y sus IOPS), snapshots, backups y logs.

Transferencia de datos
la gran olvidada

Salida a Internet, tráfico entre AZ y regiones, y procesamiento del NAT Gateway.

La transferencia es la partida que más sorprende, porque no aparece en ningún «recurso» de la consola. La regla: **entrar es gratis y salir cuesta**; **cruzar zonas de disponibilidad cuesta en los dos sentidos**; **entre regiones paga la de origen**; y lo que atraviesa un **NAT Gateway** paga además su procesamiento por GB, vaya a Internet o a S3. Dentro de una AZ, con IP privadas, es gratis, y servir contenido por CloudFront suele salir más barato que hacerlo desde EC2 o S3.

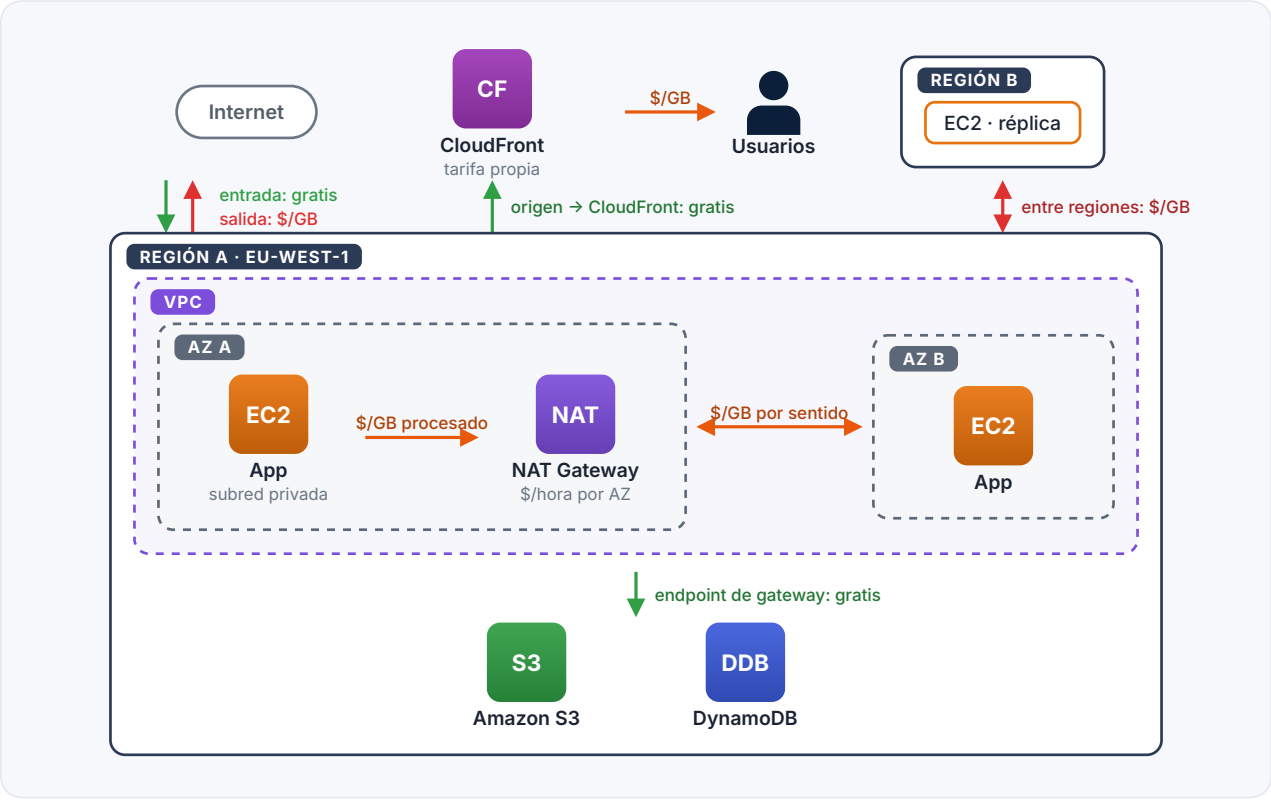


Figura 100 · Qué tráfico se paga: en verde, gratis; en naranja, pago por GB o por hora; en rojo, salida a Internet o entre regiones. Los importes varían por región y volumen; lo importante es el patrón.

17.3 Herramientas de visibilidad y control

Herramienta	Para qué sirve
Billing and Cost Management	Facturas, pagos y créditos. Con Organizations, la **facturación consolidada** suma el uso (y los descuentos por volumen) de todas las cuentas.
Cost Explorer	Explorar el gasto por servicio, cuenta, región o etiqueta, con granularidad diaria o mensual, previsiones y recomendaciones de Savings Plans y reservas.
AWS Budgets	Presupuestos de coste, uso o cobertura con alertas (correo, SNS) cuando el gasto real o **previsto** supera un umbral; las **budget actions** pueden aplicar políticas o detener instancias.
Data Exports (CUR 2.0)	El detalle línea a línea de la factura (el antiguo Cost and Usage Report, también en formato FOCUS) entregado en S3 para consultarlo con Athena o una herramienta de BI.
Cost Anomaly Detection	Aprendizaje automático que detecta gastos inusuales (un servicio que se dispara) y avisa, sin umbrales fijos.
Pricing Calculator	Estimar el coste de una arquitectura **antes** de construirla.
Compute Optimizer	Recomendaciones de tamaño para EC2, Auto Scaling, EBS, Lambda, ECS en Fargate y RDS, basadas en métricas reales.
Trusted Advisor y **Cost Optimization Hub**	Comprobaciones de recursos ociosos o infrautilizados (las de coste requieren soporte Business o superior) y un panel que reúne todas las recomendaciones de ahorro.

Configúralo el primer día en cada cuenta nueva: un presupuesto mensual con alertas al 80 % del gasto real y al 100 % del previsto, y Cost Anomaly Detection activado.

Etiquetas de asignación de costes

Las etiquetas (*tags*) responden a «¿cuánto cuesta este proyecto?». Define un esquema común —`Proyecto`, `Entorno`, `Equipo`, `CentroDeCoste`—, aplícalo desde la IaC (con `default_tags`, como en el capítulo de Infraestructura como código) y hazlo obligatorio con **tag policies** de Organizations o SCP que denieguen crear recursos sin etiquetas. Las **Cost Categories** agrupan cuentas, servicios y etiquetas en categorías de negocio.

> **CUIDADO**
>
> Tus etiquetas no aparecen en Cost Explorer ni en Data Exports hasta que las **activas** como etiquetas de asignación de costes en la consola de facturación (en la cuenta de administración). Y no todo el gasto es etiquetable: el plan de soporte, por ejemplo, no lleva etiquetas.

17.4 Estrategias de optimización

Rightsizing
ajustar el tamaño

Compute Optimizer y CloudWatch delatan lo sobredimensionado. Baja de talla **antes** de comprometerte.

Apagar fuera de horario
dev y test

Sin noches ni fines de semana eliminas casi dos tercios de las horas (EventBridge Scheduler, Instance Scheduler).

Graviton
procesadores Arm de AWS

Instancias más baratas por hora que sus equivalentes x86 y con mejor relación precio-rendimiento; también en Lambda, Fargate y RDS.

Clases de S3 y lifecycle
datos fríos, precio frío

Reglas que mueven los datos a Standard-IA o Glacier (o Intelligent-Tiering) y expiran versiones antiguas y subidas multipart incompletas.

gp3 en lugar de gp2
EBS

Más barato por GB, con 3000 IOPS y 125 MiB/s de base sea cual sea el tamaño; se cambia en caliente.

Serverless si es intermitente
pagar por actividad

Lambda, DynamoDB on-demand o Aurora Serverless v2 (que puede bajar a 0 ACU) no cobran cómputo en reposo.

Súmales las palancas de la transferencia (endpoints de VPC en lugar de NAT, caché en CloudFront) y la retención de logs, que verás entre las trampas.

17.5 Modelos de compra

Con los recursos ya ajustados, toca comprometer el uso estable. Los modelos y la estrategia — **compromisos** para la base que nunca baja, **On-Demand** para los picos y **Spot** para lo interrumpible — se explican en el capítulo «EC2: cómputo en la nube»; aquí importa a qué servicios llega cada descuento (los Savings Plans, un compromiso de gasto en $/hora, se aplican solos):

Modelo	Descuento	Plazo	A qué se aplica
Compute Savings Plans	hasta 66 %	1 o 3 años	EC2 (cualquier familia, región o SO), **Fargate** y **Lambda**
EC2 Instance Savings Plans	hasta 72 %	1 o 3 años	Una familia de EC2 en una región (cualquier tamaño, SO o AZ)
Database Savings Plans	hasta 35 %	1 año	Aurora, RDS, DynamoDB, ElastiCache… (desde diciembre de 2025)
Reserved Instances	hasta 72 %	1 o 3 años	EC2 y también RDS, ElastiCache, OpenSearch y Redshift
Spot	hasta 90 %	—	Capacidad sobrante de EC2 y Fargate; 2 minutos de aviso

17.6 Trampas comunes

Trampa	Por qué cuesta	Cómo evitarla
NAT Gateway	Hora por cada AZ más cada GB procesado, incluido el tráfico a S3, DynamoDB o ECR.	Endpoints de gateway (y de interfaz si compensa); VPC Flow Logs para ver quién lo usa.
IPv4 públicas y Elastic IP	Desde febrero de 2024 cada IPv4 pública se cobra por hora (instancias, balanceadores, NAT), también las Elastic IP sin asociar.	Subredes privadas tras un balanceador, IPv6 donde se pueda, liberar las Elastic IP que sobren; inventario con VPC IPAM.
Logs sin retención	Los GB almacenados se acumulan mes a mes, para siempre.	Retención por grupo de logs; archivo en S3.
Snapshots y AMI huérfanos	Se pagan por GB-mes aunque el volumen original ya no exista.	Data Lifecycle Manager o AWS Backup con retención.
EBS sin adjuntar	Un volumen en estado *available* cuesta lo mismo que uno en uso.	Revisión periódica: snapshot si hace falta y borrar.
Tráfico entre AZ	Microservicios «habladores» o réplicas en otra AZ pagan cada GB en ambos sentidos.	Afinidad por AZ, cachés locales y compresión.
Recursos olvidados	Entornos de prueba, balanceadores o clústeres en otra región siguen facturando.	Etiquetas obligatorias, presupuestos y limpieza automática.

17.7 FinOps: el coste como responsabilidad compartida

FinOps es la práctica —y la cultura— de gestionar el gasto en la nube entre ingeniería, finanzas y negocio. La FinOps Foundation la organiza en tres fases que se repiten en ciclo:



Figura 101 · El ciclo FinOps: cada vuelta mejora la visibilidad, el ahorro y los procesos.

La métrica más útil es el **coste unitario** (por cliente, por pedido, por millón de peticiones): si la factura crece pero el coste unitario baja, el negocio escala bien.

RESUMEN DEL CAPÍTULO

- Pagas por lo que **aprovisionas**: un recurso ocioso cuesta lo mismo que uno ocupado.
- Los grandes conductores son cómputo, almacenamiento y **transferencia**: entrar es gratis, salir y cruzar AZ o regiones cuesta, y el NAT Gateway cobra por GB procesado.
- Cost Explorer analiza, Budgets avisa (y actúa), Data Exports da el detalle, Anomaly Detection vigila y Compute Optimizer recomienda tamaños.
- Las etiquetas de asignación de costes solo cuentan una vez **activadas**; hazlas obligatorias desde la IaC y las políticas de Organizations.
- Cubre la base con Savings Plans o RI, los picos con On-Demand y lo interrumpible con Spot (capítulo de EC2); Compute SP llega también a Fargate y Lambda.
- Palancas rápidas: rightsizing, apagar dev de noche, Graviton, gp3, lifecycle de S3, retención de logs y endpoints de VPC.

Preguntas de repaso

1. **¿Qué tráfico de datos no se cobra?**
   Respuesta: La entrada desde Internet, el tráfico dentro de la misma AZ con IP privadas, el acceso a S3 o DynamoDB por un endpoint de gateway en la misma región y la transferencia de un origen de AWS a CloudFront.
2. **Has etiquetado todo, pero las etiquetas no salen en Cost Explorer. ¿Por qué?**
   Respuesta: Hay que activarlas como etiquetas de asignación de costes en la consola de facturación; hasta entonces no aparecen en los informes.
3. **Una instancia de desarrollo funciona 24/7 al 3 % de CPU. ¿Por qué cuesta lo mismo que si fuera al 90 % y qué haces?**
   Respuesta: Porque se paga lo aprovisionado, no lo usado. Baja de talla con las recomendaciones de Compute Optimizer y apágala fuera de horario (EventBridge Scheduler o Instance Scheduler); si la carga es intermitente, valora un servicio serverless.
4. **La partida de NAT Gateway se ha disparado. ¿Qué revisas?**
   Respuesta: Qué tráfico pasa por él (VPC Flow Logs); llevar S3 y DynamoDB a endpoints de gateway y valorar endpoints de interfaz para servicios como ECR o CloudWatch.
5. **¿En qué se diferencian AWS Budgets y Cost Anomaly Detection?**
   Respuesta: Budgets avisa (o actúa) cuando el gasto real o previsto supera umbrales que tú defines; Anomaly Detection usa aprendizaje automático para detectar gastos inusuales sin umbrales fijos.

CAPÍTULO 18

Arquitecturas de referencia

Ocho diseños que verás una y otra vez, explicados pieza a pieza: el punto de partida para tus propias arquitecturas.

EN ESTE CAPÍTULO APRENDERÁS

1. Web de tres capas altamente disponible
2. Microservicios en contenedores con ECS
3. API serverless y procesamiento por eventos
4. Sitios estáticos y pipelines de datos
5. Estrategia multicuenta con Organizations
6. Recuperación ante desastres multirregión

SERVICIOS PROTAGONISTAS

Amazon Route 53 · Amazon CloudFront · Application Load Balancer · Amazon ECS en Fargate · AWS Lambda · Amazon DynamoDB · Amazon SQS · Amazon EventBridge · Amazon Kinesis · AWS Organizations

Este capítulo junta las piezas de toda la guía en ocho arquitecturas habituales. En cada diagrama, los números siguen el recorrido de una petición o de un dato y coinciden con los pasos, que explican qué hace cada pieza y **por qué** está ahí. Son puntos de partida: contrástalos con la checklist Well-Architected del final.

18.1 Aplicación web de tres capas altamente disponible

El clásico: presentación, lógica y datos separados en capas, cada una repartida en dos zonas de disponibilidad para sobrevivir a la caída de una de ellas.

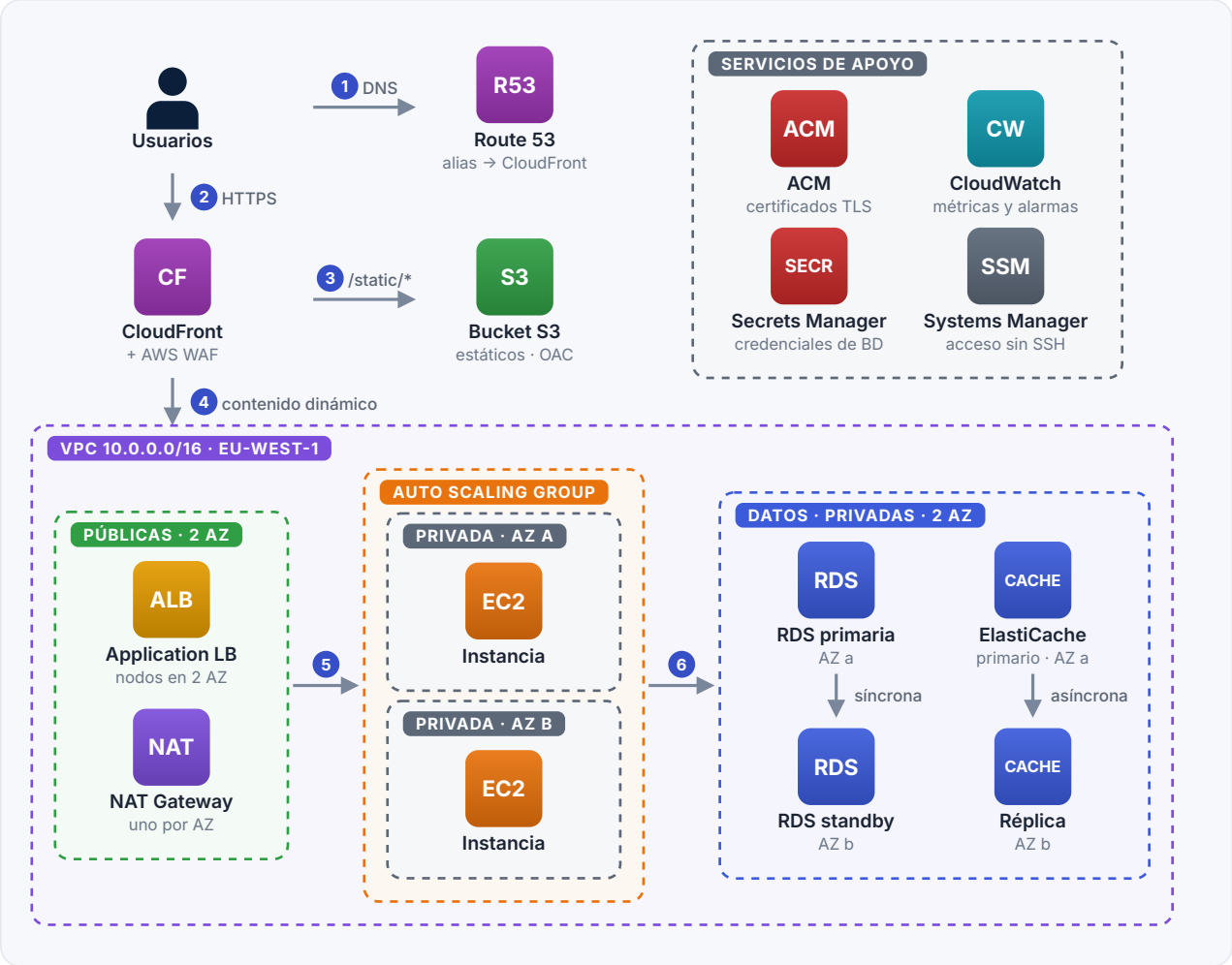


Figura 102 · Web de tres capas: borde global (Route 53, CloudFront, WAF), balanceador en subredes públicas, Auto Scaling en subredes privadas y datos Multi-AZ.

1. **Route 53** resuelve el dominio con un registro *alias* hacia CloudFront (las consultas alias a recursos de AWS no se cobran).
2. **CloudFront** termina HTTPS con un certificado de ACM, sirve desde la caché del borde y aplica **AWS WAF** contra inyecciones, bots y abusos.
3. Las rutas estáticas van a un **bucket S3 privado** al que solo accede CloudFront mediante OAC.
4. El resto se reenvía al **ALB**, único punto de entrada a la VPC, cuyos nodos viven en las subredes públicas de ambas AZ.
5. El ALB reparte entre las instancias sanas del **Auto Scaling group**, que mantiene capacidad en dos AZ y escala según la carga; las instancias, sin estado, salen a Internet por el NAT de su AZ.
6. La aplicación lee de **ElastiCache** (sesiones y consultas frecuentes) y escribe en **RDS Multi-AZ**, cuya standby síncrona toma el relevo en uno o dos minutos si cae la primaria.

VARIANTES

Con **CloudFront VPC Origins** el ALB puede ser interno, sin exposición directa a Internet. Cambia EC2 por contenedores (siguiente arquitectura) o RDS por **Aurora**, con réplicas de lectura y conmutación más rápida.

18.2 Microservicios en contenedores con ECS Fargate

Varios servicios pequeños, cada uno con su equipo, su ciclo de despliegue y sus datos, ejecutados como contenedores sin gestionar servidores.

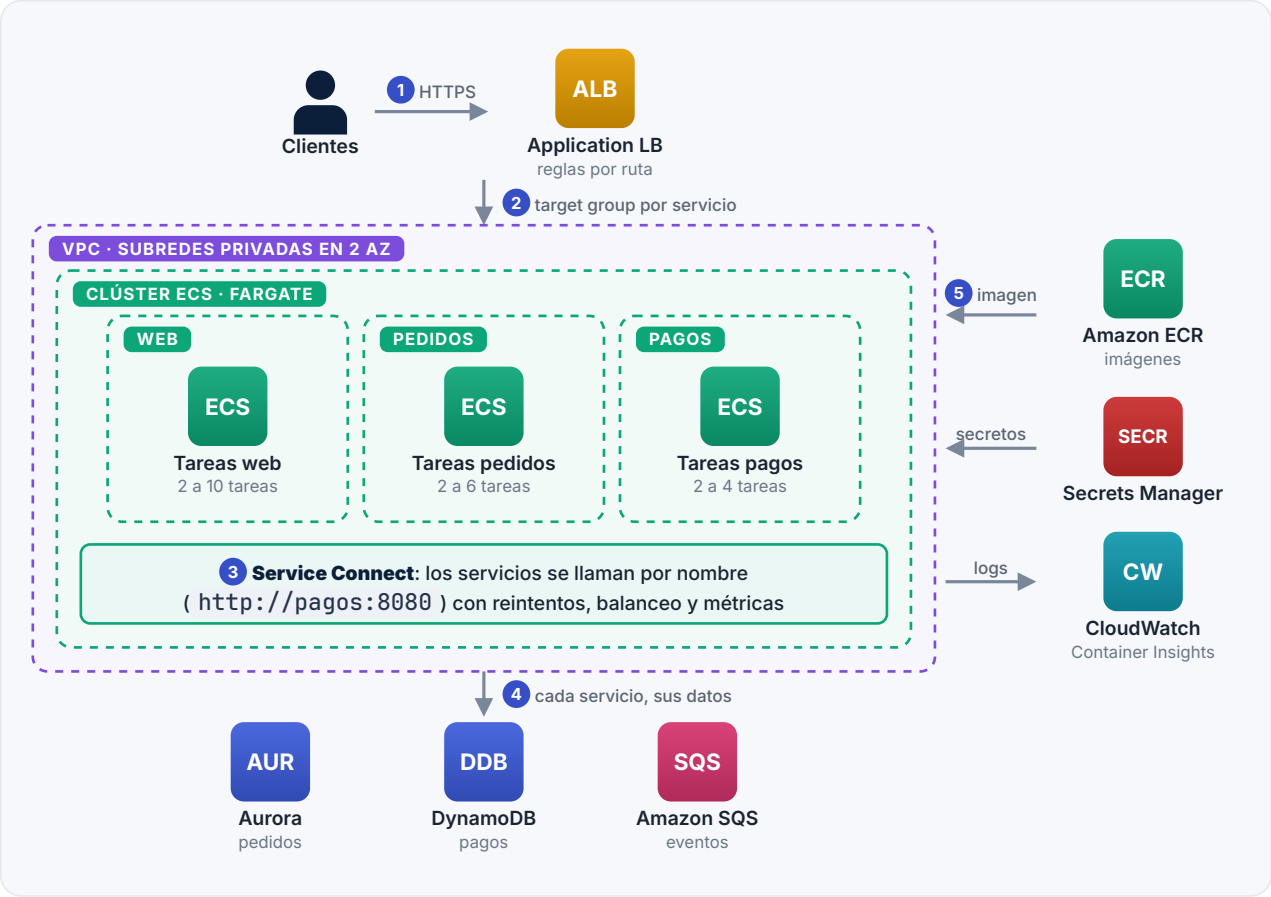


Figura 103 · Microservicios en ECS con Fargate: el ALB enruta por ruta a cada servicio, Service Connect comunica los servicios entre sí y cada servicio es dueño de sus datos.

1. El **ALB** recibe HTTPS y enruta por ruta o cabecera (`/pedidos/*`, `/pagos/*`) al *target group* de cada servicio.
2. Cada microservicio es un **servicio de ECS sobre Fargate**: sin servidores que parchear, con tareas repartidas en dos AZ (cada una con su propia IP en la VPC) y Auto Scaling por CPU o por peticiones.
3. Entre servicios, **ECS Service Connect** resuelve nombres cortos y añade reintentos, balanceo y métricas sin tocar el código.
4. Cada servicio es **dueño de sus datos** (Aurora, DynamoDB) y publica eventos en **SQS** para desacoplar los procesos lentos.
5. Al arrancar, cada tarea descarga su imagen de **ECR** y recibe sus secretos de **Secrets Manager** como variables de entorno; logs y métricas van a **CloudWatch**.

VARIANTES

Amazon EKS (con Auto Mode) es la alternativa si tu organización ya trabaja con Kubernetes o necesita su ecosistema (Helm, operadores). Para cargas grandes y estables, ECS puede usar **instancias EC2** como capacidad; despliega con los **blue/green o canary nativos de ECS**.

18.3 API serverless

Una API REST o HTTP sin un solo servidor: pagas por petición, escala de cero a miles de peticiones por segundo y AWS se ocupa de la disponibilidad.

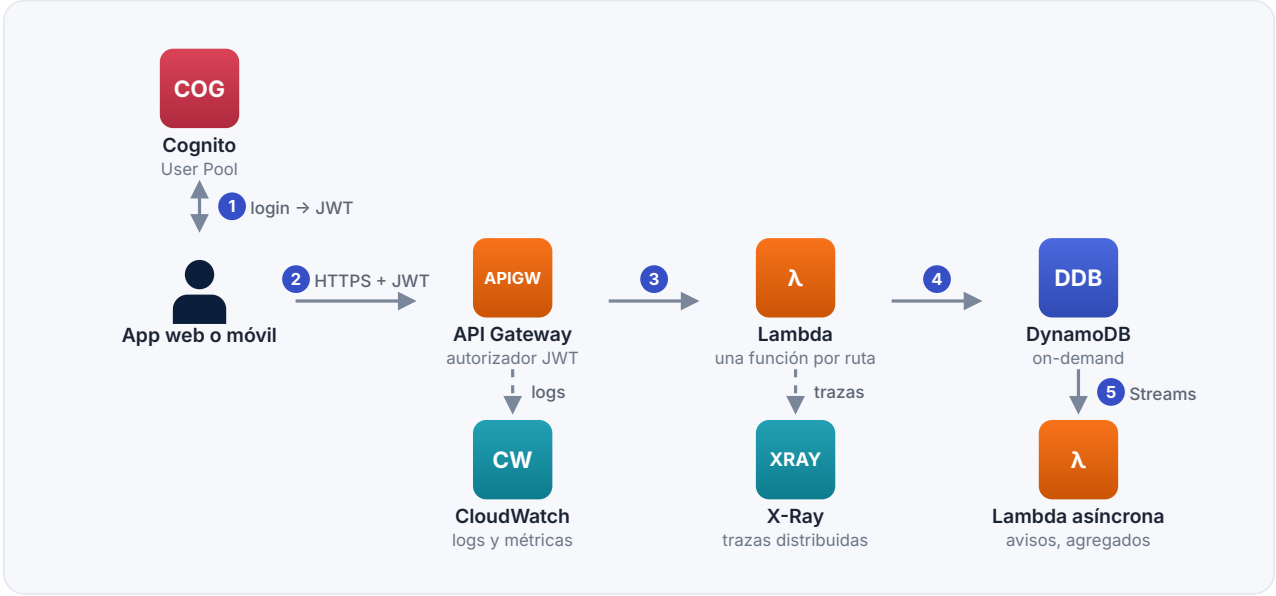


Figura 104 · API serverless: Cognito autentica, API Gateway valida el token y limita el tráfico, Lambda ejecuta la lógica y DynamoDB guarda los datos.

1. El usuario inicia sesión en un **User Pool de Cognito** (registro, MFA, acceso con Google o SAML) y recibe tokens JWT.
2. La app llama a **API Gateway** con el token: un **autorizador JWT** valida firma y caducidad antes de ejecutar nada, y el *throttling* protege el backend.
3. Cada ruta invoca una función **Lambda** pequeña, con un rol IAM de mínimo privilegio, que escala sola.
4. **DynamoDB** en modo on-demand guarda los datos con latencia de milisegundos, sin conexiones que agotar ni capacidad que dimensionar.
5. **DynamoDB Streams** dispara funciones asíncronas (notificaciones, agregados, indexación) sin ralentizar la API.

VARIANTES

Una **API HTTP** es más sencilla y barata; una **API REST** añade WAF, caché y planes de uso. Para flujos de varios pasos, **Step Functions**; si necesitas SQL, **Aurora Serverless v2** o **Aurora DSQL**; contra el *cold start*, SnapStart o concurrencia aprovisionada.

18.4 Procesamiento asíncrono orientado a eventos

Cuando un trabajo tarda (procesar un fichero, generar un informe), no lo hagas dentro de la petición: encólalo y deja que los *workers* lo procesen a su ritmo.

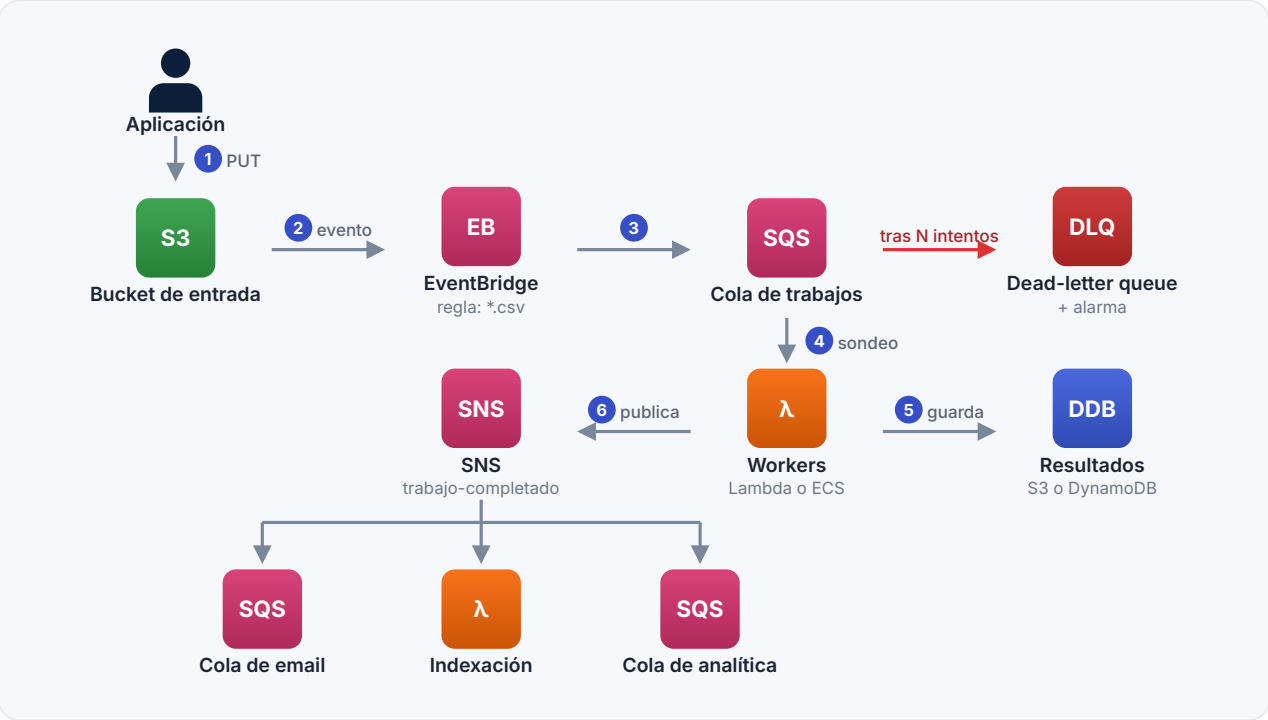


Figura 105 · Procesamiento asíncrono: S3 emite eventos, EventBridge los filtra, SQS absorbe los picos (con DLQ para los fallidos) y SNS reparte el resultado a varios suscriptores.

1. La aplicación sube un fichero al **bucket de entrada** (directamente o con una URL prefirmada).
2. S3 emite el evento *Object Created* a **EventBridge**, cuya regla filtra (solo ciertos prefijos o extensiones) y enruta.
3. El destino es una **cola SQS**, que absorbe los picos y guarda cada trabajo hasta que se procesa.
4. Los **workers** (Lambda, o tareas de ECS que escalan con la profundidad de la cola) procesan y borran el mensaje; si fallan, reaparece tras el *visibility timeout* y, tras `maxReceiveCount` intentos, pasa a la **DLQ**, vigilada por una alarma.
5. El resultado se guarda en **S3 o DynamoDB**.
6. El worker publica un aviso en **SNS**, que hace *fan-out*: cada suscriptor (dos colas y una función) recibe su copia y avanza a su ritmo.

> **VARIANTES**
>
> Con un único consumidor, S3 → SQS directamente es más simple; para orquestar pasos, **Step Functions**; para orden por clave, **SQS FIFO**; para grandes volúmenes con relectura, **Kinesis**. Y haz los workers **idempotentes**: SQS estándar entrega **al menos una vez**.

18.5 Sitio estático o SPA

Una web corporativa, una documentación o el front-end de una aplicación de una sola página (React, Vue, Angular): ficheros estáticos servidos desde el borde, sin servidores. El patrón base —bucket privado con OAC, certificado y DNS— se construye paso a paso en el capítulo «CloudFront, Route 53 y el borde de la red»; aquí lo completamos con la API y el despliegue continuo.

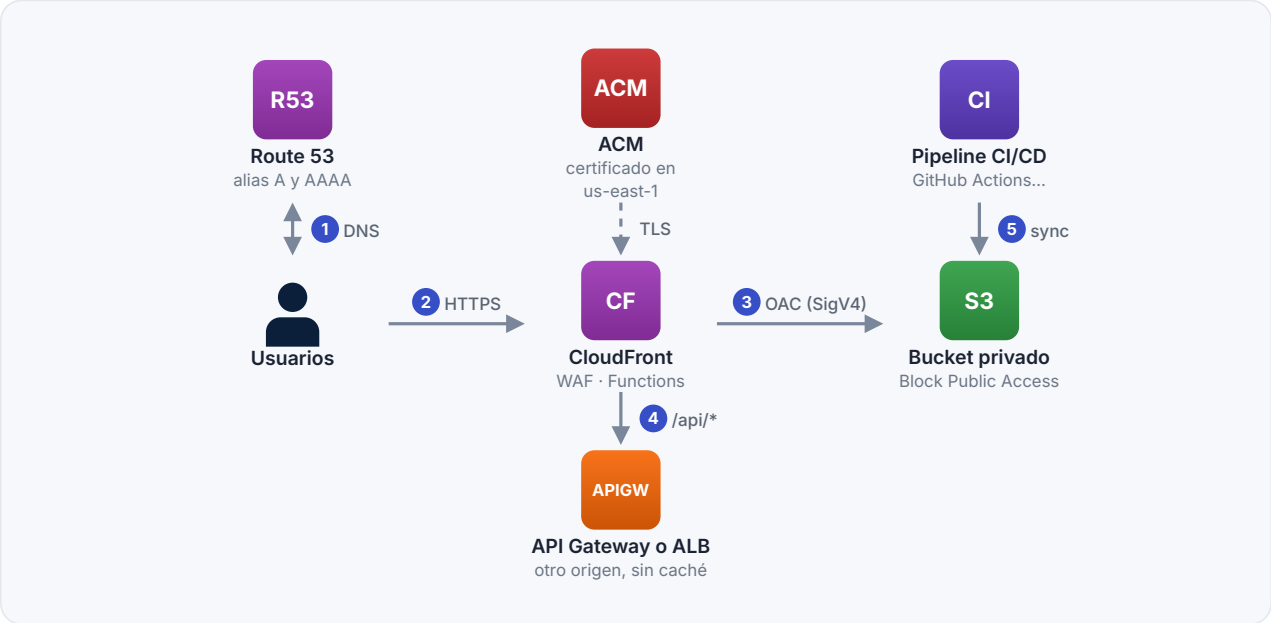


Figura 106 · Sitio estático: CloudFront sirve el contenido desde la caché del borde y lee de un bucket privado mediante OAC; las llamadas a la API van a otro origen.

1. **Route 53** responde con registros alias (A y AAAA) que apuntan a la distribución.
2. **CloudFront** termina HTTPS con un certificado de **ACM**, que para CloudFront debe estar en **us-east-1**; WAF filtra el tráfico y una **CloudFront Function** reescribe las rutas de la SPA (`/perfil` → `/index.html`) o añade cabeceras de seguridad.
3. Si el objeto no está en caché, CloudFront lo lee del **bucket privado** mediante **OAC**; el bucket sigue sin ningún acceso público.
4. Las peticiones `/api/*` usan otro *behavior* con otro origen (API Gateway o un ALB) y sin caché: una sola distribución y un solo dominio para el front-end y la API, así que el navegador no necesita CORS (capítulo de Serverless).
5. El pipeline publica con `aws s3 sync` y crea una invalidación (o, mejor, usa nombres de fichero con *hash* para no tener que invalidar).

VARIANTES

Si necesitas renderizado en servidor, **AWS Amplify Hosting**; con tráfico previsible, los planes de tarifa plana de CloudFront.

18.6 Pipeline de datos y analítica

Recoger eventos de aplicaciones, dispositivos o logs, guardarlos en un *data lake* barato y consultarlos con SQL y paneles, sin gestionar clústeres.

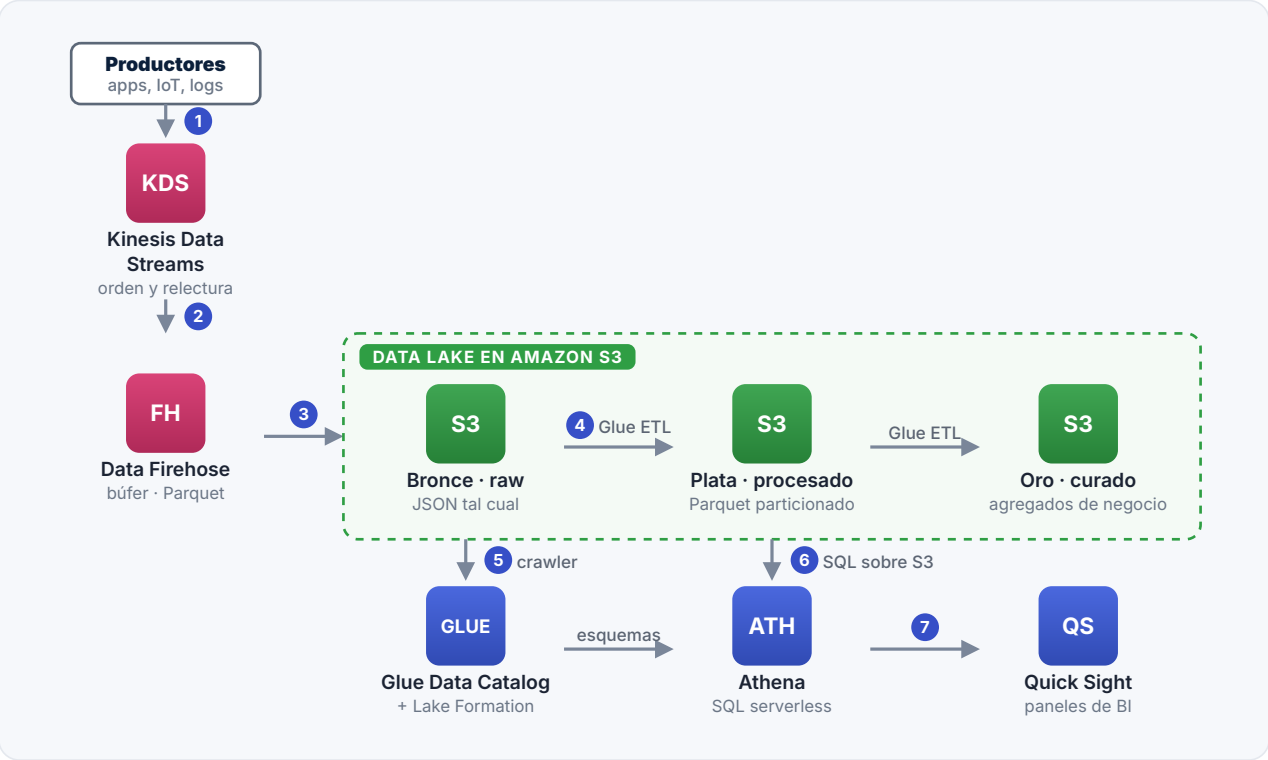


Figura 107 · Pipeline de analítica: ingesta en streaming, data lake en S3 por zonas (bronce, plata y oro), catálogo y transformación con Glue, consultas con Athena y paneles con Quick Sight.

1. Aplicaciones y dispositivos envían eventos a **Kinesis Data Streams**, que los ordena por clave de partición, admite varios consumidores y permite releer (24 h por defecto, hasta 365 días).
2. **Amazon Data Firehose** consume el stream, acumula por tamaño o tiempo, puede transformar con Lambda y convierte a **Parquet**.
3. Firehose escribe por lotes en la zona **bronce** (*raw*) del data lake: almacenamiento barato, duradero y separado del cómputo.
4. Trabajos de **AWS Glue** (Spark serverless) limpian, deduplican y escriben las zonas **plata** (procesada) y **oro** (curada) en formato columnar y particionado.
5. Un *crawler* registra esquemas y particiones en el **Glue Data Catalog**; **Lake Formation** da permisos por tabla, columna o fila.
6. **Athena** consulta con SQL directamente sobre S3 (pagas por datos escaneados: Parquet y particiones lo abaratan mucho).
7. **Amazon Quick Sight** (antes QuickSight) publica los paneles para negocio.

> **VARIANTES**
>
> Sin varios consumidores, las apps pueden escribir directamente en Firehose. Para tiempo real, **Managed Service for Apache Flink**; para un *data warehouse* muy concurrente, **Redshift**; las tablas **Apache Iceberg** (también en **S3 Tables**) añaden transacciones al lake.

18.7 Estrategia multicuenta con Organizations

Una sola cuenta para todo mezcla permisos, costes y riesgos. La práctica recomendada es una **landing zone**: muchas cuentas pequeñas, agrupadas por función y gobernadas desde un único punto.

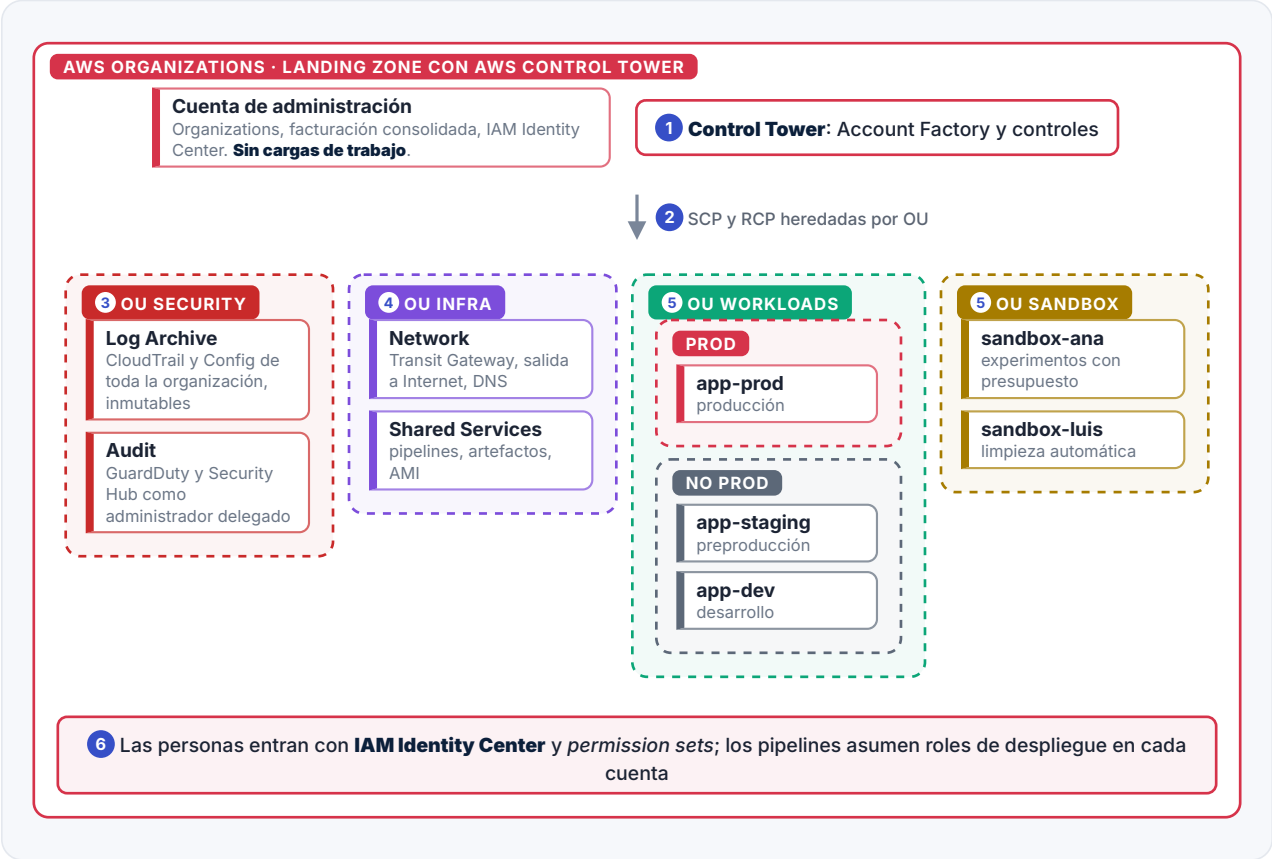


Figura 108 · Landing zone típica: cuenta de administración sin cargas de trabajo, cuentas agrupadas en OU con SCP heredadas, seguridad y red centralizadas, y una cuenta por aplicación y entorno.

1. La **cuenta de administración** solo gestiona Organizations, la facturación e Identity Center; **AWS Control Tower** monta la landing zone y crea cuentas configuradas con **Account Factory**.
2. Las cuentas se agrupan en **OU**. Las **SCP** (límites para las identidades) y las **RCP** (límites para los recursos) se heredan y fijan el máximo permitido: prohibir regiones, impedir desactivar CloudTrail…
3. La OU **Security** reúne **Log Archive**, que recibe los logs de toda la organización, y **Audit**, administradora delegada de los servicios de seguridad.
4. La OU **Infrastructure** («Infra») centraliza la red (Transit Gateway, salida a Internet, DNS) y los servicios compartidos.
5. Cada aplicación tiene una cuenta por entorno en **Workloads**, y los equipos experimentan en **Sandbox** con presupuesto y limpieza automática.
6. Nadie usa usuarios IAM: las personas entran con **IAM Identity Center** y *permission sets*; los pipelines, con roles.

VARIANTES

Las organizaciones grandes añaden OU como **Suspended** (cuentas en baja) o **Policy Staging**, o separan por unidad de negocio. Sin Control Tower, la misma estructura se monta con IaC (**Landing Zone Accelerator on AWS**, StackSets).

18.8 Recuperación ante desastres multirregión

Multi-AZ te protege de la caída de un centro de datos; para sobrevivir a la de una región entera necesitas una segunda región. El RPO, el RTO y las cuatro estrategias de recuperación (*backup & restore*,

pilot light, *warm standby* y *multi-site* activo-activo) se presentaron en el capítulo de Fundamentos. Esta figura aplica *warm standby* a la web de tres capas:

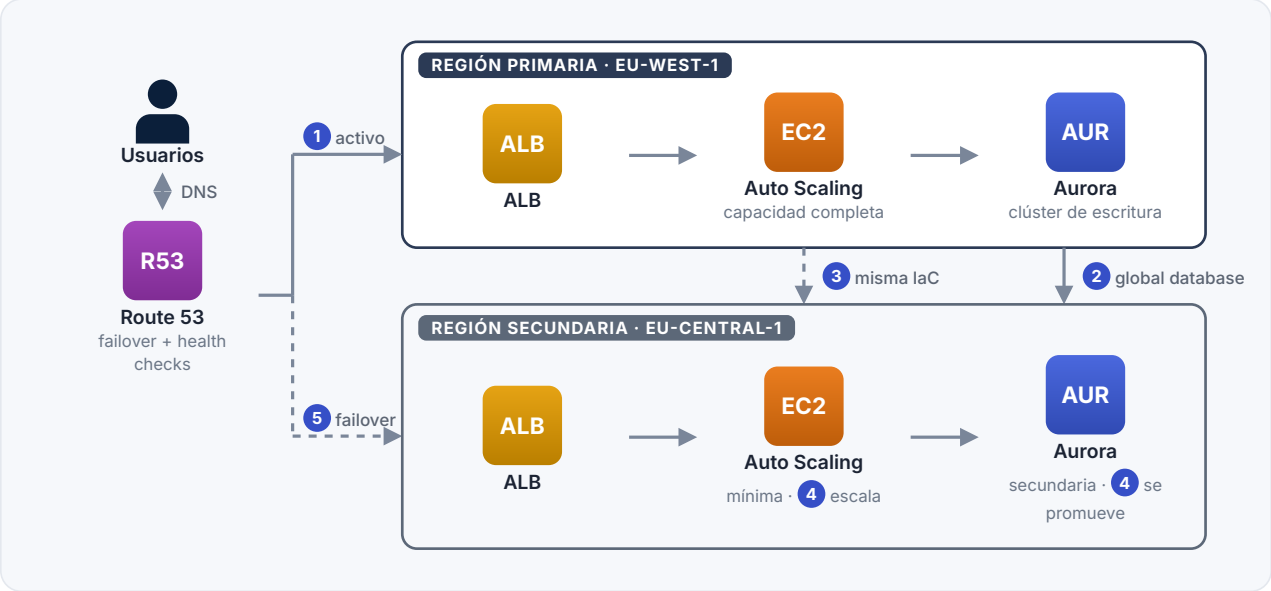


Figura 109 · *Warm standby* entre dos regiones: Route 53 envía el tráfico a la primaria, los datos se replican de forma continua y, ante un desastre, la secundaria se promueve, escala y recibe el tráfico.

1. En condiciones normales, **Route 53** envía el tráfico a la región primaria mediante un registro de *failover* con health check.
2. Los datos se replican de forma continua: **Aurora Global Database** (normalmente con menos de un segundo de retraso), **DynamoDB global tables**, **S3 Cross-Region Replication** y copias de **AWS Backup** entre regiones.
3. La región secundaria tiene la misma infraestructura, desplegada con la **misma IaC**, pero a escala mínima.
4. Ante un desastre, el clúster secundario de Aurora se **promueve** a escritura y el Auto Scaling group crece hasta la capacidad de producción.
5. Route 53 conmuta el DNS, automáticamente por health check o de forma controlada con **Application Recovery Controller**, y el tráfico pasa a la secundaria.

Con otra estrategia, la arquitectura es la misma; cambia lo que vive en la región secundaria:

- **Backup & restore**: solo copias de AWS Backup y la IaC; todo se crea al activar el plan.
- **Pilot light**: Aurora sigue replicando, pero el Auto Scaling group está a capacidad 0.
- **Multi-site**: las dos regiones sirven tráfico (Route 53 por latencia) y aceptan escrituras (DynamoDB global tables, Aurora DSQL).

Un plan de DR que no se ensaya no existe: programa simulacros y mide el RTO real. Y recuerda que la replicación también copia los errores; conserva además copias inmutables (AWS Backup Vault Lock).

18.9 Checklist de revisión Well-Architected

Antes de dar por buena una arquitectura, repásala con los seis pilares del **AWS Well-Architected Framework** (capítulo de Fundamentos), a mano o con la Well-Architected Tool:

Pilar	Comprueba que…
Excelencia operativa	✔ Todo se despliega con IaC y pipelines ✔ Despliegues pequeños, frecuentes y reversibles ✔ Alarmas accionables con *runbooks* ✔ Revisiones post-incidente sin culpables
Seguridad	✔ Multi-cuenta y acceso con Identity Center, sin claves de larga duración ✔ Mínimo privilegio y SCP ✔ Cifrado en reposo (KMS) y en tránsito (TLS) ✔ CloudTrail, GuardDuty y Security Hub activos; WAF en lo público
Fiabilidad	✔ Multi-AZ en todas las capas ✔ Auto Scaling y health checks ✔ Backups probados restaurándolos ✔ RPO y RTO definidos y DR ensayado ✔ Cuotas vigiladas
Eficiencia del rendimiento	✔ Servicios gestionados o serverless donde encajen ✔ Cachés (CloudFront, ElastiCache, DAX) ✔ Pruebas de carga ✔ Tipos de instancia y Graviton elegidos con datos
Optimización de costes	✔ Etiquetas, presupuestos y alertas ✔ Rightsizing y apagado de no-prod ✔ Savings Plans para la base ✔ Lifecycle en S3, retención de logs y endpoints en lugar de NAT
Sostenibilidad	✔ Capacidad ajustada a la demanda real ✔ Graviton y serverless ✔ Borrado de datos y recursos sin uso ✔ Regiones y horarios con menor impacto

RESUMEN DEL CAPÍTULO

- **Web de tres capas**: cada capa en dos AZ, con CloudFront y WAF en el borde, ALB, Auto Scaling y RDS Multi-AZ.
- **Microservicios**: ECS Fargate tras un ALB, Service Connect entre servicios y datos propios; EKS si ya usas Kubernetes.
- **API serverless**: Cognito, API Gateway, Lambda y DynamoDB, con pago por uso y escalado automático.
- **Asíncrono**: EventBridge, SQS con DLQ y SNS para el fan-out, con workers idempotentes.
- **Sitio estático**: CloudFront con un bucket privado vía OAC, certificado de ACM en us-east-1 y un *behavior* `/api/*` hacia la API.
- **Datos**: Kinesis y Firehose hacia un data lake en S3 por zonas, catalogado con Glue y consultado con Athena y Quick Sight.
- **Multi-cuenta**: landing zone con OU, SCP, Log Archive, Audit e Identity Center.
- **DR**: la estrategia de Fundamentos que pidan el RPO, el RTO y el presupuesto —aquí, *warm standby* con Aurora Global Database y failover de Route 53—, siempre ensayada.

Preguntas de repaso

1. **¿Qué hace que la web de tres capas sobreviva a la caída de una AZ?**
   Respuesta: Cada capa está en dos AZ: el ALB tiene nodos en ambas, el Auto Scaling group repone instancias y RDS Multi-AZ conmuta a su réplica síncrona.
2. **¿Para qué sirve ECS Service Connect?**
   Respuesta: Para que los servicios se llamen por nombre dentro del clúster, con descubrimiento, reintentos, balanceo y métricas, sin cambiar el código.
3. **En la arquitectura asíncrona, ¿qué pasa con un mensaje que falla una y otra vez?**
   Respuesta: Reaparece en la cola al vencer el visibility timeout y, al superar `maxReceiveCount`, SQS lo mueve a la DLQ, donde una alarma avisa para revisarlo.
4. **¿Cómo sirve una sola distribución de CloudFront el front-end estático y la API?**
   Respuesta: Con dos *behaviors*: el de por defecto lleva al bucket privado (con caché) y `/api/*`, a API Gateway o a un ALB, sin caché. Front-end y API comparten dominio, así que no hace falta CORS.
5. **¿Qué cambiarías en el *warm standby* para convertirlo en *pilot light*, y qué ganas y pierdes?**
   Respuesta: Dejar a capacidad 0 el Auto Scaling group de la región secundaria y mantener la réplica de Aurora. Pagas menos, pero el RTO crece: hay que arrancar y escalar las instancias antes de conmutar el tráfico.

APÉNDICE A

Glosario

Los términos y siglas que aparecen a lo largo de la guía, con una definición breve. Cada sigla empieza con su desarrollo en cursiva; dentro de una definición, la cursiva señala un término que tiene su propia entrada.

A

ABAC · *Attribute-Based Access Control*. Control de acceso basado en atributos: las políticas comparan etiquetas del principal y del recurso (`aws:PrincipalTag`, `aws:ResourceTag`) en lugar de enumerar recursos.
Access key · Clave de acceso (ID y secreto) de un *usuario IAM* para la CLI o los SDK. Credencial de larga duración: mejor *roles* o *IAM Identity Center*.
Account Factory · Función de *Control Tower* que crea cuentas nuevas ya configuradas según la *landing zone*.
ACL · *Access Control List*. En VPC, la *NACL* filtra el tráfico de una subred; en S3, las ACL son un mecanismo heredado, desactivado en los buckets nuevos.
ACM · *AWS Certificate Manager*. Emite y renueva certificados *TLS*; los públicos son gratuitos con servicios integrados como ELB, CloudFront o API Gateway.
ALB · *Application Load Balancer*. Balanceador de capa 7 (HTTP, HTTPS, gRPC, WebSocket) con reglas por ruta, host o cabeceras hacia *target groups*.
Alias · En *Route 53*, registro que apunta a un recurso de AWS (CloudFront, ELB…), incluso en el dominio raíz, sin coste por consulta. En Lambda y KMS, nombre de una versión o de una clave que se reasigna sin tocar a los clientes.
AMI · *Amazon Machine Image*. Plantilla para lanzar instancias EC2 (volumen raíz, permisos, mapeo de dispositivos); es regional, pero se puede copiar a otras.
API Gateway · Servicio para publicar API REST, HTTP y WebSocket con autorización, *throttling* y transformación de peticiones.
AppConfig · Configuración dinámica y *feature flags* con validación, despliegue gradual y reversión automática por alarmas.
ARN · *Amazon Resource Name*. Identificador único de un recurso, con la forma `arn:partición:servicio:región:cuenta:recurso`.
ASG · *Auto Scaling group*. Grupo de instancias EC2 con capacidad mínima, deseada y máxima que sustituye las no sanas y escala según políticas.
Assumed role · Sesión con credenciales temporales obtenida al asumir un *rol* (`sts:AssumeRole`); su ARN tiene la forma `arn:aws:sts::cuenta:assumed-role/rol/sesión`.
Athena · Consultas SQL serverless sobre datos guardados en S3; se paga por datos escaneados.
Aurora · Base de datos relacional compatible con MySQL y PostgreSQL, con almacenamiento en tres AZ; ofrece Serverless v2 (escala en ACU, incluso a 0), Global Database (multirregión) y DSQL (distribuida).
AZ · *Availability Zone*. Uno o más centros de datos con energía y red independientes dentro de una *región*. Su nombre (`eu-west-1a`) puede variar entre cuentas; su AZ ID (`euw1-az1`), no.

B

Backup (AWS Backup) · Copias de seguridad centralizadas de muchos servicios, con planes, bóvedas y copias entre cuentas y regiones.
Backup & restore · La estrategia de recuperación ante desastres más barata y lenta (RTO de horas): solo copias en otra región, y todo se recrea al activar el plan.
Bastión · Instancia de salto para llegar a recursos privados; hoy suele sustituirse por *Session Manager*.
Behavior · Regla de una distribución de CloudFront que asocia un patrón de ruta a un origen y a una política de caché.
Block Public Access · Ajuste de S3 que impide exponer públicamente buckets y objetos; activado por defecto en los buckets nuevos.
Blue/green · Despliegue que levanta la versión nueva completa en paralelo y cambia todo el tráfico de una vez, con vuelta atrás inmediata.
Bucket · Contenedor de objetos de S3, con nombre único global, ubicado en una región.
Bucket policy · Política de recurso de un bucket S3: quién puede acceder a él y en qué condiciones.
Budgets · Presupuestos de coste o uso con alertas y acciones si el gasto real o previsto supera un umbral.

C

Canary · Despliegue que envía primero un pequeño porcentaje del tráfico a la versión nueva y, si las métricas son buenas, el resto.
CDK · *AWS Cloud Development Kit*. Define infraestructura con TypeScript, Python, Java, C#, Go o JavaScript a partir de piezas llamadas constructs; genera plantillas de *CloudFormation*.
Change set · Vista previa de los cambios que CloudFormation aplicará a un *stack*, incluido si algún recurso se reemplazará.
CIDR · Notación de rangos IP con prefijo (`10.0.0.0/16`): cuanto mayor es el prefijo, menor es el rango.
CloudFormation · IaC nativa de AWS: plantillas YAML o JSON que crean *stacks*; las funciones intrínsecas (`!Ref`, `!GetAtt`…) calculan valores al desplegar.
CloudFront · Red de entrega de contenidos (CDN) de AWS, con cientos de *edge locations* en todo el mundo.
CloudFront Functions · Funciones JavaScript muy ligeras que se ejecutan en el borde para reescribir URL, redirigir o añadir cabeceras.

CloudHSM · *HSM* dedicados y de un solo inquilino en tu VPC, con control exclusivo de las claves; puede servir a *KMS* como almacén de claves personalizado.
CloudTrail · Auditoría de las llamadas a la API. Guarda 90 días de eventos de gestión sin configurar nada; un trail los entrega a S3 para conservarlos más tiempo.
CloudWatch · Métricas, logs, alarmas y paneles para recursos y aplicaciones.
CodeBuild · Compilación y pruebas gestionadas: ejecuta un `buildspec.yml` en contenedores efímeros.
CodeCommit · Repositorios Git gestionados de AWS. Cerró a nuevos clientes en julio de 2024 y volvió a abrirse a ellos el 25 de noviembre de 2025.
CodeDeploy · Automatiza despliegues en EC2, servidores on-premises, Lambda y ECS: in-place, blue/green, canary o lineal.
CodePipeline · Orquesta el CI/CD por etapas: origen, compilación, pruebas, aprobación y despliegue.
Cognito · Identidad para los usuarios de tus aplicaciones: user pools (registro e inicio de sesión) e identity pools (credenciales de AWS).
Cold start · Arranque en frío: latencia extra cuando Lambda crea un entorno de ejecución nuevo; se mitiga con *SnapStart* o *provisioned concurrency*.
Compute Optimizer · Recomendaciones de tamaño para EC2, Auto Scaling, EBS, Lambda, ECS y RDS basadas en el uso real.
Config (AWS Config) · Inventario e historial de la configuración de los recursos, con reglas que evalúan su cumplimiento.
Confused deputy · «Ayudante confundido»: alguien engaña a un servicio o a un tercero con permisos legítimos para que actúe sobre recursos ajenos; lo evitan el *External ID* y condiciones como `aws:SourceArn`.
Control plane · Plano de control de Kubernetes (API server, etcd, scheduler…), que gestiona el clúster; en EKS lo opera AWS.
Control Tower · Monta y gobierna una *landing zone* con controles preventivos, detectivos y proactivos.
Cost Explorer · Análisis y previsión del gasto por servicio, cuenta, región o etiqueta.
Cuenta de administración · *Management account*. La que crea la organización en *Organizations*, paga la factura consolidada y gestiona las políticas; las *SCP* no la limitan, así que no aloja cargas de trabajo.
Cuota de servicio · Límite de recursos o peticiones por cuenta y región; muchas se pueden ampliar en Service Quotas.
CUR · *Cost and Usage Report*. Detalle línea a línea de la factura; hoy se obtiene con Data Exports (CUR 2.0 o FOCUS).

D

Data key · Clave de datos: clave simétrica que genera *KMS* (`GenerateDataKey`) para cifrar datos en local; se guarda cifrada junto a ellos.
Data lake · Repositorio central, normalmente en S3, con datos en bruto y procesados listos para analítica.
DAX · *DynamoDB Accelerator*. Caché en memoria compatible con la API de DynamoDB, con latencias de microsegundos.
DeletionPolicy · Atributo de CloudFormation que decide si un recurso se borra, se conserva o se guarda en un snapshot al eliminar el stack.
Deployment · Objeto de Kubernetes que mantiene N réplicas de un *pod* sin estado y las actualiza poco a poco (para aplicaciones con estado, StatefulSet; un pod por *nodo*, DaemonSet).
Direct Connect · Conexión de red dedicada y privada entre tu centro de datos y AWS.
Distribución · Configuración de CloudFront con sus orígenes, *behaviors*, certificados y dominios.
DLQ · *Dead-letter queue*. Cola que recibe los mensajes que no se han podido procesar tras varios intentos, para analizarlos aparte.
Drift · Diferencia entre la configuración declarada en la IaC y la real, casi siempre por cambios manuales.
DynamoDB · Base de datos NoSQL clave-valor y de documentos, serverless, con latencia de milisegundos a cualquier escala e ítems de hasta 400 KB. Las global tables la replican en varias regiones.
DynamoDB Streams · Registro ordenado de los cambios de una tabla durante 24 horas, que puede disparar funciones Lambda.

E

EBS · *Elastic Block Store*. Volúmenes de bloque persistentes para EC2, ligados a una AZ (gp3, io2…).
EC2 · *Elastic Compute Cloud*. Máquinas virtuales (instancias) de multitud de tipos y tamaños.
ECR · *Elastic Container Registry*. Registro privado y público de imágenes de contenedor.
ECS · *Elastic Container Service*. Orquestador de contenedores de AWS: ejecuta tareas y servicios sobre *Fargate* o EC2.
Edge location · Punto de presencia de la red de AWS donde CloudFront cachea contenido y Route 53 responde consultas.
EFS · *Elastic File System*. Sistema de ficheros NFS gestionado y elástico, compartible entre muchas instancias y AZ.
EKS · *Elastic Kubernetes Service*. Kubernetes gestionado: AWS opera el plano de control y, con Auto Mode, también los nodos.
EKS Pod Identity · Forma de dar un rol IAM a los pods de EKS mediante asociaciones y un agente en el nodo; más sencilla que *IRSA*.
Elastic IP · IPv4 pública estática reservada para tu cuenta (a veces abreviada EIP); como toda IPv4 pública, se factura por hora.
ElastiCache · Caché en memoria gestionada compatible con Valkey, Redis OSS y Memcached.
ELB · *Elastic Load Balancing*. Familia de balanceadores: *ALB*, *NLB*, *GWLB* y el Classic, heredado.
Encryption context · Pares clave-valor no secretos vinculados a una operación de *KMS*: hay que repetirlos al descifrar y quedan registrados en CloudTrail.
Endpoint de VPC · Acceso privado a servicios de AWS sin pasar por Internet: de gateway (S3 y DynamoDB, gratuitos) o de interfaz (*PrivateLink*).
ENI · *Elastic Network Interface*. Interfaz de red con IP privada, *Security Groups* y, opcionalmente, IP pública.

Envelope encryption · Cifrado envolvente: los datos se cifran con una *data key* y esta, con una *KMS key*; así los datos nunca viajan a KMS.
Event source mapping · Recurso de Lambda que lee de SQS, Kinesis o DynamoDB Streams e invoca la función con lotes de registros.
EventBridge · Bus de eventos serverless que enruta eventos de AWS, de SaaS y de tus aplicaciones mediante reglas; incluye un programador (Scheduler).
Execution role · En Lambda, el rol que asume la función; en ECS, el del agente, que descarga la imagen, lee secretos y envía logs (el código usa el *task role*).
External ID · Valor que un tercero debe enviar al asumir un rol en tu cuenta; protege del *confused deputy*.

F

Failover · Conmutación por error: el servicio pasa automáticamente a una réplica o a otra región cuando falla la principal (RDS Multi-AZ, registros de failover de Route 53).
Fan-out · Patrón en el que un mensaje se replica a varios consumidores, por ejemplo un tema SNS suscrito por varias colas SQS.
Fargate · Motor serverless para contenedores en ECS y EKS: pagas por la vCPU y la memoria de cada tarea, sin gestionar instancias.
Feature flag · Interruptor que activa una funcionalidad sin redesplegar, por ejemplo con *AppConfig*.
Federación · Acceso con identidades de un proveedor externo (IdP) mediante *SAML* u *OIDC*, sin crear *usuarios IAM*: el IdP autentica y AWS entrega credenciales temporales.
FIFO (cola) · Cola de SQS que conserva el orden dentro de cada grupo de mensajes y descarta duplicados (procesamiento exactamente una vez).
FinOps · Práctica de gestionar el coste de la nube de forma compartida entre ingeniería, finanzas y negocio.
Firehose · *Amazon Data Firehose* (antes Kinesis Data Firehose). Entrega datos en streaming a S3, Redshift u OpenSearch, con búfer y transformación.

G

Glacier · Clases de archivo de S3 (Instant Retrieval, Flexible Retrieval y Deep Archive), muy baratas y con distintos tiempos de recuperación.
Global Accelerator · Red global de AWS con IP anycast estáticas que acelera el tráfico TCP/UDP y conmuta entre regiones.
Glue · Integración de datos serverless: catálogo, crawlers que descubren esquemas y ETL sobre Spark.
Grant · Permiso puntual sobre una *KMS key* para un *principal* concreto, que se crea y revoca por API sin tocar la *key policy*; los servicios lo usan para cifrar y descifrar en tu nombre.
Graviton · Procesadores Arm de AWS, con mejor relación precio-rendimiento en EC2, Lambda, Fargate, RDS y otros servicios.
Grupo IAM · Conjunto de *usuarios IAM* que heredan las políticas del grupo. No es un *principal*: no puede asumir roles ni aparecer en una política de recurso.
GSI y LSI · Índices secundarios de DynamoDB: el global (GSI) usa otra clave de partición y se crea en cualquier momento; el local (LSI) comparte la de la tabla y solo se define al crearla.
GuardDuty · Detección de amenazas con aprendizaje automático sobre CloudTrail, VPC Flow Logs, DNS y otras fuentes.
GWLB · *Gateway Load Balancer*. Inserta de forma transparente appliances de red de terceros (cortafuegos, IDS).

H

Health check · Comprobación periódica del estado de un destino (ELB, Route 53, Auto Scaling) para dejar de enviarle tráfico si falla.
Helm · Gestor de paquetes de Kubernetes: instala aplicaciones como charts parametrizables.
Hosted zone · Zona alojada: contenedor de los registros DNS de un dominio en Route 53, pública o privada (asociada a VPC).
HSM · *Hardware Security Module*. Dispositivo resistente a manipulaciones que genera y guarda claves y opera con ellas por dentro; *KMS* y *CloudHSM* usan HSM validados FIPS 140.

I

IaC · Infraestructura como código: definir la infraestructura en ficheros versionados que una herramienta aplica de forma repetible.
IAM · *Identity and Access Management*. Servicio global que controla quién puede hacer qué sobre qué recursos.
IAM Access Analyzer · Detecta recursos compartidos fuera de tu cuenta u organización, valida políticas y genera las de *mínimo privilegio* según la actividad real.
IAM Identity Center · Acceso centralizado de las personas a varias cuentas y aplicaciones mediante *permission sets*; es el antiguo AWS SSO.
Idempotencia · Propiedad de una operación que da el mismo resultado aunque se repita; imprescindible con entregas «al menos una vez».
IGW · *Internet Gateway*. Conecta una VPC con Internet; una subred es pública si su tabla de rutas apunta a él.
IMDS · *Instance Metadata Service*. En `169.254.169.254`, da a la instancia sus metadatos y credenciales temporales; IMDSv2 exige un token de sesión.
Ingress · Reglas HTTP de entrada a los servicios de un clúster de Kubernetes; en EKS, el AWS Load Balancer Controller las convierte en un ALB.
Inline policy · Política IAM incrustada en un solo usuario, grupo o rol, que se borra con él; frente a ella, la *managed policy* es reutilizable.
Inspector · Escaneo continuo de vulnerabilidades en instancias EC2, imágenes de ECR y funciones Lambda.
Instance profile · Contenedor que da un *rol* IAM a una instancia EC2 para que obtenga credenciales temporales.
Instance store · Discos locales de algunos tipos de instancia EC2, muy rápidos pero efímeros: sus datos se pierden al parar o terminar la instancia.

Invalidación · Orden para que CloudFront descarte objetos de su caché antes de que expire su *TTL*.
IOPS · Operaciones de entrada/salida por segundo de un volumen de almacenamiento.
IRSA · *IAM Roles for Service Accounts*. Asocia roles IAM a cuentas de servicio de Kubernetes mediante el proveedor *OIDC* del clúster de EKS.

J

JWT · *JSON Web Token*. Token firmado con afirmaciones (claims) sobre un usuario; Cognito los emite y API Gateway los valida.

K

Karpenter · Autoescalador de nodos de Kubernetes, de código abierto, que lanza la instancia más adecuada para los pods pendientes; EKS Auto Mode lo integra.
Key policy · Política de recurso de una *KMS key*: cada clave tiene exactamente una y es la base de su acceso.
Kinesis Data Streams · Streaming en tiempo real, ordenado por clave de partición, con varios consumidores y relectura; su capacidad se mide en *shards*.
KMS · *Key Management Service*. Crea y gestiona claves de cifrado y las usa desde los servicios de AWS, con cada uso auditado en CloudTrail.
KMS key · Clave lógica de KMS (antes CMK) cuyo material nunca sale del servicio sin cifrar: AWS owned (del servicio), AWS managed (`aws/servicio`) o customer managed (la controlas tú).
kubelet · Agente que se ejecuta en cada *nodo* de Kubernetes y arranca y vigila los contenedores de los pods asignados.
Kubernetes · Orquestador de contenedores de código abierto; en AWS se ofrece gestionado con *EKS*.

L

Lake Formation · Gobierno de *data lakes* en S3 con permisos finos por base de datos, tabla, columna y fila.
Lambda · Funciones serverless que se ejecutan por eventos, hasta 15 minutos y 10 240 MB de memoria, con pago por uso.
Lambda@Edge · Funciones Lambda que se ejecutan en las ubicaciones regionales de CloudFront para personalizar peticiones y respuestas.
Landing zone · Entorno multicuenta bien diseñado (cuentas, OU, políticas y logs centralizados) desde el que crecer de forma segura.
Launch template · Plantilla versionada con la configuración de lanzamiento de instancias (AMI, tipo, red, *user data*) que usa Auto Scaling.
Lifecycle (S3) · Reglas que mueven objetos a clases más baratas o los eliminan según su antigüedad.
Listener · Proceso de un balanceador que escucha en un puerto y protocolo y aplica reglas para reenviar a *target groups*.
Log group · Agrupación de flujos de logs de CloudWatch Logs con su retención, sus permisos y sus filtros; se consulta con Logs Insights.
Long polling · Recepción de mensajes de SQS que espera hasta 20 segundos a que lleguen, lo que reduce las peticiones vacías.

M

Macie · Descubre datos sensibles (como datos personales) en S3 mediante aprendizaje automático.
Managed policy · Política IAM independiente y reutilizable, creada por AWS o por el cliente, que se adjunta a varias identidades.
MFA · Autenticación multifactor: un segundo factor (app TOTP, llave de seguridad, passkey) además de la contraseña.
Mínimo privilegio · Principio de conceder solo los permisos imprescindibles para cada tarea.
Modelo de responsabilidad compartida · AWS protege la infraestructura de la nube; el cliente, lo que pone en ella: datos, configuración, identidades y aplicaciones.
Multi-AZ · Despliegue repartido en varias zonas de disponibilidad para tolerar la caída de una de ellas.
Multi-site · Estrategia de recuperación ante desastres activo-activo: dos o más regiones sirven tráfico real a la vez, con RTO y RPO casi nulos y el mayor coste.
Multipart upload · Subida a S3 en partes enviadas en paralelo; obligatoria por encima de 5 GB.

N

NACL · *Network ACL*. Cortafuegos sin estado a nivel de subred, con reglas numeradas de permitir y denegar.
Namespace · En Kubernetes, partición lógica de un clúster para aislar recursos, cuotas y permisos; en CloudWatch, contenedor de métricas (`AWS/EC2`…).
NAT Gateway · Permite que los recursos de subredes privadas salgan a Internet sin ser accesibles desde fuera; se paga por hora y por GB procesado.
Nested stack · Stack de CloudFormation incluido como recurso dentro de otro para reutilizar componentes.
NLB · *Network Load Balancer*. Balanceador de capa 4 (TCP, UDP, TLS) de latencia muy baja, con IP estática por AZ.
Nodo · Máquina de un clúster de Kubernetes (instancia EC2 o capacidad de Fargate) en la que corren los pods.

O

OAC · *Origin Access Control*. Mecanismo con el que CloudFront firma sus peticiones a un bucket S3 privado; sustituye a OAI.
OIDC · *OpenID Connect*. Estándar de identidad sobre OAuth 2.0; permite a GitHub Actions o a EKS obtener credenciales de AWS sin claves.
On-Demand · Modelo de compra de pago por uso, sin compromiso.
Organizations · Gestión centralizada de varias cuentas desde la *cuenta de administración*, con *OU*, facturación consolidada y políticas como las *SCP* y *RCP*.
Origen · Servidor del que CloudFront obtiene el contenido: un bucket S3, un ALB, API Gateway o cualquier servidor HTTP.
OU · *Organizational Unit*. Grupo de cuentas de Organizations; sus políticas se heredan.

P

Parameter Store · Almacén jerárquico de configuración y secretos de Systems Manager.

Partition key · Clave de partición: atributo con el que DynamoDB o Kinesis reparten los datos.
Peering de VPC · Conexión privada entre dos VPC, incluso de cuentas o regiones distintas; no es transitiva.
Permission set · Plantilla de permisos de IAM Identity Center que se materializa como un *rol* en cada cuenta asignada.
Permissions boundary · Límite de permisos: política gestionada que fija el máximo que puede tener un usuario o rol, aunque sus políticas concedan más.
Pilot light · Estrategia de recuperación ante desastres con datos replicados e infraestructura mínima en otra región, sin servidores de aplicación en marcha.
Placement group · Colocación de instancias EC2: `cluster` (baja latencia), `spread` (separadas) o `partition`.
Pod · Unidad mínima de Kubernetes: uno o varios contenedores que comparten red y almacenamiento.
Política de recurso · Política de un recurso (bucket, cola, *KMS key*, rol) que indica qué *principals* pueden usarlo.
Política IAM · Documento JSON con declaraciones `Effect`, `Action`, `Resource` y `Condition` que concede o deniega permisos.
Principal · Quien hace una petición a AWS: usuario, rol, servicio, cuenta o identidad federada; en una política de recurso, a quién se aplica.
PrivateLink · Expone y consume servicios de forma privada mediante endpoints de interfaz en tu VPC.
Provisioned concurrency · Entornos de Lambda preinicializados que eliminan el *cold start* a cambio de un coste fijo.

Q

Quick Sight · Servicio de BI y paneles de AWS (antes QuickSight).

R

RCP · *Resource Control Policy*. Política de Organizations (2024) que limita los permisos máximos sobre los recursos de las cuentas, sea quien sea el solicitante.
RCU y WCU · Unidades de capacidad aprovisionada de DynamoDB: 1 RCU da una lectura fuerte por segundo de hasta 4 KB (o dos eventuales); 1 WCU, una escritura por segundo de hasta 1 KB.
RDS · *Relational Database Service*. Bases de datos relacionales gestionadas: PostgreSQL, MySQL, MariaDB, Oracle, SQL Server y Db2.
Read replica · Réplica de lectura: copia asíncrona de una base de datos para escalar lecturas o como base de recuperación.
Redshift · *Data warehouse* columnar gestionado para analítica a gran escala, también en versión serverless.
Región · Área geográfica con varias AZ aisladas entre sí; la mayoría de servicios son regionales.
Reserved Instance · Descuento a cambio de comprometerse a usar una configuración concreta durante 1 o 3 años.
Rightsizing · Ajustar el tamaño de los recursos a su uso real.
Rol IAM · Identidad sin credenciales permanentes que asumen usuarios, servicios o cuentas para obtener credenciales temporales.
Rolling · Despliegue por lotes en el que la versión antigua y la nueva conviven durante el proceso.
Root user · Usuario root (o raíz): identidad creada con la cuenta, con acceso total. Protégelo con MFA y no lo uses en el día a día.
Route 53 · DNS gestionado de AWS, con registro de dominios, *health checks* y políticas de enrutamiento.
Route table · Tabla de rutas: reglas que deciden a dónde se envía el tráfico que sale de una subred.
RPO y RTO · *Recovery Point Objective* (cuántos datos se pueden perder) y *Recovery Time Objective* (cuánto tiempo sin servicio) ante un desastre.

S

S3 · *Simple Storage Service*. Almacenamiento de objetos diseñado para once nueves de durabilidad, con clases según coste y frecuencia de acceso (Standard, Standard-IA, Intelligent-Tiering, *Glacier*...).
SAM · *Serverless Application Model*. Extensión de CloudFormation y CLI para aplicaciones serverless.
SAML · Estándar de *federación* basado en XML: el proveedor de identidad corporativo (Entra ID, Okta...) autentica a la persona y AWS o IAM Identity Center confían en su aserción.
Savings Plans · Descuento a cambio de un compromiso de gasto por hora durante 1 o 3 años según el tipo (Compute, EC2 Instance, Database...).
SCP · *Service Control Policy*. Política de Organizations que fija los permisos máximos de las identidades de las cuentas afectadas; no concede permisos.
Secrets Manager · Secretos (contraseñas, claves de API) cifrados con KMS y con rotación automática.
Security Group · Cortafuegos con estado a nivel de interfaz de red; solo admite reglas de permitir.
Security Hub · Panel central de hallazgos de seguridad (findings) y de comprobaciones de cumplimiento frente a estándares.
Serverless · Modelo en el que no gestionas servidores, la capacidad escala sola y pagas por uso.
Service (Kubernetes) · Punto de acceso estable (IP y nombre DNS) a un conjunto de pods; puede exponerse con un balanceador.
Service Connect · Función de ECS para que los servicios se descubran y se comuniquen por nombre, con reintentos y métricas.
Service-linked role · Rol predefinido vinculado a un servicio de AWS, que el servicio crea y usa para actuar en tu nombre; solo él puede asumirlo.
Session Manager · Función de Systems Manager para abrir sesiones de shell en instancias sin SSH ni bastión, con auditoría.
Session policy · Política que se pasa al asumir un rol o al federarse y que recorta los permisos de esa sesión; nunca los amplía.
Shard · Unidad de capacidad y de orden de Kinesis Data Streams: cada uno admite 1 MB/s (o 1000 registros/s) de escritura y 2 MB/s de lectura.
Shield · Protección contra DDoS: Standard, gratuita y automática, y Advanced, de pago, con equipo de respuesta y protección de costes.

Snapshot · Copia puntual e incremental de un volumen EBS o de una base de datos.
SnapStart · Optimización de Lambda que restaura una instantánea del entorno ya inicializado para reducir el *cold start*.
SNS · *Simple Notification Service*. Mensajería pub/sub: un mensaje publicado en un tema llega a todos sus suscriptores.
Sort key · Segundo atributo de la clave primaria de DynamoDB: ordena los ítems de una partición.
Spot · Capacidad sobrante de EC2, muy barata, que AWS puede recuperar con 2 minutos de aviso.
SQS · *Simple Queue Service*. Colas de mensajes gestionadas, estándar o *FIFO*, para desacoplar productores y consumidores.
SSE · Cifrado del lado del servidor: el servicio cifra los datos en reposo. En S3, SSE-S3 se aplica por defecto, SSE-KMS usa claves de KMS y las Bucket Keys reducen sus llamadas a KMS.
Stack · Conjunto de recursos de CloudFormation que se crean, actualizan y borran como una unidad.
StackSets · Despliegan el mismo stack de CloudFormation en varias cuentas y regiones a la vez.
Stateful y stateless · Un cortafuegos con estado (*Security Group*) recuerda cada conexión y deja pasar sola la respuesta; uno sin estado (*NACL*) evalúa cada paquete y necesita reglas en ambos sentidos.
Step Functions · Orquestación serverless de flujos de trabajo definidos como máquinas de estados.
STS · *Security Token Service*. Emite credenciales temporales (AssumeRole, federación) para identidades de AWS.
Subred · Rango de IP de una VPC ubicado en una sola AZ; es pública si su tabla de rutas apunta a un *IGW*.
Systems Manager · Conjunto de herramientas para operar flotas: *Session Manager*, *Parameter Store*, Patch Manager, Run Command…

T

Tag · Etiqueta: par clave-valor de un recurso para organizarlo, controlar el acceso (*ABAC*) y asignar costes.
Target group · Conjunto de destinos (instancias, IP, funciones Lambda) a los que un balanceador envía tráfico y cuya salud comprueba.
Task definition · Plantilla de ECS que describe los contenedores de una tarea: imagen, CPU, memoria, puertos, roles y logs.
Task role · Rol IAM que usa el código de una tarea de ECS para llamar a otros servicios; no confundir con el *execution role*.
Terraform · IaC multiproveedor de HashiCorp, con lenguaje HCL y un fichero de estado.
Throttling · Limitación de la tasa de peticiones: por encima, el servicio responde con errores (HTTP 429).
TLS · *Transport Layer Security*. Protocolo que cifra y autentica las comunicaciones: la base de HTTPS.
Transit Gateway · Hub regional que interconecta VPC y redes on-premises de forma transitiva.
Trust policy · Política de confianza: política de recurso de un *rol* que indica qué *principals* pueden asumirlo.
Trusted Advisor · Comprobaciones automáticas de buenas prácticas de coste, seguridad, rendimiento, tolerancia a fallos y cuotas.
TTL · *Time To Live*. Tiempo que un dato puede conservarse en caché (DNS, CloudFront) o tras el que expira (DynamoDB).

U

UpdateReplacePolicy · Atributo de CloudFormation que decide qué hacer con el recurso antiguo cuando una actualización lo reemplaza.
URL prefirmada · URL temporal, firmada con las credenciales de quien la genera, para descargar o subir un objeto concreto de S3 sin abrir el bucket.
User data · Script que se ejecuta al arrancar por primera vez una instancia EC2.
Usuario IAM · Identidad con credenciales de larga duración (contraseña o *access keys*); hoy se prefieren *roles* e *IAM Identity Center*.

V

Versionado (S3) · Conserva todas las versiones de cada objeto para recuperar borrados y sobrescrituras.
Visibility timeout · Tiempo durante el que un mensaje de SQS recibido queda oculto al resto de consumidores; si no se borra, vuelve a la cola.
VPC · *Virtual Private Cloud*. Red virtual aislada, dentro de una región, en la que despliegas tus recursos.
VPC Flow Logs · Registros con los metadatos del tráfico IP de una VPC, subred o interfaz, enviados a CloudWatch Logs, S3 o Firehose.
VPN (Site-to-Site) · Túneles IPsec cifrados sobre Internet entre tu red y una VPC o un Transit Gateway.

W

WAF · *AWS WAF*. Cortafuegos web para CloudFront, ALB, API Gateway y otros: una web ACL reúne reglas propias o gestionadas y límites de tasa.
Warm standby · Estrategia de recuperación ante desastres con una copia completa funcionando a escala reducida en otra región.
Well-Architected · Marco de buenas prácticas de AWS en seis pilares: excelencia operativa, seguridad, fiabilidad, eficiencia del rendimiento, optimización de costes y sostenibilidad.

X

X-Ray · Trazas distribuidas para seguir una petición a través de varios servicios y localizar latencias y errores; también admite OpenTelemetry (ADOT).

Z

Zona de disponibilidad · Véase *AZ*.

APÉNDICE B

Chuleta de referencia rápida

Cifras que conviene tener a mano, los comandos de AWS CLI más útiles y una guía rápida para elegir servicio.

Cifras y límites clave

Valores por defecto o máximos vigentes al redactar esta guía (2026). Un asterisco (*) indica una cuota **ajustable** desde Service Quotas; «car.» son caracteres. Antes de diseñar al límite, comprueba la documentación.

Lambda

Duración máxima	**15 min**
Memoria	**128–10 240 MB**
/tmp efímero	**512 MB–10 GB**
Payload síncrono	**6 MB**
Payload asíncrono	**1 MB**
Zip / sin comprimir	**50 / 250 MB**
Imagen de contenedor	**10 GB**
Concurrencia	**1000***

SQS

Mensaje	**1 MiB**
Retención	**1 min–14 días**
… por defecto	**4 días**
Visibility timeout	**0 s–12 h**
… por defecto	**30 s**
Long polling	**hasta 20 s**
Retardo (delay)	**hasta 15 min**
FIFO sin lotes	**300 msg/s**

DynamoDB

Ítem	**400 KB**
1 RCU: lectura fuerte/s	**4 KB**
1 WCU: escritura/s	**1 KB**
Query / Scan	**1 MB/página**
BatchWrite / Get	**25 / 100 ítems**
Transacción	**100 acciones**
GSI / LSI	**20* / 5**
Streams	**24 h**

S3

Objeto máximo	**50 TB**
Antes de dic. 2025	**5 TB**
Un solo PUT	**5 GB**
Multipart	**10 000 partes**
Tamaño de parte	**5 MiB–5 GiB**
Escrituras/prefijo	**3500/s**
Lecturas/prefijo	**5500/s**
Buckets por cuenta	**10 000***

VPC

CIDR de VPC y subred	**/16 a /28**
IP reservadas/subred	**5**
VPC por región	**5***
SG por ENI	**5***
Reglas por SG	**60 + 60***
Reglas por NACL	**20 por sentido***
Peering	**no transitivo**

EC2, EBS y ELB

Aviso de Spot	**2 min**
Linux: por segundo	**mín. 60 s**
IMDS	**169.254.169.254**
gp3: IOPS base	**3000**
gp3: rendimiento base	**125 MiB/s**
ALB idle timeout	**60 s**
Deregistration	**300 s**

IAM y Organizations

Usuarios IAM	**5000**
Grupos por usuario	**10**
Sesión de un rol	**1 h (máx. 12 h)**
Roles encadenados	**máx. 1 h**
Política gestionada	**6144 car.**
SCP por destino	**5**
Tamaño de SCP	**5120 car.**
Niveles de OU	**5**

Observabilidad

CloudTrail: historial	**90 días**
Métricas: retención	**15 meses**
Resolución estándar	**1 min**
Alta resolución	**1 s**
EC2 básico / detall.	**5 / 1 min**
CloudWatch Logs	**sin caducidad**

KMS

Encrypt directo	**4 KB**
Espera de borrado	**7–30 días**
… por defecto	**30 días**
Rotación automática	**anual**
Configurable	**90–2560 días**

API Gateway y streaming

API GW: timeout	**29 s***
API GW: payload	**10 MB**
Step Functions Std.	**1 año**
Step Functions Expr.	**5 min**
Kinesis: retención	**24 h–365 días**
Shard: escritura	**1 MB/s**
Shard: lectura	**2 MB/s**

RDS y Aurora

Backups automáticos	**hasta 35 días**
Réplicas de lectura	**hasta 15**
Failover Multi-AZ	**1–2 min**
Copias de Aurora	**6 en 3 AZ**
Global Database	**< 1 s típico**

Borde y costes

Cert. de CloudFront	**ACM, us-east-1**
TTL por defecto (CF)	**24 h**
Alias de Route 53	**gratis**
Salida gratis	**100 GB/mes**
IPv4 pública	**por hora**

Comandos de AWS CLI más útiles

BASH

```
# Identidad y perfiles
aws configure sso                                # perfil con IAM Identity Center
aws sso login --profile prod
aws sts get-caller-identity                      # ¿con qué identidad estoy?

# EC2 y Systems Manager
aws ec2 describe-instances --filters Name=instance-state-name,Values=running \
  --query 'Reservations[].Instances[].[InstanceId,InstanceType]' --output table
aws ssm start-session --target i-0123456789abcdef0     # shell sin SSH ni bastión

# S3
aws s3 ls s3://mi-bucket/datos/ --recursive --human-readable --summarize
aws s3 sync ./dist s3://mi-web --delete
aws s3 presign s3://mi-bucket/informe.pdf --expires-in 3600   # URL temporal

# Lambda y CloudWatch Logs
aws lambda invoke --function-name procesador \
  --cli-binary-format raw-in-base64-out --payload '{"id": 42}' salida.json
aws logs tail /aws/lambda/procesador --follow --since 15m
```

BASH

```
# Contenedores
aws ecr get-login-password | docker login --username AWS \
  --password-stdin 111111111111.dkr.ecr.eu-west-1.amazonaws.com
aws ecs update-service --cluster prod --service api --force-new-deployment
aws eks update-kubeconfig --name prod --region eu-west-1

# Mensajería y datos
aws sqs send-message --queue-url "$COLA" --message-body '{"pedido": 42}'
aws sqs get-queue-attributes --queue-url "$COLA" \
  --attribute-names ApproximateNumberOfMessages
aws dynamodb get-item --table-name Pedidos --key '{"pk": {"S": "PEDIDO#42"}}'

# IaC, CDN y costes
aws cloudformation deploy --template-file plantilla.yaml --stack-name app \
  --capabilities CAPABILITY_NAMED_IAM --parameter-overrides Entorno=prod
aws cloudformation detect-stack-drift --stack-name app
aws cloudfront create-invalidation --distribution-id E1A2B3C4D5 --paths "/*"
aws ce get-cost-and-usage --time-period Start=2026-10-01,End=2026-11-01 \
  --granularity MONTHLY --metrics UnblendedCost \
  --group-by Type=DIMENSION,Key=SERVICE
```

CONSEJO

Añade `--query` (JMESPath) y `--output table` o `text` para filtrar la salida, `--dry-run` en EC2 para comprobar permisos sin ejecutar nada, y `--debug` para ver qué petición firma la CLI cuando algo falla.

¿Qué servicio uso para...?

Necesito...	Usa	Alternativa o matiz
Ejecutar código por eventos, en tareas cortas	Lambda	Contenedores si dura más de 15 min o la carga es constante
Contenedores sin gestionar servidores	ECS con Fargate	EKS si ya trabajas con Kubernetes
Kubernetes gestionado	EKS (Auto Mode)	ECS si no necesitas Kubernetes
Máquinas virtuales con control del SO	EC2 + Auto Scaling	Spot para lotes tolerantes a interrupciones
Orquestar un flujo de varios pasos	Step Functions	EventBridge para coreografía por eventos

Necesito...	Usa	Alternativa o matiz
Base de datos relacional gestionada	RDS o Aurora	Aurora DSQL para SQL distribuido multirregión
Clave-valor a gran escala con latencia de ms	DynamoDB	DAX si necesitas microsegundos
Caché en memoria	ElastiCache	DAX, específica para DynamoDB
Analítica SQL a gran escala (warehouse)	Redshift	Athena para consultas puntuales sobre S3
Consultar con SQL datos en S3	Athena	Redshift Spectrum desde el warehouse
Objetos: estáticos, backups, data lake	S3	Clases Glacier para archivo
Disco de bloque para una instancia	EBS	Instance store para datos temporales
Sistema de ficheros compartido	EFS	FSx (Windows, Lustre, ONTAP, OpenZFS)
Desacoplar con una cola	SQS	SQS FIFO si importa el orden
Un mensaje para muchos suscriptores	SNS	EventBridge si necesitas reglas más ricas o más destinos
Bus de eventos con reglas, SaaS o horarios	EventBridge	SNS para fan-out simple
Streaming en tiempo real con relectura	Kinesis Data Streams	Amazon MSK si necesitas Kafka
Cargar streams en S3 o Redshift sin código	Data Firehose	Glue para ETL por lotes
Publicar una API	API Gateway	ALB si ya hay contenedores o EC2 detrás
Balancear HTTP(S) con reglas por ruta	ALB	NLB para TCP/UDP, IP fija o latencia mínima
CDN y caché global	CloudFront	Global Accelerator para TCP/UDP sin caché
Conectar muchas VPC entre sí	Transit Gateway	VPC peering si son pocas
Ofrecer un servicio privado a otras VPC	PrivateLink	VPC Lattice para comunicación entre servicios
Conectar con el centro de datos	Site-to-Site VPN	Direct Connect para un enlace dedicado
Acceso de personas a varias cuentas	IAM Identity Center	Usuarios IAM solo como excepción
Registro e inicio de sesión de tus usuarios	Cognito	Incluye federación con Google, Apple o SAML
Secretos con rotación automática	Secrets Manager	Parameter Store para configuración
Claves de cifrado	KMS	CloudHSM para un HSM dedicado
Proteger una web de ataques de capa 7	AWS WAF	Shield Advanced frente a DDoS grandes
Auditar quién hizo qué	CloudTrail	AWS Config para cambios de configuración
Detectar amenazas y vulnerabilidades	GuardDuty e Inspector	Macie para datos sensibles en S3
Métricas, logs y alarmas	CloudWatch	X-Ray para trazas distribuidas
Infraestructura como código	CloudFormation o CDK	Terraform si usas varios proveedores
Copias de seguridad centralizadas	AWS Backup	Vault Lock para copias inmutables
Entrar en una instancia sin SSH	Session Manager	EC2 Instance Connect (SSH con claves temporales)